

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2026 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
інформаційних систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Вебзастосунок для організації хакатонів з динамічним модулем оцінювання

Виконав студент IV курсу, групи ІП-24
(шифр групи)

Малий Олександр Сергійович
(прізвище, ім'я, по батькові) _____ (підпис)

Керівник ст. викл., Марченко О. І.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Консультант асистент, Шулькевич Т. В.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Рецензент доц. каф. КІОНС, к.т.н., Мураховський С. А.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Засвідчую, що у цьому дипломному проєкті немає запозичень з праць інших авторів без відповідних посилань.

Студент _____
(підпис)

Київ – 2026

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії
Рівень вищої освіти – перший (бакалаврський)
Спеціальність – 121 Інженерія програмного забезпечення
Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

Едуард ЖАРІКОВ

(підпис)

(ім'я прізвище)

“ ” 2026 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Малому Олександр Сергійовичу

(прізвище, ім'я, по батькові)

1. Тема проєкту Вебзастосунок для організації хакатонів з динамічним модулем оцінювання

керівник проєкту Марченко Олена Іванівна, ст. викл.

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «03» червня 2026 р. №2055-с

2. Термін подання студентом проєкту «14» червня 2026 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Передпроектне обстеження предметної області: постановка завдання дипломного проєктування, аналіз предметної області, аналіз існуючих рішень, аналіз та моделювання бізнес-процесів.

2) Розроблення вимог до програмного забезпечення: варіанти використання програмного забезпечення, розроблення функціональних та нефункціональних вимог, аналіз системних вимог, аналіз економічних показників програмного забезпечення, постановка завдання на розробку програмного забезпечення.

3) Конструювання та розроблення програмного забезпечення: архітектура програмного забезпечення, архітектурні рішення та обґрунтування вибору засобів розробки, конструювання програмного забезпечення, аналіз безпеки даних.

4) Аналіз якості та тестування програмного забезпечення: аналіз якості ПЗ, опис процесів тестування, опис контрольного прикладу, висновки до розділу.

5) Розгортання та супровід програмного забезпечення: розгортання програмного забезпечення, супровід програмного забезпечення.

6) Висновки.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використань

2) Схема структурна компонентів програмного забезпечення

3) Схема бази даних

4) Схема структурна класів програмного забезпечення

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «15» березня 2026 року

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	15.02.2026	
2	Аналіз існуючих методів розв'язання задачі	20.02.2026	
3	Постановка та формалізація задачі	10.03.2026	
4	Розробка інформаційного забезпечення	25.03.2026	
5	Алгоритмізація задачі	30.03.2026	
6	Обґрунтування вибору використаних технічних засобів	10.04.2026	
7	Розробка програмного забезпечення	05.05.2026	
8	Налагодження програми	10.05.2026	
9	Виконання графічних документів	13.05.2026	
10	Оформлення пояснювальної записки	15.06.2026	
11	Подання ДП на попередній захист	18.06.2026	
12	Подання ДП рецензенту	10.06.2026	
13	Подання ДП на основний захист	14.06.2026	

Студент

(підпис)

Олександр МАЛИЙ

(ініціали, прізвище)

Керівник

(підпис)

Олена МАРЧЕНКО

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з п'яти розділів, містить 64 таблиці, 58 рисунків та 21 джерело – загалом 122 сторінки.

Дипломний проєкт присвячений розробці вебзастосунку «h.linker» для комплексної організації хакатонів, ІТ-змагань та освітніх конкурсів із вбудованим динамічним модулем оцінювання.

Мета роботи – створення єдиного інтерактивного середовища для автоматизації процесів реєстрації учасників, формування, подачі проєктів та їх суддівського оцінювання за допомогою динамічної системи критеріїв

У першому розділі проведено передпроєктне обстеження предметної області, проаналізовано існуючі програмні продукти та технічні рішення на ринку, а також виконано аналіз і моделювання ключових процесів системи.

Другий розділ присвячений розробленню вимог до програмного забезпечення. У ньому описано варіанти використання, сформовано детальні функціональні, нефункціональні та системні вимоги, а також проведено розрахунок економічних показників і метрик вартості розробки за допомогою методу функціональних точок.

У третьому розділі описано процеси конструювання та розроблення програмного забезпечення. Обґрунтовано вибір клієнт-серверної архітектури на базі монорепозіторію Nx, обрано технологічний стек (Angular, NestJS, PostgreSQL, ORM Prisma), описано структуру БД та алгоритми безпеки.

Четвертий розділ присвячений аналізу якості та тестуванню створеного продукту. Описано застосування комплексного підходу: статичного аналізу коду, автоматизованого модульного тестування за допомогою Jest, навантажувального тестування бекенду, аудиту продуктивності та виконання мануальних тест-кейсів контрольного прикладу.

У п'ятому розділі розглянуто питання розгортання програмного забезпечення у хмарному середовищі за допомогою контейнеризації Docker та визначено регламент подальшого супроводу системи.

Результати роботи пройшли апробацію на науково-практичній конференції «Роль науки у вирішенні глобальних викликів сучасності» та опубліковано в збірнику конференції.

КЛЮЧОВІ СЛОВА: ХАКАТОН, ВЕБЗАСТОСУНОК, УПРАВЛІННЯ КОМАНДАМИ, ОЦІНЮВАННЯ, ВІДГУКИ, РЕЙТИНГ, РЕКОМЕНДАЦІЇ, GITHUB, GOOGLE, МОНОРЕПОЗИТОРІЙ, NODE.JS, КЛІЄНТ-СЕРВЕРНА АРХІТЕКТУРА, MICROSOFT AZURE.

ABSTRACT

The explanatory note of the diploma project consists of five sections, contains 64 tables, 58 figures and 21 sources – in total 122 pages.

The diploma project is dedicated to the development of the «h.linker» web application for the comprehensive organization of hackathons, IT competitions, and educational contests with a built-in dynamic evaluation module.

The purpose of the diploma project is the creation of a unified interactive environment for automating the processes of participant registration, team formation, project submission, and their judging evaluation using a dynamic criteria system.

In the first section, a pre-project survey of the subject area is conducted, existing software products and technical solutions on the market are analyzed, and the analysis and modeling of the key processes of the system are performed.

The second section is dedicated to the development of software requirements. It describes use cases, forms detailed functional, non-functional, and system requirements, and also conducts the calculation of economic indicators and development cost metrics using the functional point method.

The third section describes the processes of software construction and development. The choice of client-server architecture based on the Nx monorepo is justified, the technology stack (Angular, NestJS, PostgreSQL, ORM Prisma) is selected, and the database structure and security algorithms are described.

The fourth section is dedicated to the quality analysis and testing of the created product. The application of a comprehensive approach is described: static code analysis, automated modular (unit) testing using Jest, backend load testing, performance audit, and the execution of manual test cases of the control example.

The fifth section considers the issues of software deployment in a cloud environment using Docker containerization and defines the regulations for further system maintenance.

The results of the work underwent approbation at the scientific and practical conference «The Role of Science in Solving Contemporary Global Challenges» and are published in the conference proceedings.

KEYWORDS: HACKATHON, WEB APPLICATION, TEAM MANAGEMENT, EVALUATION, REVIEWS, RATING, RECOMMENDATIONS, GITHUB, GOOGLE, MONOREPO, NODE.JS, CLIENT-SERVER ARCHITECTURE, MICROSOFT AZURE.

[illegible]

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

«ЗАТВЕРДЖЕНО»

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ВЕБЗАСТОСУНОК ДЛЯ ОРГАНІЗАЦІЇ ХАКАТОНІВ З
ДИНАМІЧНИМ МОДУЛЕМ ОЦІНЮВАННЯ**

Технічне завдання

КПІ.ПІ-2419.045440.01.91

«ПОГОДЖЕНО»

Керівник проєкту:

_____ Олена МАРЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

Виконавець:

_____ Олександр МАЛИЙ

Київ – 2026

ЗМІСТ

1	НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	3
2	ПІДСТАВА ДЛЯ РОЗРОБКИ.....	4
3	ПРИЗНАЧЕННЯ РОЗРОБКИ	5
4	ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
4.1	Вимоги до функціональних характеристик	6
4.1.1	Користувацького інтерфейсу	6
4.1.2	Облік користувачів:.....	16
4.2	Вимоги до надійності	17
4.3	Умови експлуатації.....	18
4.3.1	Вид обслуговування	18
4.3.2	Обслуговуючий персонал.....	18
4.4	Вимоги до складу і параметрів технічних засобів	18
4.5	Вимоги до інформаційної та програмної сумісності	19
4.5.1	Вимоги до вхідних даних	19
4.5.2	Вимоги до вихідних даних	19
4.5.3	Вимоги до мови розробки.....	19
4.5.4	Вимоги до середовища розробки.....	20
4.5.5	Вимоги до представленню вихідних кодів	20
4.6	Вимоги до маркування та пакування	20
4.7	Вимоги до транспортування та зберігання	20
4.8	Спеціальні вимоги.....	20
5	ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ	21
5.1	Попередній склад програмної документації	21
5.2	Спеціальні вимоги до програмної документації.....	21
6	СТАДІЇ І ЕТАПИ РОЗРОБКИ	22
7	ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ	23

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Вебзастосунок для організації хакатонів з динамічним модулем оцінювання.

Галузь застосування:

Наведене технічне завдання поширюється на розробку вебзастосунку для організації хакатонів h.linker [КП.ІІІ-2419.045440.01.91], котра використовується для автоматизації повного життєвого циклу заходу: від реєстрації учасників і формування команд до збору проєктів та підрахунку результатів за гнучкими критеріями та призначена для використання в освітніх закладах, ІТ-компаніях та громадських організаціях, що займаються проведенням командних змагань (хакатонів, студентських олімпіад, воркшопів).

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки вебзастосунку для організації хакатонів з динамічним модулем оцінювання є завдання на дипломне проєктування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для забезпечення комплексної функціональної підтримки процесів організації та проведення хакатонів, включаючи реєстрацію учасників, автоматизоване формування команд, збір проєктів через інтеграцію з GitHub API, а також надання інструментарію для прозорого оцінювання робіт членами журі. ПЗ повинно бути зручним у використанні, забезпечувати швидкий доступ до аналітичних даних та результатів, щоб відповідати потребам організаторів, учасників та експертів.

Метою розробки є мінімізація кількості помилок, спричинених людським фактором при підрахунку балів, підвищення точності та об'єктивності оцінювання завдяки використанні зважених коефіцієнтів, оптимізація процесів взаємодії між учасниками та скорочення часу на зведення результатів змагання завдяки автоматизації розрахункових операцій.

Мета дослідження – створити вебзастосунок з динамічним модулем оцінювання, який суттєво скоротить час, необхідний для адміністрування хакатону, налаштування критеріїв та обробки фінальних результатів. Завдяки автоматизації обчислень на стороні сервера та інтерфейсу для всіх ролей користувачів, застосунок забезпечить ефективніше використання часу як організаторами, так і журі, перевищуючи показники швидкості та гнучкості порівняно з традиційними підходами на основі електронних таблиць.

Поставлена мета обов'язково повинна бути досягнута, досягнення мети повинно чітко визначатись у висновках дипломної роботи.

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1 Користувацького інтерфейсу

– реєстрація та авторизація (рис. 4.1);

Користувач може створити обліковий запис або увійти в систему за допомогою стандартної форми або інтеграції з сервісами GitHub та Google.

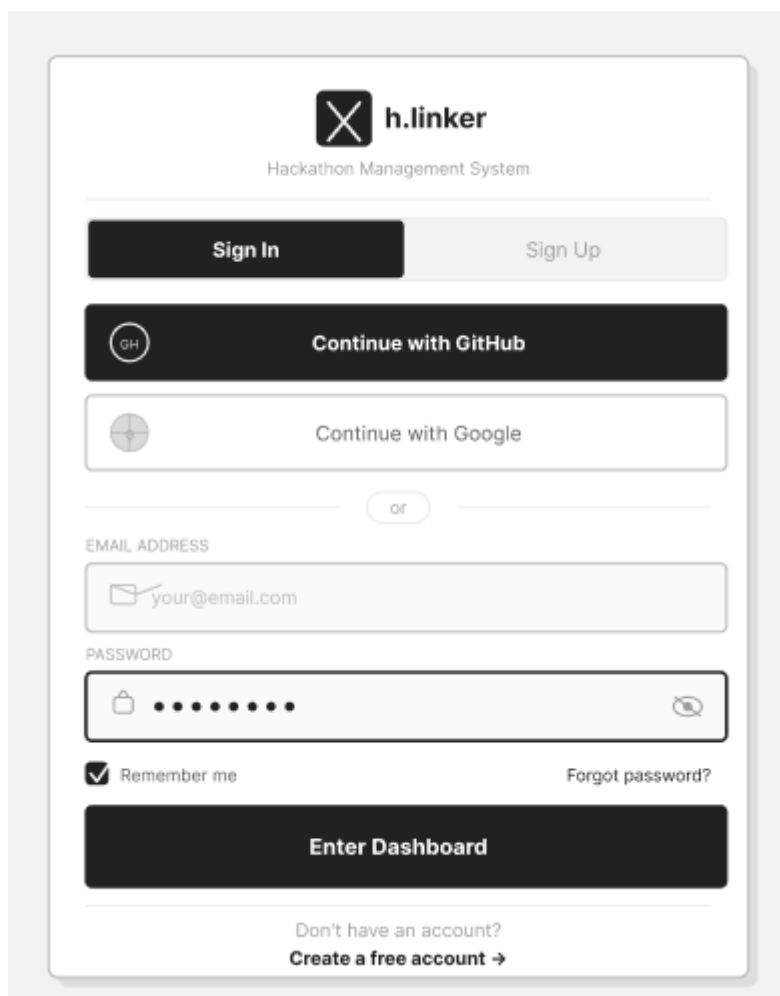
The image shows a login interface for 'h.linker Hackathon Management System'. At the top is the logo and name. Below are two buttons: 'Sign In' (dark) and 'Sign Up' (light). Then, there are two large buttons for social login: 'Continue with GitHub' (dark with GitHub icon) and 'Continue with Google' (light with Google icon). A small 'or' separator is between them. Below is a form with 'EMAIL ADDRESS' and 'PASSWORD' labels. The email field contains 'your@email.com'. The password field is masked with dots and has a toggle icon. There is a 'Remember me' checkbox (checked) and a 'Forgot password?' link. At the bottom is a large dark button 'Enter Dashboard'. At the very bottom, it says 'Don't have an account? Create a free account ->'.

Рисунок 4.1 – Сторінка авторизації користувача

– перегляд та пошук хакатонів (рис. 4.2);

Головна сторінка дозволяє користувачу переглядати перелік активних та майбутніх подій з можливістю фільтрації за категоріями.

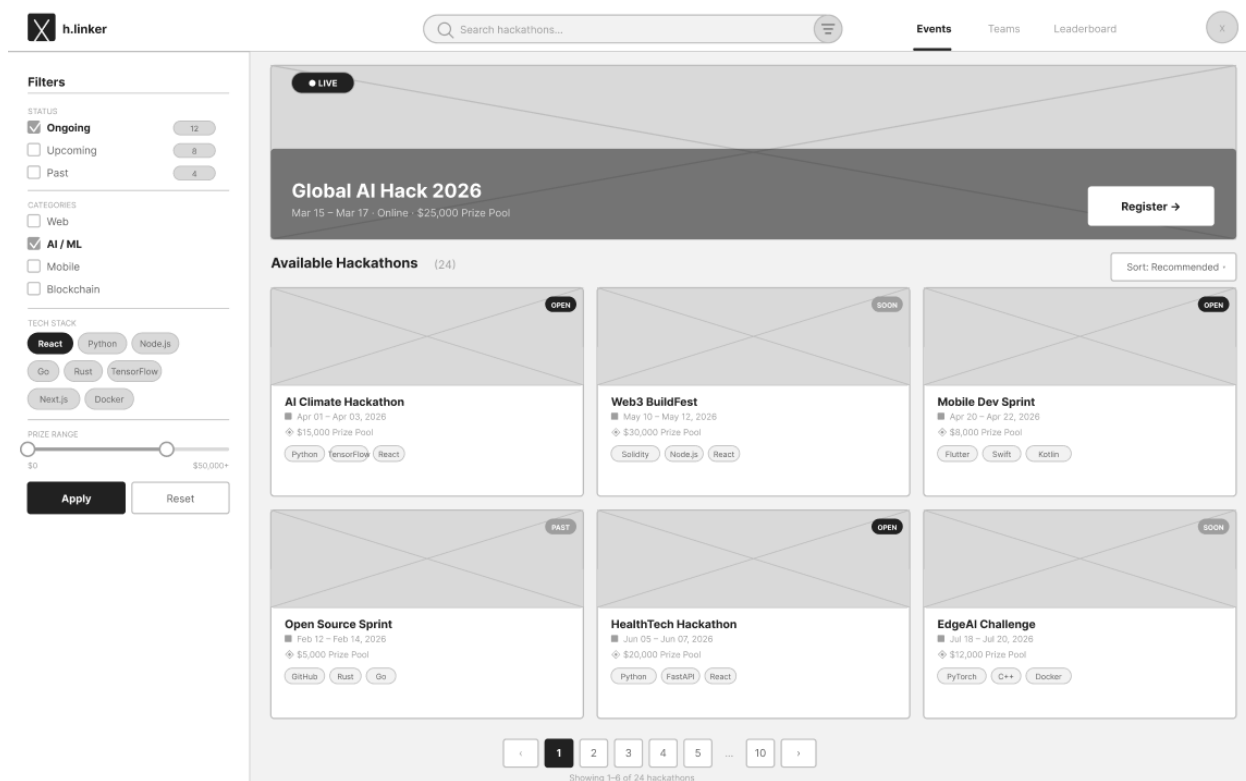


Рисунок 4.2 – Сторінка перегляду списку хакатонів

– детальна інформація про хакатон (рис. 4.3);

Сторінка містить опис події, регламент, призовий фонд та кнопки для реєстрації або пошуку команди.

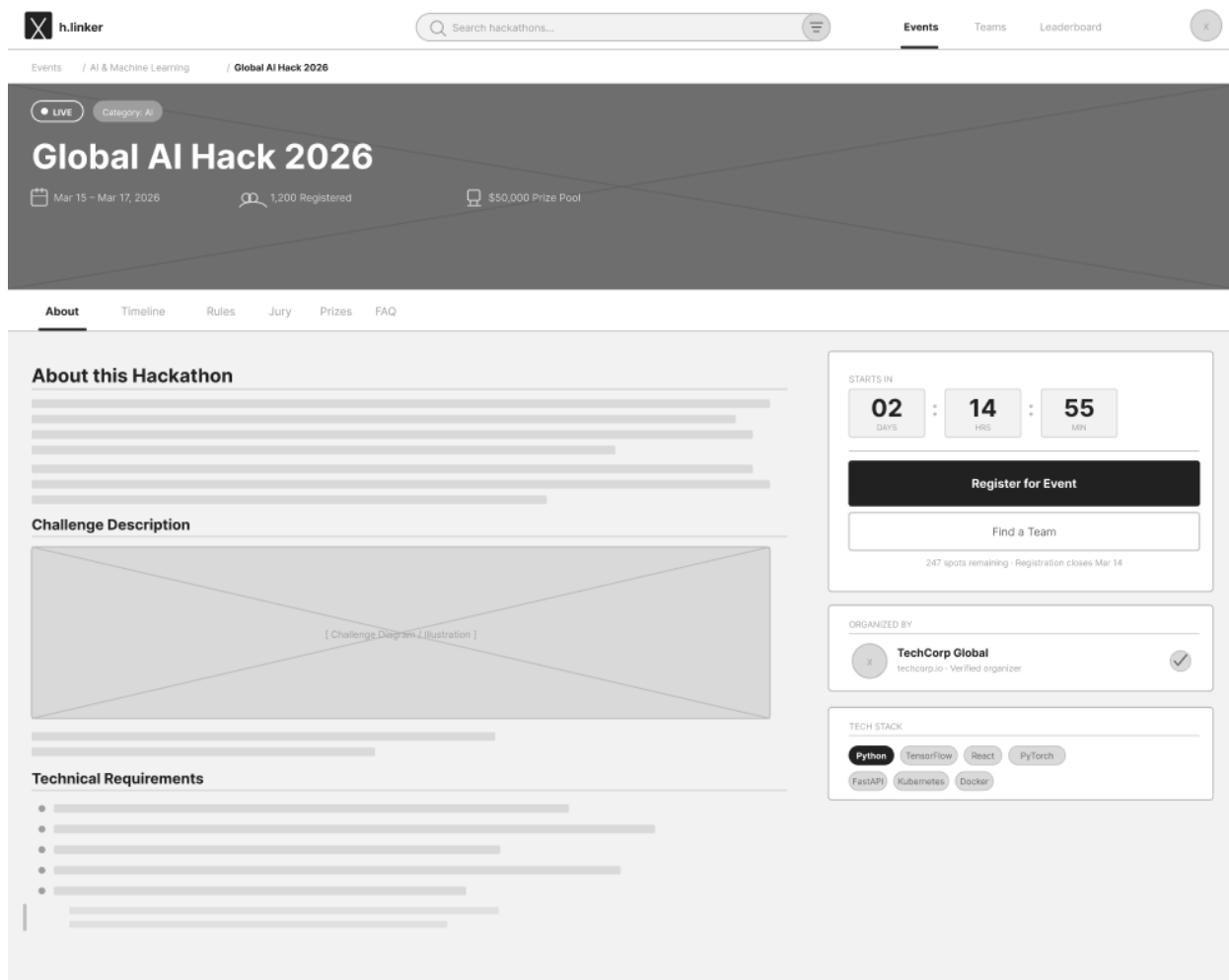


Рисунок 4.3 – Сторінка хакатону

— особистий профіль та налаштування (рис. 4.4 – 4.5);

Користувач має змогу редагувати біографію, налаштовувати технічний стек та синхронізувати дані з GitHub.

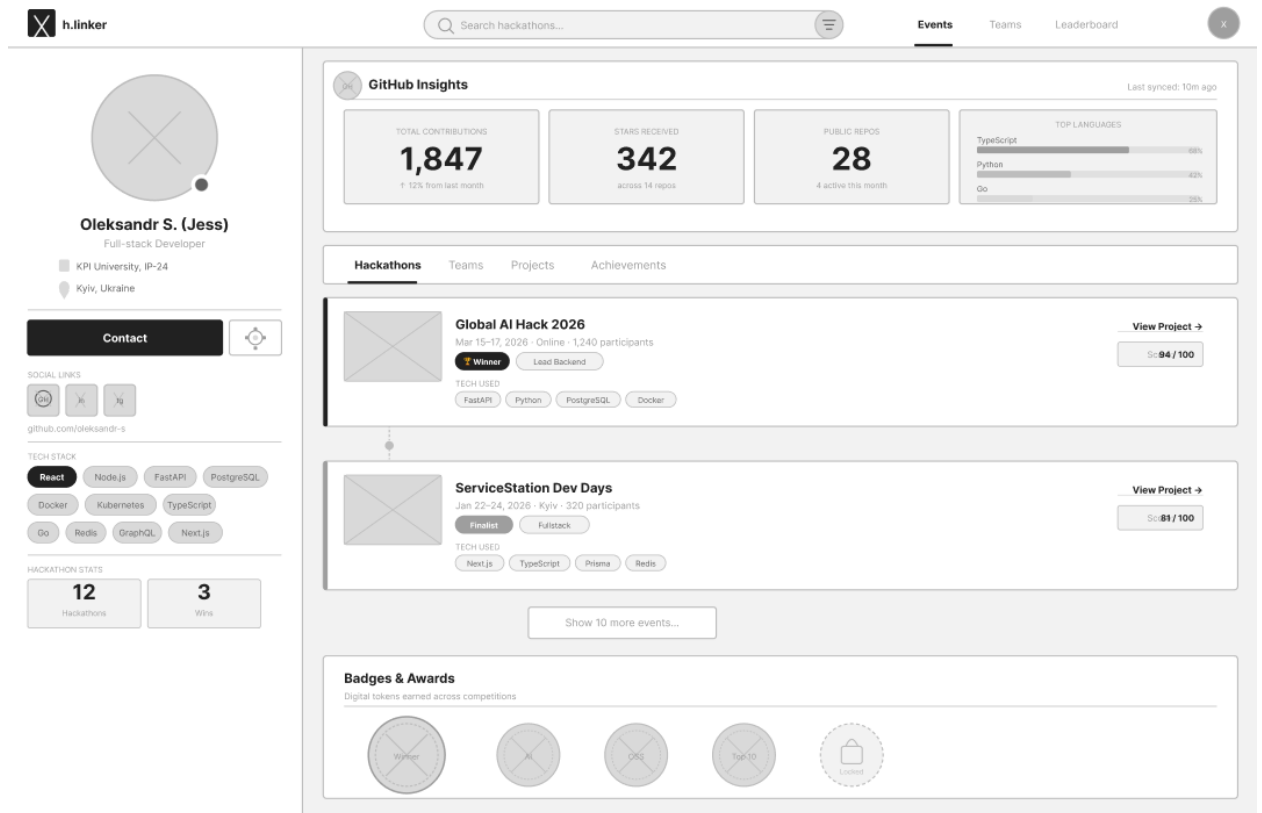


Рисунок 4.4 – Сторінка профілю користувача

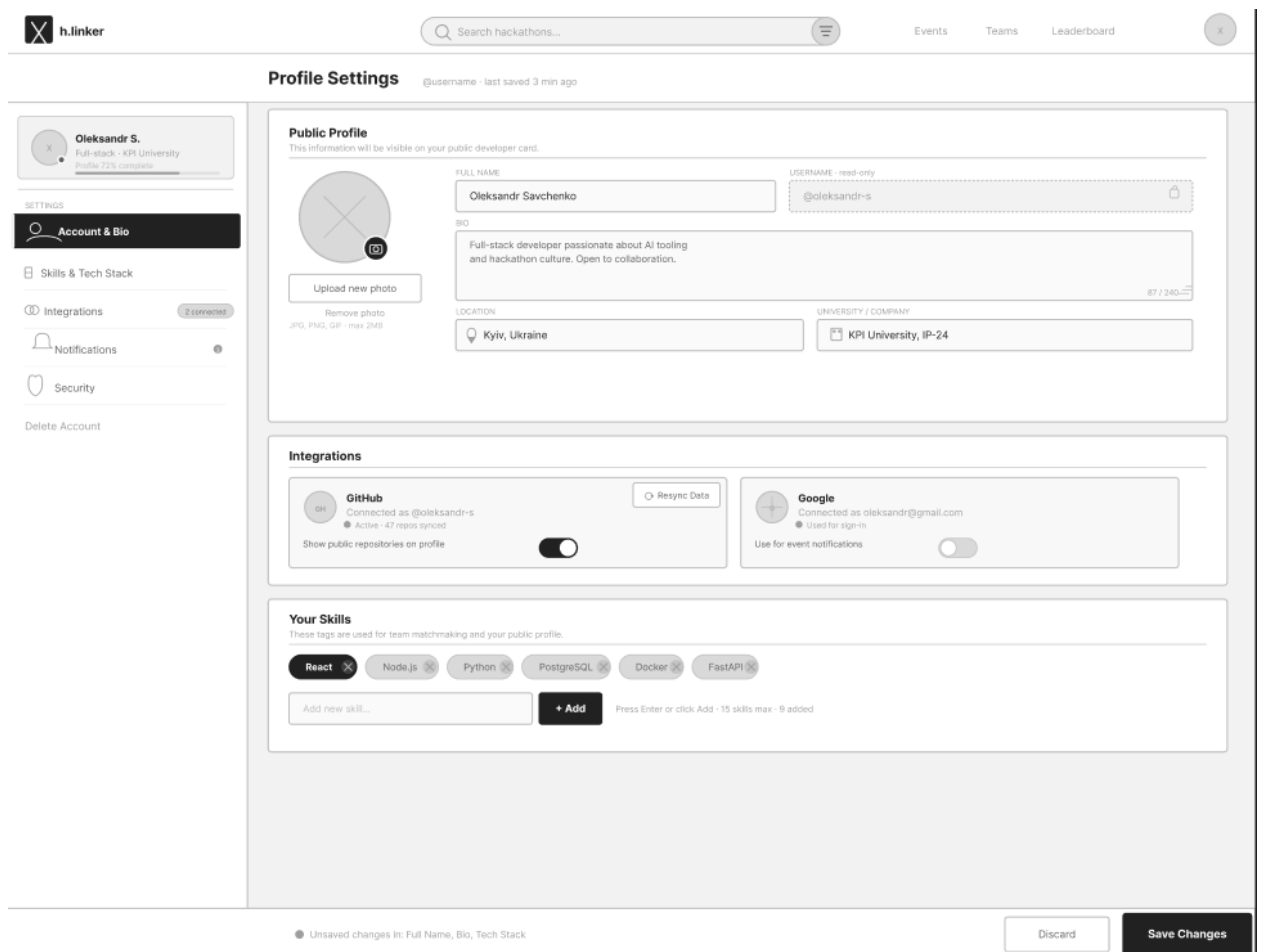


Рисунок 4.5 – Сторінка налаштувань профілю

- пошук учасників та команд (Matchmaking) (рис. 4.6 – 4.7);
- Спеціалізовані інтерфейси для підбору тіммейтів за навичками або пошуку вільних місць у існуючих командах.

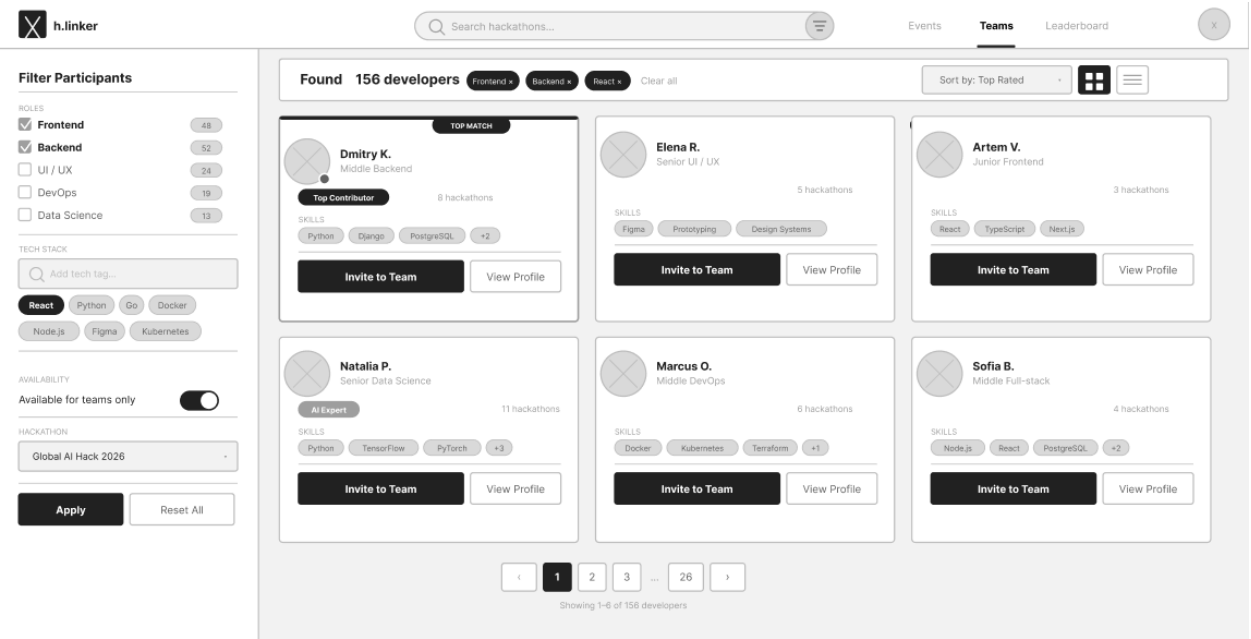


Рисунок 4.6 – Сторінка пошуку учасників

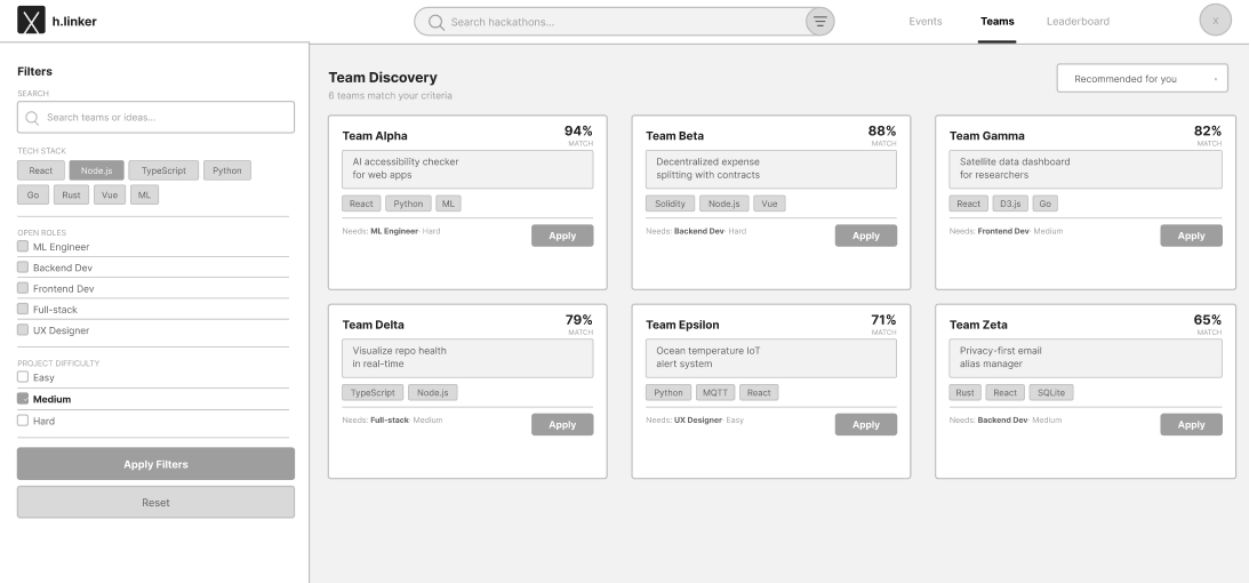


Рисунок 4.7 – Сторінка пошуку команд

- робочий простір команди (рис. 4.8);
- Центр керування проєктом, де лідер команди може прив’язати GitHub-репозиторій, керувати складом учасників та переглядати посилання на зовнішній чат.

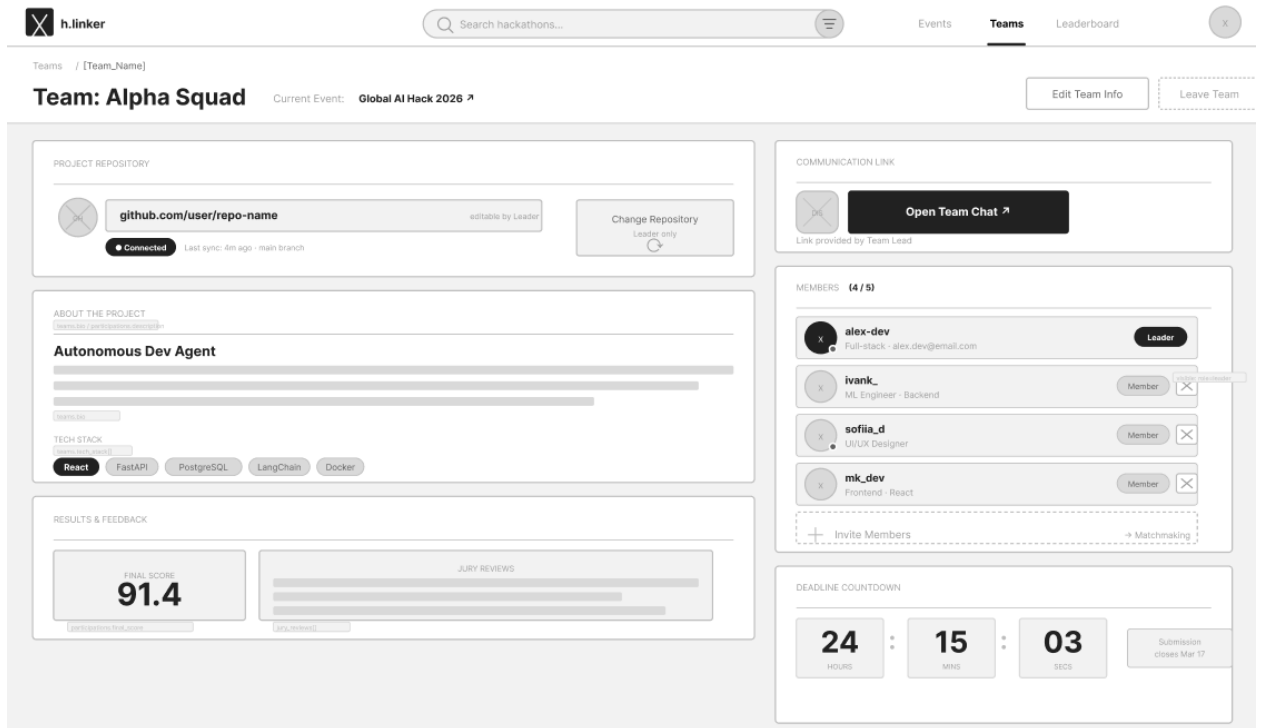


Рисунок 4.8 – Сторінка команди

— панель поточного хакатону (рис. 4.9);

Інформаційний дашборд учасника з таймером до дедлайну, анонсами від організаторів та кнопкою подання проєкту.

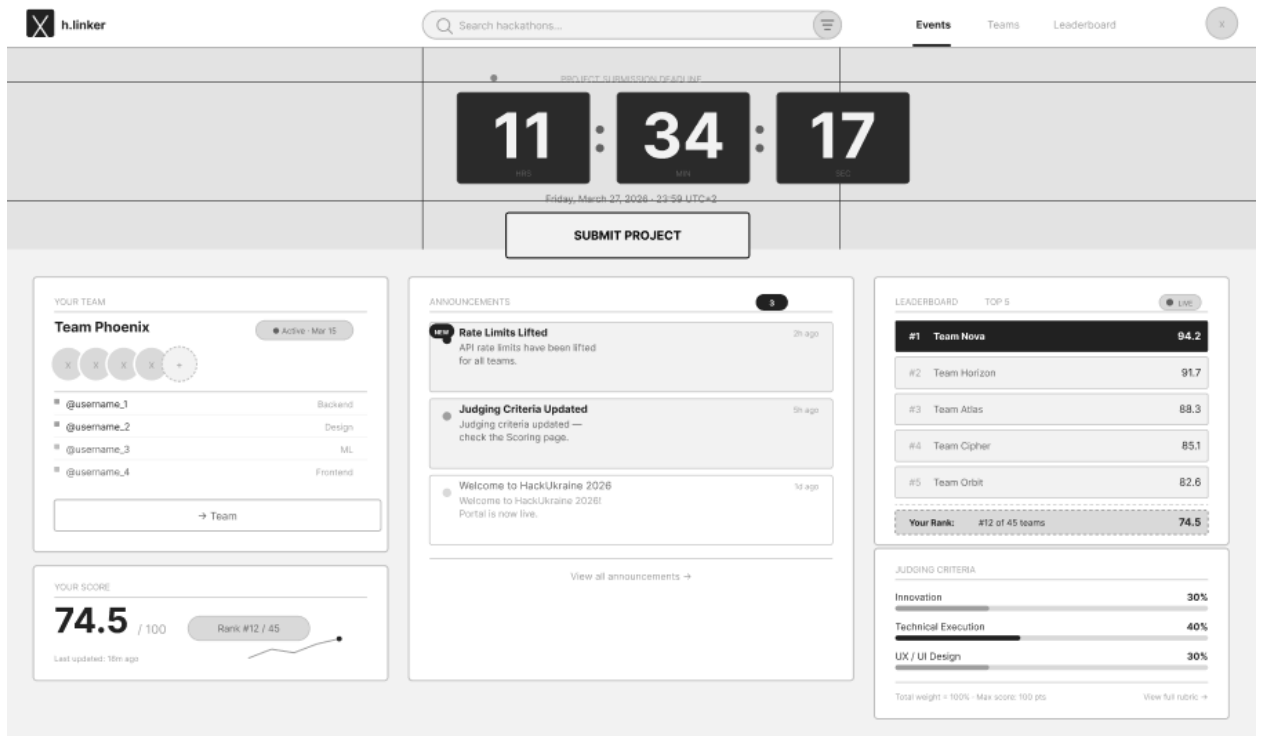


Рисунок 4.9 – Сторінка поточного хакатону

— подання проєкту (рис. 4.10 – 4.12);

Форма здачі роботи, що передбачає введення посилання на репозиторій та опис реалізованого функціоналу.

Project Submission
Team Phoenix - HackUkraine 2024

1 INFO 2 REPOSITORY 3 FINALIZE

Project Info

PROJECT TITLE *

Enter your project title...

PROJECT DESCRIPTION * 0 / 500

Describe your project: the problem it solves, your approach, and key features...

CATEGORIES / TAGS

AI / ML Web3 DevTools Accessibility Climate Tech EdTech HealthTech Security Data Viz Open Source

Selected: AI / ML, DevTools

Fields marked * are required before continuing.

Save Draft Continue -> Repository

Рисунок 4.10 – Сторінка подання проєкту

Link GitHub Repository

Connected as @username- 5 repositories fetched

SELECT REPOSITORY *

username/project-backend

username/react-dashboard Selected

username/ml-pipeline

username/node-auth

username/redis-wrapper

OR PASTE URL MANUALLY

https://github.com/your-org/repo-name

AUTO-FETCHED REPO STATS

312 Commits 4 Contributors 2h ago Last Push

Back Save Draft Continue -> Finalize

Рисунок 4.11 – Сторінка вибору репозиторію

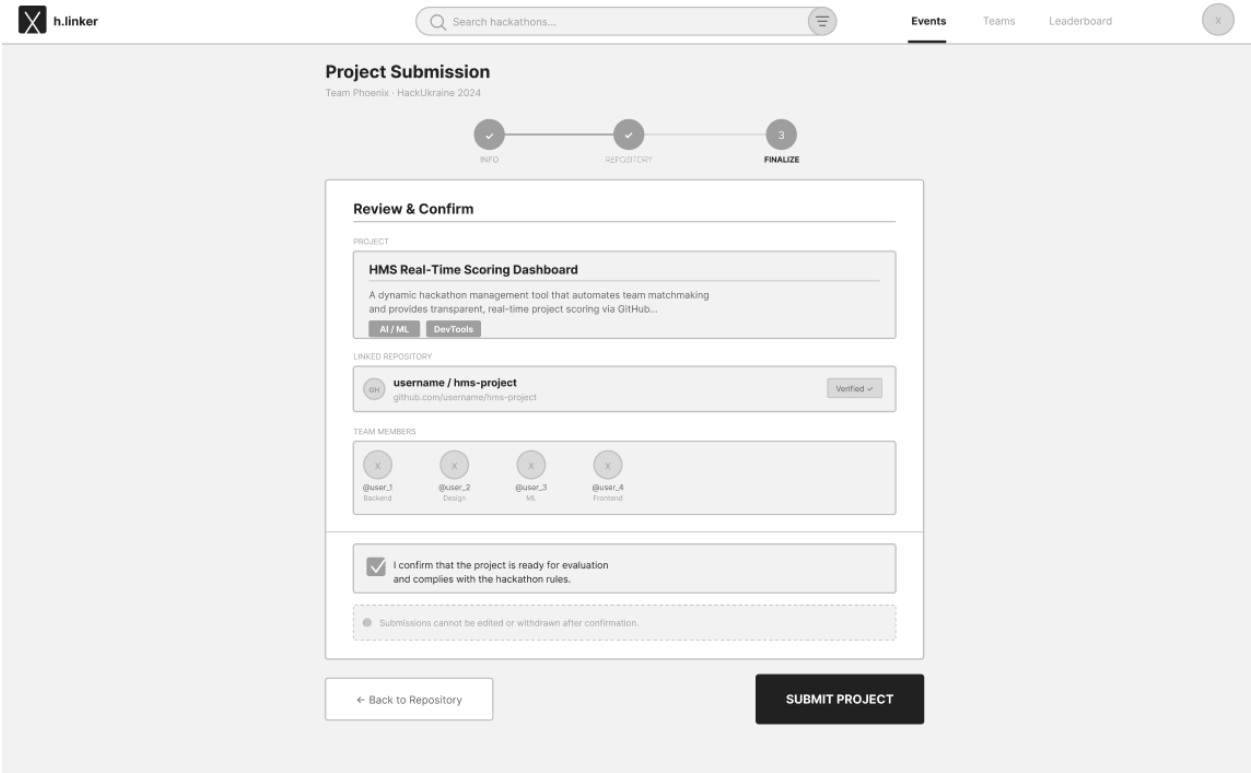


Рисунок 4.12 – Сторінка підтвердження

— оцінювання та результати (рис. 4.13 – 4.14);

Портал для журі з можливістю виставлення балів та глобальна таблиця результатів (Leaderboard).

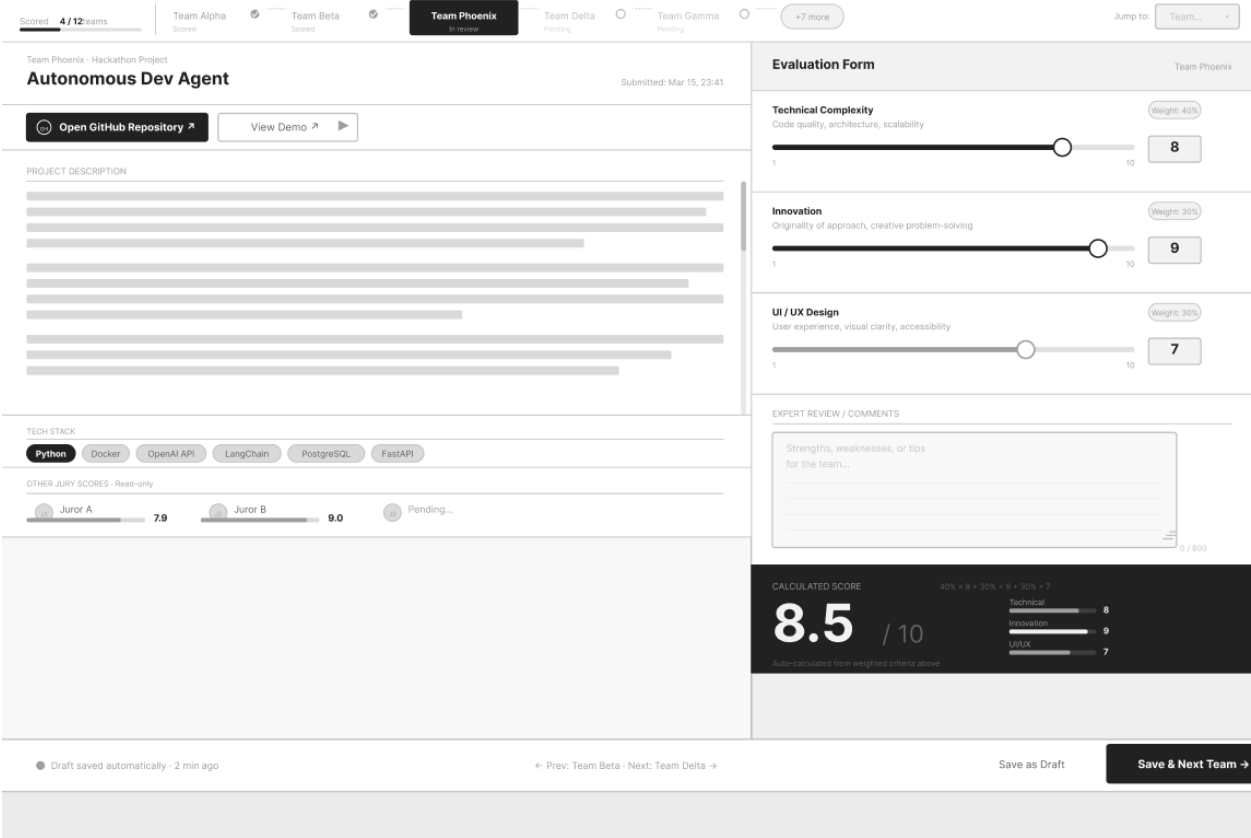


Рисунок 4.13 – Сторінка оцінювання журі

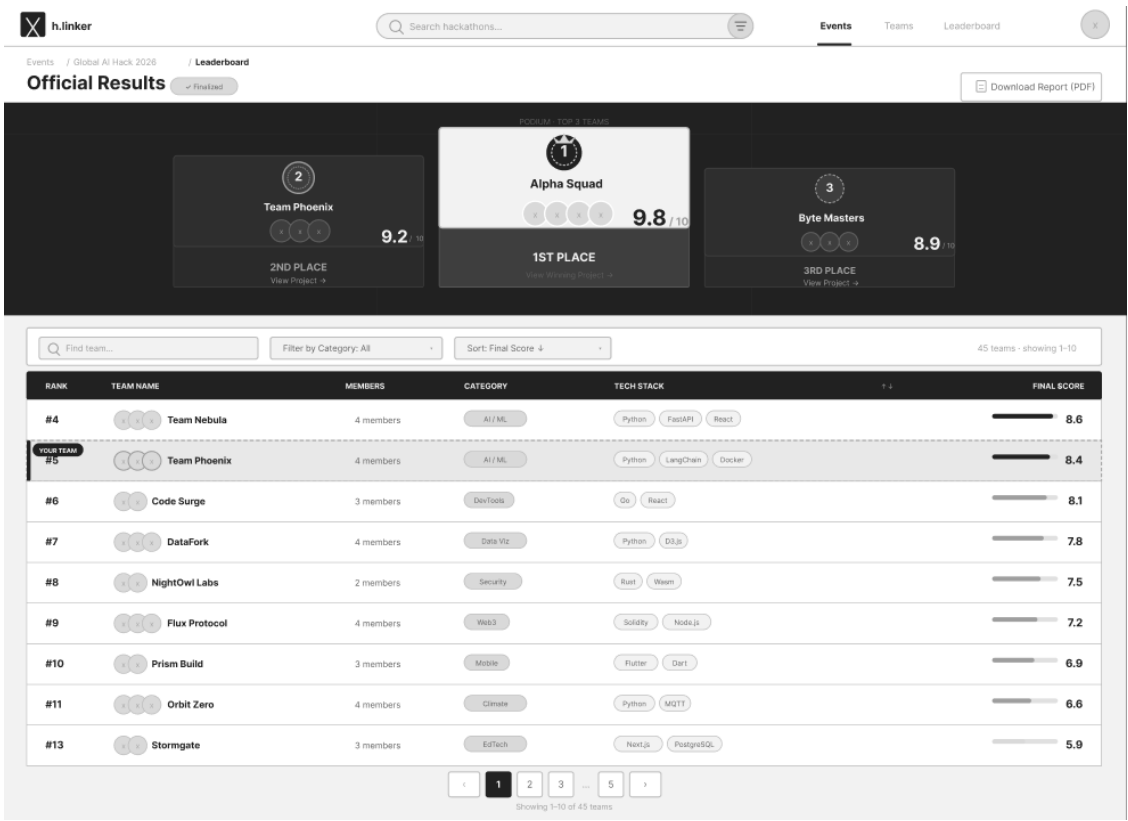


Рисунок 4.14 – Сторінка лідерборду

– архів найкращих проєктів (рис. 4.15);

Галерея Showcase для перегляду успішних кейсів минулих подій.

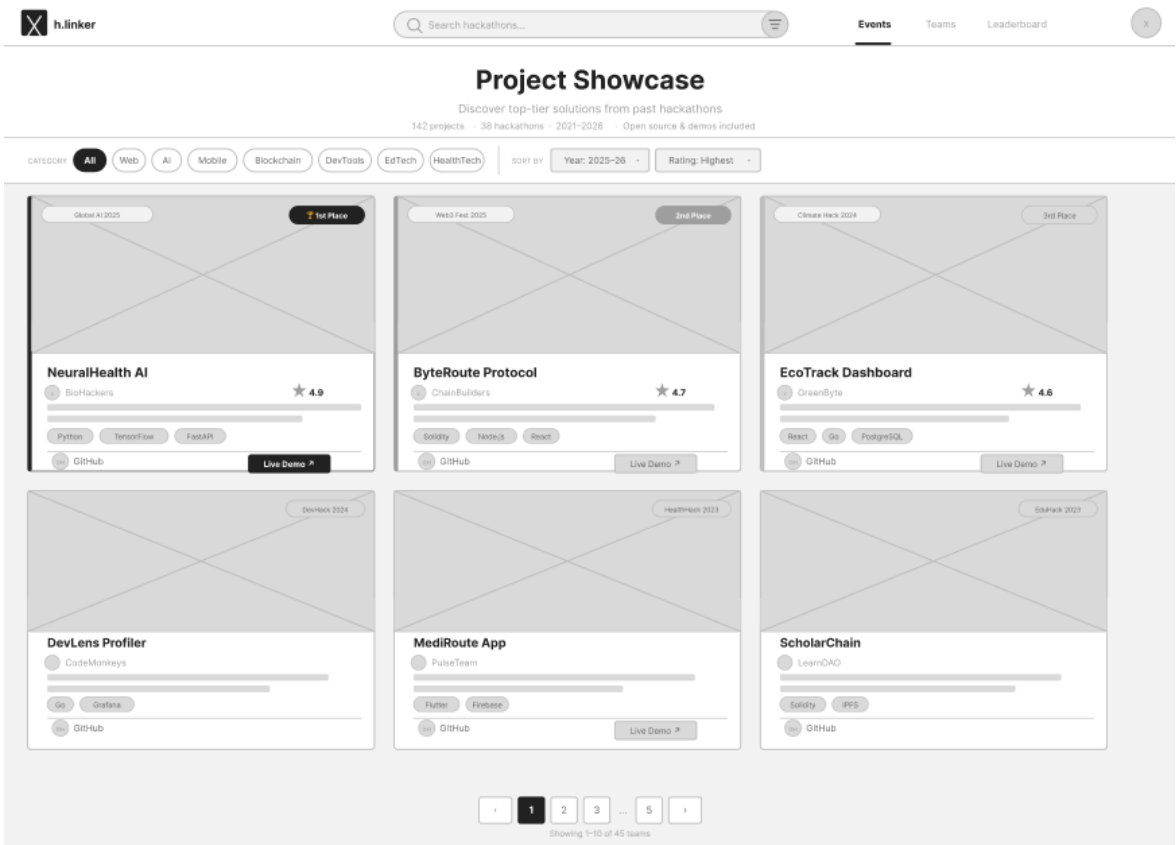


Рисунок 4.15 – Сторінка найкращих проєктів

— панель адміністратора (рис. 4.16);

Центральний вузол керування хакатоном, що відображає статистику реєстрацій, поточний статус події та дозволяє модерацію списків команд та журі.

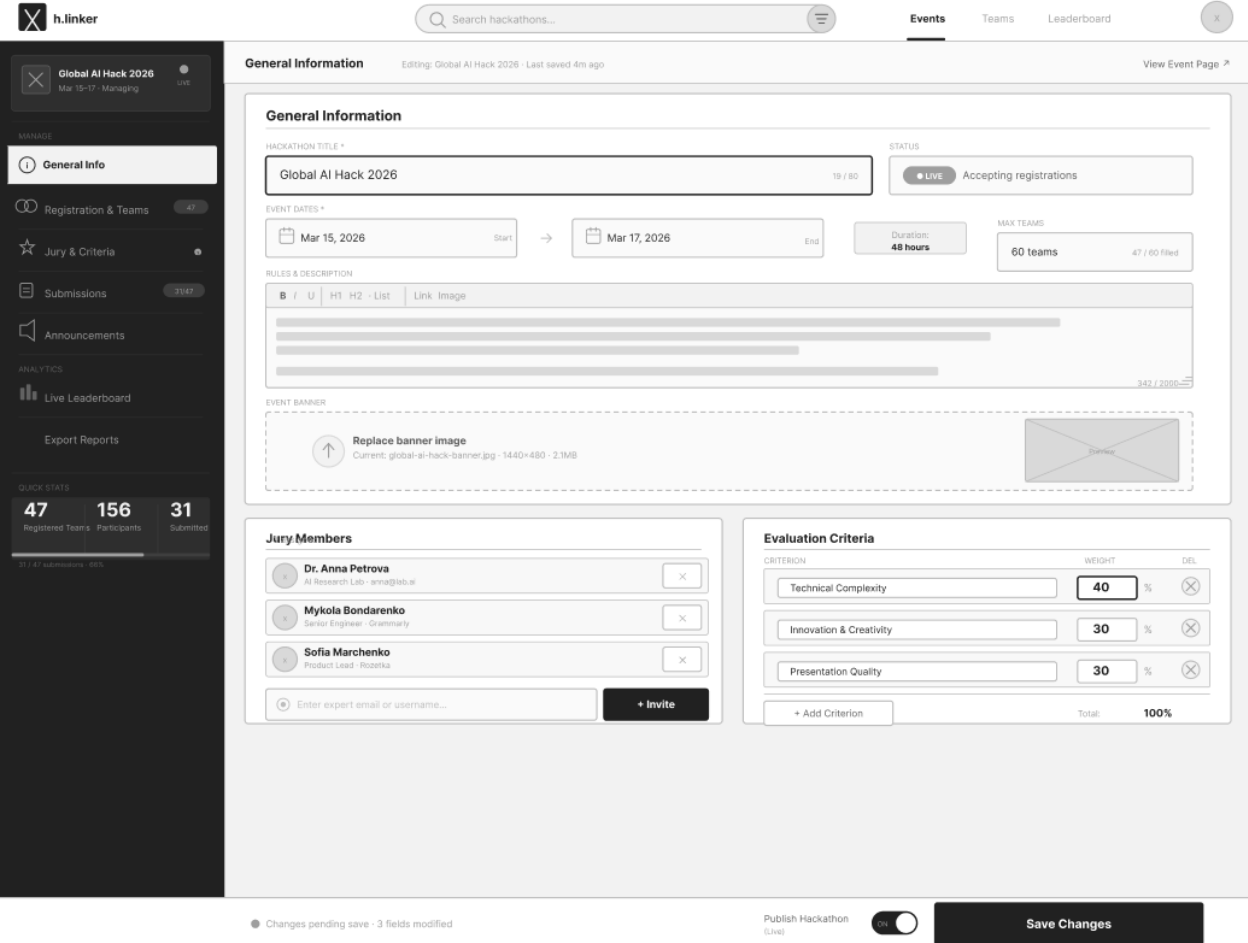


Рисунок 4.16 – Сторінка панелі адміністратора

— конструктор створення події (рис. 4.17).

Спеціалізована форма для введення метаданих хакатону: назви, призового фонду, налаштування вагових коефіцієнтів критеріїв оцінювання та часових рамок етапів.

Рисунок 4.17 – Сторінка створення хакатону

4.1.2 Облік користувачів:

4.1.2.1 Для користувача (Учасник/Лідер):

- реєстрація та створення облікового запису;
- авторизація;
- вихід з акаунту;
- оновлення та редагування особистої інформації у профілі;
- вибір навичок для рекомендацій підходящих команд та хакатонів;
- формування команд: створення нової команди, пошук учасників, надсилання запрошень та запитів на вступ, видалення учасників;
- пошук та перегляд хакатонів;

- реєстрація на хакатон;
- подання проєкту;
- перегляд оцінок та текстових відгуків від журі;
- перегляд найкращих проєктів.

4.1.2.2 Для творця хакатону (Організатор):

- створення хакатону із зазначенням назви, опису, дат проведення та призового фонду;
- формування списку журі та надання їм прав доступу до оцінювання робіт;
- створення критеріїв оцінки з визначенням їхньої «ваги» у відсотках для динамічного оцінювання;
- моніторинг та статистика;
- зміна статусів хакатону (реєстрація, активний, оцінювання, завершено).

4.1.2.3 Для журі:

- виставлення балів за кожним критерієм для команд у зазначеній шкалі (наприклад 1-10);
- написання та редагування рецензій до поданих проєктів у вигляді коментарів (загально, сильні та слабкі сторони).

4.1.2.4 Додаткові вимоги:

- авторизація через інтеграцію з Google та GitHub;
- автоматизація розрахунків зваженого балу команди на основі формули організатора;
- автоматичне підтягування метаданих репозиторію через GitHub Арі (мови програмування, README, останній коміт).

4.2 Вимоги до надійності

Передбачити контроль введення інформації та захист від некоректних дій користувача. Забезпечити цілісність інформації в базі даних.

Забезпечення конфіденційності даних користувачів передбачає використання сучасних методів шифрування при зберіганні та передачі чутливої інформації.

Заблокувати можливості зміни критеріїв оцінювання після початку роботи журі для забезпечення прозорості та цілісності результатів.

Передбачити обробку виняткових ситуацій під час роботи із зовнішніми Арі.

4.3 Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.1 Вид обслуговування

Обслуговування системи під час експлуатації полягає у періодичному моніторингу працездатності серверної частини, оновленні залежностей безпеки для Node.js та бібліотек, а також створенні резервних копій бази даних PostgreSQL згідно з регламентом надійності. Взаємодія із зовнішніми АРІ (GitHub, Google, ImgBB) не потребує обслуговування з боку розробника, оскільки підтримується відповідними сервісними провайдерами.

4.3.2 Обслуговуючий персонал

Обслуговуючий персонал буде необхідний для виконання основних операцій обслуговування, таких як моніторинг системи, оновлення програмного забезпечення, виправлення помилок та забезпечення безпеки даних.

4.4 Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на ІВМ-сумісних персональних комп'ютерах.

Мінімальна конфігурація технічних засобів:

- операційна система: Windows 10;

- об'єм ОЗП: 4 Гб;
- браузер: підтримка сучасних браузерів
- підключення до мережі Інтернет зі швидкістю від 20 мегабіт;

Рекомендована конфігурація технічних:

- операційна система: Windows 10;
- тип процесору: Ryzen 7 5700x;
- об'єм ОЗП: 32 Гб;
- браузер: найновіші версії браузерів з підтримкою HTML5 і

JavaScript;

- підключення до мережі Інтернет зі швидкістю від 100 мегабіт;

4.5 Вимоги до інформаційної та програмної сумісності

ПЗ повинно бути сумісним з основними операційними системами, як Windows, Linux, macOS, і браузерами: Google Chrome, Microsoft Edge, Opera.

4.5.1 Вимоги до вхідних даних

Вхідні дані повинні бути представлені в наступному форматі:

- формат зображень: png, jpg, gif, svg.

4.5.2 Вимоги до вихідних даних

Вимоги до вихідних даних не висуваються.

4.5.3 Вимоги до мови розробки

Розробку вебзастосунку виконати на наступних мовах програмування та технологіях

- front-end: мова програмування TypeScript з використанням з використанням фреймворку Angular для створення динамічного та інтерактивного користувацького інтерфейсу, CSS;

- back-end: мова програмування TypeScript на базі платформи Node.js з використанням фреймворку Nest для обробки HTTP-запитів і реалізації логіки серверної частини;
- база даних: PostgreSQL як основна система управління базами даних, яка забезпечує надійність і високу продуктивність для зберігання інформації. Зв'язок між серверною частиною та БД за допомогою Prisma ORM.

4.5.4 Вимоги до середовища розробки

Розробку виконати за допомогою середовища WebStorm.

4.5.5 Вимоги до представленню вихідних кодів

Вихідний код програми має бути представлений у вигляді кодів у системі контролю версій Git, з наданням доступу до репозиторію на GitHub

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

4.8 Спеціальні вимоги

Спеціальні вимоги до програмного забезпечення включають:

- контейнеризація: розробка інсталяційної версії має бути виконана у вигляді набору Docker-образів для кожного компонента системи;
- автоматизація розгортання: підготовка docker-compose.yaml файлу, який забезпечить розгортання однією командою;
- конфігураційні файли: надання шаблонів файлів оточення.

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- пояснювальна записка;
- технічне завдання;
- текст програми;
- керівництво користувача.

Графічна частина повинна бути виконана на аркушах формату А3 та містити наступні документи:

- схема структурна варіантів використання;
- схема структурна компонентів програмного забезпечення ;
- схема бази даних;
- схема структурна класів програмного забезпечення.

5.2 Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою проєкту	15.02	
2.	Розробка технічного завдання	20.02	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	26.03	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	25.04	Схема структурна програмного забезпечення та специфікація компонентів (діаграма класів, схема алгоритму)
5.	Програмна реалізація програмного забезпечення	10.05	Тексти програмного забезпечення
6.	Тестування програмного забезпечення	20.05	Тести, результати тестування
7.	Розробка матеріалів текстової частини проєкту	25.05	Пояснювальна записка
8.	Розробка матеріалів графічної частини проєкту	30.05	Графічний матеріал проєкту
9.	Оформлення технічної документації проєкту	05.06	Технічна документація

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до «Програми та методики тестування».

**Пояснювальна записка
до дипломного проєкту**

на тему: **Вебзастосунок для організації хакатонів з динамічним модулем
оцінювання**

КПІ.ПІ-2419.045440.02.81

ЗМІСТ

1	ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	7
1.1	Постановка завдання дипломного проектування	7
1.2	Аналіз предметної області	8
1.3	Аналіз існуючих рішень	9
1.3.1	Аналіз відомих програмних продуктів	10
1.3.2	Аналіз відомих алгоритмічних та технічних рішень	16
1.4	Аналіз та моделювання бізнес-процесів	20
1.4.1	Процес управління конфігурацією та динамічним оцінюванням	21
1.4.2	Процес реєстрації та формування команд	21
1.4.3	Процес реєстрації та формування команд	22
1.5	Висновки до розділу	23
2	РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	24
2.1	Варіанти використання програмного забезпечення	24
2.2	Розроблення функціональних вимог	43
2.3	Розроблення нефункціональних вимог	46
2.4	Аналіз системних вимог	47
2.5	Аналіз економічних показників програмного забезпечення	48
2.5.1	Аналіз функціональних точок	48
2.5.1.1	Аналіз функцій даних	48
2.5.1.2	Аналіз транзакційних функцій	50
2.5.1.3	Розрахунок нормованої кількості функціональних точок	53
2.5.2	Аналіз основних характеристик системи GSC	53
2.5.3	Обчислення метрик вартості	54
2.6	Постановка завдання на розробку програмного забезпечення	55
2.7	Висновки до розділу	56
3	КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	57
3.1	Архітектура програмного забезпечення	57

3.2	Архітектурні рішення та обґрунтування вибору засобів розробки	60
3.2.1	Ключові архітектурні рішення.....	60
3.2.2	Обґрунтування технологічного стеку	61
3.2.3	Допоміжні програмні засоби.....	62
3.3	Конструювання програмного забезпечення.....	63
3.3.1	Опис структури бази даних	64
3.3.2	Опис сторонніх бібліотек та утиліт	74
3.4	Аналіз безпеки даних.....	76
3.5	Висновки до розділу	78
4	АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	79
4.1	Аналіз якості ПЗ.....	79
4.2	Опис процесів тестування.....	85
4.3	Опис контрольного прикладу	95
4.4	Висновки до розділу	102
5	РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	104
5.1	Розгортання програмного забезпечення.....	104
5.2	Супровід програмного забезпечення	109
5.3	Висновки розділу	111
	ВИСНОВКИ.....	113
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	115
	ДОДАТОК А ЗВІТ ПОДІБНОСТІ.....	117

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated Development Environment – інтегроване середовище розробки.
API	– Application programming interface, прикладний програмний Інтерфейс.
ER	– Entity-Relation diagram.
OC	– Операційна система.
БД	– База даних.
JWT	– JSON Web Token
REST	– REpresentational State Transfer.
СУБД	– Система керування базами даних.
ORM	– Object-Relational Mapping.
BPMN	– Business Process Model and Notation.
MVC	– Model-View-Controller.
DET	– Data Element Type.
RET	– Record Element Type.
ILF	– Internal Logical File – логічно пов'язана група даних, що визначається користувачем і знаходиться всередині кордонів ПЗ.
EIF	– External Interface File – логічно пов'язана група даних, або блоки керуючої інформації, на які посилається продукт, але які підтримуються поза продукту
EI	– External Input – процес введення даних і керуючої інформації в ПЗ.
EO	– External Output – процес, що генерує дані або керуючу інформацію, які надходять на вихід ПЗ.
EQ	– External Inquiry - діалогове введення, яке призводить до негайної відповіді ПЗ в формі діалогового виведення.
FTR	– File Types Referenced.

UFPC	– Unadjusted Function Point Count, ненормована кількість функціональних точок.
AFPC	– Adjusted Function Point Count, коригована кількості функціональних точок.
GSC	– General System Characteristics, основні характеристики системи.
DI	– Dependency Injection, впровадження залежностей.
SSO	– Single Sign-On.
DTO	– Data Transfer Object – об'єкт, який використовується для передачі даних між різними частинами програми.
SPA	– Single Page Application, односторінковий застосунок.
SSR	– Server-Side Rendering.
VM	– Virtual Machine

ВСТУП

Організація та проведення хакатонів відіграє ключову роль у сучасному процесі створення інновацій та пошуку талановитих спеціалістів, проте ефективність таких заходів часто знижується через фрагментарність використовуваних інструментів та значний вплив людського фактору на етапі оцінювання проєктів. У сучасному цифровому світі автоматизація цих процесів є актуальною та необхідною умовою для забезпечення прозорості та об'єктивності результатів змагань. Платформи, що дозволяють об'єднати реєстрацію, формування команд, збір робіт та підрахунок балів в єдину екосистему, стають стандартом для освітніх установ та ІТ-компаній.

Глобальні тенденції в області управління змаганнями свідчать про активне впровадження спеціалізованих вебрішень, що дозволяють мінімізувати операційне навантаження на організаторів. Провідні світові платформи демонструють успішні приклади автоматизації збору проєктів та комунікації. Сучасний стан розробок у цій сфері часто пропонує або надто жорсткі шаблони критеріїв, або вимагає ручного зведення даних у сторонніх таблицях, що створює ризики помилок при зміні вагових коефіцієнтів під час фінальних розрахунків.

Вебзастосунок, що розробляється в межах даної роботи, покликаний розв'язати проблему негнучких систем оцінювання шляхом впровадження динамічного модуля, який дозволяє організаторам самостійно конструювати формулу успіху для кожної події. Можливі сфери застосування створеного продукту включають не лише ІТ-хакатони, а й студентські олімпіади, наукові конкурси та внутрішні корпоративні воркшопи. Автоматизація розрахунків зваженого балу та інтеграція з GitHub API для перевірки робіт забезпечує вищий рівень надійності та якості обслуговування учасників порівняно з традиційними методами адміністрування.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Постановка завдання дипломного проєктування

Метою дипломного проєкту є створення спеціалізованої вебплатформи, що забезпечує автоматизацію організаційних етапів хакатонів та реалізацію інструментарію для гнучкого динамічного оцінювання проєктів. Для досягнення поставленої мети дипломне проєктування передбачає виконання наступних завдань:

- аналіз предметної області організації та управління змаганнями типу «хакатон», опис ключових бізнес-процесів взаємодії між організаторами, учасниками та журі, а також визначення загального завдання розробки;
- аналіз існуючих програмних платформ для проведення хакатонів, таких як DevPost та hacktribe, з метою виявлення їхніх недоліків у контексті гнучкості налаштування критеріїв оцінювання;
- моделювання бізнес-процесів реєстрації, формування команд, організації хакатону, подання проєктів та ітеративного оцінювання робіт експертами;
- розроблення функціональних, нефункціональних та системних вимог до програмного забезпечення;
- аналіз економічної доцільності та показників ефективності впровадження спеціалізованого програмного забезпечення порівняно з ручними методами обробки даних;
- постановка завдання на розробку програмного забезпечення ДП;
- розроблення архітектури програмного забезпечення;
- розроблення архітектурних рішень та обґрунтування вибору засобів розробки програмного забезпечення;
- конструювання та програмна реалізація серверної та клієнтської частин вебзастосунку з акцентом на модульній структурі;

- аналіз безпеки даних, розроблення механізмів автентифікації через протокол OAuth2 (Google, GitHub) та забезпечення цілісності результатів оцінювання;
- аналіз якості та тестування програмного забезпечення;
- розгортання та супровід програмного забезпечення;
- створення супроводжувальної документації до розробленого програмного забезпечення.

1.2 Аналіз предметної області

Предметна область цього дослідження охоплює процеси організації, управління та проведення хакатонів – короткострокових командних змагань, під час яких фахівці з різних галузей програмної інженерії спільно працюють над розв’язанням певної прикладної задачі або створенням інноваційного продукту. Основними етапами життєвого циклу хакатону є анонсування події, реєстрація учасників, формування команд, розробка проєктів та фінальне оцінювання результатів експертною комісією. Ключовою метою таких заходів є стимулювання креативності, обмін досвідом та розробка прототипів програмних рішень у стислі терміни.

Ключові бізнес-процеси предметної області включають адміністрування бази учасників, координацію взаємодії між членами команд, збір результатів роботи та багатофакторний процес виставлення балів журі. Належна організація цих процесів потребує автоматизації, оскільки фрагментарність даних на будь-якому етапі призводить до операційного перевантаження організаторів та погіршення досвіду користувачів.

На поточному етапі розвитку ІТ-технологій імплементація предметної області найчастіше реалізується за допомогою набору неінтегрованих інструментів: Google Forms для збору заявок, соціальних мереж та месенджерів для комунікації, а також електронних таблиць Google Sheets для підведення підсумків. Також часто створюється окремий сайт під кожен хакатон. Незважаючи на доступність цих засобів, такий підхід має критичні

недоліки. Процес оцінювання в таблицях є ручним, непрозорим і схильним до помилок, а будь-яка зміна вагових коефіцієнтів критеріїв вимагає перерахунку, що робить систему негнучкою. Більшість існуючих спеціалізованих платформ орієнтовані на великі комерційні заходи та мають фіксовану логіку, яка не дозволяє адаптувати формулу оцінювання під специфіку локальних університетських змагань.

Для покращення ситуації доцільно розробляти інтегровані вебплатформи, що забезпечують єдиний простір для всіх ролей користувачів. Системи нового покоління повинні пропонувати:

- автоматизацію повного життєвого циклу події від анонсу до лідерборду;
- гнучкий конструктор критеріїв оцінювання з можливістю динамічного обрахунку балів;
- автоматизований збір проєктів через інтеграцію з системами контролю версій;
- впровадження механізмів підбору тиммейтів на основі навичок учасників.

У рамках даного дипломного проєкту було обрано шлях розробки спеціалізованого вебсервісу з динамічним рушієм оцінювання. Рішення фокусується на створенні гнучкої backend-системи, яка дозволяє організатору самостійно конструювати математичну модель оцінювання, забезпечуючи прозорість та об'єктивність результатів, недоступну при використанні традиційних методів.

1.3 Аналіз існуючих рішень

Проаналізуємо відоме на сьогодні алгоритмічне забезпечення у даній області та технічні рішення, що допоможуть у реалізації h.linker. Далі будуть розглянуті готові програмні рішення, допоміжні програмні засоби та засоби розробки.

1.3.1 Аналіз відомих програмних продуктів

Першим розглянутим рішенням є платформа Devpost [1], яка є найбільш впізнаваним сервісом для проведення хакатонів у світі (рис. 1.1). Вона пропонує комплексний інструментарій для створення сторінки події, реєстрації учасників та формування галереї проєктів.

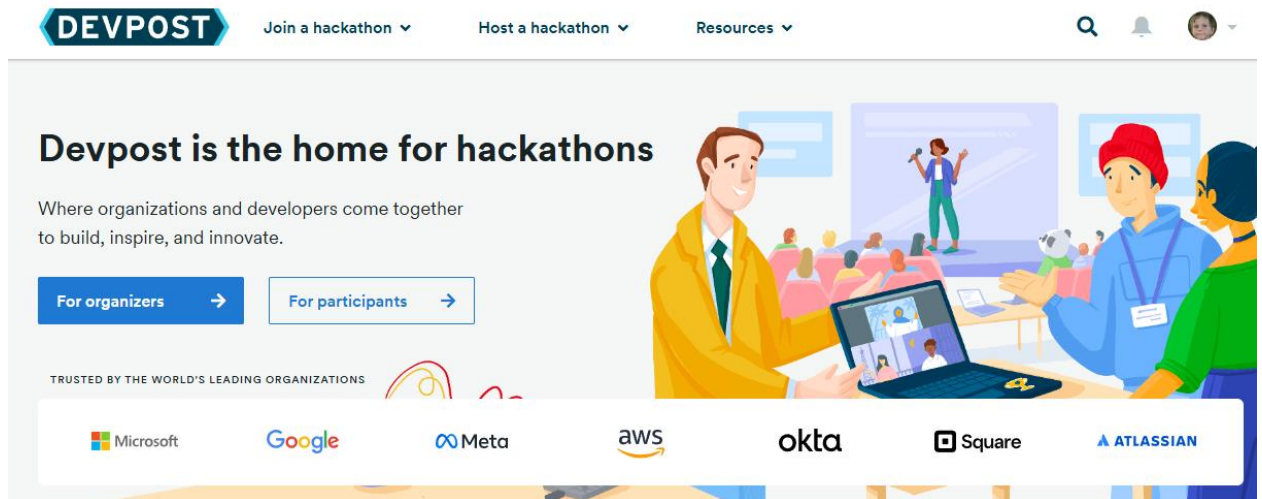


Рисунок 1.1 – Платформа Devpost

Основною перевагою системи є автоматизація маркетингової складової та зручний інтерфейс для подання робіт. Проте, з точки зору внутрішньої логіки оцінювання, Devpost має суттєві обмеження. Зокрема, вбудована система онлайн-голосування обмежує журі суворою шкалою від 1 до 5 балів для кожного критерію, що не дозволяє використовувати більш детальні метрики оцінювання. Крім того, організатори обмежені лише переліком назв критеріїв, тоді як можливість створення математичних моделей із динамічними ваговими коефіцієнтами для кожного окремого параметра відсутня (рис. 1.2). Це означає, що всі критерії мають однакову значущість при підрахунку фінального результату, що робить платформу менш ефективною для технічно складних змагань, де важлива точність розрахунків та пріоритетність окремих характеристик проєкту.

*** Criteria***Add criteria to help participants understand how they'll be judged.*

* Criteria Title

Interesting

Description

Save Cancel

Рисунок 1.2 – Меню додавання критерію оцінювання

Також процес адміністрування оцінювання потребує значного ручного втручання, оскільки адміністратор має самостійно активувати та завершувати періоди голосування, що знижує загальний рівень автоматизації системи (рис. 1.3).

Start the Judging Period

Before you can start the judging period, a few things need to be completed:

The submission period must be over

✗ 12 days remaining

You must turn on the gallery

✗ The gallery is off — [turn on the gallery](#)

All submissions that are visible in the gallery must be set to *Judge* or *Don't judge*

✓ You have moderated all the submissions

Start judging

Рисунок 1.3 – Необхідність запустити оцінювання

Другим аналогом є Hacktribe [2] – професійна комерційна платформа, що пропонує широку модульну структуру залежно від масштабу та потреб заходу (рис. 1.4).

HACKTRIBE

Use Cases

Pricing

Blog

Free Pitch Timer

Contact us

Book a demo!

Innovation program and Hackathon management platform

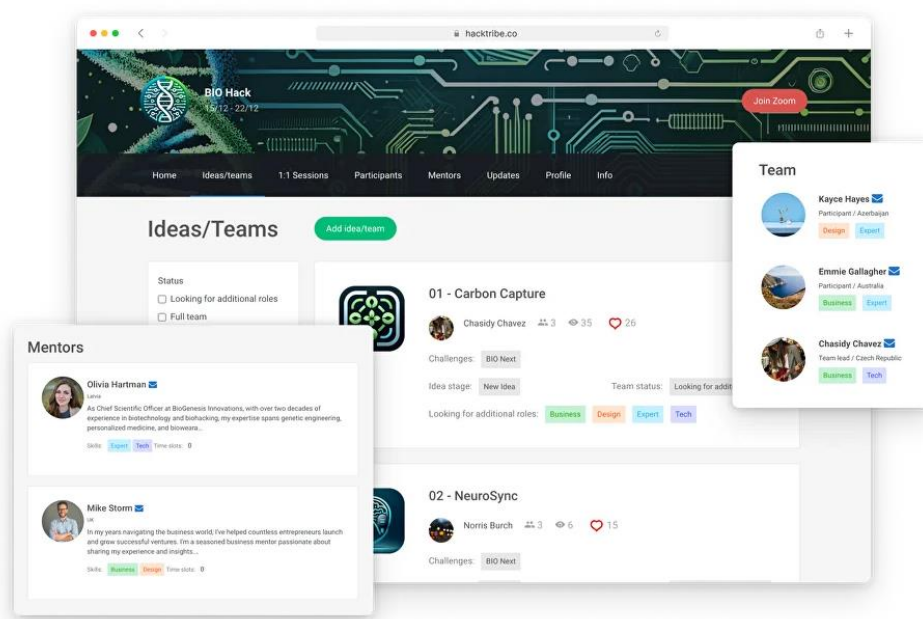


Рисунок 1.4 – Платформа Hacktribe

У межах базового варіанту система забезпечує лише фундаментальний набір функцій: керування спільнотою, форми реєстрації, менеджмент ідей та інструменти для формування команд. Більш просунуті плани (Pro та Premium) розширюють можливості за рахунок інтелектуального підбору команд за допомогою AI Matchmaking Assistant, системи оцінювання та голосування, планувальника менторських сесій та автоматизованої генерації сертифікатів. Усі можливості можна переглянути на рисунку 1.5.

Basic	Pro	Premium
- Customizable Landing Page - Community Management - Registration forms - Ideas Management - Team Formation - Announcements - Discussions - Wiki Pages - Data Analytics - Staff Training From 900 EUR per Event Get More Info	- All Basic Plan features 1:1 Mentoring Scheduler - Workshop Attendance - Evaluation and Voting - Feedback and Surveys - Custom Forms - Certificates - AI Matchmaking Assistant Get More Info	- All Pro Plan features + API, SSO, SMS - For organizations with unique requirements or large-scale events, we offer a customizable plan. Get More Info

Рисунок 1.5 – Доступні функції планів Hacktribe

Попри значний функціональний обсяг, Hacktribe має суттєвий недолік у вигляді високої вартості входу від 900 EUR за одну подію, що робить платформу економічно недоступною для більшості локальних, волонтерських або студентських ініціатив. Штучний поділ інструментарію на тарифні плани призводить до того, що критично важливі для якісного проведення хакатону модулі, такі як «Evaluation and Voting» та «Feedback and Surveys», стають доступними лише у дорогих версіях системи. Це змушує організаторів малих заходів або знову використовувати сторонні безкоштовні інструменти, або погоджуватися на спрощені моделі взаємодії з учасниками.

Крім того, закритість програмного коду та комерційна модель розповсюдження не дозволяють адаптувати логіку роботи системи під специфічні потреби організаторів без придбання індивідуального плану Premium. У таких умовах розробка власного відкритого рішення стає доцільною альтернативою, що поєднує професійну автоматизацію з повною гнучкістю налаштувань.

Третім рішенням є так званий «традиційний метод», який полягає у використанні зв'язки Google Forms для збору заявок та Google Sheets для

підрахунку результатів. Цей метод забезпечує найвищий рівень гнучкості у створенні розрахункових формул, оскільки дозволяє організаторам прописувати довільні математичні вирази без обмежень програмного інтерфейсу. Проте такий підхід повністю позбавлений автоматизації та інтеграції з професійними розробницькими інструментами, що робить його критично вразливим при збільшенні кількості учасників. Організатори змушені вручну перевіряти посилання на GitHub на предмет їх валідності, самотійно розсилати індивідуальні посилання на форми членам журі та власноруч контролювати цілісність даних у таблицях, що часто призводить до випадкового видалення формул або дублювання записів.

Крім того, за такого підходу неможливо забезпечити автоматичне підтягування метаданих репозиторію. Процес зведення результатів від різних членів журі в одну фінальну таблицю вимагає побудови зв'язків чи синхронізації через google-таблиці, що призводить до перевантаження документа даними та ускладнює навігацію (рис. 1.5). Відсутність автоматизованого «рушія» оцінювання також унеможлиблює надання швидкого зворотного зв'язку учасникам, оскільки формування текстових звітів із відгуками журі перетворюється на ще одну трудомістку ручну задачу. Велика кількість ручних операцій не лише затягує процес оголошення переможців, а й суттєво знижує прозорість та об'єктивність заходу.

Hackathon table numbers									
Файл Змінити Вигляд Вставити Формат Дані Інструменти Gemini Розширення Довідка									
Q Меню 100% \$ % 123 Arial 10 B I A									
J2									
	A	B	C	D	E	F	G	H	I
	Table1								
1	ID	Назва Проекту	Посилання на GitHub	#	Суддя 1 (Креатив)	Суддя 2 (Тех. Рівень)	Суддя 3 (Комерц.)	Сума Балів (ВАРТО)	Статус / Помилка
2	1	EcoTracker	[URL]		8	7	9	8.1	OK
3	2	SmartHome	bit.ly/3xyz564		5	6		ERROR	Суддя 3 не проголосував!
4	3	HealthSync	[URL]		9	9	10	9.4	OK
5	4	--	[empty]		7	8	7	7.4	??
6	5	CryptoWallet	[URL]		10	9	9	9.5	OK
7	6	FoodApp	[URL]		4	5	"Seven"		FIX
8	7	AI Recruiter	[URL]		8	8	8	8	OK
9	8	Travel	[broken link]		6	7	6		DEAD
10	9	FitPal	[URL]		7	6	8	7.1	OK
11	10	PetAdopt	[URL]		9	10	9	9.3	OK
12	11	MusicStream	[empty]		5	4	6	5.1	--
13	12	EventManager	[URL]		8	7	7	7.4	OK
14	13	TaskMaster	[URL]		6	6	5	5.7	Late
15	14	WeatherNow	[URL]		7	8	9	8.1	OK
16	15	RecipeBook	[URL]		9	8	8		Error
17	16	BudgetPlan	[URL]		4	5	4	4.3	Bad
18	17	MovieRate	[URL]		8	9	8	8.4	OK
	Вилалили п'ялок								

Рисунок 1.5 – Приклад завантаженої таблиці

Для порівняння проекту з аналогом можна скористатись таблицею 1.1.

Таблиця 1.1 – Порівняння з аналогами

Функціонал	h.linker	Devpost	Hacktribe	Традиційний метод	Пояснення
Реєстрація та автентифікація (OAuth)	+	+	+	+/-	Використання Google та GitHub для швидкого входу
Matchmaking на основі навичок	+	-	+ (AI)	-	Автоматичні рекомендації команд згідно з профілем користувача, також з використанням фільтрів
Автоматизований збір проєктів (GitHub)	+	+/-	-	-	Автоматичне підтягування метаданих репозиторію через API
Динамічний конструктор критеріїв	+	-	+/- (Pro)	+	Організатор сам визначає перелік та зміст критеріїв

Продовження таблиці 1.1

Функціонал	h.linker	Devpost	Hacktribe	Традиційний метод	Пояснення
Використання вагових коефіцієнтів	+	-	-	+	Можливість задати пріоритетність кожного параметра оцінювання
Автоматичний розрахунок зваженого бала	+	-	-	-	Обчислення результату на стороні сервера без зовнішніх формул
Гнучка шкала оцінювання (напр. 1-10)	+	+/- (1-5)	+/-	+	Налаштування довільного діапазону балів для членів журі.
Централізована панель журі	+	+	+	-	Єдиний інтерфейс для оцінювання всіх закріплених команд
Система текстових рецензій та відгуків	+	+	+/- (Pro)	-	Можливість залишати детальний зворотний зв'язок учасникам
Валідація та цілісність даних (Back-end)	+	+	+	-	Захист від некоректного введення та помилок типу #REF!
Керування статусами події (Workflow)	+	+/- (ручне)	+	-	Автоматизація переходів між фазами реєстрації та оцінювання
Галерея та фільтрація найкращих робіт	+	+	+	-	Візуалізація результатів та перегляд успішних кейсів
Моніторинг та резервне копіювання	+	+	+	-	Забезпечення стабільності роботи та збереження бази дани
Економічна доступність	+	+	- (900€+)	+	Відсутність ліцензійних платежів за проведення кожного заходу

1.3.2 Аналіз відомих алгоритмічних та технічних рішень

Для реалізації вебзастосунку необхідно обрати оптимальні алгоритмічні та архітектурні підходи, що забезпечать гнучкість налаштування критеріїв оцінювання та надійну роботу в умовах навантаження під час фіналу заходу.

Однією з ключових задач проєкту є розрахунок фінального результату команд на основі оцінок різних членів журі з урахуванням пріоритетності критеріїв. Алгоритм простого середнього арифметичного – найпростіший підхід, де всі оцінки додаються та діляться на їх кількість. Недолік – неможливість виділити важливіші аспекти (наприклад, технічна складність має важити більше за дизайн). Тому буде обрано модель зваженої суми, що дозволяє враховувати вагові коефіцієнти для кожного критерію. Фінальний бал S розраховується за формулою:

$$S = \sum_{i=1}^n \omega_i \cdot c_i \quad (1.1)$$

де ω_i – вага критерію, а c_i – оцінка за цим критерієм.

Для реалізації вебзастосунку `h.linker` використовуються сучасні технічні рішення, що забезпечують високу надійність, гнучкість налаштувань та можливість масштабування системи під потреби конкретних заходів. Одним із засадничих підходів є використання архітектури на базі фреймворку NestJS [3], яка дозволяє організувати логіку за принципом мікросервісного розподілу функціональності. Хоча сервіси можуть не виступати як повністю відокремлені інфраструктурні одиниці, такий підхід дозволяє чітко розмежувати відповідальність між окремими модулями – такими як модуль автентифікації, рушій динамічного оцінювання та система управління командами. Це забезпечує легкість підтримки коду та можливість незалежного вдосконалення кожної частини системи.

Взаємодія між клієнтською та серверною частинами реалізується через архітектурний стиль REST API. Це рішення дозволяє створити стандартизовану систему передачі даних у форматі JSON, що є критично важливим для інтеграції з зовнішніми сервісами, наприклад, з GitHub API для автоматичного збору інформації про проєкти учасників. Використання REST API забезпечує чітке визначення кінцевих точок для виконання операцій створення, читання та оновлення даних, що сприяє зручному масштабуванню та прозорості архітектури.

Безпека передачі даних та контроль доступу базуються на використанні технології JWT. Цей підхід дозволяє реалізувати безсесійну автентифікацію, де вся необхідна інформація про права користувача зберігається безпосередньо у токени на стороні клієнта. Це суттєво знижує навантаження на базу даних, оскільки відпадає необхідність постійної перевірки стану сесії на сервері при кожному запиті, що особливо актуально під час пікових навантажень у моменти активного голосування журі.

На рівні фронтенду використовується фреймворк Angular [4], побудований на мові програмування TypeScript. Вибір даного інструменту зумовлений його компонентною архітектурою, що дозволяє реалізувати інтерфейс системи `h.linker` як набір незалежних та перевикористовуваних модулів. Це важливо для екранів, таких як «Creator Dashboard» або «Jury Portal», де необхідно забезпечити чітку логіку взаємодії та високу швидкість рендерингу. Використання бібліотеки RxJS [5] дозволяє ефективно керувати асинхронними потоками даних, що забезпечує миттєве оновлення інформації в лідербордах та динамічне відображення таймерів без необхідності повного перезавантаження сторінок. Завдяки вбудованим механізмам валідації форм та суворій типізації, Angular гарантує цілісність вхідних даних ще на етапі їх введення користувачем, що значно підвищує загальну відмовостійкість клієнтської частини застосунку.

Для збереження даних обрано реляційну систему керування базами даних PostgreSQL [6], яка вирізняється високою продуктивністю та повною підтримкою ACID-транзакцій. Це є критично важливим для забезпечення цілісності оцінок журі та запобігання втраті даних при одночасних запитах. Взаємодія з базою даних здійснюється через сучасну ORM Prisma [7], яка забезпечує зручний інтерфейс для побудови складних запитів та автоматичну синхронізацію схем даних із кодом застосунку. Такий комплексний вибір технологій дозволяє створити стійку до відмов систему, яка за своїми технічними характеристиками перевершує традиційні методи організації хакатонів.

Таблиця 1.2 – Порівняння технічних рішень

Технічне рішення	Дипломний проєкт	Аналогічні рішення	Пояснення
Архітектура Бекенд частини	Мікросервісна shared database	Монолітна, database per service	Дозволяє ізолювати логіку оцінювання в окремі модулі, що полегшує підтримку та тестування
Мова програмування	TypeScript	JavaScript, PHP або формули	Забезпечує сувору типізацію, що унеможливорює помилки типів
Фронтенд-фреймворк	Angular	React, Vue або Template Engines	Забезпечує сувору типізацію та компонентну структуру, що полегшує підтримку великих форм.
Бекенд-фреймворк	NestJS	Express, Fastify, Koa	Використання Dependency Injection та декораторів забезпечує чітку структуру коду
Керування даними	PostgreSQL + Prisma ORM	Власні бази даних або Google Sheets	Гарантує цілісність оцінок журі та автоматичну синхронізацію схем даних
Авторизація	JWT	session-based	Дозволяє реалізувати безпечний, stateless-доступ до системи для великої кількості журі одночасно

Продовження таблиці 1.2

Технічне рішення	Дипломний проєкт	Аналогічні рішення	Пояснення
Архітектура API	REST API	SOAP, gRPC, WebSocket, GraphQL	REST API є універсальним рішенням, яке легко інтегрується
Інтеграція	GitHub API	Ручне копіювання посилань	Автоматизує збір даних про проєкт безпосередньо в інтерфейс журі
Інфраструктура та розгортання	Docker & Docker Compose	Ручне налаштування серверів	Гарантує ідентичність середовища, спрощує розгортання

Дана робота фокусується не на створенні оригінальних математичних апаратів, а на високоякісній імплементації перевірених технологічних підходів. Застосування обраного стеку технологій гарантує побудову системи, яка забезпечує високу швидкість обробки даних та відповідає сучасним вимогам до масштабованих вебзастосунків.

1.4 Аналіз та моделювання бізнес-процесів

Для забезпечення ефективної автоматизації хакатонів необхідно чітко визначити ключові операційні цикли, які складають основу життєвого циклу заходу. З загальної множини процесів для моделювання було обрано три найбільш критичні, що демонструють взаємодію користувачів із системою h.linker: налаштування параметрів події, формування команд та фінальне оцінювання проєктів.

Для наочного представлення логіки цих процесів використано нотацію BPMN, яка дозволяє відобразити послідовність дій та розподіл відповідальності між користувачами та програмним забезпеченням.

1.4.1 Процес управління конфігурацією та динамічним оцінюванням

Процес описує підготовку заходу адміністратором (рис 1.6). Основна увага приділяється гнучкості налаштування «математичного ядра» системи.

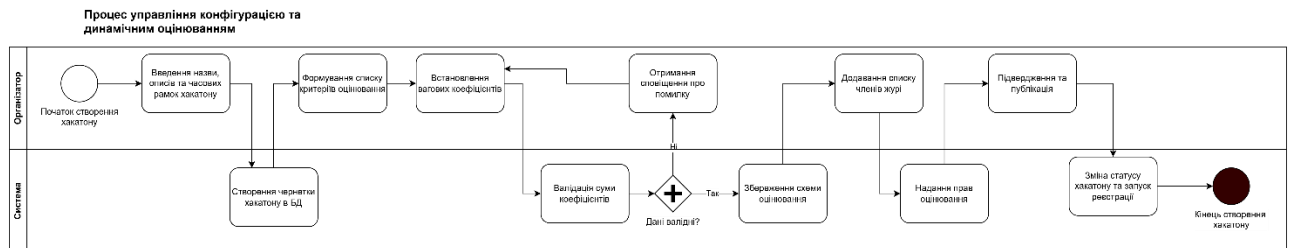


Рисунок 1.6 – Модель бізнес-процесу налаштування хакатону

Опис моделі бізнес-процесу конфігурації:

- організатор вводить назву, опис та часові рамки заходу;
- система створює чернетку події в базі даних;
- організатор формує перелік критеріїв оцінювання та встановлює для них вагові коефіцієнти;
- система перевіряє валідність суми вагових коефіцієнтів;
- якщо дані некоректні, організатор отримує сповіщення та повертається до редагування;
- організатор додає список членів журі та публікує конфігурацію.

1.4.2 Процес реєстрації та формування команд

Цей процес відображає шлях користувача від авторизації до закріплення у складі команди (рис. 1.7).

Ресстрація та формування команд

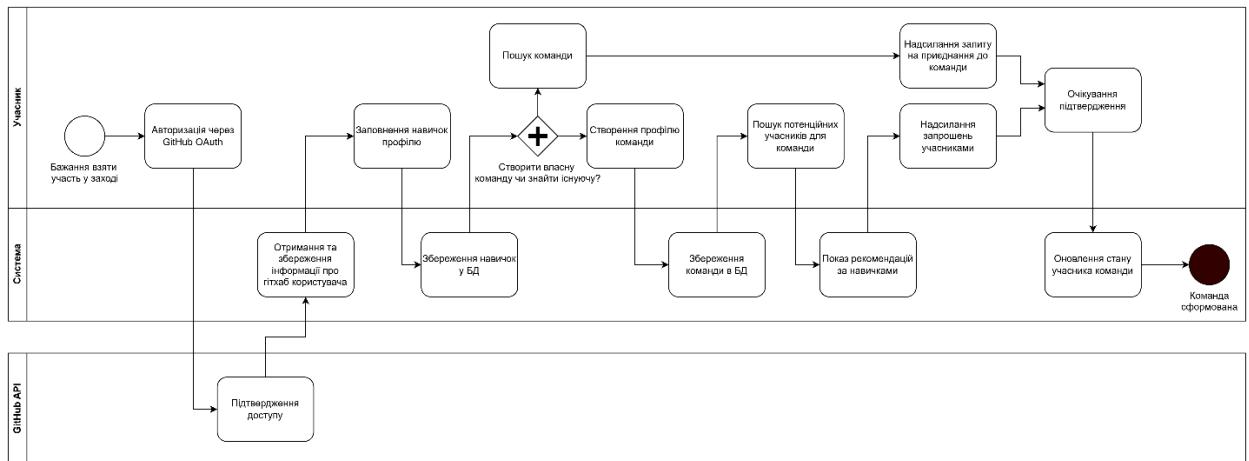


Рисунок 1.7 – Модель процесу реєстрації та формування команд

Опис моделі бізнес-процесу:

- учасник авторизується через GitHub OAuth, що дозволяє системі автоматично синхронізувати дані про його розробницький досвід через GitHub API;
- користувач обирає стратегію участі: створення власної команди або пошук існуючої за допомогою модуля Matchmaking;
- система фіксує членство учасника у складі команди та оновлює відповідні записи в базі даних.

1.4.3 Процес реєстрації та формування команд

Цей етап описує автоматизований перехід від виставлення індивідуальних балів до формування підсумкового рейтингу команд (рис. 1.8).

Експертне оцінювання та підрахунок результатів

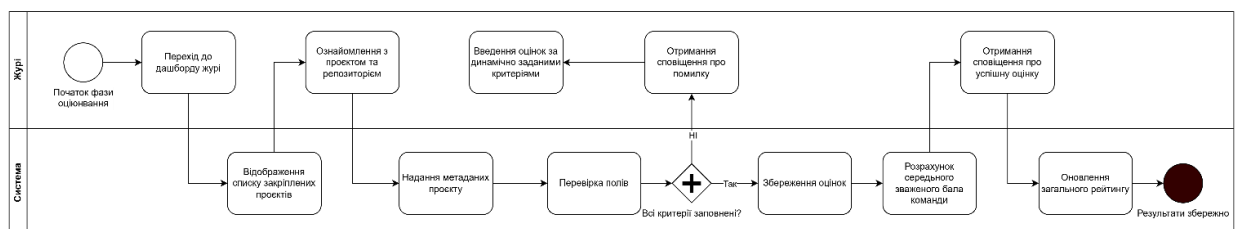


Рисунок 1.8 – Модель процесу експертного оцінювання

Опис моделі бізнес-процесу:

- експерт використовує персональну панель для оцінювання проєктів, де система автоматично підтягує актуальні метадані репозиторіїв для верифікації роботи;
- журі вводить бали за динамічними критеріями, після чого серверна частина на базі NestJS валідує дані та виконує розрахунок зваженого результату за заданою формулою;
- система миттєво оновлює загальний лідерборд, що забезпечує прозорість заходу та виключає ризики помилок ручного підрахунку.

1.5 Висновки до розділу

У першому розділі проведено аналіз предметної області та існуючих засобів автоматизації хакатонів, який виявив критичні недоліки традиційних методів у вигляді фрагментарності даних та обмеженість комерційних платформ у налаштуванні гнучких критеріїв оцінювання. Побудовані BPMN-моделі ключових бізнес-процесів підтвердили необхідність автоматизації етапів реєстрації через GitHub API та впровадження обчислювального «рушія» для безпомилкового підрахунку результатів у реальному часі.

На основі проведеного дослідження обґрунтовано доцільність розробки системи h.linker із використанням моделі зваженої суми, що забезпечує математичну точність та об'єктивність змагань. Обраний технологічний стек, що гарантує високу продуктивність, сувору типізацію та цілісність даних, що дозволить створити масштабоване рішення, яке за функціональними можливостями перевершує існуючі аналоги.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Діаграма варіантів використання з ключовими акторами та основними функціями показана на рисунку 2.1.



Рисунок 2.1 – Діаграма варіантів використання

В таблицях 2.1-2.25 наведено опис варіантів використання програмного забезпечення.

Таблиця 2.1 – Варіант використання UC-01

Use case name	Реєстрація користувача
Use case ID	UC-01
Goals	Створення нового облікового запису для доступу до функцій платформи

Продовження таблиці 2.1

Actors	Гість (неавторизований користувач)
Trigger	Користувач натискає кнопку «Sign up»
Pre-conditions	Користувач має відкриту сторінку реєстрації
Flow of Events	Користувач вводить персональні дані, включаючи ім'я, прізвище, унікальний нікнейм та адресу електронної пошти, після чого встановлює та підтверджує пароль. Далі він переходить до секції вибору технічних навичок, де вказує свій основний стек технологій. Після натискання кнопки завершення реєстрації, система валідує дані та вносить запис про нового користувача до бази даних.
Extension	UC-01.1: Реєстрація через сторонні сервіси (Google / GitHub).
Post-Condition	Обліковий запис успішно створено, користувач отримує доступ до авторизації.

Таблиця 2.2 – Варіант використання UC-02

Use case name	Авторизація в системі
Use case ID	UC-02
Goals	Отримання прав доступу до персонального кабінету та функцій керування
Actors	Користувач
Trigger	Введення логіна та пароля на сторінці входу «Log in»
Pre-conditions	Наявність попередньо зареєстрованого акаунту в системі
Flow of Events	Користувач вводить ідентифікаційні дані у відповідні поля форми авторизації. Після натискання кнопки входу серверна частина системи h.linker виконує перевірку наданих даних на відповідність записам у базі. У разі успішної верифікації система генерує JWT-токен та перенаправляє авторизованого користувача на головну сторінку застосунку.
Extension	UC-02.1: Вхід через Google / GitHub акаунт. У разі невірного пароля система виводить повідомлення.
Post-Condition	Користувач авторизований, токен збережено в браузері для підтримки сесії.

Таблиця 2.3 – Варіант використання UC-03

Use case name	Перегляд головної сторінки
Use case ID	UC-03
Goals	Ознайомлення з актуальними подіями та навігація по системі
Actors	Користувач
Trigger	Перехід за кореневою адресою застосунку або натискання на логотип
Pre-conditions	-
Flow of Events	Система завантажує посадкову сторінку з центральним Hero-блоком, який описує призначення платформи. На екрані візуалізуються три картки основних функціональних модулів (Team Management, GitHub Sync та Hackathons), а в нижній частині відображаються глобальні лічильники метрик системи в реальному часі. Користувач вивчає інформацію та використовує елементи заклику до дії (наприклад, кнопки «Get Started» чи «Sign In») для переходу до інших розділів.
Extension	-
Post-Condition	Користувач зорієнтований у структурі платформи та готовий до вибору цільової дії.

Таблиця 2.4 – Варіант використання UC-04

Use case name	Перегляд найкращих проєктів
Use case ID	UC-04
Goals	Вивчення архіву успішних робіт минулих хакатонів
Actors	Користувач
Trigger	Перехід до розділу «Showcase» у головному меню
Pre-conditions	Наявність у системі хоча б одного завершеного хакатону з визначеними переможцями

Продовження таблиці 2.4

Flow of Events	Система формує галерею карток проєктів, які посіли призові місця в попередніх заходах. Користувач прокручує список, ознайомлюючись із назвами, короткими описами та візуальними матеріалами робіт. При натисканні на конкретну картку система відкриває модальне вікно або окрему сторінку з детальними характеристиками проєкту та посиланням на його вихідний код.
Extension	Можливість фільтрації галереї за категоріями хакатонів або технічним стеком.
Post-Condition	Користувач отримав інформацію про рівень робіт та технічні рішення переможців.

Таблиця 2.5 – Варіант використання UC-05

Use case name	Перегляд переліку хакатонів
Use case ID	UC-05
Goals	Ознайомлення зі списком усіх подій платформи, їх фільтрація за категоріями та вибір хакатону
Actors	Користувач
Trigger	Натискання на вкладку «Events» у глобальному хедері застосунку
Pre-conditions	-
Flow of Events	Система завантажує сторінку каталогу змагань. У центрі відображається закріплений головний банер актуальної події, а нижче - сітка карток усіх доступних хакатонів із сортуванням за новизною. Ліворуч візуалізується бічна панель фільтрів, яка дозволяє динамічно відсіювати за фільтрами. Користувач взаємодіє з інтерфейсом для пошуку потрібного івенту.
Extension	UC-05.1: Фільтрація переліку хакатонів за критеріями. UC-06: Перегляд деталей хакатону.
Post-Condition	Користувач знайшов цікаву подію та готовий перейти до вивчення її деталей.

Таблиця 2.6 – Варіант використання UC-06

Use case name	Перегляд деталей хакатону
Use case ID	UC-06
Goals	Ознайомлення з повною інформацією про подію, її регламентом, складом журі, а також перехід до відповідного робочого простору залежно від статусу та ролі користувача
Actors	Користувач
Trigger	Вибір конкретного хакатону в каталозі або перехід за прямим URL-маршрутом події
Pre-conditions	-
Flow of Events	Система завантажує сторінку події, відображаючи головний банер із назвою, статусом, часовими межами, лічильником зареєстрованих і призом. У центрі доступна навігація: вкладка «About» виводить опис та вагові коефіцієнти критеріїв оцінювання, «Timeline» - графік, «Jury» - судді із посиланнями на профілі. У бічній панелі працює таймер зворотного відліку та блок дій. Для незареєстрованих активна кнопка реєстрації, для вже сформованих команд - перехід до дашборду учасника та лідерборду.
Extension	UC-06.1: Перегляд таблиці результатів UC-18: Реєстрація на хакатон UC-21: Редагування даних хакатону UC-23: Перегляд надісланих робіт
Post-Condition	Користувач ознайомився з регламентом події та перейшов до цільової взаємодії згідно зі своєю ролю

Таблиця 2.7 – Варіант використання UC-07

Use case name	Перегляд профілів користувачів
Use case ID	UC-07
Goals	Ознайомлення з персональною інформацією, технічними навичками, аналітикою GitHub та історією активності користувача на платформі
Actors	Користувач

Продовження таблиці 2.7

Trigger	Натискання на профіль, аватар або ім'я розробника у будь-якому розділі системи (кабінет команди, списки журі) або перехід до власної сторінки
Pre-conditions	-
Flow of Events	Система завантажує сторінку обраного профілю. У лівій бічній панелі відображаються базові дані (аватар, статус, опис та стек технічних навичок). У центральній частині динамічно візуалізується віджет «GitHub Insights» з детальною статистикою репозиторіїв (загальна кількість внесків, зірки, публічні репозиторії та кругова діаграма мов програмування). Нижче відображаються таби для перегляду створених користувачем хакатонів, команд та проєктів.
Extension	UC-08: Редагування даних власного профілю.
Post-Condition	Користувач отримав актуальну інформацію про свій статус у системі.

Таблиця 2.8 – Варіант використання UC-08

Use case name	Редагування власного профілю
Use case ID	UC-08
Goals	Модифікація даних, керування зовнішніми інтеграціями, оновлення стека навичок та обробка запитів від команд
Actors	Користувач
Trigger	Активація кнопки редагування у власному профілі
Pre-conditions	Користувач авторизований у системі
Flow of Events	Система завантажує поточні дані користувача з бази даних. Користувач може модифікувати персональну інформацію, керувати переліком технологій у своєму стеку шляхом додавання або видалення тегів, а також ініціювати синхронізацію із зовнішнім профілем GitHub. Також користувач опрацьовує наявні запрошення до команд та може оновити поточний пароль після верифікації старого. Система валідує введені дані і зміни на сервер або скасовує редагування, відновлюючи початковий стан.

Продовження таблиці 2.8

Extension	UC-08.1: Синхронізація з профілем GitHub UC-08.2: Прийняття або відхилення запрошення до команди UC-08.3: Зміна пароля облікового запису UC-08.4: Видалення облікового запису
Post-Condition	Змінені дані успішно оновлені в базі даних PostgreSQL, поточний стан користувача на клієнті синхронізовано.

Таблиця 2.9 – Варіант використання UC-09

Use case name	Вихід із системи
Use case ID	UC-09
Goals	Безпечне завершення поточної сесії та видалення даних доступу з пристрою
Actors	Користувач
Trigger	Натискання кнопки «Logout» у глобальному хедері застосунку
Pre-conditions	Наявність активної авторизованої сесії
Flow of Events	При активації тригера система миттєво очищує локальний стан авторизації у localStorage та скидає стан поточного користувача в null. Після цього ініціюється запит до бекенду для анулювання сесії на стороні сервера. Незалежно від результату виконання запиту, система перенаправляє користувача на сторінку авторизації (/login).
Extension	-
Post-Condition	Сесія успішно завершена, доступ до функцій учасника обмежено.

Таблиця 2.10 – Варіант використання UC-10

Use case name	Перегляд переліку команд
Use case ID	UC-10
Goals	Ознайомлення зі списком усіх команд на платформі, їх фільтрація за приналежністю користувача та управління статусом участі
Actors	Користувач

Продовження таблиці 2.10

Trigger	Перехід до розділу «Teams» у глобальному хедері
Pre-conditions	-
Flow of Events	Система завантажує з бази даних загальний перелік команд із відображенням кількості знайдених відповідностей. Для кожної команди відображається її назва, опис, кількість учасників та індивідуальний статус поточного користувача щодо неї (власник, учасник, наявність активної або відхиленої заявки на вступ). Користувач може застосувати фільтрацію для відображення лише тих об'єднань, де він є лідером або безпосереднім членом. Система динамічно оновлює дані та лічильники на основі обраних. Користувач взаємодіє з інформацією для вибору подальшої цільової дії..
Extension	UC-11: Перегляд деталей команди UC-10.1: Фільтрація команд UC-10.2: Подача запиту на вступ до команди (або повторна відправка при відхиленні) UC-12: Створення команди
Post-Condition	Нова команда створена, користувач набув статусу Лідера команди.

Таблиця 2.11 – Варіант використання UC-11

Use case name	Перегляд деталей команди
Use case ID	UC-11
Goals	Ознайомлення з інформацією про команду, її складом, підключеними проєктними репозиторіями, історією виступів на хакатонах та управління статусом свого членства
Actors	Користувач (може виступати як Гість, Учасник команди або Лідер)
Trigger	Вибір конкретної команди в загальному списку або перехід за прямим URL-маршрутом
Pre-conditions	Авторизація (опційна, необхідна для відкриття функцій)

Продовження таблиці 2.11

Flow of Events	Система завантажує дані команди з бази даних. На основі ідентифікатора поточного користувача система визначає його статус (власник, підтверджений учасник, користувач із надісланою чи відхиленою заявкою, або сторонній переглядач). Користувачу відображається опис команди, повний список її членів із виділенням керівника, перелік посилань на пов'язані GitHub-репозиторії та історія участі у змаганнях із зафіксованими оцінками та результатами проєктів. Для членів команди активується доступ до посилання на зовнішній комунікаційний чат. Залежно від статусу, система надає відповідні функціональні опції дій.
Extension	UC-06: Перехід до сторінки конкретного хакатону з історії виступів. UC-13: Редагування даних команди (для лідера). UC-14: Подача або повторна відправка запиту на вступ до команди. UC-15: Вихід зі складу команди (для учасника). UC-17: Пошук нових учасників. UC-11.1: Перехід до зовнішнього комунікаційного чату.
Post-Condition	Користувач ознайомився з профілем команди та ініціював цільову дію згідно зі своїми правами доступу.

Таблиця 2.12 – Варіант використання UC-12

Use case name	Створення команди
Use case ID	UC-12
Goals	Формування нової командної структури для участі у хакатоні
Actors	Користувач (отримує роль Лідера)
Trigger	Активація кнопки «Create Team» у розділі команд
Pre-conditions	Авторизація
Flow of Events	Користувач заповнює форму створення, вказуючи назву команди, короткий опис проєкту та посилання для комунікації. Після натискання кнопки підтвердження система створює новий запис у базі даних, автоматично призначає творця лідером цієї команди. Відтепер користувач отримує розширені права на модерацію складу та команди.

Продовження таблиці 2.12

Extension	-
Post-Condition	Нова команда створена, користувач набув статусу Лідера.

Таблиця 2.13 – Варіант використання UC-13

Use case name	Редагування даних команди
Use case ID	UC-13
Goals	Модифікація загальної інформації про команду, адміністрування складу учасників та вхідних заявок, передача прав власності або повне розформування спільноти
Actors	Лідер команди
Trigger	Натискання кнопки управління на сторінці деталей команди
Pre-conditions	Користувач авторизований у системі та має права власника (лідера) для цієї команди
Flow of Events	Система завантажує актуальний профіль команди, списки її членів та активних запитів із бази даних. Лідер може змінювати базові текстові дані (назву, опис, комунікаційне посилання), видаляти діючих розробників зі складу команди, а також опрацьовувати нові запити на вступ. Система автоматично перевіряє коректність заповнення полів форми.
Extension	UC-13.1: Видалення учасника зі складу команди UC-13.2: Схвалення або відхилення запиту на вступ до команди UC-13.3: Призначення іншого лідера UC-13.4: Розформування (повне видалення) команди Валідація посилання на зовнішні чати на коректність формату URL
Post-Condition	Конфігурації команди, її склад чи статус успішно оновлені в базі даних, зміни синхронізовано на клієнті.

Таблиця 2.14 – Варіант використання UC-14

Use case name	Надсилання запиту на вступ до команди
Use case ID	UC-14

Продовження таблиці 2.14

Goals	Виявлення зацікавленості в участі у конкретній команді та ініціація запиту на вступ
Actors	Користувач
Trigger	Активація кнопки «Send Request» у профілі команди
Pre-conditions	Авторизація; користувач не є членом жодної команди в межах поточної події
Flow of Events	Користувач переглядає перелік команд на сторінці «Matchmaking» та обирає колектив. Після натискання кнопки з'являється модальне меню з вибором ролі та повідомленням запиту. Система створює запис у статусі очікування у базі даних та надсилає автоматичне сповіщення лідеру обраної команди. Користувач отримує підтвердження, що його запит успішно відправлено на розгляд.
Extension	Система блокує можливість надсилання запиту, якщо команда вже повністю укомплектована.
Post-Condition	Запит на вступ зареєстровано, лідер команди отримав сповіщення про нового кандидата.

Таблиця 2.15 – Варіант використання UC-15

Use case name	Вихід з команди
Use case ID	UC-15
Goals	Добровільне припинення членства в команді
Actors	Користувач (учасник команди)
Trigger	Натискання кнопки «Leave Team» у налаштуваннях
Pre-conditions	Користувач є офіційним членом команди та не є її лідером
Flow of Events	Учасник переходить до розділу керування своєю участю та ініціює процедуру виходу з колективу. Система відображає модальне вікно для підтвердження дії, після чого видаляє зв'язок між користувачем та командою в базі. Користувач втрачає доступ до закритої документації команди та знову отримує статус вільного учасника хакатону.
Extension	Якщо учасник є лідером, система вимагає спочатку призначити нового лідера перед виходом.

Post-Condition	Користувач припинив участь у команді, місце учасника стало вакантним.
-----------------------	---

Таблиця 2.16 – Варіант використання UC-16

Use case name	Перегляд коментарів (оцінки) до роботи команди
Use case ID	UC-16
Goals	Отримання зворотного зв'язку від експертів за результатами розробки
Actors	Користувач (член команди)
Trigger	Перехід до розділу «Dashboard» на сторінці хакатону
Pre-conditions	Журі завершило етап оцінювання
Flow of Events	Користувач відкриває дашборд своєї команди, після чого система завантажує та відображає бали. Разом із числовими показниками система виводить текстові рецензії, залишені членами журі, де описані сильні та слабкі сторони проєкту. Користувач ознайомлюється з фідбеком для аналізу своєї роботи.
Extension	Відображення загального середньозваженого балу на основі встановлених організатором коефіцієнтів.
Post-Condition	Користувач отримав вичерпну інформацію про оцінку свого проєкту експертами.

Таблиця 2.17 – Варіант використання UC-17

Use case name	Пошук нових учасників
Use case ID	UC-17
Goals	Пошук розробників за компетенціями, ознайомлення з їхнім професійним стеком та надсилання офіційних запрошень до складу своєї команди
Actors	Лідер
Trigger	Ініціація пошуку розробників через відповідний навігаційний розділ або через інтерфейс управління командою
Pre-conditions	Користувач авторизований та має створену команду

Продовження таблиці 2.17

Flow of Events	Система завантажує загальний список зареєстрованих користувачів, автоматично виключаючи з вибірки поточного авторизованого актора. Для кожного спеціаліста відображаються його ідентифікаційні дані (ім'я, нікнейм, текстовий опис профілю), повний перелік зафіксованих технологічних навичок та статус верифікації облікового запису GitHub. Користувач може вводити пошукові запити, вибирати технологічні категорії та встановлювати обмеження на відображення лише тих спеціалістів, які мають активну інтеграцію із GitHub. Система динамічно обчислює збіги та оновлює перелік доступних карток розробників.
Extension	UC-07: Перегляд профілів користувачів UC-17.1: Фільтрація UC-17.2: Надсилання запрошення у команду з визначенням ролі спеціаліста та супровідного повідомлення
Post-Condition	Користувач знайшов необхідних спеціалістів та надіслав запити на приєднання.

Таблиця 2.18 – Варіант використання UC-18

Use case name	Реєстрація на хакатон
Use case ID	UC-18
Goals	Офіційне підтвердження участі команди у конкретному заході
Actors	Лідер
Trigger	Натискання кнопки «Register for Event» на сторінці події
Pre-conditions	Наявність відкритої фази реєстрації в налаштуваннях хакатону
Flow of Events	Користувач відкриває сторінку обраного хакатону та ініціює процедуру реєстрації. Система перевіряє статус користувача та пропонує обрати команду. Після вибору та підтвердження реєстрації система вносить запис до таблиці залученості учасників.
Extension	Система блокує реєстрацію, якщо термін подання заявок завершився або ліміт команд вичерпано.
Post-Condition	Команда офіційно занесена до списку учасників хакатону.

Таблиця 2.19 – Варіант використання UC-19

Use case name	Здача проєкту
Use case ID	UC-19
Goals	Внесення інформації про розроблений продукт, підключення кодової бази та фінальне відправлення проєкту на оцінювання журі хакатону
Actors	Лідер
Trigger	Ініціація процедури надсилання або редагування матеріалів проєкту з кабінету учасника
Pre-conditions	Користувач авторизований, а його команда має підтверджений статус реєстрації на відповідну подію
Flow of Events	Система перевіряє статус реєстрації та динамічно завантажує дані вже існуючого сабміту (якщо проєкт редагується повторно). Процес виконується послідовно: спочатку користувач заповнює ідентифікаційні дані (назву та розширений опис функціоналу проєкту), після чого вносить перевірену URL-адресу публічного репозиторію коду на платформі GitHub. На завершальному етапі система формує зведену картку перевірки із переліком авторів проєкту та вимагає підтвердження готовності матеріалів і згоди з регламентом змагань. Після фінальної команди система валідує всю інформацію та відправляє сформований пакет даних на сервер.
Extension	UC-19.1: Динамічне підтягування полів раніше збереженого проєкту для редагування. Автоматична валідація синтаксису посилання на репозиторій GitHub. Блокування відправки форми у разі невідповідності обмеженням довжини тексту.
Post-Condition	Дані проєкту успішно збережені або оновлені в базі даних та прив'язані до поточної участі команди в хакатоні.

Таблиця 2.20 – Варіант використання UC-20

Use case name	Створення хакатону
Use case ID	UC-20

Продовження таблиці 2.20

Goals	Ініціалізація та первинне налаштування нової ІТ-події з визначенням її регламенту, тем, складу експертів та критеріїв оцінювання проєктів
Actors	Користувач (який стає Організатором події)
Trigger	Вибір опції створення нової події на сторінці «Events»
Pre-conditions	Користувач авторизований у системі h.linker
Flow of Events	Користувач вносить первинні метадані події (назву, унікальний URL-slug, опис, призовий фонд та завантажує банер). Далі визначаються часові межі чотирьох основних етапів змагання: початок реєстрації, старт хакатону, дедлайн здачі проєктів та офіційне завершення. Після збереження цих даних система ініціалізує подію в базі даних та створює унікальний ідентифікатор. Після цього користувач вказує фокусні технологічні категорії події, формує склад колегії журі за допомогою пошуку користувачів та визначає перелік критеріїв оцінювання з обов'язковим розподілом вагових коефіцієнтів (сумарно 100%). На фінальному етапі подія зберігається у статусі чернетки..
Extension	УС-20.1: Вибір членів журі – пошук та призначення експертів за допомогою електронної пошти або внутрішнього ідентифікатора (нікнейма); УС-20.2: Вибір категорій – визначення тематичних напрямків та пріоритетних технологій хакатону для кращої фільтрації проєктів; УС-20.3: Виставлення критеріїв оцінки – формування переліку параметрів, за якими журі аналізуватиме роботи; УС-20.4: Зазначення призу – опис нагородного фонду, спонсорських бонусів та умов їх отримання переможцями; УС-20.5: Зазначення часу проведення – детальне налаштування таймлайну події. Блокування переходу на етап запуску у разі невідповідності суми вагових коефіцієнтів нормі
Post-Condition	Новий хакатон успішно створено, заповнено конфігураційними даними та збережено в базі даних.

Таблиця 2.21 – Варіант використання UC-21

Use case name	Редагування даних хакатону
Use case ID	UC-21
Goals	Модифікація параметрів, часових меж, складу експертів, критеріїв оцінювання та життєвого циклу вже існуючого хакатону
Actors	Організатор (творець хакатону)
Trigger	Вибір опції редагування на сторінці деталей створеного хакатону
Pre-conditions	Користувач авторизований та є безпосереднім творцем обраного хакатону
Flow of Events	Система завантажує повну конфігурацію існуючого хакатону за його ідентифікатором, перевіряючи права доступу користувача. Організатор вносить зміни у текстові поля, коригує дедлайни етапів, додає нових чи вилючає діючих суддів, або змінює назви й ваги критеріїв оцінювання (із обов'язковим дотриманням балансу 100%). Також користувач може змінити поточний статус життєвого циклу події (перевести у фазу реєстрації, активного змагання чи завершити івент). Система валідує внесені зміни і за командою користувача відправляє оновлені пакети даних на сервер..
Extension	UC-21.1: Оперативне вилучення користувачів зі складу журі із анулюванням їхніх прав доступу до оцінювання робіт події UC-21.2: Керування етапами хакатону UC-06: Перехід до публічного перегляду оновленої сторінки івенту Перевірка прав доступу на рівні компонента з блокуванням інтерфейсу у разі спроби підміни ідентифікатора хакатону в URL.
Post-Condition	Дані хакатону оновлені та синхронізовані для всіх користувачів.

Таблиця 2.22 – Варіант використання UC-21.3

Use case name	Керування етапами хакатону
Use case ID	UC-21.3

Продовження таблиці 2.22

Goals	Послідовне перемикання фаз життєвого циклу події для автоматичної зміни бізнес-правил та прав доступу користувачів
Actors	Організатор
Trigger	Ініціація зміни етапу події в конструкторі хакатону
Pre-conditions	Доступ до Creator Dashboard конкретного хакатону
Flow of Events	Система зчитує поточний стан події з бази даних. Організатор вибирає новий статус із суворого переліку доступних етапів системи: DRAFT (прихована чернетка для налаштування параметрів), REGISTRATION (відкриття прийому заявок від команд), ACTIVE (активна фаза змагання, триває розробка) або FINISHED (офіційне завершення події та старт роботи журі). Система валідує запит і відправляє оновлене значення на сервер, де автоматично змінюються умови доступу.
Extension	Автоматичне блокування функцій реєстрації команд на події при виході зі статусу REGISTRATION. Активація робочого простору оцінювання журі та повне блокування можливості здачі чи редагування проєктів учасниками при переході у статус FINISHED.
Post-Condition	Статус хакатону змінено, змінено права доступу користувачів до функцій події.

Таблиця 2.23 – Варіант використання UC-22

Use case name	Перегляд статистики хакатону
Use case ID	UC-22
Goals	Отримання аналітичних даних про активність учасників та стан розробки
Actors	Організатор
Trigger	Перехід на вкладку статистики хакатону
Pre-conditions	Хакатон опублікований та має зареєстрованих учасників

Продовження таблиці 2.23

Flow of Events	Організатор відкриває аналітичний дашборд, де система в реальному часі формує звіти на основі даних. Користувач бачить загальну кількість зареєстрованих людей, кількість сформованих команд, а також детальну статистику щодо зданих проєктів та активності журі. Система відображає графіки залученості та дозволяє організатору оцінити, чи всі команди встигають до дедлайну.
Extension	-
Post-Condition	Організатор володіє актуальною інформацією для оперативного керування івентом.

Таблиця 2.24 – Варіант використання UC-24

Use case name	Перегляд надісланих робіт
Use case ID	UC-23
Goals	Ознайомлення з проєктними матеріалами команд та моніторинг балів, виставлених іншими членами колегії журі
Actors	Журі
Trigger	Перехід до робочого простору оцінювання хакатону («Jury Workspace»)
Pre-conditions	Користувач авторизований, наділений роллю судді для цієї події, а хакатон перебуває у фазі оцінювання робіт
Flow of Events	Система завантажує загальний перелік команд-учасників із індикаторами їх поточного статусу перевірки та лічильником загального прогресу оцінювання. При виборі команди система підтягує з бази даних інформацію про проєкт: назву, час здачі та детальний опис вирішуваної проблеми. Користувач може ініціювати прямий перехід до зовнішнього публічного репозиторію коду на GitHub. Одночасно система виконує вибірку та візуалізує перелік зафіксованих оцінок від колег по журі із відображенням їхніх профілів та аватарів.
Extension	UC-23.1: Перегляд оцінок журі (колег по колегії) UC-24: Оцінка роботи
Post-Condition	Суддя отримав повну інформацію про технічну складову проєкту та поточний стан його оцінювання іншими експертами.

Таблиця 2.25 – Варіант використання UC-25

Use case name	Оцінка роботи
Use case ID	UC-24
Goals	Диференційоване виставлення балів за встановленими критеріями, формування текстового відгуку та фіксація підсумкового середньозваженого результату проєкту
Actors	Журі
Trigger	Зміна значень у формі оцінювання або активація текстового поля рецензії
Pre-conditions	Хакатон у статусі «ACTIVE»; користувач призначений організатором до списку журі
Flow of Events	Система підтягує критерії оцінювання події з бази даних. Суддя виставляє бали за кожним параметром у межах його максимального значення та заповнює поле експертного відгуку. Система в реальному часі прораховує підсумкову оцінку на основі відсоткової ваги кожного критерію та динамічно зберігає проміжний стан форми у локальне сховище браузера для захисту від втрати даних. При підтвердженні фінального оцінювання система надсилає пакет даних (бали + коментар) на сервер, очищує локальну чернетку, оптимістично маркує команду в загальному списку як перевірену та автоматично перемикає інтерфейс на наступний проєкт.
Extension	UC-24.1: Оцінка кожного окремого критерію з кроком 0.5 за допомогою інтерактивних регуляторів UC-24.2: Коментування (написання текстового експертного відгуку) UC-24.3: Автоматичне збереження чернетки оцінювання у локальне сховище UC-24.4: Редагування коментаря та перезапис раніше опублікованих балів у базі даних
Post-Condition	Індивідуальні бали та відгук експерта успішно збережені в базі даних, лічильник перевірених робіт оновлено. Позиція учасника оновлюється автоматично.

2.2 Розроблення функціональних вимог

Програмне забезпечення h.linker розділене на функціональні модулі, кожен з яких відповідає за певну сферу взаємодії користувача з платформою. У таблиці 2.26 наведено загальну модель вимог, яка визначає пріоритетність та рівень ризику реалізації кожного функціонального блоку. Деталізований опис кожної окремої вимоги, що розкриває її технічну сутність, представлено у таблиці 2.27. Для підтвердження повноти проєктування та встановлення чітких зв'язків між функціоналом і сценаріями взаємодії розроблено матрицю трасування вимог, яку наведено на рисунку 2.2.

Таблиця 2.26 – Загальна модель вимог

№	Назва	ID	Пріоритет	Ризики
1	Керування доступом	FR-1	Високий	Середній
1.1	Реєстрація користувача	FR-1.1	Високий	Високий
1.2	Авторизація в системі	FR-1.2	Високий	Середній
1.3	Вихід із системи	FR-1.3	Низький	Низький
1.4	Перегляд та редагування профілю	FR-1.4	Середній	Низький
2	Інформаційні функції	FR-2	Середній	Низький
2.1	Перегляд головної сторінки	FR-2.1	Низький	Низький
2.2	Перегляд галереї кращих проєктів	FR-2.2	Низький	Низький
2.3	Перегляд та фільтрація хакатонів	FR-2.3	Високий	Низький
3	Керування командою	FR-3	Високий	Середній
3.1	Створення команди	FR-3.1	Високий	Середній
3.2	Редагування даних команди	FR-3.2	Середній	Низький
3.3	Видалення команди	FR-3.3	Низький	Низький
3.4	Призначення іншого лідера	FR-3.4	Низький	Низький
3.5	Вихід із команди	FR-3.5	Середній	Низький
4	Взаємодія та Matchmaking	FR-4	Високий	Середній
4.1	Пошук учасників та рекомендації	FR-4.1	Середній	Середній
4.2	Надсилання запиту на вступ	FR-4.2	Середній	Середній
4.3	Робота із запрошеннями (прийом/відмова)	FR-4.3	Високий	Середній
5	Участь та подання робіт	FR-5	Високий	Високий
5.1	Реєстрація на хакатон	FR-5.1	Високий	Середній
5.2	Прив'язка GitHub-репозиторію	FR-5.2	Середній	Високий
5.3	Фінальна задача проєкту	FR-5.3	Високий	Середній

Продовження таблиці 2.26

№	Назва	ID	Пріоритет	Ризики
5.4	Перегляд оцінок та фідбеку	FR-5.4	Середній	Низький
6	Організація хакатонів	FR-6	Високий	Середній
6.1	Створення хакатону	FR-6.1	Високий	Середній
6.2	Редагування параметрів хакатону	FR-6.2	Середній	Низький
6.3	Керування етапами хакатону	FR-6.3	Високий	Середній
6.4	Перегляд статистики хакатону	FR-6.4	Низький	Низький
7	Модуль журі	FR-7	Високий	Середній
7.1	Оцінювання робіт за критеріями	FR-7.1	Високий	Середній
7.2	Надання текстового фідбеку	FR-7.2	Середній	Низький

Таблиця 2.27 – Перелік функціональних вимог

Назва	Опис
FR-1	Комплекс функцій для ідентифікації користувача, забезпечення безпечного входу в систему та управління персональними даними.
FR-1.1	Створення облікового запису через форму або OAuth-сервіси (Google/GitHub) із вибором початкових технічних навичок.
FR-1.2	Надання доступу до приватних розділів платформи після верифікації даних та генерації сесійного JWT-токена.
FR-1.3	Безпечне завершення сеансу шляхом видалення ідентифікаційних даних із локального сховища браузера.
FR-1.4	Актуалізація персональної інформації, зміна пароля та керування списком технічних компетенцій розробника.
FR-2	Забезпечення публічного доступу до даних про платформу, активні заходи та успішні кейси минулих хакатонів.
FR-2.1	Відображення загальної інформації про h.linker, актуальної статистики та анонсів найближчих подій.
FR-2.2	Доступ до архіву переможців минулих.
FR-2.3	Відображення списку подій із можливістю пошуку та сортування за статусом, датами та призовим фондом.
FR-3	Набір інструментів для створення, адміністрування та модерації командних структур у межах платформи.
FR-3.1	Ініціалізація нової командної одиниці для обраного хакатону з наданням автору прав лідера.
FR-3.2	Оновлення назви, опису проєкту, логотипа та посилань.
FR-3.3	Повне розформування команди та анулювання її участі в заході до моменту подання фінальної роботи.

Продовження таблиці 2.27

Назва	Опис
FR-3.4	Офіційна передача адміністративних повноважень іншому зареєстрованому учаснику команди.
FR-3.5	Можливість для учасника залишити склад команди та відновити статус вільного розробника.
FR-4	Функції для комунікації між учасниками та автоматизованого пошуку партнерів за технічними критеріями.
FR-4.1	Підбір спеціалістів на основі технічного стека та перегляд списку вільних розробників.
FR-4.2	Ініціація запиту на приєднання до команди з очікуванням підтвердження від її лідера.
FR-4.3	Механізм обробки лідером заявок від кандидатів або користувачем – запрошень від команд.
FR-5	Функціонал для офіційної участі в хакатонах, інтеграції коду та фіксації результатів розробки.
FR-5.1	Закріплення команди за подією для отримання доступу до активної фази та технічних завдань.
FR-5.2	Інтеграція з GitHub API для збору статистики (коміти, мови програмування) та верифікації активності.
FR-5.3	Подання підсумкової заявки з посиланням на код та описом результатів для розгляду журі.
FR-5.4	Відображення балів та текстових рецензій експертів після завершення етапу оцінювання.
FR-6	Панель інструментів для створення нових подій та моніторингу їх проведення в реальному часі.
FR-6.1	Використання конструктора для налаштування таймлайну, призового фонду, складу журі та критеріїв.
FR-6.2	Коригування налаштувань поточної події організатором (зміна описів, дат або списку журі).
FR-6.3	Перемикання фаз заходу (Реєстрація, Активна фаза, Оцінювання, Завершено).
FR-6.4	Аналітика кількості реєстрацій, зданих проєктів та активності журі в реальному часі.
FR-7	Спеціалізований інтерфейс для експертного оцінювання поданих робіт за встановленими правилами.
FR-7.1	Виставлення числових балів за кожним параметром з урахуванням вагових коефіцієнтів.
FR-7.2	Написання розгорнутих рецензій та коментарів до проєктів для надання зворотного зв'язку учасникам.

ID Вимоги	UC-01	UC-02	UC-03	UC-04	UC-05	UC-06	UC-07	UC-08	UC-09	UC-10	UC-11	UC-12	UC-13	UC-14	UC-15	UC-16	UC-17	UC-18	UC-19	UC-20	UC-21	UC-22	UC-23	UC-24
FR-1	+	+					+	+	+															
FR-1.1	+																							
FR-1.2		+																						
FR-1.3									+															
FR-1.4							+	+																
FR-2			+	+	+	+																		
FR-2.1			+																					
FR-2.2				+																				
FR-2.3					+	+																		
FR-3										+	+	+	+		+									
FR-3.1											+													
FR-3.2													+											
FR-3.3													+											
FR-3.4													+											
FR-3.5														+										
FR-4							+		+	+		+	+			+								
FR-4.1																+								
FR-4.2									+	+			+											
FR-4.3							+					+												
FR-5							+								+		+	+						
FR-5.1																	+							
FR-5.2							+											+						
FR-5.3																		+						
FR-5.4															+									
FR-6																			+	+	+			
FR-6.1																			+					
FR-6.2																				+				
FR-6.3																					+			
FR-6.4																						+		
FR-7																							+	+
FR-7.1																								+
FR-7.2																								+

Рисунок 2.2 – Матриця трасування вимог

2.3 Розроблення нефункціональних вимог

Нефункціональні вимоги визначають критерії якості, обмеження та експлуатаційні характеристики системи h.linker, які не стосуються безпосередньої бізнес-логіки, але є критичними для її стабільного функціонування. На основі аналізу предметної області та обраного технічного стека визначено наступні групи вимог:

- продуктивність та швидкість відгуку: система повинна забезпечувати час відгуку на основні користувацькі запити не більше 1.5–2 секунд при нормальному рівні навантаження. Архітектура сервера на базі NestJS та оптимізована структура бази даних повинні витримувати пікові навантаження, особливо під час етапу масової здачі проєктів, забезпечуючи стабільну обробку до 100 одночасних запитів без суттєвої деградації продуктивності. Час генерації фінальної таблиці результатів з урахуванням усіх зважених коефіцієнтів та критеріїв оцінювання не повинен перевищувати 1 секунди;
- безпека та захист даних: автентифікація та авторизація користувачів у системі має базуватися на використанні технології JWT. Система

повинна реалізовувати рольову модель доступу, розмежовуючи права та доступні функції між звичайним учасником, лідером команди, членом журі та організатором хакатону. Конфіденційні дані, такі як паролі користувачів та токени доступу до GitHub API, мають зберігатися в базі даних у зашифрованому вигляді з використанням сучасних алгоритмів хешування;

- сумісність та адаптивність: клієнтська частина застосунку, реалізована на фреймворку Angular, повинна бути повністю адаптивною, забезпечуючи коректне та зручне відображення інтерфейсу на пристроях із різною роздільною здатністю екрана, включаючи десктопи, планшети та мобільні телефони. Користувацький інтерфейс має стабільно та без помилок функціонувати в усіх сучасних та популярних веббраузерах: Google Chrome, Mozilla Firefox, Safari, Microsoft Edge;
- зручність використання: інтерфейс системи має бути інтуїтивно зрозумілим, забезпечуючи доступ до будь-якої ключової функції не більше ніж за 3 кліки. Програма повинна видавати чіткі та змістовні повідомлення про помилки;
- надійність та підтримка: архітектура коду та використання Docker гарантують стабільну роботу системи в різних середовищах та легкість її подальшого масштабування. Обрана база даних PostgreSQL забезпечує повну цілісність даних.

2.4 Аналіз системних вимог

Програмне забезпечення повинно функціонувати на IBM-сумісних персональних комп'ютерах.

Мінімальна конфігурація технічних засобів:

- операційна система: Windows 10;
- об'єм ОЗП: 4 Гб;
- браузер: підтримка сучасних браузерів

- підключення до мережі Інтернет зі швидкістю від 20 мегабіт;

Рекомендована конфігурація технічних:

- операційна система: Windows 10;
- тип процесору: Ryzen 7 5700x;
- об'єм ОЗП: 32 Гб;
- браузер: найновіші версії браузерів з підтримкою HTML5 і JavaScript;
- підключення до мережі Інтернет зі швидкістю від 100 мегабіт;

2.5 Аналіз економічних показників програмного забезпечення

2.5.1 Аналіз функціональних точок

Метод функціональних точок [8] дозволяє оцінити розмір програмного забезпечення, спираючись на його функціональність, визначену у вимогах користувача, незалежно від технологій реалізації. Процес аналізу передбачає ідентифікацію функцій даних та транзакційних функцій з їх наступним зважуванням за факторами складності

2.5.1.1 Аналіз функцій даних

Функціональність даних оцінюється шляхом аналізу логічних груп даних, які підтримуються або використовуються програмним продуктом. У межах системи h.linker виділяють два типи груп даних:

- внутрішні логічні файли – групи даних, що знаходяться всередині кордонів ПЗ та обслуговуються його транзакціями. Детальний перелік та розрахунок балів для ILF наведено у таблиці 2.28;
- зовнішні інтерфейсні файли – групи даних, на які посилається продукт, але які підтримуються зовнішніми системами. Детальний перелік та розрахунок балів для EIF наведено у таблиці 2.29.

Складність кожної групи даних визначається DET та RET. Для всіх ідентифікованих файлів системи h.linker встановлено низький рівень

складності, що відповідає ваговим коефіцієнтам у 7 балів для ILF та 5 балів для EIF.

Таблиця 2.28 – Перелік та оцінка ILF

№	Назва ILF	Опис	RET	DET	Складність	UFPC
1	Користувачі (users)	Профілі, біографії та дані автентифікації.	1	10	Низька	7
2	Команди (teams)	Назви, описи та посилення для зв'язку команд.	1	5	Низька	7
3	Склад команд (users_teams)	Дані про поточне членство та ролі в командах.	1	8	Низька	7
4	Хакатони (hackathons)	Параметри, терміни та статуси заходів.	1	12	Низька	7
5	Реєстрація участі (participations)	Реєстрація команд на події та підсумкові бали.	1	9	Низька	7
6	Критерії оцінювання (criteria)	Правила оцінювання та вагові коефіцієнти.	1	5	Низька	7
7	Склад журі (jury)	Список експертів, закріплених за заходами.	1	3	Низька	7
8	Оцінки за критеріями (scores)	Бали журі за окремими критеріями оцінки.	1	6	Низька	7
9	Текстові відгуки (reviews)	Текстові рецензії та фідбек журі.	1	7	Низька	7
10	Категорії (categories)	Довідник технічних стеків та тематик.	1	1	Низька	7
11	Довідник ролей (roles)	Довідник функціональних ролей у команді.	1	1	Низька	7
	Разом					77

Відповідно до стандарту IFPUG, прості сполучні таблиці зв'язків «багато до багатьох» без додаткових бізнес-атрибутів (такі як users_categories та hackathons_categories) враховані як логічні зв'язки

(RET) всередині основних сутностей і не виділяються в окремі ILF, щоб уникнути штучного завищення складності архітектури. Таким чином, сумарна оцінка інформаційної цінності внутрішніх логічних файлів платформи становить 77 балів.

Таблиця 2.29 – Перелік та оцінка EIF

№	Назва EIF	Опис	RET	DET	Складність	UFP
1	Google OAuth Identity	Профіль користувача Google для автентифікації.	1	4	Low	5
2	GitHub Data (Profile & Repo)	Технічні дані про розробника та його репозиторії для верифікації активності.	2	8	Low	5
3	ImgBB Image Metadata	Дані про завантажені зображення, що зберігаються на зовнішньому хостингу.	1	4	Low	5
	Разом					15

2.5.1.2 Аналіз транзакційних функцій

Транзакційні функції представляють собою елементарні процеси, що забезпечують взаємодію користувача з даними системи h.linker. Згідно з методикою функціональних точок, вони поділяються на три типи:

- зовнішні введення EI – процеси, що змінюють стан внутрішніх логічних файлів або керують поведінкою системи. Перелік та оцінка EI наведені у таблиці 2.30;
- зовнішні запити EQ – процеси, що полягають у простому зчитуванні та відображенні даних без виконання складних розрахунків. Розрахунок для EQ наведено у таблиці 2.31;
- зовнішні виведення EO – процеси виведення даних, що обов’язково включають математичну обробку, розрахунки або створення похідних даних. Розрахунок для EO наведено у таблиці 2.32.

Складність транзакцій визначається кількістю DET та кількістю залучених логічних файлів FTR.

Таблиця 2.30 – Перелік та оцінка EI

№	Назва транзакції (EI)	Пов'язані UC	FTR	DET	Складність	Бали
1	Реєстрація та редагування профілю	UC-01, UC-08	1	10	Низька	3
2	Авторизація в системі (Вхід)	UC-02	1	2	Низька	3
3	Вихід із системи (Анулювання сесії)	UC-09	1	1	Низька	3
4	Створення нової команди	UC-12	1	4	Низька	3
5	Редагування даних команди	UC-13	1	3	Низька	3
6	Надсилання запиту або запрошення	UC-14, UC-17	1	5	Низька	3
7	Прийняття або відхилення запиту/запрошення	UC-08.2, UC-13.2	1	2	Низька	3
8	Зміна ролі учасника або лідера команди	UC-13.1, UC-13.3	1	3	Низька	3
9	Створення хакатону	UC-20	1	9	Низька	3
10	Редагування параметрів та критеріїв хакатону	UC-21	2	4	Низька	3
11	Керування етапами хакатону (Зміна статусу)	UC-21.3	1	2	Низька	3
12	Реєстрація команди на захід	UC-18	1	2	Низька	3
13	Подання фінальної роботи (Submission)	UC-19	1	4	Низька	3
14	Введення числових балів за критеріями	UC-24	1	4	Низька	3
15	Публікація текстового відгуку (Review)	UC-24	1	4	Низька	3
	Разом					45

Таблиця 2.31 – Перелік та оцінка EQ

№	Назва транзакції (EQ)	Пов'язані UC	FTR	DET	Складність	Бали
1	Перегляд головної сторінки	UC-03	2	5	Низька	3
2	Перегляд переліку хакатонів та фільтрація	UC-05	1	6	Низька	3
3	Перегляд деталей хакатону (таби, таймлайн)	UC-06	3	12	Низька	3
4	Перегляд профілів користувачів	UC-07	1	8	Низька	3
5	Перегляд переліку команд та фільтрація	UC-10	1	5	Низька	3
6	Перегляд деталей команди (члени, репозиторії)	UC-11	3	10	Низька	3
7	Пошук нових учасників (талантів) за стеком	UC-17	1	7	Низька	3
8	Перегляд надісланих робіт (робочий простір журі)	UC-23	3	9	Низька	3
	Разом					24

Таблиця 2.32 – Перелік та оцінка EO

№	Назва транзакції (EO)	Пов'язані UC	FTR	DET	Складність	Бали
1	Перегляд найкращих проєктів (Showcase)	UC-04	2	6	Низька	4
2	Перегляд оцінок та фідбеку (Дашборд команди)	UC-16	3	7	Низька	4
3	Перегляд аналітичної статистики хакатону	UC-22	3	8	Низька	4
	Разом					12

Всі транзакції ідентифіковані на основі розроблених раніше варіантів використання, що забезпечує повне покриття функціональних вимог до системи h.linker.

2.5.1.3 Розрахунок нормованої кількості функціональних точок

Загальна кількість ненормованих функціональних точок UFPC [8] розраховується як сума зважених значень для всіх ідентифікованих функцій даних та транзакцій, що були деталізовані у таблицях 2.28 – 2.32:

$$UFPC = \sum ILF + \sum EIF + \sum EI + \sum EQ + \sum EO \quad (2.1)$$

$$UFPC = 77 + 15 + 45 + 24 + 12 = 173$$

2.5.2 Аналіз основних характеристик системи GSC

Для врахування нефункціональних вимог та особливостей середовища розробки системи h.linker використовується коефіцієнт коригування значення VAF. Він базується на оцінці 14 загальних характеристик системи GSC [8; 9], що наведені у таблиці 2.33.

Таблиця 2.33 – Характеристики GSC

№	Характеристика	Оцінка	Обґрунтування
1	Обмін даними	4	Використання REST API для інтеграції з GitHub, Google та ImgBB.
2	Розподілена обробка даних	2	Розподіл логіки між клієнтом (Angular) та сервером (NestJS).
3	Продуктивність	3	Важливість швидкості завантаження лідербордів та важких списків.
4	Обмеження по апаратних ресурсів	2	Стандартні вимоги до хмарних ресурсів або VPS.
5	Транзакційне навантаження	3	Висока активність користувачів у моменти здачі проєктів (дедлайнів).
6	Інтенсивність взаємодії з користувачем	5	Система повністю побудована на формах (створення команд, оцінювання).
7	Ергономіка	4	Оптимізація інтерфейсу для швидкої роботи журі та організаторів.
8	Інтенсивність зміни даних	4	Необхідність миттєвого оновлення статусів та балів у базі даних.
9	Складність обробки	3	Алгоритми розрахунку середньозважених балів та логіка прав доступу.

Продовження таблиці 2.33

№	Характеристика	Оцінка	Обґрунтування
10	Повторне використання	2	Модульна структура для автентифікації та роботи з медіа-файлами.
11	Зручність інсталяції	3	Використання Docker для швидкого розгортання оточення.
12	Зручність адміністрування	3	Наявність адміністративної панелі та логування процесів.
13	Можливість портування	0	Платформа розгортається як єдиний вебресурс.
14	Зручність	3	Можливість додавання нових типів оцінювання без зміни ядра системи.
	Сумарний вплив (TDI)	41	

2.5.3 Обчислення метрик вартості

Розрахунок VAF виконується за формулою [8]:

$$VAF = 0.65 + (TDI \cdot 0.01) = 0.65 + (0.01 \cdot 41) = 1.06 \quad (2.2)$$

Кінцевий функціональний розмір програмного продукту у нормованих функціональних точках AFPC становить [8]:

$$AFPC = VAF \cdot UFPC = 173 \cdot 1.06 = 183.38 \quad (2.3)$$

На основі отриманого показника AFPC виконується прогноз обсягу вихідного коду (SLOC) [8]. Для мови програмування TypeScript (стек NestJS та Angular) середній показник Language Multiplier за даними QSM становить $LM = 53$ рядків на одну точку [10].

$$SLOC = AFPC \cdot LM = 183.38 \cdot 53 \approx 9719 \text{ рядків} \quad (2.4)$$

Враховуючи необхідність роздільної реалізації логіки на боці сервера та інтерфейсу користувача, очікуваний обсяг коду системи становитиме від 12 000 до 15 000 рядків.

На основі розрахованого значення AFPC визначається загальна трудомісткість розробки. Для цього використовується показник середньої продуктивності розробки P .

Згідно з базовими метриками (табл. 16), продуктивність для JavaScript становить 8.29 FP/міс [11, с. 49]. Проте, враховуючи використання сучасних фреймворків як Nest.js та Angular, типізації TypeScript, засобів швидкої розробки та інструментів, коефіцієнт продуктивності для даного проєкту було скориговано до $P = 15$ FP/міс.

$$E = \frac{AFPC}{P} = \frac{183.38}{15} \approx 12.225 \text{ людино-місяці} \quad (2.5)$$

Оцінка вартості розробки базується на розрахованій трудомісткості та середньомісячній заробітній платі Middle-розробника на ринку України, що приблизно дорівнює 2000\$ [12].

$$C = E \cdot S_{avg} = 12.225 \cdot 2000 = 24450\$ \quad (2.6)$$

2.6 Постановка завдання на розробку програмного забезпечення

Об'єктом розробки у даному дипломному проєкті є вебзастосунок h.linker, призначена для організації та проведення онлайн-хакатонів. Система має забезпечувати повний життєвий цикл технічного заходу: від ініціації події будь-яким користувачем до фінального оцінювання проєктів експертами.

Метою розробки є створення гнучкої, надійної та масштабованої екосистеми, яка автоматизує рутинні процеси управління хакатоном, спрощує формування команд на основі технічних навичок та забезпечує прозору систему оцінювання результатів.

- реалізувати систему автентифікації
- впровадити інструменти формування команд на основі технічних навичок та ролей користувачів;
- розробити покроковий конструктор для повного налаштування параметрів хакатону, включаючи таймлайн, склад журі та гнучку систему критеріїв оцінювання;
- забезпечити інтеграцію з GitHub API;
- впровадити рушій автоматичного розрахунку зваженого балу та систему відображення аналітичної статистики і результатів.

2.7 Висновки до розділу

У другому розділі було проведено всебічне розроблення вимог до програмного забезпечення вебзастосунку «h.linker», що включало моделювання варіантів використання, аналіз функціональних, нефункціональних та системних вимог, а також детальний розрахунок економічних показників майбутнього продукту.

На основі аналізу предметної області проведено моделювання сценаріїв взаємодії, що дозволило чітко розмежувати ролі та права доступу основних акторів системи: неавторизованих гостей, зареєстрованих учасників, лідерів команд, організаторів хакатонів та членів журі. Розроблено 25 сценаріїв варіантів використання, які охоплюють повний життєвий цикл заходу – від реєстрації профілю через GitHub OAuth до складних процесів динамічного оцінювання проєктів за допомогою зважених коефіцієнтів та автоматичного формування лідербордів у реальному часі. Ці сценарії лягли в основу переліку функціональних вимог, що забезпечують роботу Matchmaking-системи для пошуку учасників за навичками, інтеграцію з GitHub API для верифікації коду та гнучке керування етапами проведення змагань.

Проведено аналіз економічних показників проєкту методом функціональних точок. На основі ідентифікації 11 ILF та 3 EIF логічних файлів, а також оцінки системних характеристик, визначено нормований функціональний розмір системи – 183.38 точки (AFPC). Прогнозований обсяг вихідного коду для обраного стеку (NestJS/Angular) становить 12 000 – 15 000 рядків. Розрахована трудомісткість розробки складає 12 людино-місяців, а загальна вартість проєкту за актуальними ринковими ставками оцінюється у 24450 USD. Отримані результати підтверджують економічну доцільність реалізації системи.

За результатами розділу сформовано технічне завдання на розробку програмного забезпечення.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

Для реалізації системи `h.linker` була обрана багаторівнева клієнт-серверна архітектура. Даний підхід забезпечує високу масштабованість, надійність та чіткий розподіл відповідальності між клієнтською частиною (Frontend) та серверною частиною (Backend). Клієнтська частина, реалізована на базі фреймворку Angular, відповідає за рендеринг інтерфейсу, обробку станів користувача та взаємодію з кінцевим споживачем. Серверна частина на базі NestJS забезпечує виконання бізнес-логіки, валідацію даних, безпеку та взаємодію з базою даних.

Вибір архітектурного рішення для серверної частини базується на використанні модульної структури, нативної для фреймворку NestJS. Архітектура платформи побудована на принципах інверсії управління та DI, що гарантує високу тестованість та гнучкість компонентів. Завдяки розподілу на модулі, контролери та сервіси, система дозволяє ізолювати логіку управління хакатонами від функціонала командної взаємодії, що спрощує подальшу підтримку та розширення продукту.

Для візуалізації структури системи та процесів взаємодії використано модель C4, яка дозволяє поетапно деталізувати архітектуру від загального контексту до конкретних компонентів.

Діаграма системного контексту (C4 Level 1) відображає місце системи `h.linker` в інформаційному середовищі та її зв'язки із зовнішніми акторами та сервісами (рис. 3.1). Основними користувачами системи є учасники хакатонів, лідери команд, організатори та члени журі. Для забезпечення повноцінного функціонала система інтегрується з наступними зовнішніми сервісами:

- Google OAuth API – для ідентифікації та автентифікації користувачів;
- GitHub API – для верифікації профілів розробників, аналізу репозиторіїв та збору технічних метрик поданих проєктів;

- ImgBB API – для хмарного зберігання зображень (логотипів команд, обкладинок хакатонів та аватарів).

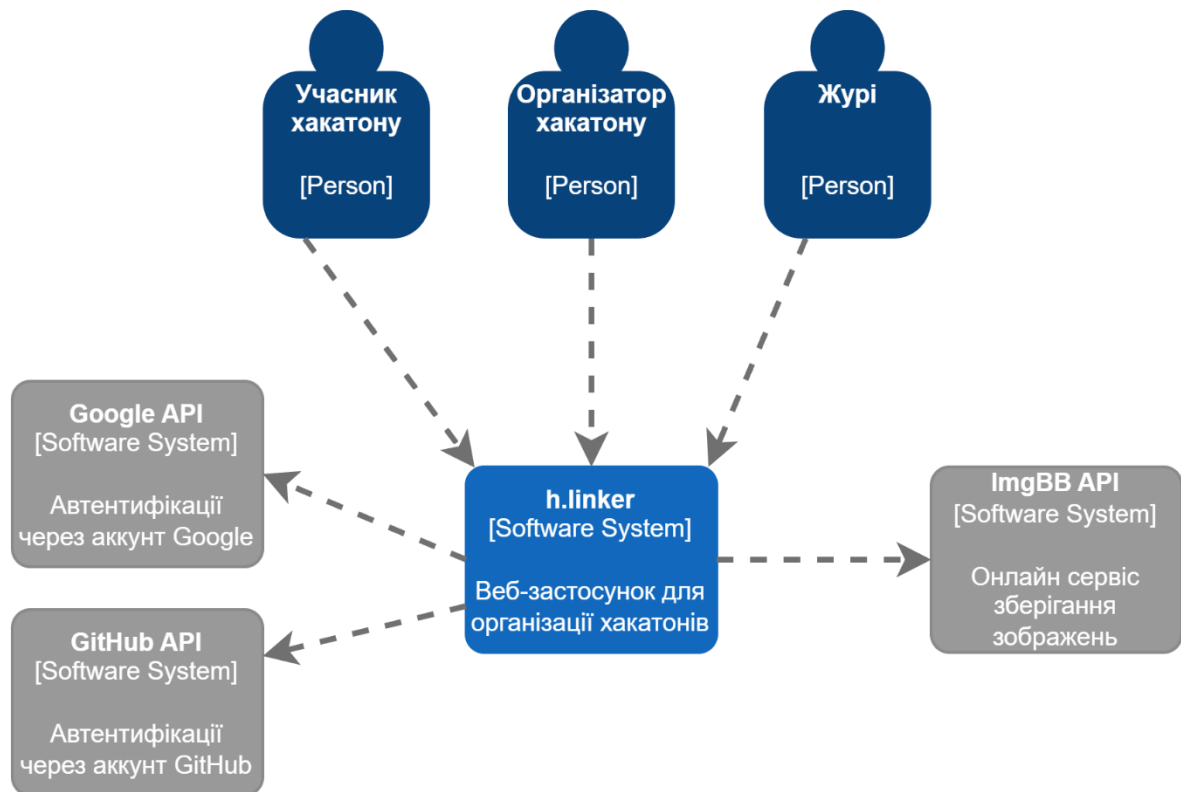


Рисунок 3.1 – Діаграма C4 першого рівня

Діаграма контейнерів (C4 Level 2) деталізує внутрішню структуру платформи, виділяючи три ключові технічні компоненти (рис. 3.2). Основним елементом взаємодії з користувачем є Web Client – односторінковий застосунок на базі фреймворку Angular, який виконується безпосередньо у браузері та забезпечує високу динамічність інтерфейсу. Клієнтська частина надсилає запити до API Server, що являє собою серверний застосунок на NestJS. Цей контейнер виступає центральним вузлом обробки бізнес-логіки, реалізує RESTful API та відповідає за виконання алгоритмів модуля оцінювання. Усі дані системи, включаючи інформацію про користувачів, команди, транзакції та результати змагань, надійно зберігаються у контейнері Database, який представлений реляційною базою даних PostgreSQL.

Інтеграція із зовнішніми програмними системами реалізована з урахуванням оптимального розподілу функцій. Клієнтський застосунок бере на себе пряму взаємодію з ImgBB API. Натомість серверна частина на NestJS

забезпечує безпечну комунікацію з Google API для автентифікації користувачів, а також з GitHub API для автоматизованого збору технічних метрик репозиторіїв та верифікації профілів розробників.

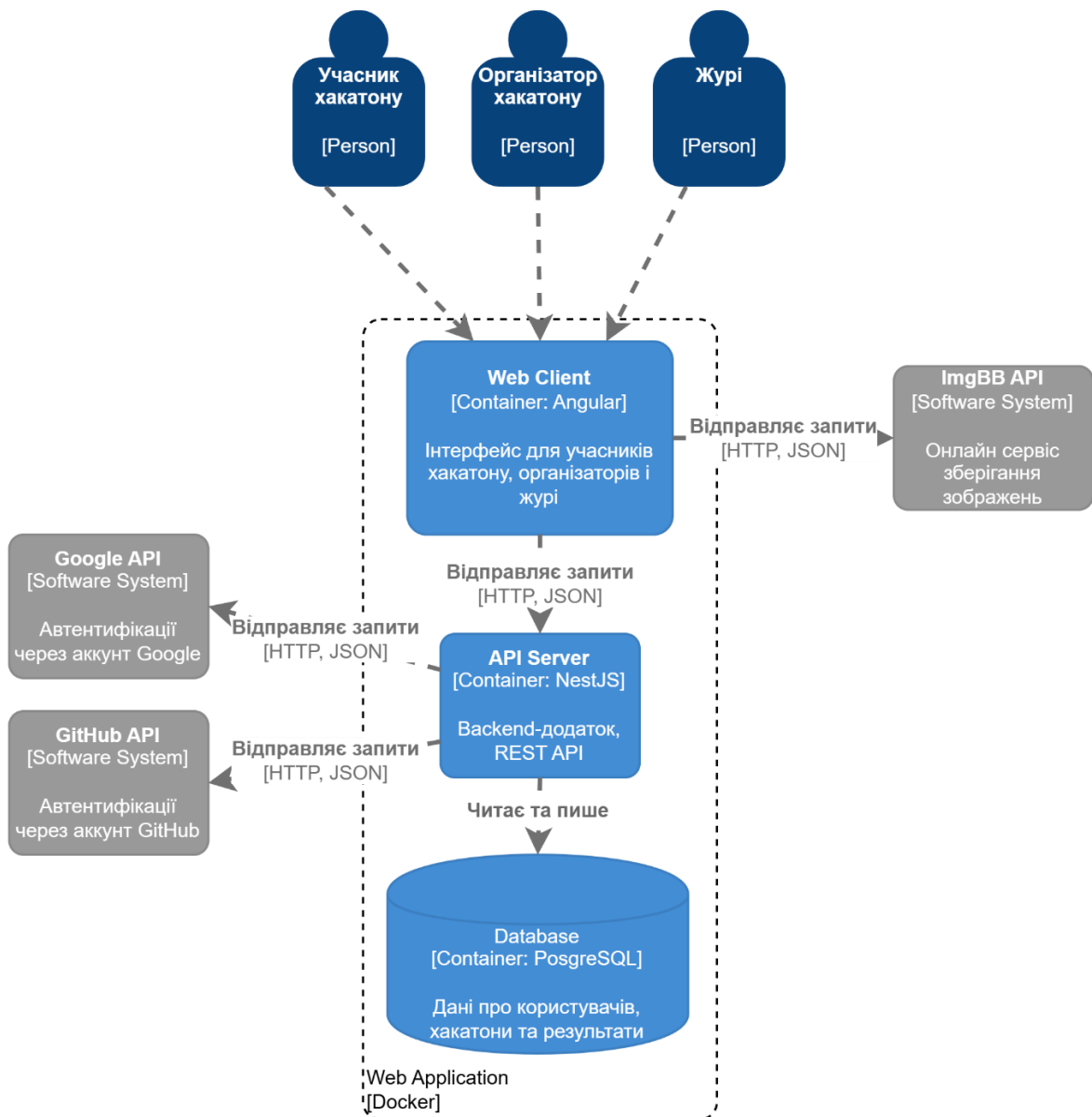


Рисунок 3.2 – Діаграма C4 другого рівня

Діаграма компонентів (C4 Level 3) деталізує внутрішню структуру контейнерів платформи (рис. 3.3). На цьому рівні серверний застосунок декомпоновано на функціональні модулі NestJS, що відповідають за автентифікацію, управління хакатонами, роботу команд та динамічне оцінювання. Внутрішня структура контейнера повністю декомпонована на шість функціональних модулів NestJS: AuthModule, UserModule, TeamModule, HackathonModule, CategoryModule та PrismaModule. Взаємодія між

компонентами відбувається через внутрішні сервіси, а зв'язок із базою даних забезпечується засобами Prisma ORM.

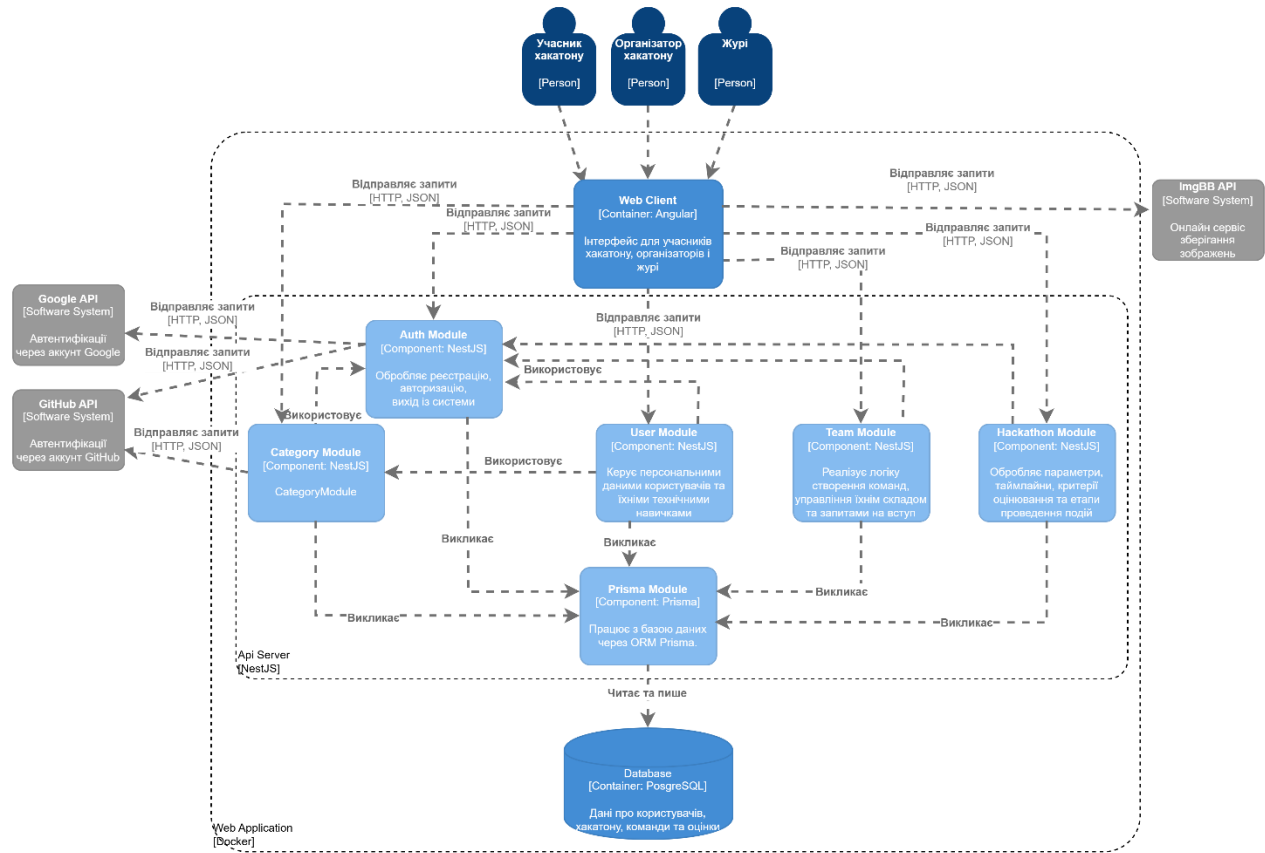


Рисунок 3.3 – Діаграма C4 третього рівня

3.2 Архітектурні рішення та обґрунтування вибору засобів розробки

Нижче розглянуто основні архітектурні підходи та інструменти розробки, обрані для платформи «h.linker». Обґрунтування цих рішень базується на необхідності створити стабільну, безпечну та масштабовану систему.

3.2.1 Ключові архітектурні рішення

Процес управління користувачами та їх автентифікація в системі реалізуються за гібридною моделлю, що поєднує стратегію SSO та традиційну реєстрацію. Використання сторонніх ідентифікаторів через Google та GitHub на базі протоколу OAuth 2.0 дозволяє технічним фахівцям швидко заходити в систему без створення нових паролів. Водночас у системі передбачено

можливість реєстрації через електронну пошту з використанням локального пароля.

Інтеграція із зовнішніми програмними системами виконується через стандартні інтерфейси HTTPS/REST. Сервіс Google API задіяний виключно для ідентифікації осіб, тоді як можливості GitHub API використовуються глибше – для автоматизованої верифікації профілів та аналізу технічних метрик репозиторіїв, що є основою для модуля оцінювання. Робота з графічним контентом делегована сервісу ImgBB, що дозволяє винести ресурсомісткі операції зі зберігання та обробки зображень на спеціалізовану хмарну платформу, зменшуючи навантаження на основний сервер застосунку.

Виконання нефункціональних вимог до продуктивності та безпеки забезпечується впровадженням механізму JWT-токенів. Використання безсесійної архітектури дозволяє серверній частині працювати у форматі «stateless», що критично для горизонтального масштабування системи за умов великої кількості одночасних підключень. На рівні збереження даних обрано реляційну модель, реалізовану в СУБД PostgreSQL. Такий вибір зумовлений високою складністю зв'язків між сутностями (команди, хакатони, критерії та бали).

3.2.2 Обґрунтування технологічного стеку

Для розробки системи обрано стек технологій на основі мови TypeScript, що дозволяє забезпечити типізацію від бази даних до інтерфейсу користувача. Використання TypeScript мінімізує кількість помилок на етапі компіляції та спрощує підтримку складних алгоритмів розрахунку середньозважених балів.

Серверна частина базується на фреймворку NestJS, який порівняно з аналогами пропонує чітку модульну архітектуру та вбудовану підтримку ін'єкції залежностей. Клієнтська частина реалізована за допомогою Angular. Даний фреймворк обрано через його комплексність та наявність потужних інструментів для створення складних інтерфейсів управління, зокрема бібліотеки RxJS для реактивної обробки потоків даних.

Структура проєкту організована за принципом монорепозиторію. Це рішення дозволяє зберігати вихідний код фронтенду та бекенду в одному сховищі, що значно спрощує спільне розробку. Завдяки такому підходу вдалося уникнути дублювання коду, полегшити управління спільними бібліотеками та забезпечити швидку синхронізацію змін між клієнтською та серверною частинами системи.

Для управління інформацією в системі обрано реляційний тип бази даних, а саме PostgreSQL. Такий вибір зумовлений наявністю великої кількості зв'язків між сутностями. Реляційна модель дозволяє гарантувати цілісність даних та забезпечує високу точність при виконанні аналітичних запитів.

Взаємодія програмного коду з базою даних реалізується через Prisma ORM. На відміну від класичних рішень, Prisma автоматично генерує типи на основі схеми даних, що виключає невідповідність між структурами в коді та в БД. Це рішення забезпечує високу швидкість розробки та надійність запитів при формуванні динамічних лідербордів.

3.2.3 Допоміжні програмні засоби

Вибір інструментарію розробника ґрунтується на необхідності глибокої інтеграції з обраними фреймворками. Буде використано WebStorm IDE [15]. Дане середовище забезпечує підтримку рефакторингу TypeScript-коду та має розвинені вбудовані засоби візуального налагодження для Angular-компонентів та сервісів NestJS.

Для забезпечення якості програмного коду використовується фреймворк Jest [16], за допомогою якого реалізовано автоматизовані юніт-тести для перевірки логіки модуля оцінювання. Налагодження взаємодії між клієнтом та сервером виконується у середовищі Postman [17], що дозволяє тестувати REST-ендпоінти в ізоляції та документувати структуру запитів.

3.3.1 Опис структури бази даних

База даних розробленої системи призначена для збереження інформації про облікові записи користувачів, конфігурації хакатонів, дані про сформовані команди та бали, виставлені членами журі. Логічна структура БД у вигляді ER-діаграми та її фізична модель наведені на рисунках 3.5 та 3.6 відповідно.



Рисунок 3.5 – ER діаграма сутностей

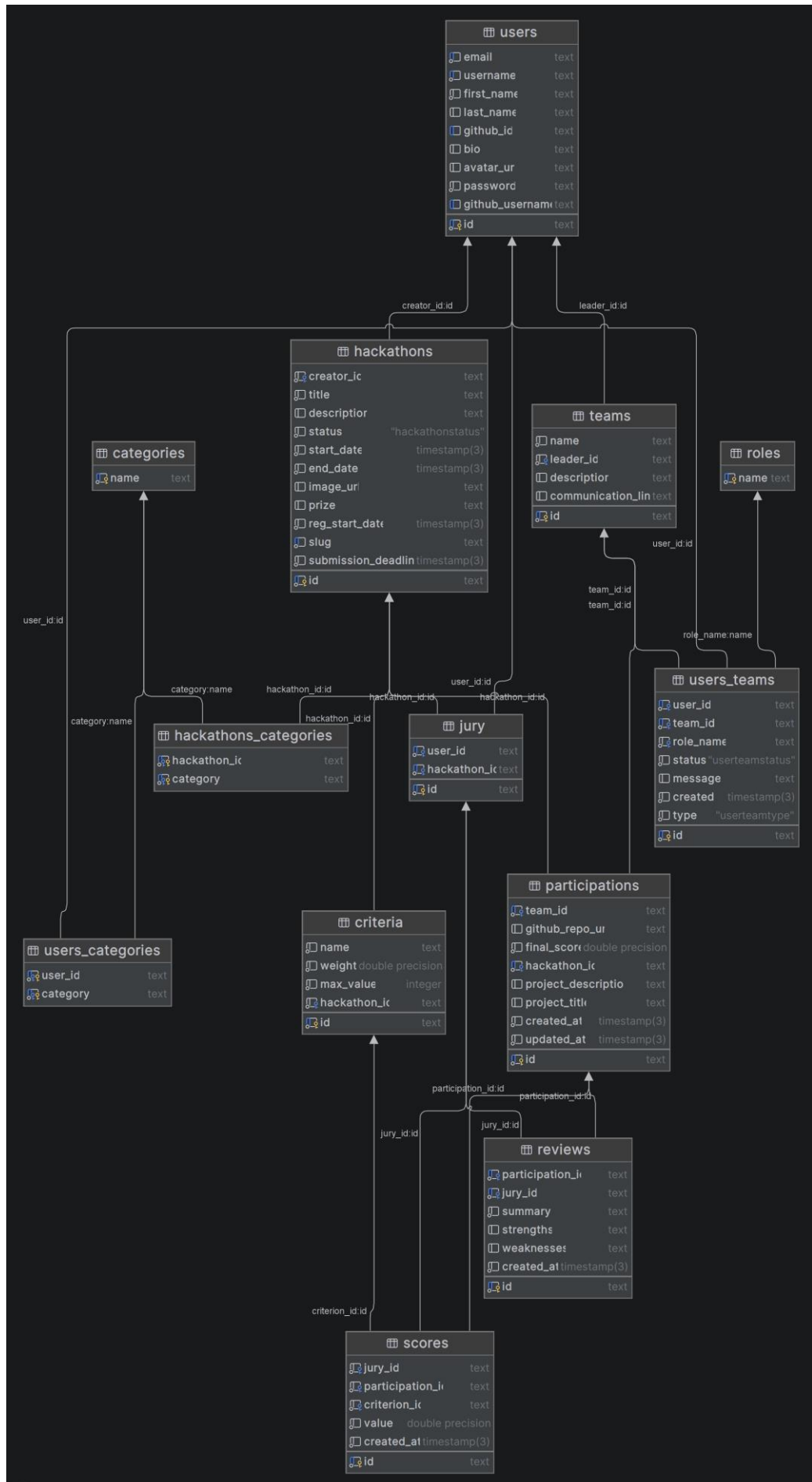


Рисунок 3.6 – Фізична схема БД

Для збереження даних про користувачів, параметри хакатонів, склад команд та результати оцінювання у системі «h.linker» застосовано СУБД PostgreSQL. Детальний опис кожної таблиці з переліком атрибутів, типів даних та описів полів представлено у таблицях 3.1 – 3.13.

Таблиця 3.1 – Опис таблиці users

Назва поля	Тип даних	Опис
id	varchar(36)	Первинний ключ, унікальний ідентифікатор користувача (UUID)
email	varchar (255)	Електронна адреса користувача
username	varchar(100)	Унікальний нікнейм користувача в системі
first_name	varchar (100)	Ім'я користувача
last_name	varchar (100)	Прізвище користувача
github_id	varchar (100)	Унікальний ідентифікатор профілю в системі GitHub
github_username	varchar(100)	Нікнейм користувача на платформі GitHub (опціональне)
bio	text	Коротка біографія або опис навичок користувача
avatar_url	varchar(255)	Посилання на зображення профілю (аватар)
password	text	Хешований пароль користувача для локальної автентифікації

Таблиця 3.2 – Опис таблиці teams

Назва поля	Тип даних	Опис
id	varchar(36)	Первинний ключ, унікальний ідентифікатор команди (uuid)
name	varchar(255)	Назва команди
leader_id	varchar(36)	Зовнішній ключ, ідентифікатор капітана команди (посилання на users.id)
description	text	Опис команди, її мети або спеціалізації
communication_link	varchar(255)	Посилання на канал зв'язку (Discord, Telegram, Slack тощо)

Таблиця 3.3 – Опис таблиці users_teams

Назва поля	Тип даних	Опис
id	varchar(36)	Зовнішній ключ, ідентифікатор користувача (посилання на users.id)
user_id	varchar(36)	Зовнішній ключ, ідентифікатор користувача (посилання на users.id)
team_id	varchar(36)	Зовнішній ключ, ідентифікатор команди (посилання на teams.id)
status	Status	Перерахування, що визначає стан запиту. Доступні значення: pending (очікування), accepted (прийнято), declined (відхилено)

Продовження таблиці 3.3

Назва поля	Тип даних	Опис
type	RequestType	Визначає напрямок ініціативи: request (запит від користувача) або invite (запрошення від лідера команди)
role_name	varchar(50)	Зовнішній ключ, роль користувача у складі команди (посилання на roles.name)
message	text	Текст супровідного повідомлення, що додається до запиту або запрошення
created	timestamp	Дата та час створення запису (час надсилання)

Таблиця 3.4 – Опис таблиці participations

Назва поля	Тип даних	Опис
id	varchar(36)	Первинний ключ, унікальний ідентифікатор запису про участь (UUID)
team_id	varchar(36)	Зовнішній ключ, ідентифікатор команди (посилання на teams.id)
hackathon_id	varchar(36)	Зовнішній ключ, ідентифікатор хакатону (посилання на hackathons.id)
project_title	varchar(255)	Назва розробленого командою проєкту (опціональне)
project_description	text	Розширений опис функціоналу та особливостей проєкту (опціональне)

Продовження таблиці 3.4

Назва поля	Тип даних	Опис
github_repo_url	varchar(255)	Посилання на публічний GitHub-репозиторій із вихідним кодом проєкту (опціональне)
final_score	double precision	Підсумковий середньозважений бал команди за результатами оцінювання журі (за замовчуванням 0)
created_at	timestamp	Дата та час реєстрації команди на хакатон
updated_at	timestamp	Дата та час останнього оновлення матеріалів проєкту

Таблиця 3.5 – Опис таблиці hackathons

Назва поля	Тип даних	Опис
id	varchar(36)	Первинний ключ, унікальний ідентифікатор хакатону (UUID)
creator_id	varchar(36)	Зовнішній ключ, ідентифікатор організатора заходу (посилання на users.id)
title	varchar(255)	Назва хакатону
slug	varchar(255)	Унікальний текстовий URL-ідентифікатор хакатону для маршрутизації (унікальне поле)
description	text	Детальний опис заходу, цілі та правила проведення (опціональне)

Продовження таблиці 3.5

Назва поля	Тип даних	Опис
prize	varchar(255)	Інформація про призовий фонд, спонсорські бонуси або нагороди (опціональне)
status	HackathonStatus	Перерахування, що визначає етап проведення заходу. Доступні значення: DRAFT (чернетка), REGISTRATION (реєстрація), ACTIVE (активний), FINISHED (завершений)
reg_start_date	timestamp	Дата та час офіційного початку реєстрації команд на захід
start_date	timestamp	Дата та час офіційного початку хакатону (старту розробки)
submission_deadline	timestamp	Кінцева дата та час (дедлайн) подання проєктних робіт командами
end_date	timestamp	Дата та час офіційного завершення хакатону (закриття оцінювання та фіналу)
image_url	varchar(255)	Посилання на завантажений банер або промо-зображення хакатону (опціональне)

Таблиця 3.6 – Опис таблиці categories

Назва поля	Тип даних	Опис
name	varchar(100)	Первинний ключ, назва технологічного напрямку або категорії (наприклад: web, ai, blockchain)

Таблиця 3.7 – Опис таблиці users_categories

Назва поля	Тип даних	Опис
user_id	varchar(36)	Частина первинного ключа, ідентифікатор користувача (посилання на users.id)
category	varchar(100)	Частина первинного ключа, назва технологічної категорії (посилання на categories.name)

Таблиця 3.8 – Опис таблиці hackathons_categories

Назва поля	Тип даних	Опис
hackathon_id	varchar(36)	Частина первинного ключа, ідентифікатор хакатону (посилання на hackathons.id)
category	varchar(100)	Частина первинного ключа, назва категорії за напрямом заходу (посилання на categories.name)

Таблиця 3.9 – Опис таблиці criteria

Назва поля	Тип даних	Опис
id	varchar(36)	Первинний ключ, унікальний ідентифікатор критерію (uuid)
hackathon_id	varchar(36)	Зовнішній ключ, ідентифікатор хакатону, для якого встановлено критерій (посилання на hackathons.id)

Продовження таблиці 3.10

Назва поля	Тип даних	Опис
name	varchar(255)	Назва критерію (наприклад: «Чистота коду», «Інноваційність», «Дизайн»)
weight	double precision	Вагова частка критерію у загальному розрахунку підсумкового бала
max_value	integer	Максимально можлива оцінка за цим критерієм (за замовчуванням 10)
hackathon_id	varchar(36)	Зовнішній ключ, ідентифікатор хакатону, для якого встановлено критерій (посилання на hackathons.id)

Таблиця 3.10 – Опис таблиці jury

Назва поля	Тип даних	Опис
id	varchar(36)	Первинний ключ, унікальний ідентифікатор члена журі (uuid)
user_id	varchar(36)	Зовнішній ключ, ідентифікатор користувача, призначеного суддею (посилання на users.id)
hackathon_id	varchar(36)	Зовнішній ключ, ідентифікатор хакатону, у якому бере участь суддя (посилання на hackathons.id)

Таблиця 3.11 – Опис таблиці scores

Назва поля	Тип даних	Опис
id	varchar(36)	Первинний ключ, унікальний ідентифікатор запису про бал (uuid)
jury_id	varchar(36)	Зовнішній ключ, ідентифікатор судді, який виставив оцінку (посилання на jury.id)
participation_id	varchar(36)	Зовнішній ключ, ідентифікатор участі команди в заході (посилання на participations.id)
criterion_id	varchar(36)	Зовнішній ключ, ідентифікатор критерію, за яким виставлено бал (посилання на criteria.id)
value	double precision	Числове значення оцінки, виставлене суддею
createdAt	timestamp	Час виставлення оцінки

Таблиця 3.12 – Опис таблиці reviews

Назва поля	Тип даних	Опис
id	varchar(36)	Первинний ключ, унікальний ідентифікатор відгуку (uuid)
participation_id	varchar(36)	Зовнішній ключ, ідентифікатор відповідної участі команди (посилання на participations.id)
jury_id	varchar(36)	Зовнішній ключ, ідентифікатор судді, що написав відгук (посилання на jury.id)

Продовження таблиці 3.12

Назва поля	Тип даних	Опис
summary	text	Загальний короткий підсумок щодо представленого проєкту
strengths	text	Опис сильних сторін розробленого програмного продукту
weaknesses	text	Опис слабких сторін, недоліків або зауважень від журі
createdAt	timestamp	Час коментування

Таблиця 3.13 – Опис таблиці roles

Назва поля	Тип даних	Опис
name	varchar(50)	Первинний ключ, унікальна назва ролі учасника всередині команди (наприклад: leader, developer, designer, analyst)

3.3.2 Опис сторонніх бібліотек та утиліт

Для забезпечення стабільної роботи платформи, прискорення процесу написання коду та реалізації функцій безпеки було використано набір перевірених інструментів і бібліотек. У ході розробки застосовувалися засоби для керування базою даних, фреймворки для побудови інтерфейсу та серверної логіки, а також утиліти для тестування. Повний перелік основних програмних засобів, використаних у проєкті, наведено у таблиці 3.14.

Таблиця 3.14 – Опис утиліт

№ п/п	Назва утиліти	Опис застосування
1	WebStorm	Інтегроване середовище розробки (IDE) для написання, рефакторингу та відлагодження коду на TypeScript, Angular та NestJS.
2	Node.js	Програмне середовище виконання, що дозволяє запускати серверний код на базі рушія V8.
3	NestJS	Прогресивний Node.js-фреймворк, що використовується для створення модульної, ефективної та масштабованої архітектури серверної частини.
4	Angular	Платформа та фреймворк для розробки клієнтської частини застосунку у вигляді SPA (Single Page Application) з динамічним інтерфейсом.
5	Prisma ORM	Інструмент для об'єктно-реляційного відображення, що забезпечує безпечну роботу, міграції та сувору типізацію при взаємодії з СКБД PostgreSQL.
6	bcryptjs	Бібліотека для надійного криптографічного хешування паролів користувачів перед їх збереженням у базу даних.
7	Postman	Інструмент для ручного тестування RESTful API, що дозволяє надсилати HTTP-запити та верифікувати відповіді сервера.
8	TypeScript	Мова програмування зі строгою типізацією, що компілюється в JavaScript та мінімізує логічні помилки на етапі розробки.
9	passport.js	Проміжне ПЗ (middleware) для автентифікації, що інтегроване в NestJS для реалізації входу через сторонні сервіси Google та GitHub за протоколом OAuth 2.0.
10	Docker	Платформа для контейнеризації, що дозволяє ізолювати застосунок, його залежності та СКБД у стабільних ізольованих середовищах.
11	Jest	Фреймворк для автоматизованого тестування, що використовується для написання юніт-тестів та перевірки коректності ізольованої логіки сервісів.
12	Nx	Інструмент керування монорепозиторієм, який об'єднує клієнтську (Angular) та серверну (NestJS) частини в єдину кодову базу, оптимізуючи процеси збірки.
13	cookie-parser	Пакет для обробки та парсингу файлів cookie, необхідний для безпечного збереження JWT-токенів на стороні клієнта.

Продовження таблиці 3.14

№ п/п	Назва утиліти	Опис застосування
14	Database tool window	Вбудований в IDE інструмент для візуального керування базою даних, виконання SQL-запитів та перегляду схем даних в реальному часі.
15	class-validator	Бібліотека для декларативної валідації вхідних даних на основі декораторів у класах DTO (Data Transfer Object).
16	class-transformer	Пакет, що забезпечує автоматичне перетворення сирих об'єктів HTTP-запитів у екземпляри відповідних класів DTO із приведенням типів.
17	PrimeNG	Набір готових UI-компонентів для Angular, що використовується для побудови адаптивного, сучасного та уніфікованого інтерфейсу користувача.
18	RxJS	Бібліотека для реактивного програмування, що забезпечує обробку асинхронних потоків даних та подій на фронтенді за допомогою рішень Observable.
19	Axios	HTTP-клієнт, що використовується на стороні бекенду для виконання асинхронних запитів до GraphQL/REST API платформи GitHub.
20	@nestjs/jwt	Спеціалізований модуль для генерації, підпису та верифікації токенів безпеки JSON Web Tokens (JWT) під час авторизації сесій.
21	@nestjs/config	Модуль для централізованого керування конфігурацією застосунку та динамічного зчитування змінних оточення з файлів .env.

Тексти програмного коду наведені в окремому документі «Текст програми».

3.4 Аналіз безпеки даних

Захист інформації у системі «h.linker» базується на комплексному підході до безпеки персональних даних та результатів змагань. Основна увага приділяється запобіганню несанкціонованому доступу до конфіденційних ресурсів та забезпеченню цілісності технічних метрик проєктів на всіх етапах їхнього оброблення.

Для ідентифікації користувачів впроваджено гібридну модель автентифікації. Використання протоколу OAuth 2.0 (Google, GitHub) дозволяє делегувати перевірку даних надійним сервісам, а для локальних акаунтів застосовується стійке хешування бібліотекою `bcrypt`. Управління активними сесіями здійснюється через підписані JWT-токени, що гарантує безпечну передачу прав доступу між клієнтом і сервером. На рівні клієнтської архітектури захист приватних робочих просторів регулюється функціональними гвардами (Route Guards) на базі Angular Signals. Вони забезпечують безпеку SSR завдяки ізоляції виконання в контексті браузера, що запобігає передчасній ініціалізації захищених компонентів для неавторизованих гостей ще до моменту відправки запитів на сервер.

Безпека на рівні взаємодії з базою даних забезпечується використанням Prisma ORM, що автоматично унеможлиблює SQL-ін'єкції завдяки повній параметризації запитів. Додатковим фактором надійності є TypeScript і повна відмова від динамічних типів на користь об'єктів Prisma.Payload.

Валідація вхідних даних реалізована засобами NestJS за допомогою DTO та бібліотеки `class-validator`. Ця логіка комплементарно підсилена контекстуальною авторизацією всередині самих компонентів платформи. Для запобігання підміні ідентифікаторів у параметрах запиту впроваджено превентивне зіставлення ідентифікатора поточного сесійного користувача з унікальним ID власника ресурсу на етапі ініціалізації даних, що супроводжується миттєвим перериванням життєвого циклу сторінки у разі помилки. Окрім цього, у системі діє модель обмеження дій на основі поточного стану кінцевого автомата фаз хакатону, яка блокує можливість відправки або редагування матеріалів, якщо життєвий цикл події не перебуває в активній фазі.

Використання Docker дозволяє ізолювати компоненти системи та базу даних у стабільних контейнерах, захищених від прямого впливу хостової системи. Усі конфіденційні параметри, включаючи секретні ключі та паролі,

винесені у змінні оточення (.env), що виключає їх розголошення через систему контролю версій та забезпечує безпеку конфігурацій під час розгортання.

3.5 Висновки до розділу

У межах третього розділу було спроектовано технічну архітектуру веб застосунку «h.linker», що базується на клієнт-серверній моделі та використанні багаторівневих діаграм C4 для деталізації взаємодії компонентів. Обраний технологічний стек на основі мови TypeScript, фреймворків Angular та NestJS, а також архітектури монорепозиторію забезпечує високу швидкість розробки та стабільність системи.

Вибір NestJS що дозволив реалізувати роздільну логіку за принципами мікросервісного підходу в межах єдиного бекенд-додатка. Це забезпечує високу автономність функціональних модулів, спрощує їх супровід та створює можливість для легкого масштабування окремих частин системи у майбутньому без порушення цілісності всієї структури.

Розроблено фізичну модель бази даних у СУБД PostgreSQL, яка складається з 13 таблиць, адаптованих під специфіку проведення хакатонів. Структура охоплює всі ключові сутності: від облікових записів користувачів та конфігурацій змагань до динамічної системи оцінювання та управління командною взаємодією.

Також проведено аналіз безпеки та інструментів розгортання, де основна увага приділена захисту персональної інформації та стабільності середовища. Впроваджена гібридна модель автентифікації на базі OAuth 2.0 та локальних паролів із хешуванням bcrypt у поєднанні з JWT-токенами гарантує надійний захист доступу.

Використання Docker для контейнеризації та class-validator для перевірки вхідних даних забезпечує ізоляцію сервісів та стійкість системи до вразливостей, створюючи завершений фундамент для програмної реалізації проєкту.

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Аналіз якості ПЗ

Аналіз якості програмного забезпечення є важливим етапом життєвого циклу розробки, який дозволяє верифікувати відповідність створеного продукту встановленим функціональним та нефункціональним вимогам. Для платформи організації хакатонів «h.linker» забезпечення високої якості архітектури та кодової бази є особливо важливим з огляду на специфіку її використання: під час завершення етапу прийому робіт (дедлайнів) та активної фази оцінювання журі система піддається піковим навантаженням.

Для комплексної та об'єктивної оцінки якості програмного забезпечення було обрано наступні метрики:

- ергономічність та навігаційна доступність користувацького інтерфейсу;
- підтримуваність, структурованість та чистота кодової бази;
- продуктивність серверної частини та стан циклу подій;
- оптимізація та продуктивність клієнтського застосунку (Angular SPA).

Для оцінки ергономічності інтерфейсу та зручності користувацького досвіду (UX) було застосовано метрику на основі класичного «правила трьох кліків». Відповідно до неї, архітектура платформи має бути побудована таким чином, щоб користувач міг дістатися до ключових цільових сторінок не більше ніж за три переходи від головного екрана. Для перевірки цієї вимоги було згенеровано граф-діаграму навігаційного доступу (рис. 4.1). Як свідчать результати візуалізації, ієрархія маршрутизації є переважно плоскою та оптимізованою: абсолютна більшість функціональних вузлів (кабінет журі, списки хакатонів, налаштування команди) доступні за 1–3 кліки. Разом з тим, на діаграмі зафіксовано два вузли, доступ до яких потребує 4 переходів: «Здача

роботи» та «Публічний профіль користувачів». Таке відхилення є архітектурно обґрунтованим, оскільки ці сторінки не є первинними навігаційними цілями, а виступають логічним завершенням глибоких послідовних бізнес-процесів (наприклад: Головна → Хакатон → Команда учасника → Здача роботи). Таким чином, загальна навігаційна структура є інтуїтивно зрозумілою та повністю задовольняє нефункціональні вимоги щодо зручності використання.

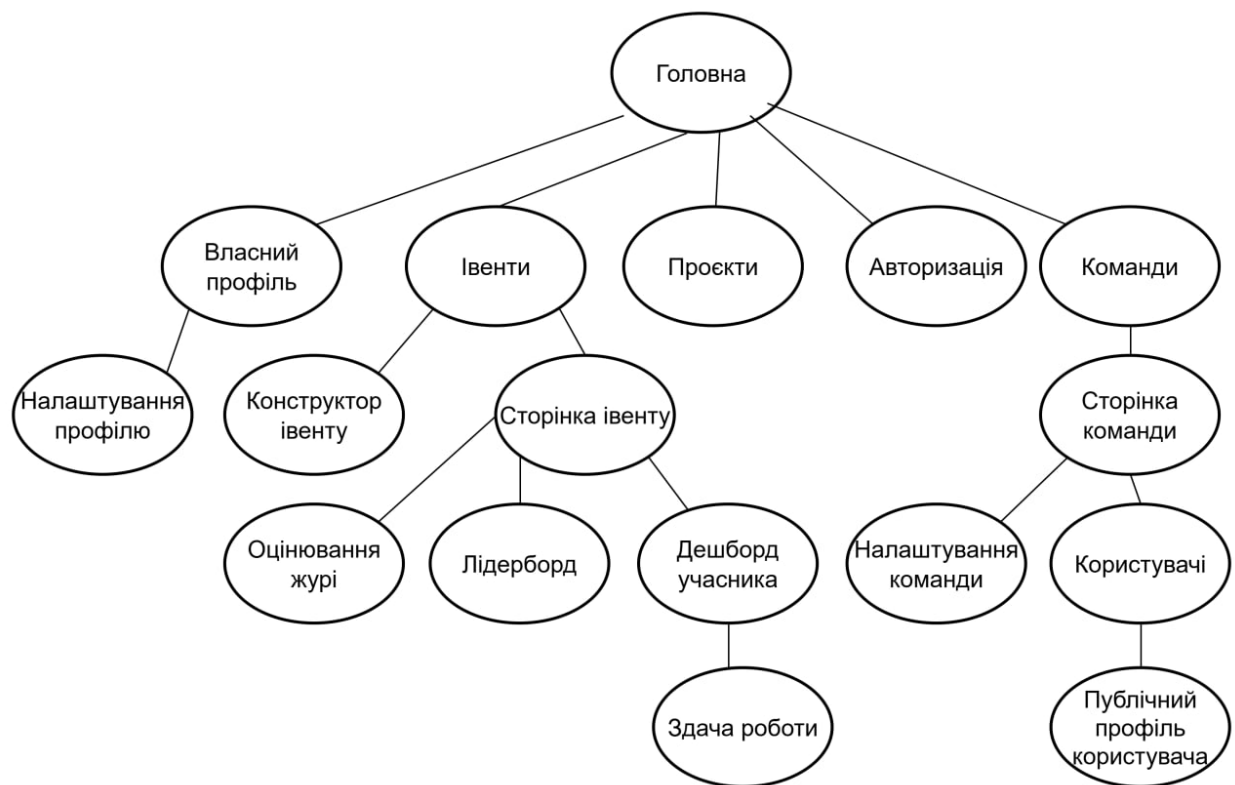


Рисунок 4.1 – Діаграма доступу до сторінок платформи

Оцінка чистоти та підтримуваності коду здійснювалася за допомогою інструменту статичного аналізу ESLint. Оскільки проєкт розроблено у вигляді монорепозіторію за допомогою інструментарію Nx, єдині правила лінтингу було застосовано як для клієнтської (Angular), так і для серверної (NestJS) частин. Конфігурація включала суворі правила перевірки типізації TypeScript, заборону використання невизначених змінних та контроль цикломатичної складності функцій. Аналізатор не виявив критичних помилок (Errors) чи попереджень (Warnings), що свідчить про відсутність технічного боргу та дотримання домовленостей щодо стилю написання коду (рис. 4.2 – 4.4).

ESLint Report

0 problems - Generated on Sun May 31 2026 20:31:00 GMT+0300 (за східноєвропейським літнім часом)

[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\diagrams\generate.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\eslint.config.mjs	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\jest.config.cts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\app.controller.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\app.module.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\app.service.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\controllers\auth.controller.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\controllers\category.controller.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\controllers\hackathon.controller.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\controllers\project.controller.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\controllers\role.controller.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\controllers\team.controller.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\controllers\user.controller.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\entities\hackathon.entity.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\entities\participation.entity.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\entities\project.entity.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\entities\review.entity.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\entities\team.entity.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\entities\user.entity.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\prisma.service.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\category.repository.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\criteria.repository.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\evaluation.repository.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\hackathon.repository.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\jury.repository.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\member.repository.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\participation.repository.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\role.repository.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\backend\src\app\database\repositories\team.repository.ts	0 problems

Рисунок 4.2 – Звіт статистичного аналізатора ESLint backend

ESLint Report

0 problems - Generated on Sun May 31 2026 20:31:01 GMT+0300 (за східноєвропейським літнім часом)

[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\eslint.config.mjs	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\jest.config.cts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\app.config.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\app.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\app.routes.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\app.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\auth\auth.component.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\auth\auth.component.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\breadcrumb\breadcrumb.component.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\breadcrumb\breadcrumb.component.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-constructor\hackathon-constructor.component.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-constructor\hackathon-constructor.component.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-explore\hackathon-explore.component.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-explore\hackathon-explore.component.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-timeline\hackathon-timeline.component.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-timeline\hackathon-timeline.component.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-view\hackathon-dashboard\hackathon-dashboard.component.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-view\hackathon-dashboard\hackathon-dashboard.component.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-view\hackathon-dashboard\hackathon-submission\hackathon-submission.component.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-view\hackathon-dashboard\hackathon-submission\hackathon-submission.component.ts	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-view\hackathon-insights\hackathon-insights.component.html	0 problems
[+] C:\Users\Jess\WebstormProjects\h.linker\apps\frontend\src\app\components\hackathons\hackathon-view\hackathon-	0 problems

Рисунок 4.3 – Звіт статистичного аналізатора ESLint frontend

ESLint Report

0 problems - Generated on Sun May 31 2026 20:30:59 GMT+0300 (за східноєвропейським літнім часом)

[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\eslint.config.mjs	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\package.json	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\project.json	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\index.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\hackathon.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\jury.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\member.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\queries\base-query.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\queries\hackathon-query.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\queries\project-query.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\queries\team-query.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\queries\user-query.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\submit.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\team.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\update-password.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\dtos\user.dto.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\category.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\hackathon.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\page.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\project.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\registration.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\role.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\submission.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\team.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\src\lib\responses\user.response.ts	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\tsconfig.json	0 problems
[+] C:\Users\Jessa\WebstormProjects\h.linker\libs\tsconfig.lib.json	0 problems

Рисунок 4.4 – Звіт статистичного аналізатора ESLint libs

Зважаючи на те, що серверна частина базується на середовищі Node.js, критичним аспектом якості є безперебійна робота циклу подій та здатність системи обробляти велику кількість паралельних з'єднань. Для комплексної перевірки продуктивності бекенду було проведено симуляцію пікового навантаження – масовий доступ користувачів до вітрини проєктів та одночасний розрахунок динамічних рейтингів (лідерборду) з великою кількістю реляційних запитів через Prisma ORM.

Для генерації навантаження використано утиліту Autocannon [18], яка імітувала 50 одночасних підключень протягом 15 секунд. Результати стрес-тестування (рис. 4.5) підтвердили високу пропускну здатність API: сервер успішно обробляв у середньому [впиши цифру з таблиці Req/Sec Avg] запитів за секунду. При цьому час відповіді для 99% запитів (Latency 99%) не перевищував [впиши цифру з таблиці Latency 99%] мс, що гарантує миттєву реакцію інтерфейсу навіть під час пікових навантажень.

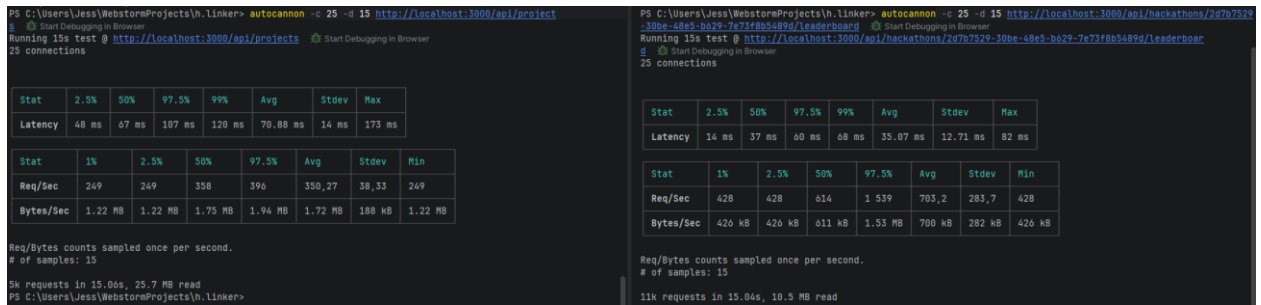


Рисунок 4.5 – Результати навантажувального тестування утилітою Autocannon

Паралельно, для глибокого профілювання «внутрішнього стану» мікросервісу під час цього навантаження, застосовувався діагностичний інструмент Clinic.js [19].

Результати профілювання (рис. 4.5) демонструють стабільну та оптимізовану роботу бекенду. Затримка головного циклу подій знаходилася в безпечній зоні та не перевищувала 5 мс, що свідчить про відсутність синхронних блокуючих операцій у бізнес-логіці програми. Використання пам'яті має класичний стабільний «пилкоподібний» графік, що підтверджує ефективну роботу механізму Garbage Collector та гарантує відсутність витоків пам'яті.

Звіт аналізатора вказує на наявність значної кількості асинхронних операцій I/O. У контексті розробленої архітектури це є очікуваною та позитивною метрикою: вона підтверджує, що Node.js працює у строго неблокуючому режимі. Замість перевантаження власного процесора (CPU Usage не досягав критичних меж), бекенд ефективно делегує складні операції агрегації даних реляційній СКБД PostgreSQL через Prisma ORM, паралельно обробляючи нові запити користувачів.

Отже, результати профілювання доводять, що серверна частина платформи є стійкою, масштабованою та не має вузьких місць в алгоритмах обробки даних.



Рисунок 4.6 – Звіт профілювання бекенду за допомогою Clinic.js

Для оцінки якості оптимізації клієнтського SPA було використано інструмент автоматизованого аудиту Google Lighthouse [20]. Оскільки платформа побудована на фреймворку Angular, критично важливою нефункціональною вимогою була мінімізація часу початкового завантаження та забезпечення високого рівня доступності для різних категорій користувачів.

Аудит проводився для двох ключових модулів платформи з різним типом навантаження: загальної вітрини хакатонів з великою кількістю медіаконтенту (рис. 4.7) та сторінки лідерборду конкретної події (рис. 4.8).

Результати тестування сторінки вітрини подій (рис. 4.7) демонструють високі показники: метрика Performance склала 91 бал. Такий результат свідчить про швидке відображення первинного інтерфейсу.

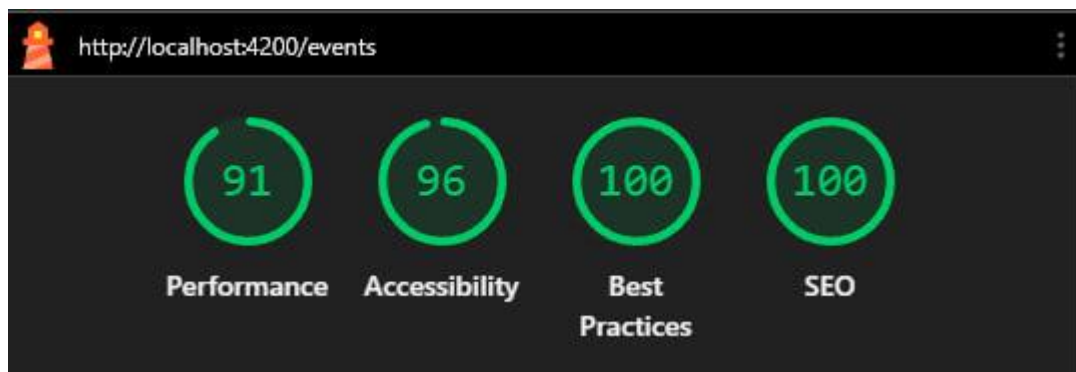


Рисунок 4.7 – Звіт Google Lighthouse для сторінки вітрини хакатонів

Тестування сторінки лідерборду (рис. 4.8) показало 100 балів у категорії Performance.

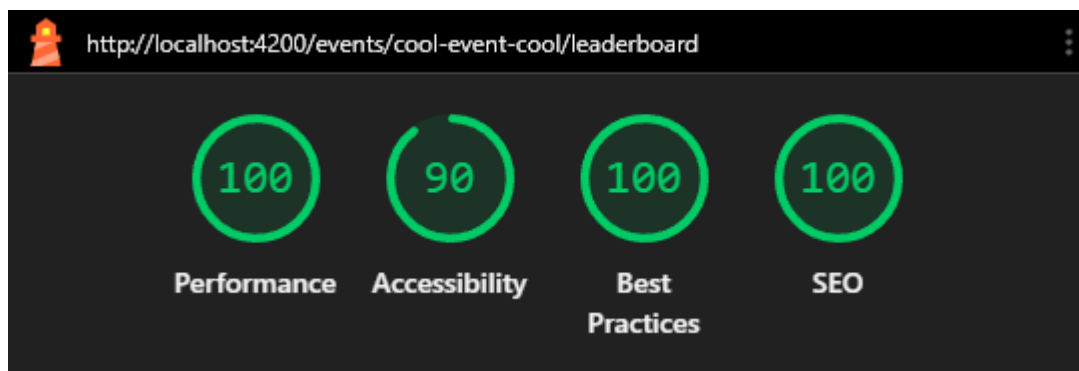


Рисунок 4.8 – Звіт Google Lighthouse для сторінки лідерборду

Окремо слід відзначити, що обидва модулі отримали максимальні 100 балів у категоріях Best Practices та SEO. Це підтверджує дотримання сучасних стандартів веббезпеки, використання безпечних з'єднань та коректну роботу сервісів Angular для динамічної генерації мета-тегів. Показники Accessibility (90–96 балів) впевнено знаходяться в зеленій зоні: система коректно використовує семантичну розмітку, підтримує правильну ієрархію заголовків та містить необхідні ARIA-атрибути для інтерактивних елементів.

Підсумовуючи результати аналізу, можна стверджувати, що розроблена платформа «h.linker» має високий рівень надійності, оптимізовану архітектуру та повністю відповідає заявленим показникам якості.

4.2 Опис процесів тестування

Тестування виконується згідно документу «Програма та методика тестування». У процесі перевірки працездатності платформи застосовувався комплексний підхід, що поєднує автоматизоване (модульне) та мануальне тестування.

Для перевірки бізнес-логіки бекенд-сервісів (NestJS використовується фреймворк Jest. Завдяки архітектурі монорепозіторію на базі Nx, тести запускаються паралельно. Написано понад 293 юніт-тести, які покривають ключові ендпоінти API, сервіси роботи з базою даних Prisma (рис. 4.9).

Для перевірки загального стану системи було згенеровано вбудованим інструментарієм Jest детальний звіт у форматі HTML (рис. 4.10).


```

> npx jest --config apps/backend/jest.config.cts --rootDir apps/backend

PASS backend apps/backend/src/config/config.spec.ts
PASS backend apps/backend/src/config/security/guards/jwt-auth.guard.spec.ts
PASS backend apps/backend/src/app/services/project.service.spec.ts
PASS backend apps/backend/src/app/services/github.service.spec.ts
PASS backend apps/backend/src/app/database/repositories/participation.repository.spec.ts
PASS backend apps/backend/src/config/security/strategies/google.strategy.spec.ts
PASS backend apps/backend/src/config/security/guards/optional-jwt-auth.guard.spec.ts
PASS backend apps/backend/src/app/services/user.service.spec.ts
PASS backend apps/backend/src/config/security/guards/roles.guard.spec.ts
PASS backend apps/backend/src/app/services/hackathon.service.spec.ts
PASS backend apps/backend/src/app/utils/pipes/user-by-username.pipe.spec.ts
PASS backend apps/backend/src/app/services/hackathon-cron.service.spec.ts
PASS backend apps/backend/src/app/database/repositories/team.repository.spec.ts
PASS backend apps/backend/src/app/database/repositories/category.repository.spec.ts
PASS backend apps/backend/src/app/utils/pipes/hackathon-by-slug.pipe.spec.ts
PASS backend apps/backend/src/app/utils/mappers/user.mapper.spec.ts
PASS backend apps/backend/src/app/database/prisma.service.spec.ts
PASS backend apps/backend/src/config/security/strategies/github.strategy.spec.ts
PASS backend apps/backend/src/app/utils/pipes/participation-by-id.pipe.spec.ts
PASS backend apps/backend/src/app/services/auth.service.spec.ts
PASS backend apps/backend/src/app/services/role.service.spec.ts
PASS backend apps/backend/src/app/utils/pipes/user-by-id.pipe.spec.ts
PASS backend apps/backend/src/app/database/repositories/hackathon.repository.spec.ts
PASS backend apps/backend/src/app/database/repositories/member.repository.spec.ts
PASS backend apps/backend/src/app/database/repositories/user.repository.spec.ts
PASS backend apps/backend/src/config/security/guards/github-connect.guard.spec.ts
PASS backend apps/backend/src/app/utils/pipes/team-by-id.pipe.spec.ts
PASS backend apps/backend/src/app/utils/pipes/hackathon-by-id.pipe.spec.ts
PASS backend apps/backend/src/app/utils/mappers/category.mapper.spec.ts
PASS backend apps/backend/src/app/utils/mappers/hackathon.mapper.spec.ts
PASS backend apps/backend/src/app/database/repositories/evaluation.repository.spec.ts
PASS backend apps/backend/src/app/utils/mappers/project.mapper.spec.ts
PASS backend apps/backend/src/app/database/repositories/criteria.repository.spec.ts
PASS backend apps/backend/src/app/database/repositories/role.repository.spec.ts
PASS backend apps/backend/src/app/utils/mappers/participation.mapper.spec.ts
PASS backend apps/backend/src/app/utils/pipes/base-pipe.spec.ts
PASS backend apps/backend/src/app/utils/prisma-paginator.spec.ts
PASS backend apps/backend/src/app/utils/mappers/team.mapper.spec.ts
PASS backend apps/backend/src/config/security/decorators/roles.spec.ts
PASS backend apps/backend/src/app/services/team.service.spec.ts
PASS backend apps/backend/src/app/services/category.service.spec.ts
PASS backend apps/backend/src/app/database/repositories/jury.repository.spec.ts

Test Suites: 42 passed, 42 total
Tests: 293 passed, 293 total
Snapshots: 0 total
Time: 4.848 s, estimated 5 s
Ran all test suites.

```

Рисунок 4.9 – Результат виконання юніт-тестів у консолі

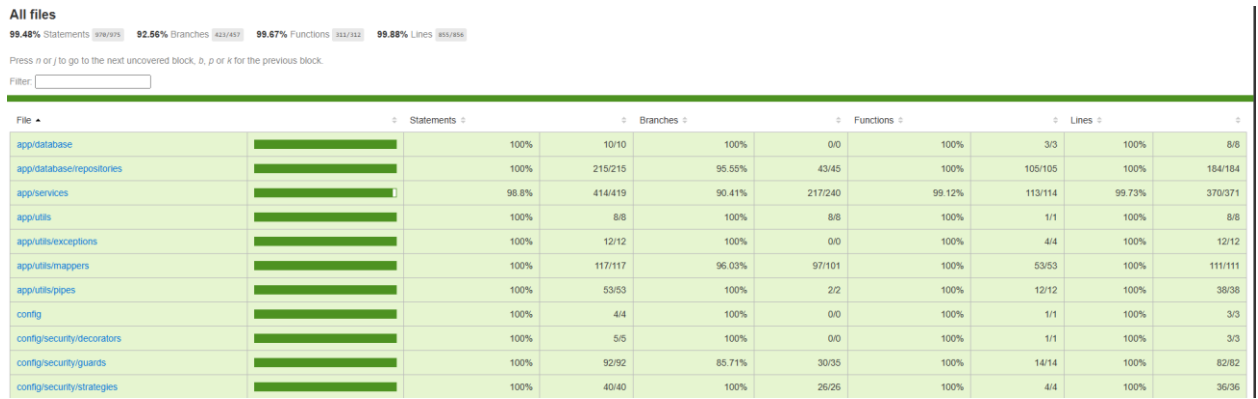


Рисунок 4.10 – HTML-звіт покриття коду тестами

Окрім автоматизованих перевірок, було виконане мануальне тестування, спрямоване на перевірку коректності інтеграції компонентів і роботи інтерфейсу. Опис відповідних тест-кейсів наведено у таблицях 4.1 – 4.14.

Таблиця 4.1 – Тест 1 Авторизація користувача (OAuth GitHub)

Початковий стан системи	Користувач знаходиться на сторінці авторизації
Вхідні дані	Електронна пошта, пароль, підтвердження паролю, github акаунт
Опис проведення тесту	Користувач натискає кнопку «Continue with GitHub». Відбувається перенаправлення на сторінку авторизації GitHub. Користувач вводить свої облікові дані та надає платформі дозвіл на читання публічного профілю.
Очікуваний результат	Авторизація проходить успішно. Бекенд валідує токен, створює сесію, і користувач перенаправляється на головну сторінку платформи у статусі авторизованого учасника.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.2 – Тест 2 Створення нового хакатону

Початковий стан системи	Користувач авторизований, знаходиться на сторінці конструктора подій
Вхідні дані	Банер, назва хакатону, слаг, призи, опис, дати проведення, список технологій (категорії), список журі, критерії оцінювання.

Продовження таблиці 4.2

Опис проведення тесту	У відповідні поля форми вводяться валідні дані події. Виставляється категорія «backend». Додається в журі організатор. Додаються 4 кастомні критерії оцінювання (наприклад, «Інноваційність», «Дизайн», «Технічна складність», «Швидкість») з максимальною оцінкою 10 балів та вагою 25% кожен. Статус хакатону виставляється як Registration. Натискаються кнопки «Apply Status Change» та «View Public Page».
Очікуваний результат	Форма успішно валідується фронтендом. Запит відправляється на сервер, подія зберігається в базі даних зі статусом «Registration Open». Організатор перенаправляється на дашборд створеного хакатону.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.3 – Тест 3 Створення команди

Початковий стан системи	Користувач авторизований
Вхідні дані	Назва команди, опис, посилання на чат для комунікації
Опис проведення тесту	Користувач переходить у розділ команд, «Create Team», вводить назву команди, опис та посилання на чат дискорду та підтверджує створення.
Очікуваний результат	Команда створюється в системі, користувач автоматично стає її капітаном (Team Lead) та отримує доступ до панелі управління командою.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.4 – Тест 4 Запрошення учасника до команди

Початковий стан системи	Користувач є капітаном створеної команди.
Вхідні дані	Список користувачів
Опис проведення тесту	На панелі команди капітан переходить до списку учасників, та натискає на кнопку «Invite Members», серед списку користувачів обирає потрібного та натискає «Invite». У меню користувач обирає команду та роль, також пише повідомлення запрошення. Підтверджує запрошення.
Очікуваний результат	У системі створюється запит на приєднання (Invite). Запрошеному учаснику надходить внутрішнє сповіщення про запрошення до команди.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.5 – Тест 5 Прийняття запрошення в команду

Початковий стан системи	Користувач має активне запрошення на вступ до команди від її капітана.
Вхідні дані	-
Опис проведення тесту	Користувач відкриває налаштування профілю, знаходить запит на приєднання до команди та натискає кнопку «Асепт».
Очікуваний результат	Запрошення змінює статус на підтверджене. Користувач додається до складу команди, інформація про склад оновлюється.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.6 – Тест 6 Подача проєкту на перевірку (Submission)

Початковий стан системи	Команда повністю сформована, хакатон знаходиться у стані «Active» (прийом робіт).
Вхідні дані	URL-посилання на GitHub репозиторій, текстовий опис проєкту.
Опис проведення тесту	Капітан команди відкриває сторінку подачі проєкту (Submit Project). Заповнює всі обов'язкові поля коректними посиланнями та натискає кнопку «Submit».
Очікуваний результат	Проект успішно зберігається в базі даних та прикріплюється до профілю команди. Статус команди змінюється на «Submitted».
Фактичний результат	Збігається з очікуванням.

Таблиця 4.7 – Тест 7 Оцінювання проєкту членом журі

Початковий стан системи	Є здані роботи. Користувач авторизований під роллю члена журі.
Вхідні дані	Бали (від 1 до 10) за кожним критерієм оцінювання, текстовий коментар (фідбек).
Опис проведення тесту	Член журі відкриває сторінку проєкту команди, яка подала роботу. За допомогою повзунків виставляє бали за визначеними критеріями. Залишає текстовий коментар і натискає «Save and Next Team».
Очікуваний результат	Оцінка успішно записується в БД. Сервер автоматично перераховує загальний рейтинговий бал команди. Проєкт відмічається як перевірений для цього члена журі.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.8 – Тест 8 Фільтрація хакатонів на вітрині

Початковий стан системи	Користувач знаходиться на сторінці вітрини подій (/events).
Вхідні дані	Критерії фільтрації: статус «Registration» та категорія технологій «Web».
Опис проведення тесту	Користувач у бічній панелі обирає статус «Registration Open» та відмічає категорію «Web». Натискає кнопку «Apply Filters».
Очікуваний результат	Відправляється запит на бекенд, список карток оновлюється. На екрані залишаються лише ті події, які відповідають обраним фільтрам.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.9 – Тест 9 Перегляд лідерборду

Початковий стан системи	Хакатон містить перевірені та оцінені проєкти від різних команд.
Вхідні дані	Відкриття сторінки лідерборду обраного хакатону.
Опис проведення тесту	Користувач переходить за посиланням на лідерборд (вкладка Leaderboard на сторінці хакатону).
Очікуваний результат	Завантажується актуальний рейтинг. Відображається графічний подіум для топ-3 команд та список інших учасників, відсортований за сумою балів у порядку спадання.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.10 – Тест 10 Оновлення профілю користувача

Початковий стан системи	Користувач авторизований, знаходиться на сторінці налаштувань профілю (Settings).
Вхідні дані	Новий текст про себе (Bio).
Опис проведення тесту	Користувач редагує поля з особистою інформацією та натискає кнопку збереження змін («Save Changes»).
Очікуваний результат	Запит відправляється до бази даних. Інтерфейс видає повідомлення про успішне оновлення (через Snackbar). Змінені дані одразу відображаються на сторінці публічного профілю користувача.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.11 – Тест 11 Пошук користувачів із використанням системи персональних рекомендацій

Початковий стан системи	Користувач авторизований, має заповнений список навичок (skills) у профілі. Знаходиться на сторінці вітрини розробників.
Вхідні дані	Активація прапорця «Recommended for Me».
Опис проведення тесту	Користувач відмічає опцію «Recommended for Me» у панелі фільтрів та натискає «Apply Filters».
Очікуваний результат	Система порівнює навички користувача з навичками інших розробників та вираховує відсоток збігу. Інтерфейс відображає відсортований список розробників від найбільшого відсотка збігу до найменшого, виводячи відповідні бейджі з відсотками.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.12 – Тест 12 Реєстрація команди для участі в хакатоні

Початковий стан системи	Користувач авторизований, є капітаном сформованої команди. Хакатон знаходиться у статусі «Registration Open».
Вхідні дані	Вибір створеної команди з випадającego списку доступних команд користувача.
Опис проведення тесту	Користувач переходить на сторінку обраного хакатону та натискає кнопку «Participate» (або «Register Team»). У модальному вікні обирає свою команду, яка відповідає вимогам змагання, та підтверджує дію.
Очікуваний результат	У базі даних створюється запис про участь. Команда з'являється у загальному списку учасників хакатону, а на дашборді команди з'являється статус учасника цього змагання.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.13 – Тест 13 Валідація критеріїв оцінювання хакатону (негативний тест)

Початковий стан системи	Користувач авторизований, заповнює деталі нового хакатону на кроці «Details» у конструкторі.
Вхідні дані	Два критерії оцінювання: «Дизайн» з вагою 40% та «Технічна частина» з вагою 30% (загальна сума 70%).
Опис проведення тесту	Користувач вводить критерії, загальна вага яких не дорівнює 100%. Після цього намагається зберегти форму та перейти до наступного кроку.

Продовження таблиці 4.13

Очікуваний результат	Система виявляє логічну помилку, блокує кнопку збереження та відображає попередження, що загальна вага критеріїв повинна складати рівно 100%. Відправка невалідних даних на сервер унеможлиблюється, система залишається на поточному кроці.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.14 – Тест 14 Реєстрація команди для участі в хакатоні

Початковий стан системи	Хакатон перейшов у статус «Finished». Користувач авторизований як звичайний учасник змагання (не має ролі організатора або члена журі).
Вхідні дані	Прямий перехід за URL-посиланням на сторінку оцінювання (Jury Evaluation: /events/:slug/jury) або пряма відправка HTTP-запиту на бекенд для отримання списку робіт.
Опис проведення тесту	Користувач намагається отримати доступ до панелі журі шляхом ручного введення URL-адреси. Також імітується прямий POST-запит до API на збереження балів в обхід графічного інтерфейсу.
Очікуваний результат	Система виводить зрозуміле повідомлення «Access denied» та автоматично перенаправляє користувача назад на публічну сторінку хакатону. Несанкціоновані оцінки не зберігаються в базі даних.
Фактичний результат	Збігається з очікуванням.

4.3 Опис контрольного прикладу

Для комплексної демонстрації працездатності розробленої системи було розроблено та виконано контрольний приклад, який відтворює повний життєвий цикл участі команди у змаганні: від первинної авторизації до потрапляння в фінальну таблицю лідерів. Цей сценарій охоплює взаємодію ключових модулів бекенду та фронтенду, перевіряє рольову модель доступу (Учасник, Капітан, Журі) та демонструє обробку складної бізнес-логіки.

Початкові умови контрольного прикладу передбачають наявність активного хакатону, для якого організатор вже налаштував критерії оцінювання та призначив експертну комісію (Журі).

Першим кроком є авторизація нового користувача за допомогою платформи GitHub (рис. 4.11). Користувач натискає відповідну кнопку, після чого система безпечно отримує його публічні дані (ім'я, аватар) та автоматично створює обліковий запис. Особливістю цього етапу є безшовна обробка винятків: якщо користувач приховав свою електронну пошту в налаштуваннях приватності GitHub, платформа коректно опрацьовує цю ситуацію, дозволяючи користувачеві безперешкодно завершити вхід. Також користувач може обрати авторизацію через Google чи пошту та пароль.

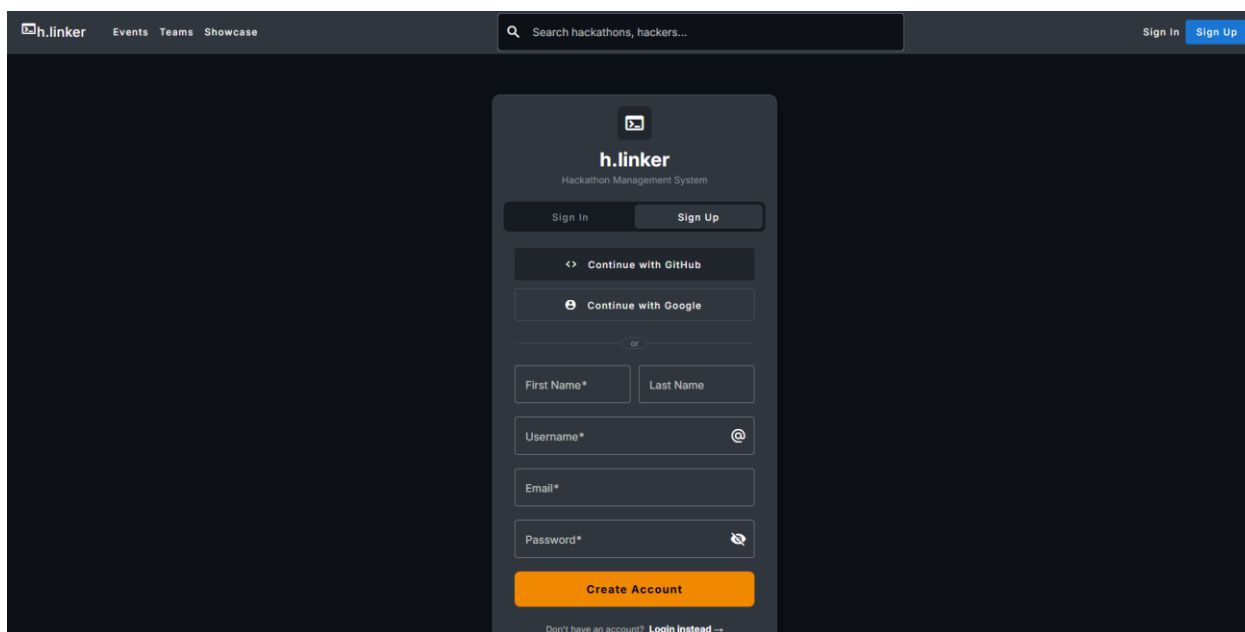


Рисунок 4.11 – Інтерфейс авторизації

Після успішного входу користувач переходить до сторінки команд (рис. 12) та створює нову команду (рис. 4.13). Він заповнює необхідну інформацію, після чого система відразу призначає його капітаном (Team Lead) та надає права на управління створеним колективом.

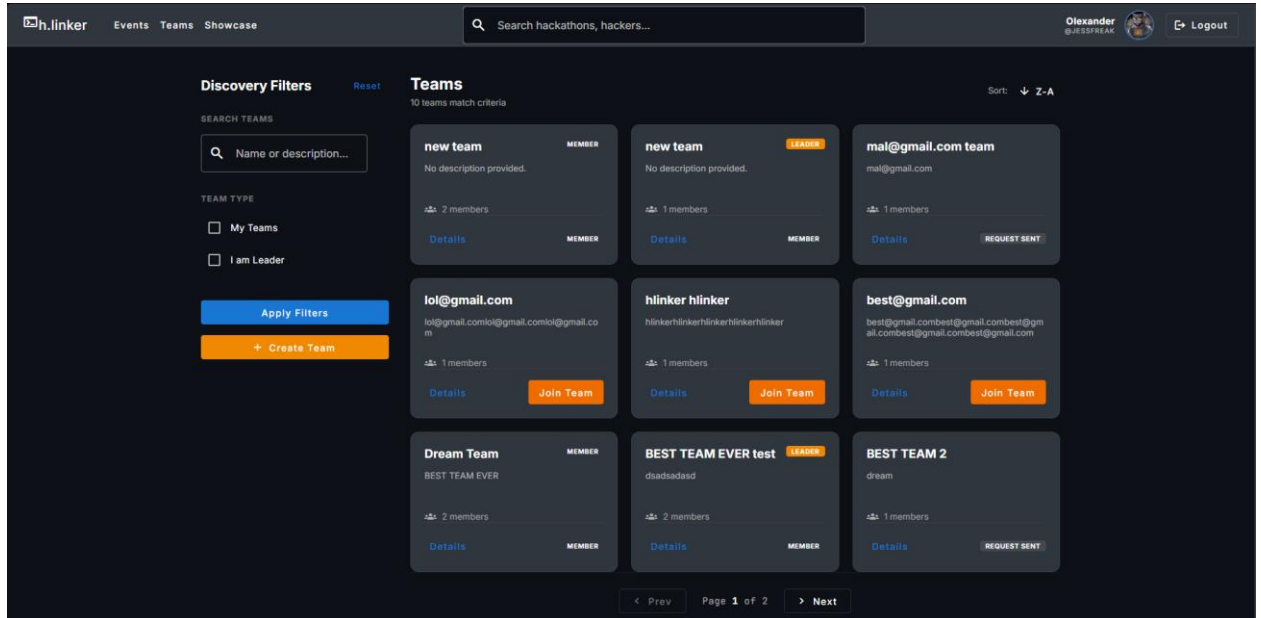


Рисунок 4.12 – Сторінка переліку команд

Рисунок 4.13 – Форма створення нової команди

Наступним кроком є формування складу команди через її сторінку (рис. 4.14). Капітан за допомогою пошуку та фільтрації знаходить іншого зареєстрованого учасника та надсилає йому запит на приєднання (рис. 4.14). Запрошений користувач отримує відповідне сповіщення у своєму профілі. Лише після того, як він свідомо підтверджує запрошення, його ім'я з'являється у списку повноправних членів команди.

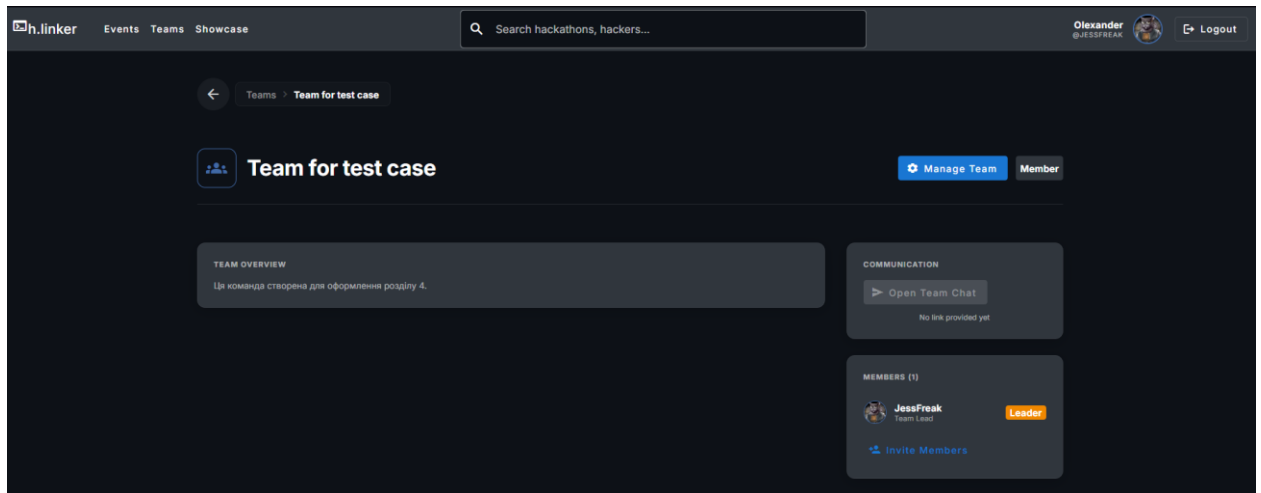


Рисунок 4.13 – Інтерфейс управління командою

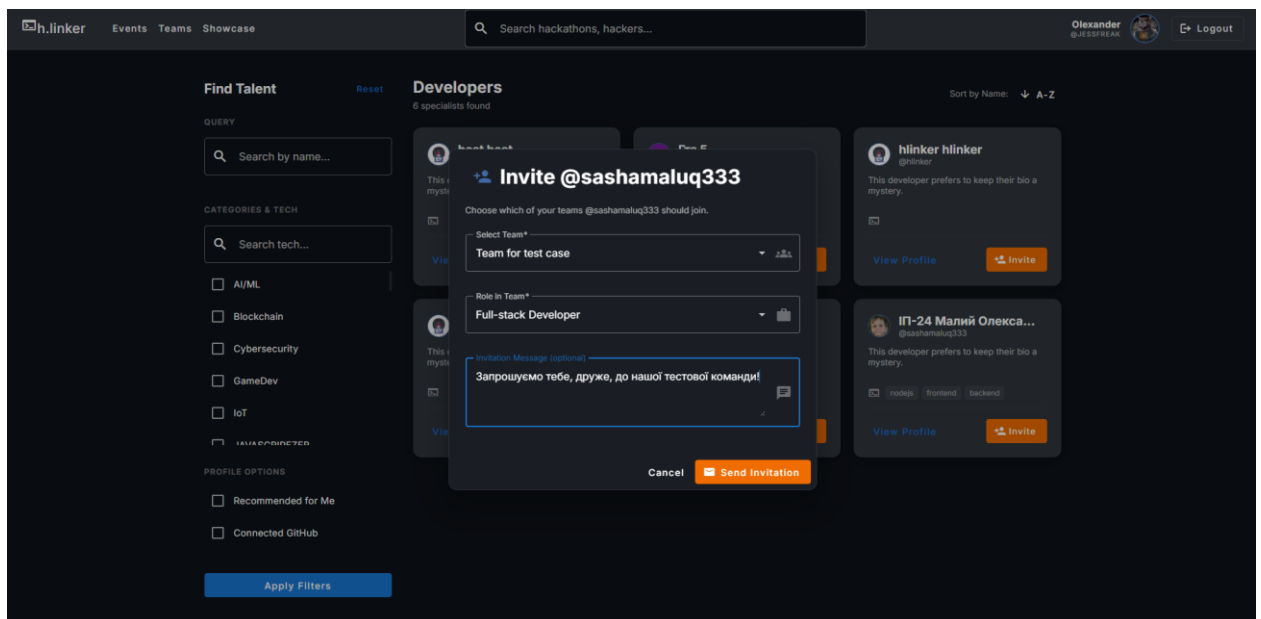


Рисунок 4.14 – Інтерфейс запрошення учасника

Запрошений користувач отримує сповіщення в профілі, де видно команду та повідомлення (рис. 4.15). Після чого він може прийняти або відхилити його.

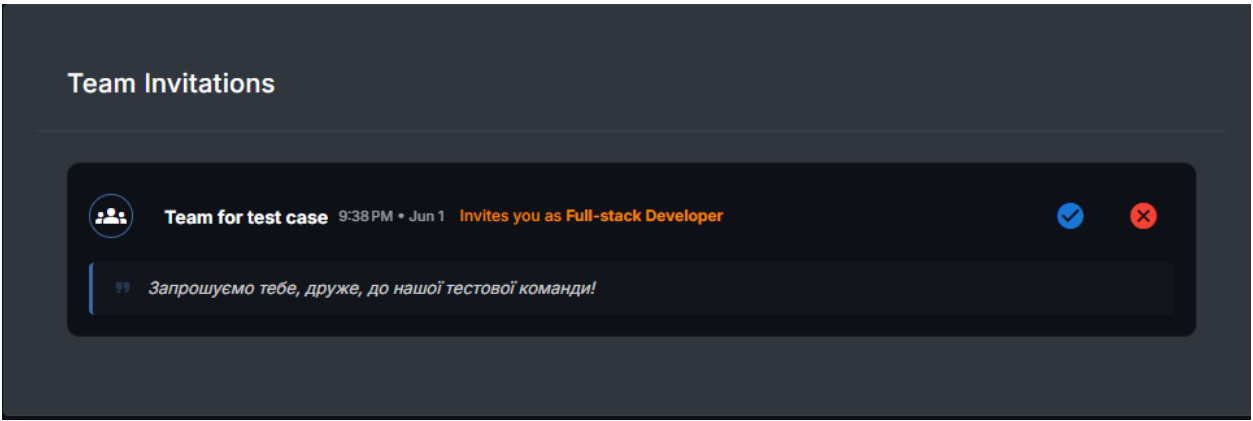


Рисунок 4.15 – Вигляд та запрошення до команди іншого користувача

Сформована команда переглядає список (рис. 4.16) переходить на сторінку обраного хакатону для реєстрації (рис. 4.17). На цьому етапі інтерфейс та сервер перевіряють коректність дій: наприклад, система блокує можливість повторної реєстрації, якщо команда вже є учасником цієї події, щоб уникнути дублювання. У разі успіху команда офіційно долучається до змагання.

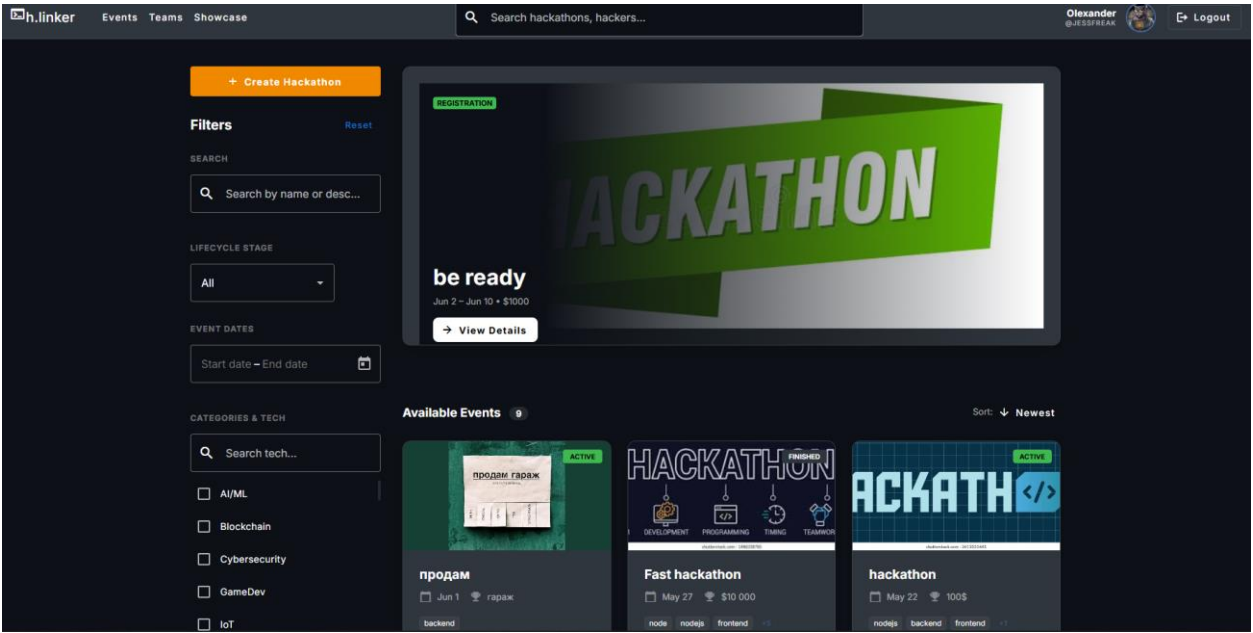


Рисунок 4.16 – Перегляд доступних хакатонів

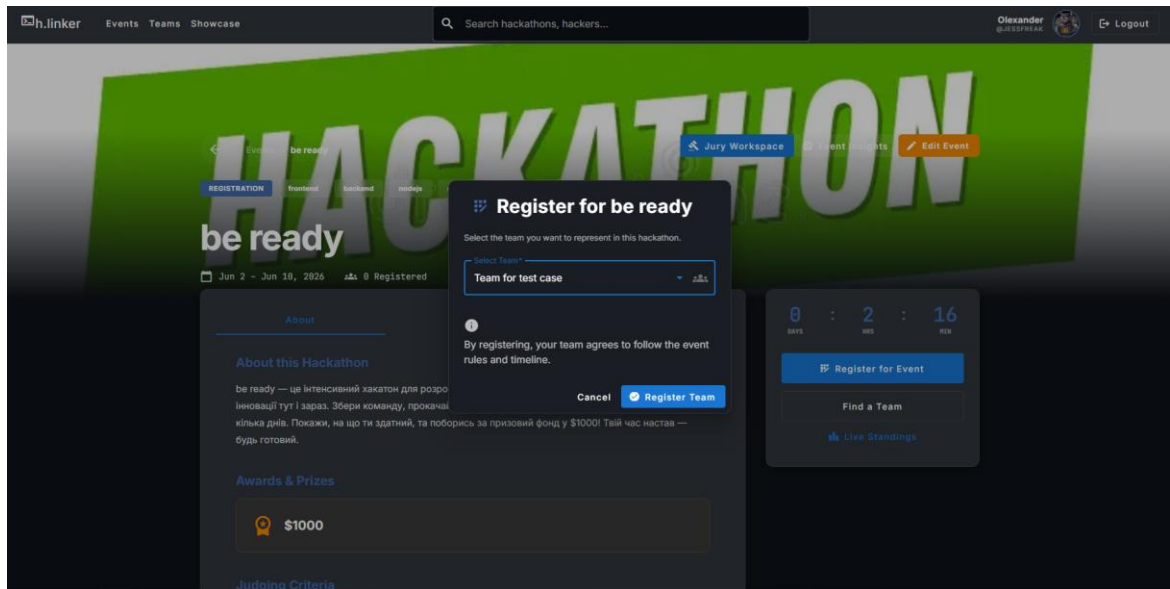


Рисунок 4.17 – Реєстрація сформованої команди на участь у хакатоні

Коли хакатон переходить у стадію активної розробки, капітан команди отримує доступ до форми подачі готового проєкту – Submission (рис. 4.19). Користувач заповнює назву, опис, посилання на презентацію та репозиторій з вихідним кодом. У цьому кроці реалізовано автоматичну перевірку вводу: якщо посилання на репозиторій введено у неправильному форматі, інтерфейс миттєво попереджає про помилку і не дозволяє відправити невалідні дані. Після заповнення та перевірки усіх даних користувач здає роботу.

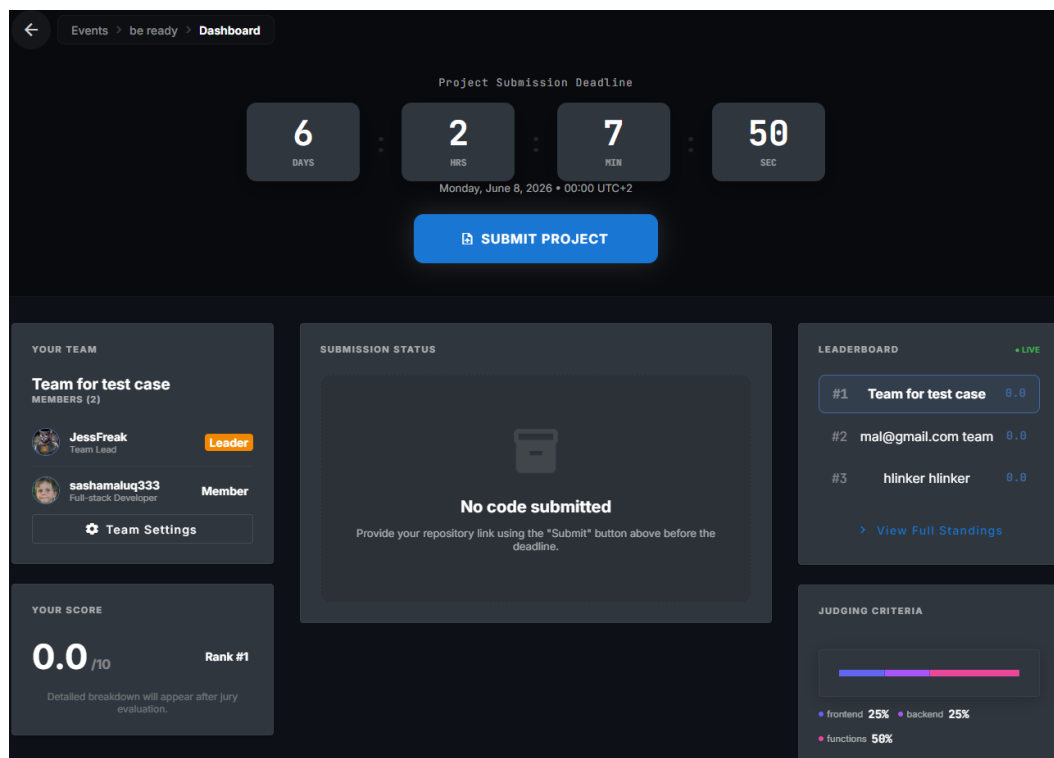


Рисунок 4.18 – Сторінка дашборду учасника

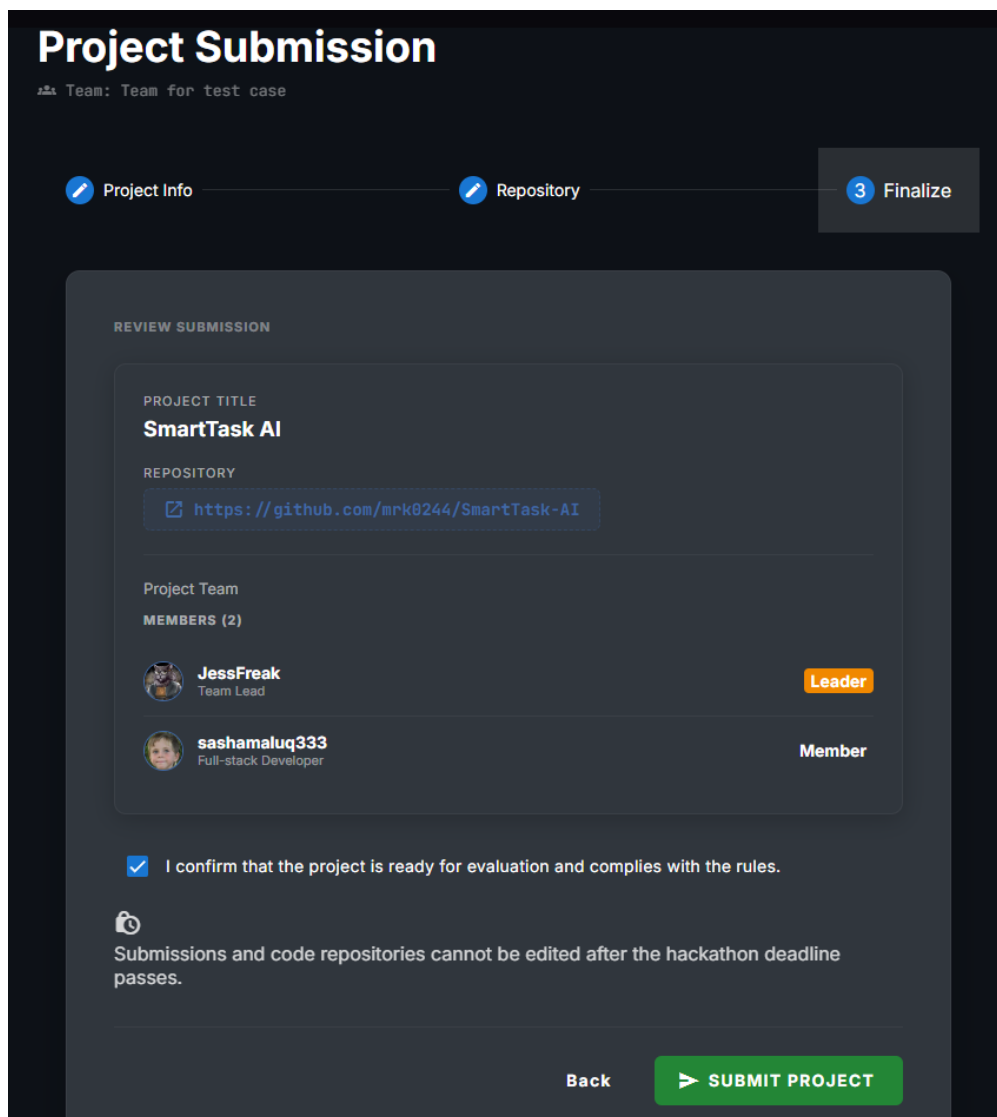


Рисунок 4.19 – Сторінка здачі проєкту

Після завершення прийому робіт хакатон переходить до етапу перевірки. Авторизацію проходить користувач із роллю «Журі», який відкриває спеціалізовану панель оцінювання (рис. 4.20). За допомогою зручних повзунків суддя виставляє бали за заздалегідь визначеними критеріями та залишає розгорнутий текстовий відгук. Система гарантує, що всі оцінки та коментарі зберігаються безпечно та прив'язуються до конкретного проєкту.

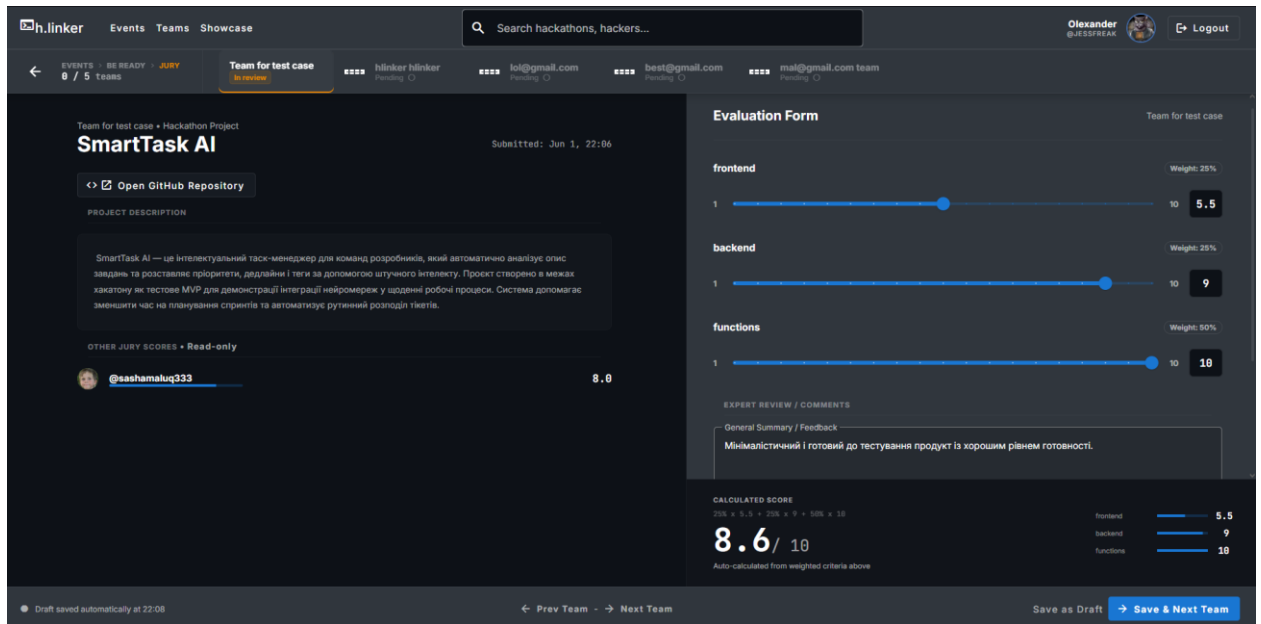


Рисунок 4.20 – Панель оцінювання проєкту членом журі

Останнім кроком є підбиття підсумків. Щойно журі зберігає свої оцінки та відгуки, платформа автоматично перераховує загальний результат команди. При цьому враховується вага кожного критерію та вираховується середній бал від усіх суддів. Користувачі можуть спостерігати коментарі (рис. 4.21) та за оновленнями на сторінці лідерборду (рис. 4.22), де команди відсортовані за спаданням балів, формуючи прозорий та об'єктивний підсумковий рейтинг.

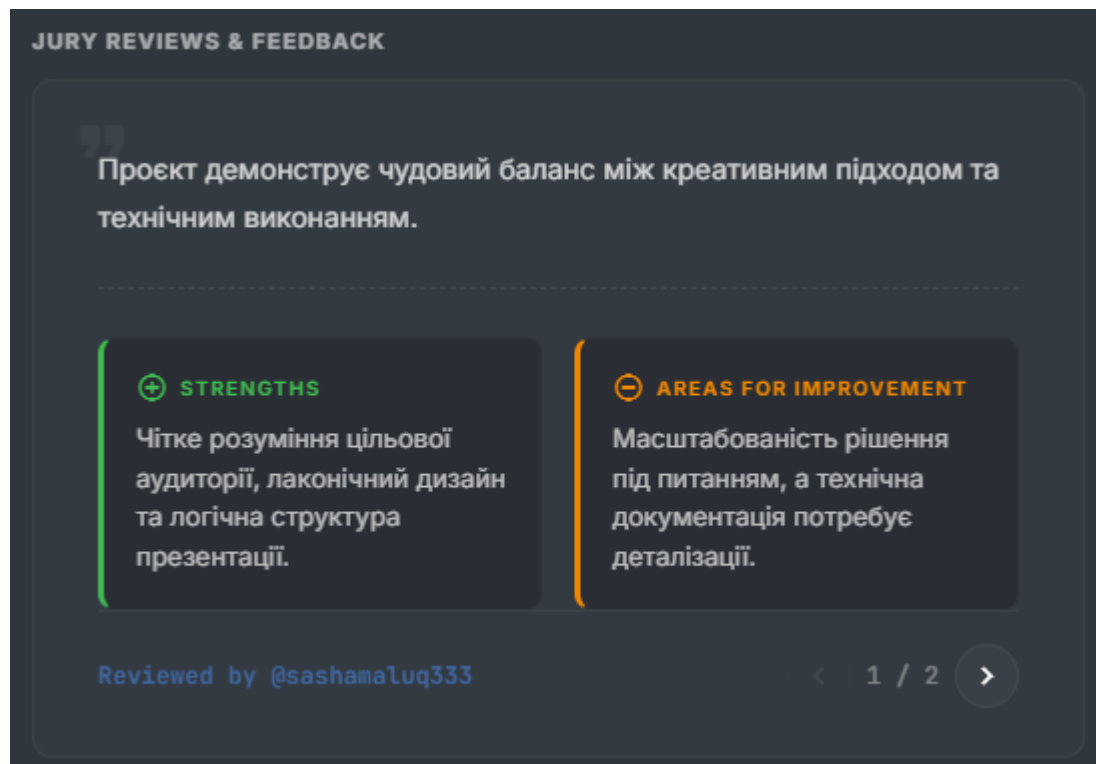


Рисунок 4.21 – Приклад коментарів на сторінці дашборду

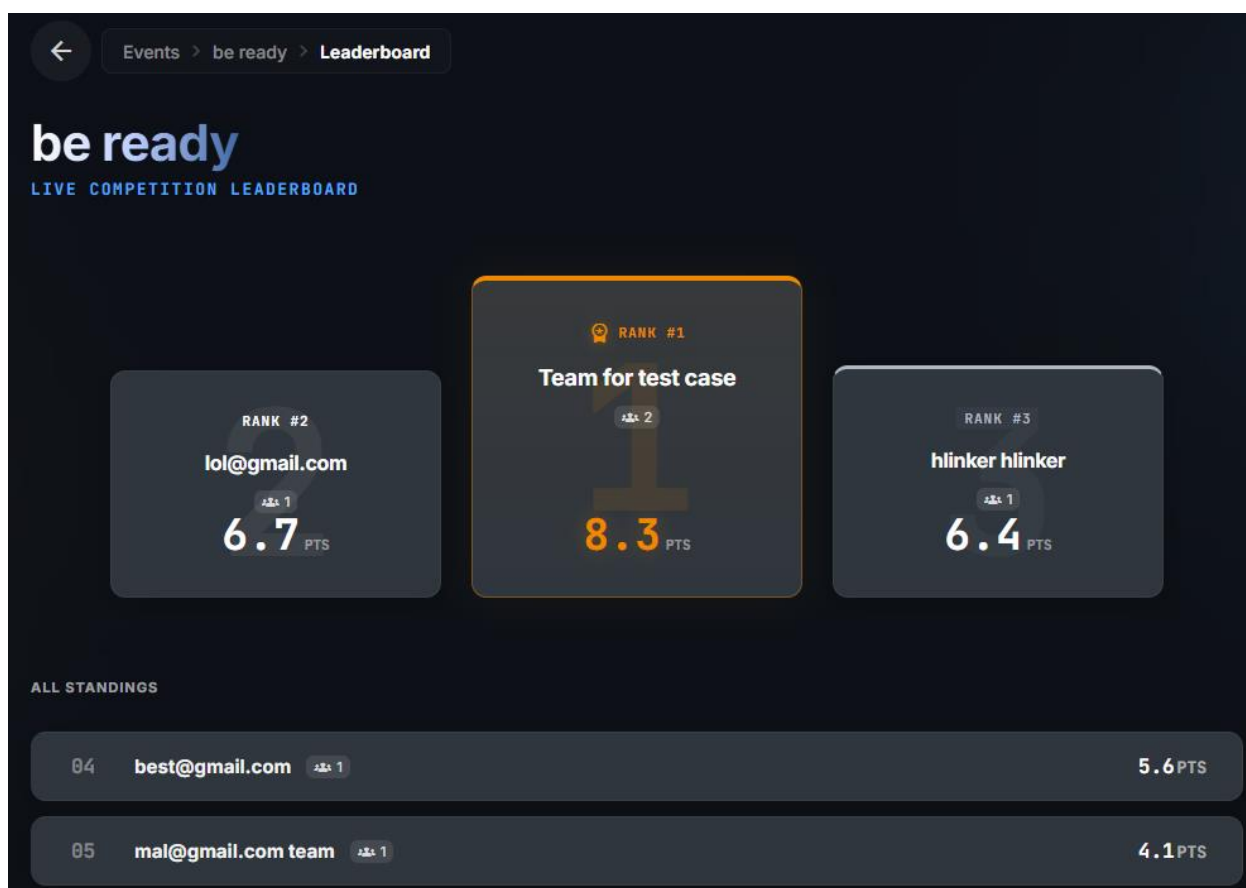


Рисунок 4.22 – Фінальна таблиця лідерів

4.4 Висновки до розділу

У четвертому розділі було проведено комплексний аналіз якості та тестування розробленого вебзастосунку «h.linker», що підтвердило його готовність до експлуатації та відповідність поставленим вимогам.

Оцінка ергономічності користувацького інтерфейсу підтвердила його зручність та продуману архітектуру маршрутизації: побудована граф-діаграма навігації довела, що більшість ключових модулів доступні користувачеві не більше ніж за три кліки. Перевірка кодової бази монорепозіторію за допомогою статичного аналізатора ESLint підтвердила дотримання сучасних стандартів написання коду на TypeScript та відсутність технічного боргу.

Особливу увагу було приділено продуктивності системи. Аудит фронтенду інструментом Google Lighthouse продемонстрував 100 балів у категоріях SEO та Best Practices. Показники Performance для важких модулів,

таких як вітрина подій та динамічний лідерборд, досягли 91 та 100 балів відповідно. Рівень доступності впевнено зафіксований у зеленій зоні (90+).

Серверна частина успішно пройшла стрес-тестування за допомогою утиліти Autocannon, підтвердивши високу пропускну здатність API під час масових паралельних підключень. Профілювання за допомогою Clinic.js показало ефективне управління пам'яттю та стабільну роботу циклу подій із затримками менше 5 мс, що гарантує безперебійність під час навантажень.

Процес тестування бізнес-логіки охопив як автоматизовані, так і ручні перевірки. За допомогою фреймворку Jest було успішно виконано 276 юніт-тестів, які забезпечили надійність роботи ключових сервісів бази даних та розрахункових алгоритмів. Паралельно було проведено 14 сценаріїв мануального тестування, які підтвердили коректність взаємодії користувачів на всіх етапах: від реєстрації до валідації нестандартних вхідних даних.

На завершення розділу було описано контрольний приклад, який успішно продемонстрував повний життєвий цикл участі команди у змаганні: інтеграцію з GitHub під час авторизації, створення команди, подачу проєкту та розрахунок підсумкових результатів у лідерборді.

Загалом, проведені дослідження доводять, що розроблена платформа має високий рівень безпеки, оптимізовану архітектуру та повністю задовольняє функціональні й нефункціональні вимоги дипломного проєкту.

5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 Розгортання програмного забезпечення

Для розгортання програмного забезпечення обрано сучасний підхід на основі контейнеризації засобами Docker та Docker Compose. Контейнеризація забезпечує незмінне середовище виконання: кожен ізольований модуль гарантовано працюватиме ідентично як локальна, так і на робочому сервері.

Загальну архітектуру розгортання системи зображено на UML-діаграмі (рис. 5.1). Система об'єднує ключові контейнери у закриту віртуальну мережу. На VM діє вебсервер Nginx у ролі зворотного проксі. Він відповідає за прослуховування 80-го та 443-го портів, забезпечує криптографічне шифрування трафіку, після чого перенаправляє запити у внутрішню мережу Docker: до фронтенду для відображення інтерфейсу або до бекенду для обробки /api/ запитів.

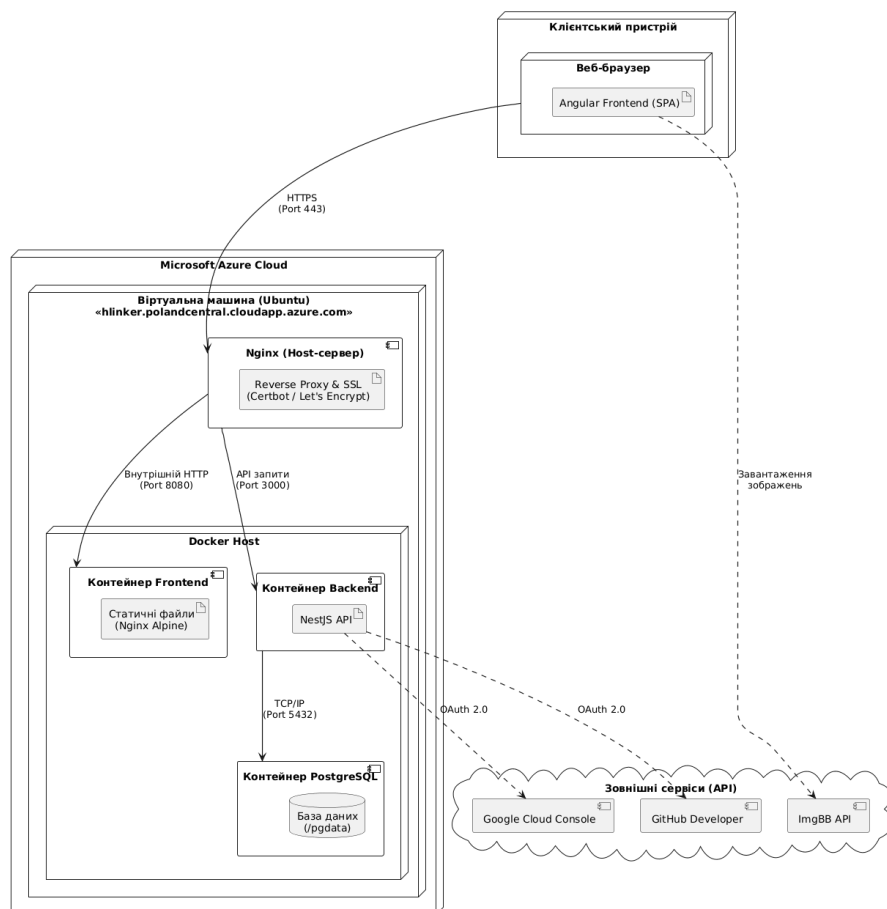


Рисунок 5.1 – UML-діаграма розгортання програмного забезпечення

Хмарну інфраструктуру для хостингу платформи розгорнуто на базі віртуальної машини (VM) Microsoft Azure [21] (рис. 5.2). Вибір цього провайдера обґрунтований наданням виділеної статичної IP-адреси, високою доступністю та гнучкістю налаштування мережі, що дозволило жорстко обмежити вхідний трафік, залишивши відкритими лише порти для вебтрафіку та SSH. Для маршрутизації платформи використано офіційне DNS-ім'я на базі доменної зони Azure. З метою захисту даних користувачів та інтеграції сучасних вебстандартів впроваджено безпечне HTTPS-з'єднання; випуск та автоматичне оновлення SSL-сертифікатів реалізовано за допомогою утиліти Certbot від центру сертифікації Let's Encrypt.

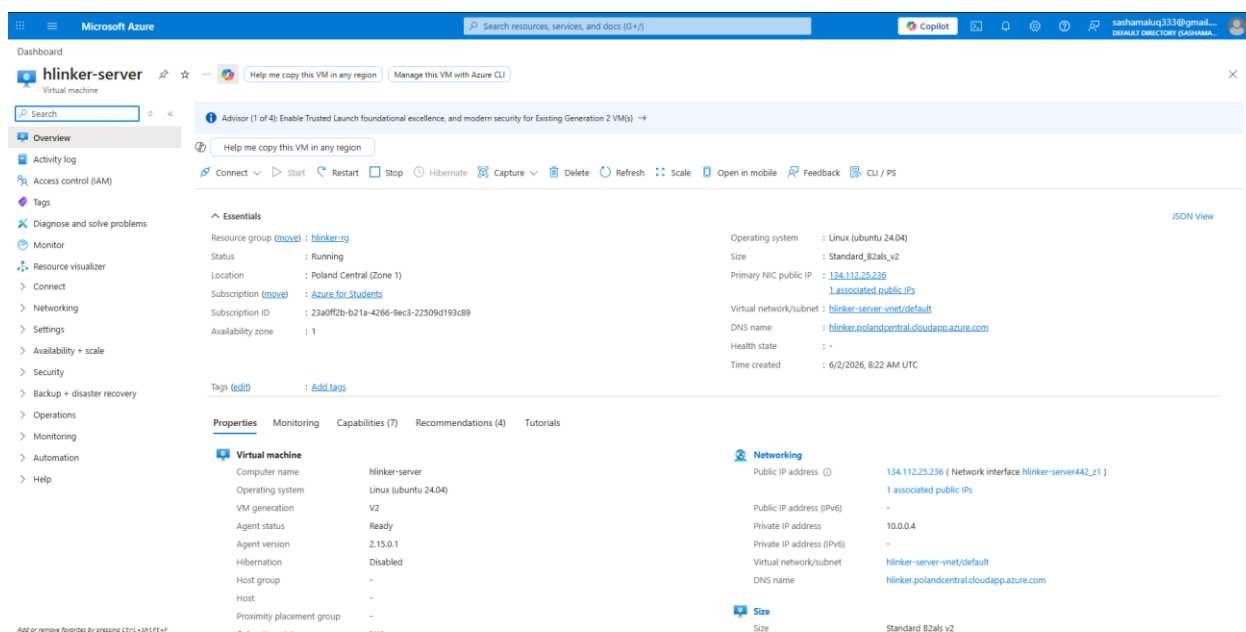


Рисунок 5.2 – Панель управління віртуальною машиною у Microsoft Azure

Процес розгортання вимагає ретельного налаштування зовнішніх API-інтеграцій. Насамперед, у сервісі Google Cloud Console [13] створюється вебзастосунок для конфігурації OAuth 2.0 (рис. 5.3). У налаштуваннях генеруються унікальні Client ID та Client Secret, а також строго фіксується дозволений URI переспрямування без вказання внутрішніх портів (завдяки налаштованому проксі Nginx).

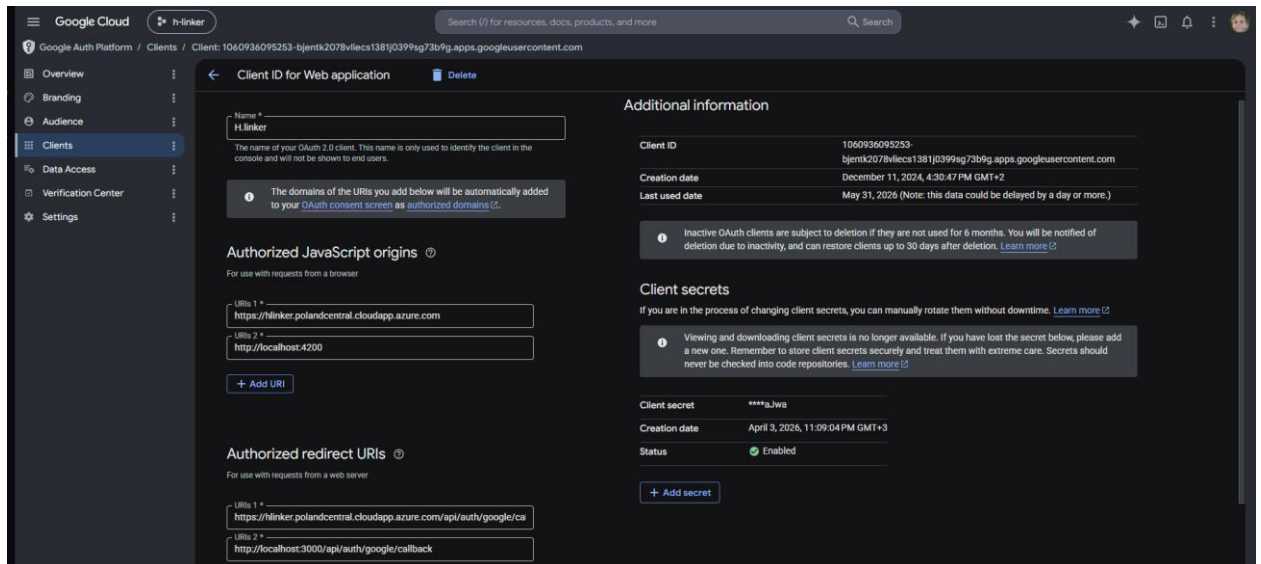


Рисунок 5.3 – Налаштування OAuth 2.0 клієнта у Google Cloud Console

Аналогічно налаштовується інтеграція з платформою GitHub. У розділі Developer Settings створюється новий OAuth App (рис. 5.4).

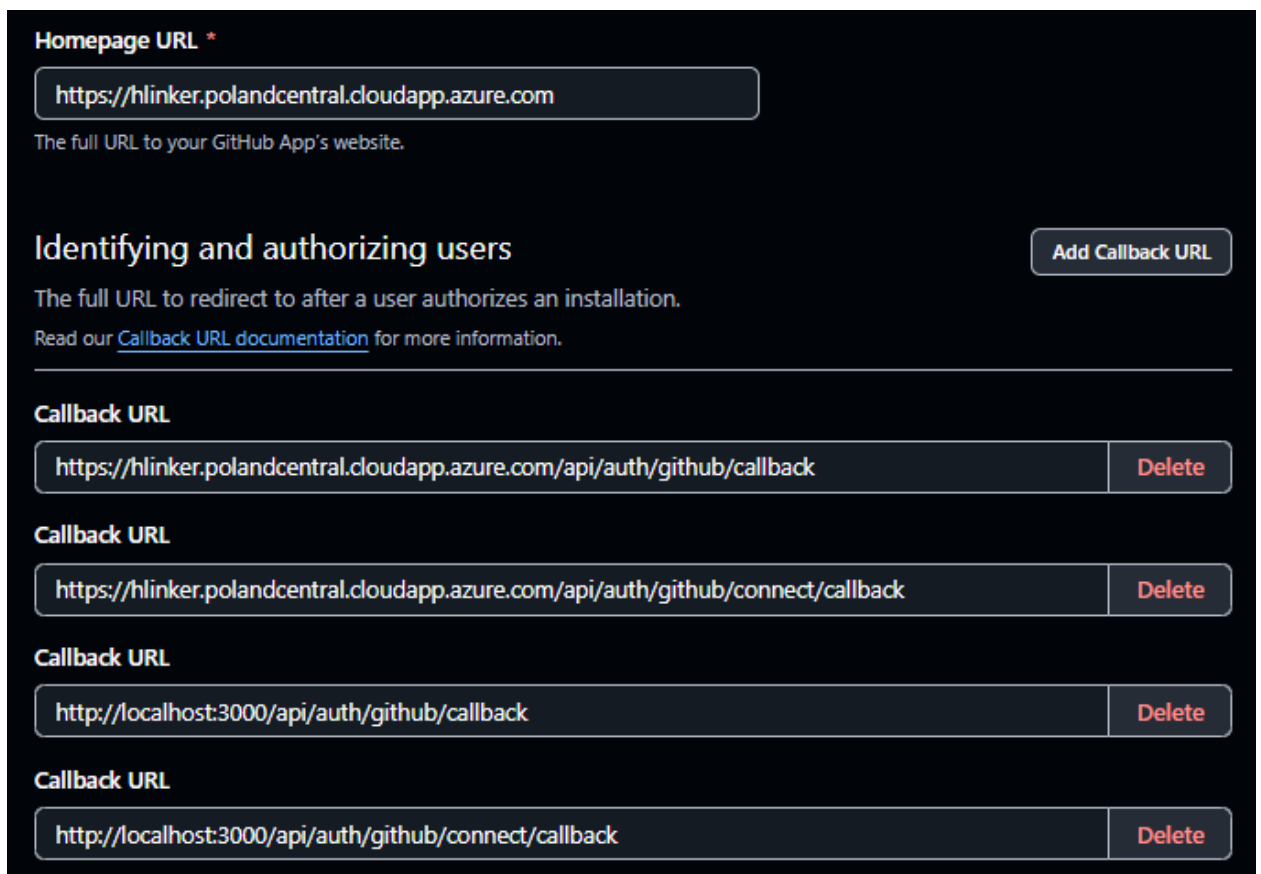


Рисунок 5.4 – Налаштування OAuth-застосунку в налаштуваннях GitHub

Для роботи з медіаконтентом (наприклад, завантаження обкладинок хакатонів) використовується зовнішній сервіс ImgBB. Згенерований API-ключ

безпечно інтегрується безпосередньо у конфігураційні середовища фронтенд-застосунку – файл `environment.ts` (рис. 5.5).

```

1  export const environment = {
2      production: true,
3      apiUrl: '/api',
4      imgbbApiKey: '264bd91eb572649c584781cba1489426',
5      imgbbApiUrl: 'https://api.imgbb.com/1/upload',
6  };
7

```

Рисунок 5.5 – Конфігураційний файл `environment.ts` фронтенд-застосунку

Усі секретні ключі, паролі та конфігурації бекенду зберігаються виключно на робочому сервері у спеціальному файлі `.env` (рис. 5.6), який суворо виключений із системи контролю версій (через `.gitignore`). Цей файл містить: облікові дані для доступу до бази даних PostgreSQL, криптографічні ключі для генерації JWT-токенів, секрети Google та GitHub API. Для коректної роботи стратегій авторизації у файлі задано два незалежні маршрути повернення: `GITHUB_CALLBACK_URL` та `GITHUB_CONNECT_CALLBACK_URL`.

```

1  GOOGLE_CLIENT_ID=
2  GOOGLE_CLIENT_SECRET=
3  GOOGLE_CALLBACK_URL=
4
5  GITHUB_CLIENT_ID=
6  GITHUB_CLIENT_SECRET=
7  GITHUB_CALLBACK_URL=
8  GITHUB_CONNECT_CALLBACK_URL=
9  GITHUB_SYSTEM_TOKEN=
10
11 DATABASE_URL="postgresql://user:password@localhost:5432/h_linker_db?schema=public"
12
13 JWT_SECRET=
14 JWT_EXPIRE=
15
16 CLIENT_URL=http://localhost

```

Рисунок 5.6 – Приклад конфігураційного файлу `.env`

Коли всі налаштування завершені, система ініціалізується командою `docker compose up -d`. Вона створює базу даних, застосовує останні міграції Prisma та піднімає всі сервери платформи у фоновому режимі (рис. 5.7).

```

1  services:
21  backend:
22    build:
23      context: .
24      dockerfile: Dockerfile.backend
25    container_name: h-linker-backend
26    restart: always
27    ports:
28      - '3000:3000'
29    env_file:
30      - .env
31    environment:
32      DATABASE_URL: "postgresql://user:password@db:5432/h_linker_db?schema=public"
33    depends_on:
34      db:
35        condition: service_healthy
36    command: >
37      sh -c "npx prisma migrate deploy && npx prisma db seed && node dist/apps/backend/main.js"
38
39  frontend:
40    build:
41      context: .
42      dockerfile: Dockerfile.frontend
43    container_name: h-linker-frontend
44    restart: always
45    ports:
46      - '8080:80'
47    depends_on:
48      - backend
49
50  volumes:
51    postgres_data:

```

Рисунок 5.7 – Файл docker-compose

Завдяки цим крокам вдається успішно задеплоїти застосунок на сервері (рис. 5.8).

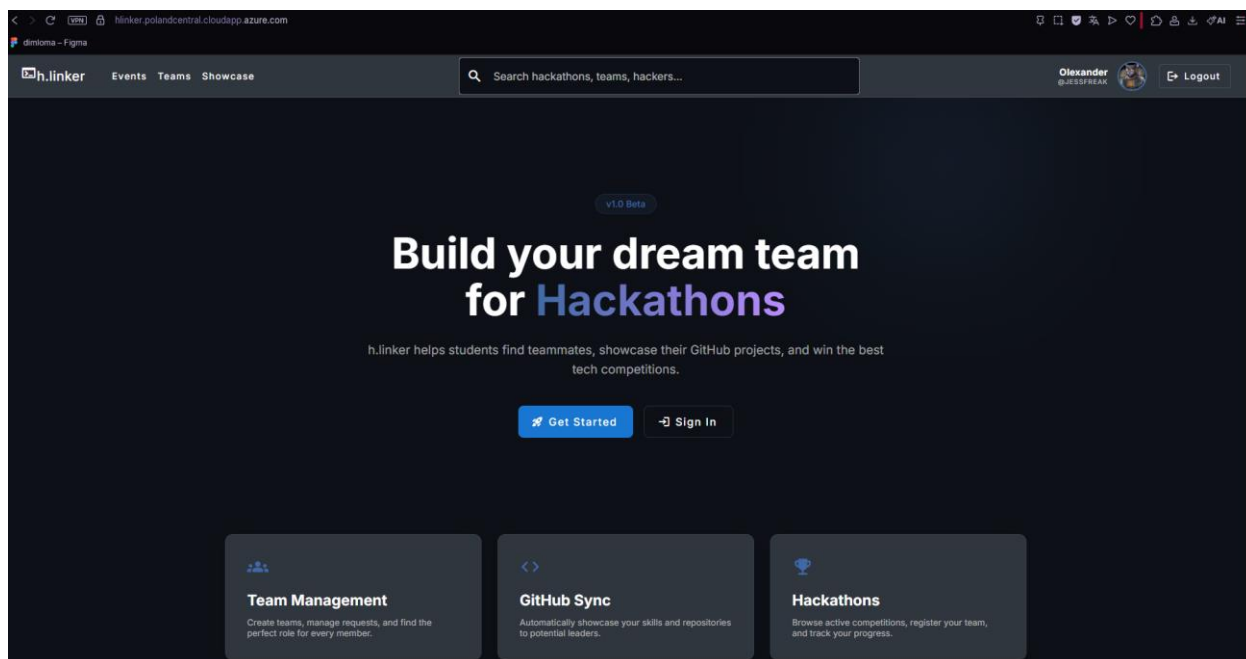


Рисунок 5.8 – Результат розгортання платформи із захищеним HTTPS-з'єднанням

5.2 Супровід програмного забезпечення

Інструкція користувача наведена в окремому документі «Керівництво користувача».

Оскільки проєкт побудовано на базі архітектури монорепозиторію з використанням інструментарію Nx, процес супроводу, оновлення та тестування коду вимагає глибокої автоматизації. Для забезпечення безперервної інтеграції та безперервного розгортання (CI/CD) платформи було впроваджено використання хмарного сервісу GitHub Actions.

Це рішення дозволяє автоматично доставляти нові версії системи на бойовий сервер Microsoft Azure відразу після злиття оновлень у головну гілку розробки без ручного втручання. Для того щоб конвеєр мав можливість підключатися до сервера по SSH, необхідні авторизаційні дані (IP-адреса віртуальної машини, ім'я користувача, SSH-ключ та пароль). Ці дані надійно зашифровані та зберігаються у розділі Repository Secrets у налаштуваннях GitHub (рис. 5.9).

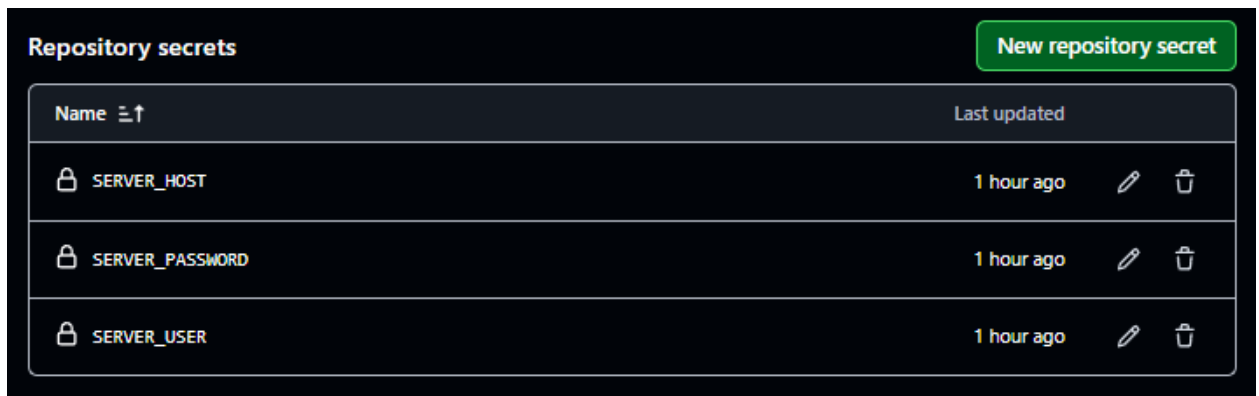


Рисунок 5.9 – Налаштування захищених змінних (Secrets) у репозиторії GitHub

Процес автоматичного супроводу (Pipeline) ініціюється автоматично та складається з трьох послідовних етапів (рис. 5.10).



Рисунок 5.10 – Структура CI/CD конвеєра у GitHub Actions

Першим виконується етап перевірки якості коду. Віртуальне середовище Ubuntu встановлює платформу Node.js, завантажує залежності та запускає статичний аналізатор ESLint для всіх проєктів монорепозіторію. З метою пришвидшення роботи лінера файли модульних тестів (.spec.ts) ігноруються.

Паралельно з перевіркою стилістики коду виконується етап тестування, що суттєво економить загальний час виконання Pipeline. На цьому етапі система автоматично генерує клієнт для взаємодії з базою даних, після чого фреймворк Jest запускає всі модульні тести.

Етап розгортання активується виключно після успішного проходження попередніх фаз. За допомогою SSH-з'єднання скрипт GitHub Actions авторизується на сервері Azure та виконує оновлення (рис. 5.10). Він отримує

найсвіжішу версію коду та ініціює примусове перезбирання Docker-образів без використання кешу, щоб гарантувати застосування будь-яких змін в .env або конфігураціях. Після рестарту контейнерів система очищує диск від старих образів.

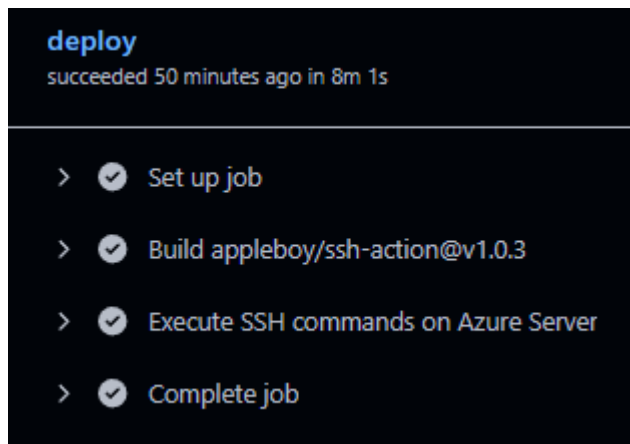


Рисунок 5.10 – Логи успішного розгортання на сервері Azure

5.3 Висновки розділу

У цьому розділі було детально розглянуто та практично реалізовано процеси розгортання та подальшого супроводу розробленого програмного забезпечення. Успішне виведення продукту в середовище реальної експлуатації стало можливим завдяки використанню сучасних DevOps-практик, хмарних технологій та засобів контейнеризації.

Для розгортання платформи було обрано підхід на основі Docker та Docker Compose. Це дозволило розділити архітектуру на незалежні ізольовані контейнери: базу даних PostgreSQL, бекенд-сервер на базі NestJS та фронтенд-застосунок. Важливою інфраструктурною ланкою є вебсервер Nginx, налаштований у ролі зворотного проксі на самій віртуальній машині. Він забезпечив впровадження захищеного HTTPS-з'єднання, а також надійний захист внутрішніх контейнерів, закривши прямий доступ до їхніх портів із зовнішньої мережі.

В якості хостинг-провайдера було обрано хмарну платформу Microsoft Azure. Розгортання на базі виділеної VM забезпечило системі статичну IP-адресу та високу доступність. Також було успішно проведено інтеграцію та

налаштування сторонніх сервісів: налаштовано OAuth 2.0 додатки у Google Cloud Console та GitHub Developer Settings для забезпечення безпечної авторизації користувачів, а також підключено API сервісу ImgBB для роботи з медіаконтентом. Важливим етапом стало розділення конфігурацій: публічні ключі інтегровано у збірку фронтенду, тоді як критичні секрети та паролі надійно ізольовано на сервері у файлі `.env`.

Для забезпечення надійного супроводу програмного забезпечення було розроблено відповідну експлуатаційну документацію («Керівництво користувача»), а також побудовано повністю автоматизований конвеєр безперервної інтеграції та розгортання (CI/CD) на базі сервісу GitHub Actions.

Оскільки проєкт має складну структуру монорепозіторію (Nx), впроваджений Pipeline складається з трьох ключових етапів. Перші два етапи – статичний аналіз коду та автоматизоване тестування – виконуються паралельно, що суттєво оптимізує час проходження перевірок. Лише після їх успішного завершення ініціюється третій етап – безпечне розгортання на сервері Azure через SSH-з'єднання. Автоматизація включає отримання найсвіжішого коду, примусове перезбирання образів без використання кешу, рестарт контейнерів та очищення дискового простору від застарілих даних. Секретні дані для доступу до сервера зберігаються у зашифрованому вигляді через GitHub Secrets.

Загалом, впроваджені інфраструктурні рішення повністю автоматизували рутинні процеси оновлення системи, унеможливили потрапляння неперевіреного коду на робочий сервер та створили надійне, безпечне і масштабоване середовище для стабільного функціонування розробленої платформи хакатонів.

ВИСНОВКИ

У ході виконання дипломного проєкту було розроблено спеціалізовану вебзастосунок «h.linker», призначену для комплексної автоматизації організаційних етапів хакатонів. Створений програмний продукт являє собою єдину екосистему, яка об'єднує модулі безпечної автентифікації користувачів, управління командами, збору проєктів та динамічного експертного оцінювання. Архітектурно застосунок побудовано за клієнт-серверною моделлю у форматі монорепозиторію, де швидко клієнтську частину реалізовано на базі фреймворку Angular, а надійну серверну – за допомогою NestJS у поєднанні з реляційною базою даних PostgreSQL та інструментом Prisma ORM.

Головна мета дипломного проєктування, яка полягала у створенні гнучкого, надійного та масштабованого інструменту для забезпечення прозорого оцінювання результатів ІТ-змагань, була досягнута в повному обсязі. Розроблена платформа успішно вирішує проблему фрагментарності даних, замінюючи розрізнені неінтегровані інструменти, такі як електронні таблиці та зовнішні форми реєстрації, на централізоване автоматизоване рішення, здатне ефективно керувати всім життєвим циклом заходу від його анонсу до формування підсумкового лідерборду.

На етапах обстеження та проєктування було успішно проаналізовано предметну область, побудовано моделі бізнес-процесів у нотації BPMN, спроектовано багаторівневу архітектуру системи за моделлю C4 та підтверджено економічну доцільність розробки. У рамках завдань конструювання програмного забезпечення реалізовано авторизацію через протокол OAuth 2.0 (API GitHub та Google), створено інструментарій для формування команд на основі навичок учасників та розроблено ядро системи – рушій автоматичного розрахунку зваженого балу на основі налаштованих організаторами критеріїв. Завдання з аналізу якості також повністю виконані: фронтенд оптимізовано до максимальних показників продуктивності та доступності за аудитом Google Lighthouse, бекенд успішно пройшов стрес-

тестування й профілювання під навантаженням, а бізнес-логіка надійно покрита автоматизованими юніт-тестами. Крім того, повністю вирішено завдання розгортання: застосунок контейнеризовано за допомогою Docker, розміщено на віртуальній машині Microsoft Azure з підключенням HTTPS та забезпечено автоматизованим CI/CD конвеєром через GitHub Actions.

Доцільність подальшого продовження розробки за даною предметною областю є високою. Завдяки закладеній гнучкій архітектурі, створений продукт має значний потенціал для комерціалізації та масштабування на ринок корпоративних та міжнародних освітніх заходів, зокрема для організації та проведення студентських і шкільних академічних олімпіад. Одним із ключових стратегічних напрямків розвитку є виведення платформи на більш глобальний рівень абстракції: трансформація з вузькоспеціалізованого сервісу виключно для ІТ-спеціалістів у багатoproфільну систему управління змаганнями, де хакатони з програмування стануть лише однією з багатьох доступних категорій. Серед перспективних векторів технічного вдосконалення системи планується поглиблення інтеграції з екосистемою GitHub для більш детального автоматизованого аналізу внеску кожного учасника, впровадження модулів штучного інтелекту для попередньої перевірки поданих робіт на плагіат, розширення інструментарію для автоматичного формування призових фондів, а також створення нативного мобільного застосунку для забезпечення оперативної комунікації між учасниками та організаторами під час проведення масштабних офлайн-заходів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Devpost - The home for hackathons. *Devpost - The home for hackathons*. URL: <https://devpost.com> (дата звернення: 30.01.2026).
- 2) HackTribe - Innovation program and Hackathon management platform. *HackTribe - Innovation program and Hackathon management platform*. URL: <https://hacktribe.co> (дата звернення: 30.01.2026).
- 3) NestJS - A progressive Node.js framework. *NestJS - A progressive Node.js framework*. URL: <https://nestjs.com> (дата звернення: 05.02.2026).
- 4) Home • Angular. *Angular*. URL: <https://angular.dev> (дата звернення: 05.02.2026).
- 5) RxJS. *RxJS*. URL: <https://rxjs.dev> (дата звернення: 05.02.2026).
- 6) PostgreSQL. *PostgreSQL*. URL: <https://www.postgresql.org> (дата звернення: 05.02.2026).
- 7) Prisma | Instant Postgres plus an ORM for simpler db workflows. *Prisma*. URL: <https://www.prisma.io> (дата звернення: 05.02.2026).
- 8) Сидоров М. О., Родіонов П. Ю., Марченко О. І. Економіка ІТ-індустрії та підприємництво: Практикум. Київ : КПІ ім. Ігоря Сікорського, 2024. URL: <https://ela.kpi.ua/server/api/core/bitstreams/5843debf-51cb-4a67-8ee3-75c1f4424cf9/content> (дата звернення: 20.03.2026).
- 9) Hai V. V., Nhung H. L. T. K., Hoc H. T. A Review of Software Effort Estimation by Using Functional Points Analysis. *Computational Statistics and Mathematical Modeling Methods in Intelligent Systems*. 2019. С. 408–422. URL: https://www.researchgate.net/publication/335939488_A_Review_of_Software_Effort_Estimation_by_Using_Functional_Points_Analysis (дата звернення: 21.03.2026).
- 10) Function Point Languages Table. *QSM Software Project Estimation*. URL: <https://www.qsm.com/resources/function-point-languages-table> (дата звернення: 25.03.2026).
- 11) Jones C. Software Economics and Function Point Metrics: Thirty years of IFPUG Progress. Namcook Analytics LLC, 2017.

URL: <https://www.ifpug.org/wp-content/uploads/2017/04/IYSM.-Thirty-years-of-IFPUG.-Software-Economics-and-Function-Point-Metrics-Capers-Jones.pdf> (дата звернення: 25.03.2026).

12) Salaries in 2025. *Спільнота програмістів | DOU*. URL: <https://jobs.dou.ua/salaries/?period=2025-12&group=1&position=all&title=3&technology=TypeScript&framework=Nest.js&specialization=Full%20Stack> (дата звернення: 27.03.2026).

13) Google Cloud Platform. *Google Cloud Platform*. URL: <https://console.cloud.google.com/> (дата звернення: 30.03.2026).

14) Upload Image – Free Image Hosting. *ImgBB*. URL: <https://api.imgbb.com> (дата звернення: 30.03.2026).

15) JetBrains. WebStorm: The JavaScript and TypeScript IDE, by JetBrains. *JetBrains*. URL: <https://www.jetbrains.com/webstorm/> (дата звернення: 30.03.2026).

16) Jest. *Jest · Delightful JavaScript Testing*. URL: <https://jestjs.io> (дата звернення: 30.03.2026).

17) Postman: The World's Leading API Platform. *Postman: The World's Leading API Platform*. URL: <https://www.postman.com> (дата звернення: 30.03.2026).

18) mcollina/autocannon: fast HTTP/1.1 benchmarking tool written in Node.js. GitHub. URL: <https://github.com/mcollina/autocannon> (дата звернення: 02.06.2026).

19) Tools to help diagnose and pinpoint Node.js performance issues. *Clinicjs*. URL: <https://clinicjs.org> (дата звернення: 31.05.2026).

20) Introduction to Lighthouse | Chrome for Developers. Chrome for Developers. URL: <https://developer.chrome.com/docs/lighthouse/overview/> (дата звернення: 31.05.2026).

21) Хмарна платформа обчислень Microsoft Azure. *Microsoft Azure*. URL: <https://azure.microsoft.com/en-us> (дата звернення: 02.06.2026).

ДОДАТОК А ЗВІТ ПОДІБНОСТІ

На рисунках А.1-А.7 наведено скріншоти зі звіту перевірки на співпадіння тексту.



1	ІП-22_Патріок_ПЗ 6/2/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	46 (0.26 %)
2	https://elja.kpi.ua/bitstreams/90b23404-4886-462c-bce3-e2e11e8ab880/download	45 (0.26 %)
3	Економіка IT-індустрії та підприємництво 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	37 (0.21 %)
4	ІП-24_Беліков_ПЗ 5/31/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	28 (0.16 %)
5	ІП-13_Пархомчук_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	27 (0.15 %)
6	ІП-13_Пархомчук_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	27 (0.15 %)
7	ІП-15_Прокопенко_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	25 (0.14 %)
8	ІП-12_Скорик_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	21 (0.12 %)
9	https://elja.kpi.ua/bitstreams/90b23404-4886-462c-bce3-e2e11e8ab880/download	20 (0.11 %)
10	ІП-11_Трикош_ПЗ 5/28/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	19 (0.11 %)
Домашня база даних (6.42 %)		
#	заголовок	кількість ідентичних слів (фрагментів)
1	ІП-22_Патріок_ПЗ 6/2/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	117 (8) (0.67 %)
2	ІП-21_Сергієнко_ПЗ 5/30/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	98 (11) (0.56 %)
3	Економіка IT-індустрії та підприємництво 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	95 (6) (0.54 %)

Рисунок А.2

National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)		
4	ІП-13_Лисенко_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	80 (9) (0.46 %)
5	ІП-13_Пархомчук_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	69 (3) (0.39 %)
6	ІП-24_Бєліков_ПЗ 5/31/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	43 (3) (0.25 %)
7	ІП-24_Кондратюк_ПЗ 6/2/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	40 (4) (0.23 %)
8	ІП-13_Качмар_ПЗ 5/29/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	39 (6) (0.22 %)
9	ІП-11_Друщенко_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	33 (4) (0.19 %)
10	ІП-11_Хамад_ПЗ 6/3/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	28 (5) (0.16 %)
11	ІП-23_Дороніна_ПЗ 6/2/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	26 (4) (0.15 %)
12	ІП-15_Прокопенко_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	25 (1) (0.14 %)
13	ІП-12_Бондарчук_ПЗ 6/10/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	24 (3) (0.14 %)
14	ІП-з11_Приходько_ПЗ 6/8/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	24 (3) (0.14 %)
15	ІП-12_Скорик_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а Інформатики та програмної інженерії)	21 (1) (0.12 %)
16	Мобільний застосунок "Антологія інтелектуальних ігор-головоломок" 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical	21 (3) (0.12 %)

Рисунок А.3

National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)		
17	ІП-12_Шоман_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	19 (3) (0.11 %)
18	ІП-11_Трикош_ПЗ 5/28/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	19 (1) (0.11 %)
19	Вебзастосунок для проведення консультацій з психологом 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	17 (2) (0.1 %)
20	ІП-15_Хливіч_ПЗ 5/28/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	16 (3) (0.09 %)
21	ІП-13_Головач_ПЗ 6/11/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	15 (1) (0.09 %)
22	Платформа моніторингу стабільності вебзастосунків 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	14 (2) (0.08 %)
23	ІП-12_Йопкін_ПЗ 6/1/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	14 (2) (0.08 %)
24	ІП-15_Лереца_ПЗ 6/6/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	14 (1) (0.08 %)
25	Архітектурне рішення веб-застосунку для цифрової дистрибуції 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	13 (1) (0.07 %)
26	ІП-15_Кондрацька_ПЗ 5/30/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	13 (2) (0.07 %)
27	ІП-12_Орищенко_ПЗ 6/11/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	13 (2) (0.07 %)
28	ІП-15_Чорний_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	12 (1) (0.07 %)
29	ІП-13_Ал_Хадям_ПЗ 6/5/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	12 (2) (0.07 %)

Рисунок А.4

національного технічного університету України Ігор Сікорський Київський політехнічний інститут (ФІОТ, К-а інформатики та програмної інженерії)		
30	ІП-15_Щербачук_ПЗ 5/31/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	12 (1) (0.07 %)
31	ІП-13_Шиманська_ПЗ 6/1/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	10 (2) (0.06 %)
32	ІП-13_Шевцова_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	10 (1) (0.06 %)
33	ІП-13_Дем'янюк_ПЗ 5/30/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	8 (1) (0.05 %)
34	Кроссплатформний файлообмінник для локальних середовищ 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	8 (1) (0.05 %)
35	Веб-сервіс аналізу відгуків на мобільні застосунки 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	8 (1) (0.05 %)
36	Програмне забезпечення для організації та планування маршрутів та поїздок 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	8 (1) (0.05 %)
37	ІП-13_Ковалик_ПЗ 6/7/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	7 (1) (0.04 %)
38	ІП-11_Головатюк_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	7 (1) (0.04 %)
39	ІП-13_Дейнега_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	6 (1) (0.03 %)
40	ІП-13_Грицина_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	6 (1) (0.03 %)
41	Сервіс сповіщення про відключення електроенергії в телеграм-боті 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	6 (1) (0.03 %)
42	Графічний інтерфейс для побудови нейронних мереж 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	6 (1) (0.03 %)

Рисунок А.5

Національний технічний університет України Ігор Сікорського (Київський політехнічний інститут) University of Ukraine Igor Sikorskyi Kyiv Politech Institute)		
43	ІП-23_Федоренко_ПЗ 6/3/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	6 (1) (0.03 %)
44	Мобільний застосунок для фіксації громадянами порушень ПДР 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	5 (1) (0.03 %)
45	ІП-15_Коваленко_ПЗ 6/11/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	5 (1) (0.03 %)
46	ІП-15_Рубалка_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	5 (1) (0.03 %)
47	ІП-15_Ткач_ПЗ 6/11/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	5 (1) (0.03 %)
48	ІП-12_Васильєв_ПЗ 6/1/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	5 (1) (0.03 %)
49	ПЗМД_ІСТ_КПІ_2025_Тригуб Д. К. 12/28/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-па інформаційних систем та технологій)	5 (1) (0.03 %)
50	Вебзастосунок для створення тестової документації 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	5 (1) (0.03 %)
51	ІП-11_Калашніков_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	5 (1) (0.03 %)
Програма обміну базами даних (0.06 %)		
#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
52	Розроблення інформаційної управляючої системи для туристичної агенції 5/18/2025 Kyiv National Economic University named after Vadym Hetman KNEU (кафедра комп'ютерних наук)	10 (1) (0.06 %)
Інтернет (3.37 %)		
#	ДЖЕРЕЛО URL	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
53	https://ela.kpi.ua/bitstreams/90b23404-4886-462c-bce3-e2e1fe5ab86f/download	240 (18) (1.37 %)

Рисунок А.6

54	https://ela.kpi.ua/bitstreams/36b06709-1403-4fe0-b623-3d35bebb19a9/download	182 (23) (1.04 %)
55	https://www.ela.kpi.ua/bitstreams/f3dd82e4-5f13-4ae7-8b93-ec818b4601c2/download	79 (9) (0.45 %)
56	https://dspace.wnu.edu.ua/bitstream/316497/39804/1/%D0%9D%D0%BE%D0%B2%D0%BE%D1%81%D0%B0%D0%B4_%D0%94%D0%9F_2018.pdf	33 (5) (0.19 %)
57	https://elartu.tntu.edu.ua/bitstream/lib/48400/1/Taras_Shchur.pdf	21 (2) (0.12 %)
58	https://ela.kpi.ua/bitstream/123456789/31143/1/Kamenskiy_bakalavr.pdf	18 (2) (0.1 %)
59	https://ela.kpi.ua/bitstream/123456789/57821/1/Kabanets_bakalavr.pdf	16 (2) (0.09 %)
Список прийнятих фрагментів		
#	ЗМІСТ	КІЛЬКІСТЬ ОДИНАКОВИХ СЛІВ (ФРАГМЕНТІВ)

Рисунок А.7

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ВЕБЗАСТОСУНОК ДЛЯ ОРГАНІЗАЦІЇ ХАКАТОНІВ З
ДИНАМІЧНИМ МОДУЛЕМ ОЦІНЮВАННЯ**

Текст програми

КПІ.ПІ-2419.045440.03.12

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена МАРЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

Виконавець:

_____ Олександр МАЛИЙ

Київ – 2026

Посилання на репозиторій з повним текстом програмного коду

<https://github.com/JessFreak/h.linker>

Файл prisma/schema.prisma

Реалізація схеми реляційної бази даних, опис моделей користувачів, хакатонів, проєктів та оцінок.

```
datasource db {
  provider = "postgresql"
}

generator client {
  provider = "prisma-client-js"
}

enum HackathonStatus {
  DRAFT
  REGISTRATION
  ACTIVE
  FINISHED
}

enum UserTeamType {
  INVITATION
  REQUEST
}

enum UserTeamStatus {
  PENDING
  ACCEPTED
  REJECTED
  LEFT
}

model Role {
  name      String   @id
  usersTeams UserTeam[]

  @@map("roles")
}

model Category {
  name      String      @id
  usersCategories   UserCategory[]
  hackathonsCategories HackathonCategory[]

  @@map("categories")
}

model User {
  id      String @id @default(uuid())
  email    String @unique
  username String @unique
  firstName String @map("first_name")
  lastName  String? @map("last_name")
  githubId  String? @unique @map("github_id")
  githubUsername String? @unique @map("github_username")
  bio       String?
  avatarUrl String? @map("avatar_url")
  password  String

  ownedTeams Team[] @relation("TeamLeader")
  createdHackathons Hackathon[] @relation("HackathonCreator")
  memberships UserTeam[]
  skills       UserCategory[]
  juryRoles    Jury[]

  @@map("users")
}

model Team {
  id      String @id @default(uuid())
  name    String
```

```

    leaderId    String @map("leader_id")
    leader      User    @relation("TeamLeader", fields: [leaderId], references: [id], onDelete: Cascade)
    description  String?
    communicationLink String? @map("communication_link")

    members      UserTeam[]
    participations Participation[]

    @@map("teams")
}

model UserTeam {
    id      String @id @default(uuid())
    userId  String @map("user_id")
    teamId  String @map("team_id")
    roleName String @map("role_name")
    status  UserTeamStatus @default(PENDING)
    type    UserTeamType
    message String?
    created DateTime @default(now())

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    team Team @relation(fields: [teamId], references: [id], onDelete: Cascade)
    role Role @relation(fields: [roleName], references: [name])

    @@unique([userId, teamId])
    @@map("users_teams")
}

model Hackathon {
    id      String @id @default(uuid())
    creatorId String @map("creator_id")
    title   String
    slug    String @unique
    description String?
    prize   String?
    status  HackathonStatus @default(DRAFT)

    registrationStartDate DateTime @map("reg_start_date")
    startDate              DateTime @map("start_date")
    endDate                DateTime @map("end_date")
    submissionDeadline     DateTime @map("submission_deadline")

    imageUrl String? @map("image_url")

    creator      User @relation("HackathonCreator", fields: [creatorId], references: [id], onDelete: Cascade)
    categories   HackathonCategory[]
    criteria     Criterion[]
    jury         Jury[]
    participations Participation[]

    @@map("hackathons")
}

model UserCategory {
    userId  String @map("user_id")
    category String
    user    User @relation(fields: [userId], references: [id], onDelete: Cascade)
    cat     Category @relation(fields: [category], references: [name], onDelete: Cascade)

    @@id([userId, category])
    @@map("users_categories")
}

model HackathonCategory {
    hackathonId String @map("hackathon_id")
    category     String
    hackathon    Hackathon @relation(fields: [hackathonId], references: [id], onDelete: Cascade)
    cat          Category @relation(fields: [category], references: [name], onDelete: Cascade)

    @@id([hackathonId, category])
    @@map("hackathons_categories")
}

model Participation {
    id      String @id @default(uuid())
    teamId  String @map("team_id")
    hackathonId String @map("hackathon_id")
    projectTitle String? @map("project_title")

```



```

projectDescription String? @map("project_description") @db.Text
githubRepoUrl      String? @map("github_repo_url")
finalScore         Float   @default(0) @map("final_score")

createdAt DateTime @default(now()) @map("created_at")
updatedAt DateTime @default(now()) @map("updated_at")

team Team @relation(fields: [teamId], references: [id], onDelete: Cascade)
hackathon Hackathon @relation(fields: [hackathonId], references: [id], onDelete: Cascade)
scores Score[]
reviews Review[]

@@unique([teamId, hackathonId])
@@map("participations")
}

model Criterion {
  id String @id @default(uuid())
  hackathonId String @map("hackathon_id")
  name String
  weight Float
  max_value Int @default(10) @map("max_value")

  hackathon Hackathon @relation(fields: [hackathonId], references: [id], onDelete: Cascade)
  scores Score[]

  @@map("criteria")
}

model Jury {
  id String @id @default(uuid())
  userId String @map("user_id")
  hackathonId String @map("hackathon_id")

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  hackathon Hackathon @relation(fields: [hackathonId], references: [id], onDelete: Cascade)
  scores Score[]
  reviews Review[]

  @@unique([hackathonId, userId])
  @@map("jury")
}

model Score {
  id String @id @default(uuid())
  juryId String @map("jury_id")
  participationId String @map("participation_id")
  criterionId String @map("criterion_id")
  value Float

  createdAt DateTime @default(now()) @map("created_at")

  jury Jury @relation(fields: [juryId], references: [id], onDelete: Cascade)
  participation Participation @relation(fields: [participationId], references: [id], onDelete: Cascade)
  criterion Criterion @relation(fields: [criterionId], references: [id], onDelete: Cascade)

  @@map("scores")
}

model Review {
  id String @id @default(uuid())
  participationId String @map("participation_id")
  juryId String @map("jury_id")
  summary String
  strengths String?
  weaknesses String?

  createdAt DateTime @default(now()) @map("created_at")

  participation Participation @relation(fields: [participationId], references: [id], onDelete: Cascade)
  jury Jury @relation(fields: [juryId], references: [id], onDelete: Cascade)

  @@unique([juryId, participationId])
  @@map("reviews")
}

```

Файл auth.service.ts

Реалізація функціональної задачі безпечної автентифікації користувачів через сторонні сервіси (Google/GitHub OAuth 2.0).

```
import { ConflictException, Inject, Injectable } from '@nestjs/common';
import * as bcrypt from 'bcryptjs';
import { JwtService } from '@nestjs/jwt';
import { LoginDTO, RegisterDTO, UpdatePasswordDTO } from '@h.linker/libs';
import { Response } from 'express';
import { NotRegisteredException } from '../utils/exceptions/not-registered.exception';
import { PasswordRepeatException } from '../utils/exceptions/password-repeat.exception';
import { InvalidPasswordException } from '../utils/exceptions/invalid-password.exception';
import { User } from '@prisma/client';
import config from '../config/config';
import { ConfigType } from '@nestjs/config';
import { GithubUser, ExternalUser } from '../utils/external-users';
import { CategoryService } from '../category.service';
import { UserService } from '../user.service';

@Injectable()
export class AuthService {
  constructor(
    @Inject(config.KEY) private configService: ConfigType<typeof config>,
    private jwtService: JwtService,
    private readonly userService: UserService,
    private readonly categoryService: CategoryService,
  ) {}

  async register(dto: RegisterDTO): Promise<User> {
    await this.userService.checkEmailUniqueness(dto.email);
    await this.userService.checkUsernameUniqueness(dto.username);

    return this.userService.create(dto);
  }

  private async validatePassword(
    password: string,
    userPassword: string,
  ): Promise<void> {
    const isPasswordsMatch = await bcrypt.compare(password, userPassword);
    if (!isPasswordsMatch) throw new InvalidPasswordException();
  }

  getToken(userId: string): string {
    return this.jwtService.sign({ sub: userId });
  }

  async login({ email, password }: LoginDTO): Promise<string> {
    const user = await this.userService.findByEmail(email);
    if (!user) throw new NotRegisteredException();

    await this.validatePassword(password, user.password);
    return this.getToken(user.id);
  }

  async validateGoogleUser(externalUser: ExternalUser): Promise<User> {
    let user = await this.userService.findByEmail(externalUser.email);

    if (!user) {
      user = await this.userService.create(externalUser);
    }

    return user;
  }

  async validateGithubUser(
    { skills, ...github }: GithubUser,
    currentUser?: User,
  ): Promise<User> {
    const user = currentUser
      ? await this.handleAccountLinking(currentUser, github.githubId, github.githubUsername)
      : await this.handleGithubAuth(github);

    if (skills?.length) {
      this.categoryService
        .syncUserSkills(user.id, skills)
        .catch((err) => console.error('GitHub Skills sync failed:', err));
    }

    return user;
  }
}
```

```

    }

    private async handleAccountLinking(
      user: User,
      githubId: string,
      githubUsername: string,
    ): Promise<User> {
      const linkedUser = await this.userService.findByGithubId(githubId);

      if (linkedUser && linkedUser.id !== user.id) {
        throw new ConflictException(
          'This GitHub account is already linked to another user',
        );
      }

      return this.userService.updateGithub(user.id, githubId, githubUsername);
    }

    private async handleGithubAuth(data: GithubUser): Promise<User> {
      let user = await this.userService.findByGithubId(data.githubId);
      if (user) return user;

      user = await this.userService.findByEmail(data.email);
      if (user) {
        return this.userService.updateGithub(user.id, data.githubId, data.githubUsername);
      }

      return this.userService.create(data);
    }

    setToken(userId: string, res: Response): void {
      const token = this.jwtService.sign({
        sub: userId,
      });

      res.cookie('access_token', token);
      res.redirect(this.configService.clientUrl);
    }

    async deleteMe(userId: string): Promise<void> {
      await this.userService.delete(userId);
    }

    async updatePassword(
      userId: string,
      { oldPassword, newPassword }: UpdatePasswordDTO,
    ): Promise<void> {
      const user = await this.userService.findById(userId);
      if (user.password) {
        await this.validatePassword(oldPassword, user.password);
      }

      if (oldPassword === newPassword) {
        throw new PasswordRepeatException();
      }

      const hashedPassword = await bcrypt.hash(newPassword, 10);
      await this.userService.updatePassword(userId, hashedPassword);
    }
  }

```

Файл team.service.ts

Реалізація функціональної задачі взаємодії з командами.

```

import {
  BadRequestException,
  Injectable,
  NotFoundException,
} from '@nestjs/common';
import { TeamRepository } from '../database/repositories/team.repository';
import {
  FullTeam,
  UserInvitationWithTeam,
} from '../database/entities/team.entity';
import { MemberRepository } from '../database/repositories/member.repository';
import {
  CreateTeamDTO,

```

```

InviteUserDTO,
JoinRequestDTO,
PageResponse,
TeamQueryDTO,
UpdateTeamDTO,
} from '@h.linker/libs';
import { Prisma, UserTeamStatus } from '@prisma/client';

@Injectable()
export class TeamService {
  constructor(
    private readonly teamRepository: TeamRepository,
    private readonly memberRepository: MemberRepository,
  ) {}

  async create(dto: CreateTeamDTO, leaderId: string): Promise<FullTeam> {
    return this.teamRepository.create({ ...dto, leaderId });
  }

  async findById(id: string): Promise<FullTeam> {
    return this.teamRepository.findById(id);
  }

  async getAll(query: TeamQueryDTO): Promise<PageResponse<FullTeam>> {
    const { search, leaderId, memberId, hackathonId, order } = query;

    const where: Prisma.TeamWhereInput = {
      ...(leaderId && { leaderId }),
      ...(memberId && { members: { some: { userId: memberId } } }),
      ...(hackathonId && { participations: { some: { hackathonId } } }),
      ...(search && {
        OR: [
          { name: { contains: search, mode: 'insensitive' } },
          { description: { contains: search, mode: 'insensitive' } },
        ],
      }),
    };

    const orderBy: Prisma.TeamOrderByWithRelationInput = {
      name: order ?? 'asc',
    };

    return this.teamRepository.findAllPaged(query, where, orderBy);
  }

  async updateById(id: string, dto: UpdateTeamDTO): Promise<FullTeam> {
    return this.teamRepository.updateById(id, dto);
  }

  async delete(id: string): Promise<void> {
    return this.teamRepository.deleteById(id);
  }

  async joinRequest(
    teamId: string,
    userId: string,
    dto: JoinRequestDTO,
  ): Promise<FullTeam> {
    await this.validateConnection(teamId, userId);

    await this.memberRepository.upsertConnection({
      teamId,
      userId,
      roleName: dto.roleName,
      message: dto.message,
      type: 'REQUEST',
      status: 'PENDING',
    });

    return this.teamRepository.findById(teamId);
  }

  async inviteUser(teamId: string, dto: InviteUserDTO): Promise<FullTeam> {
    await this.validateConnection(teamId, dto.userId);

    await this.memberRepository.upsertConnection({
      teamId,
      userId: dto.userId,
      roleName: dto.roleName,
    });
  }
}

```

```

        message: dto.message,
        type: 'INVITATION',
        status: 'PENDING',
    });

    return this.teamRepository.findById(teamId);
}

private async validateConnection(teamId: string, userId: string) {
    const existing = await this.memberRepository.findConnection(teamId, userId);

    if (existing?.status === 'ACCEPTED') {
        throw new BadRequestException('User is already a member of this team');
    }

    if (existing?.status === 'PENDING') {
        throw new BadRequestException(
            'There is already a pending request/invitation',
        );
    }
}

async removeMember(teamId: string, userId: string): Promise<FullTeam> {
    await this.memberRepository.removeMember(teamId, userId);

    return this.teamRepository.findById(teamId);
}

async respondToMemberRequest(
    teamId: string,
    userId: string,
    status: UserTeamStatus,
): Promise<FullTeam> {
    const membership = await this.memberRepository.findMember(teamId, userId);

    if (!membership) {
        throw new NotFoundException('Membership request not found');
    }

    if (membership.status !== UserTeamStatus.PENDING) {
        throw new BadRequestException(
            `Cannot change status from ${membership.status} to ${status}`,
        );
    }

    await this.memberRepository.updateMemberStatus(teamId, userId, status);

    return this.teamRepository.findById(teamId);
}

async changeLeader(id: string, newLeaderId: string): Promise<FullTeam> {
    return this.teamRepository.updateById(id, { leaderId: newLeaderId });
}

async findUserInvitations(userId: string): Promise<UserInvitationWithTeam[]> {
    return this.teamRepository.findUserInvitations(userId);
}
}

```

Файл `hackathon.cron.service.ts`

Реалізація функціональної задачі автоматизації життєвого циклу хакатонів (Cron-планувальник), що забезпечує автоматичну зміну статусів подій у часі без ручного втручання адміністратора.

```

import { Injectable, Logger } from '@nestjs/common';
import { Cron, CronExpression } from '@nestjs/schedule';
import { HackathonRepository } from '../database/repositories/hackathon.repository';

@Injectable()
export class HackathonCronService {
    private readonly logger = new Logger(HackathonCronService.name);
    constructor(private readonly hackathonRepository: HackathonRepository) {}

    @Cron(CronExpression.EVERY_HOUR)
    async handleHackathonStatuses() {
        const now = new Date();
    }
}

```

```

try {
  const toActiveResult = await this.hackathonRepository.updateToActive(now);
  if (toActiveResult.count > 0) {
    this.logger.log(
      `Hackathons transitioned to ACTIVE: ${toActiveResult.count}`,
    );
  }

  const toFinishedResult =
    await this.hackathonRepository.updateToFinished(now);
  if (toFinishedResult.count > 0) {
    this.logger.log(
      `Hackathons transitioned to FINISHED: ${toFinishedResult.count}`,
    );
  }
} catch (error) {
  this.logger.error(
    'Error during automatic hackathon status update',
    error,
  );
}
}
}

```

Файл `hackathon.service.ts`

Реалізація функціональної задачі динамічного управління хакатонами.

```

import {
  BadRequestException,
  Injectable,
  NotFoundException,
} from '@nestjs/common';
import { HackathonRepository } from '../database/repositories/hackathon.repository';
import {
  AddCommentDto,
  CreateHackathonDTO,
  CriterionDTO,
  HackathonInsightsResponse,
  HackathonQueryDTO,
  HackathonStatus,
  PageResponse,
  SubmitProjectDto,
  UpdateHackathonDTO,
} from '@h.linker/libs';
import { JuryRepository } from '../database/repositories/jury.repository';
import { CriteriaRepository } from '../database/repositories/criteria.repository';
import { CategoryRepository } from '../database/repositories/category.repository';
import { FullHackathon } from '../database/entities/hackathon.entity';
import { ParticipationRepository } from '../database/repositories/participation.repository';
import {
  LeaderboardRow,
  ParticipationWithScoresAndReviews,
  ParticipationWithTeam,
  ReviewWithJuryData,
} from '../database/entities/participation.entity';
import { EvaluationRepository } from '../database/repositories/evaluation.repository';
import { Prisma } from '@prisma/client';

@Injectable()
export class HackathonService {

```

```

constructor(
  private readonly hackathonRepository: HackathonRepository,
  private readonly juryRepository: JuryRepository,
  private readonly criteriaRepository: CriteriaRepository,
  private readonly categoryRepository: CategoryRepository,
  private readonly participationRepository: ParticipationRepository,
  private readonly evaluationRepository: EvaluationRepository,
) {}

async create(
  creatorId: string,
  dto: CreateHackathonDTO,
): Promise<FullHackathon> {
  return this.hackathonRepository.create({ ...dto, creatorId });
}

async getAll(query: HackathonQueryDTO): Promise<PageResponse<FullHackathon>> {
  const { search, status, order, categories, startDateFrom, startDateTo } =
    query;

  const where: Prisma.HackathonWhereInput = {
    ...(status && { status }),

    ...(categories &&
      categories.length > 0 && {
        categories: {
          some: {
            category: { in: categories },
          },
        },
      }),

    ...((startDateFrom || startDateTo) && {
      startDate: {
        ...(startDateFrom && { gte: startDateFrom }),
        ...(startDateTo && { lte: startDateTo }),
      },
    }),

    ...(search && {
      OR: [
        { title: { contains: search, mode: 'insensitive' } },
        { description: { contains: search, mode: 'insensitive' } },
      ],
    }),
  };

  const orderBy: Prisma.HackathonOrderByWithRelationInput = {
    registrationStartDate: order ?? 'desc',
  };

  return this.hackathonRepository.findAllPaged(query, where, orderBy);
}

```

```

async getById(id: string): Promise<FullHackathon> {
  return this.hackathonRepository.getById(id);
}

async getBySlug(slug: string): Promise<FullHackathon> {
  return this.hackathonRepository.getBySlug(slug);
}

async update(id: string, dto: UpdateHackathonDTO): Promise<FullHackathon> {
  return this.hackathonRepository.updateById(id, dto);
}

async updateStatus(
  id: string,
  status: HackathonStatus,
): Promise<FullHackathon> {
  return this.hackathonRepository.updateById(id, { status });
}

async deleteById(id: string): Promise<void> {
  return this.hackathonRepository.deleteById(id);
}

async addJuryMember(hackathonId: string, userId: string): Promise<void> {
  await this.juryRepository.add(hackathonId, userId);
}

async removeJuryMember(hackathonId: string, userId: string): Promise<void> {
  await this.juryRepository.remove(hackathonId, userId);
}

async setCriteria(
  hackathonId: string,
  criteria: CriterionDTO[],
): Promise<void> {
  const totalWeight = criteria.reduce((acc, c) => acc + c.weight, 0);
  if (totalWeight !== 100) {
    throw new BadRequestException(
      'Total weight of criteria must be exactly 100%',
    );
  }
  await this.criteriaRepository.syncCriteria(hackathonId, criteria);
}

async setCategories(
  hackathonId: string,
  categories: string[],
): Promise<void> {
  await this.categoryRepository.syncHackathonCategories(
    hackathonId,
    categories,
  );
}

```



```

    }

    async findUserRegistration(
      hackathonId: string,
      userId: string,
    ): Promise<ParticipationWithTeam> {
      return this.participationRepository.findUserParticipation(
        hackathonId,
        userId,
      );
    }
  }

  async registerTeam(
    hackathonId: string,
    teamId: string,
    userId: string,
  ): Promise<void> {
    const hackathon = await this.hackathonRepository.getById(hackathonId);
    if (hackathon.status !== HackathonStatus.REGISTRATION) {
      throw new BadRequestException('Registration is not open for this event');
    }

    const existingParticipation =
      await this.participationRepository.findByTeamAndHackathon(
        hackathonId,
        teamId,
      );
    if (existingParticipation) {
      throw new BadRequestException(
        'This team is already registered for this hackathon',
      );
    }
    const userReg = await this.participationRepository.findUserParticipation(
      hackathonId,
      userId,
    );
    if (userReg) {
      throw new BadRequestException(
        'You are already participating in this hackathon with another team',
      );
    }

    await this.participationRepository.create(hackathonId, teamId);
  }

  async submitProject(
    hackathonId: string,
    userId: string,
    dto: SubmitProjectDto,
  ): Promise<void> {
    const hackathon = await this.hackathonRepository.getById(hackathonId);

    if (hackathon.status !== HackathonStatus.ACTIVE) {

```

```

    throw new BadRequestException(
      'Submissions are only allowed for active hackathons',
    );
  }

  if (new Date() > new Date(hackathon.submissionDeadline)) {
    throw new BadRequestException('The submission deadline has passed');
  }

  const participation =
    await this.participationRepository.findUserParticipation(
      hackathonId,
      userId,
    );

  await this.participationRepository.updateSubmission(
    hackathonId,
    participation.teamId,
    dto,
  );
}

async getHackathonSubmissionsForJury(
  hackathonId: string,
): Promise<ParticipationWithScoresAndReviews[]> {
  return this.participationRepository.findAllSubmissionsByHackathonId(
    hackathonId,
  );
}

async getLeaderboard(id: string): Promise<LeaderboardRow[]> {
  return this.participationRepository.getLeaderboardData(id);
}

async setTeamScores(
  userId: string,
  hackathonId: string,
  participationId: string,
  scores: Record<string, number>,
): Promise<void> {
  const jury = await this.juryRepository.getJuryByUserAndHackathon(
    userId,
    hackathonId,
  );
  if (!jury) throw new NotFoundException('Jury member not found');

  await this.evaluationRepository.upsertScores(
    jury.id,
    participationId,
    scores,
  );

  await this.recalculateProjectFinalScore(participationId, hackathonId);
}

```

```

}

private async recalculateProjectFinalScore(
  participationId: string,
  hackathonId: string,
): Promise<void> {
  const criteria =
    await this.evaluationRepository.getCriteriaByHackathon(hackathonId);
  const allScores =
    await this.evaluationRepository.getScoresByParticipation(participationId);

  if (allScores.length === 0) return;

  const scoresByJury: Record<string, typeof allScores> = {};
  allScores.forEach((s) => {
    if (!scoresByJury[s.juryId]) scoresByJury[s.juryId] = [];
    scoresByJury[s.juryId].push(s);
  });

  let totalHackathonWeightedScore = 0;
  const totalJudgesWhoScored = Object.keys(scoresByJury).length;

  Object.values(scoresByJury).forEach((juryScores) => {
    let juryTotal = 0;
    juryScores.forEach((score) => {
      const criterion = criteria.find((c) => c.id === score.criterionId);
      if (criterion) {
        juryTotal += score.value * (criterion.weight / 100);
      }
    });
    totalHackathonWeightedScore += juryTotal;
  });

  const finalScore =
    totalJudgesWhoScored > 0
      ? totalHackathonWeightedScore / totalJudgesWhoScored
      : 0;

  await this.evaluationRepository.updateParticipationFinalScore(
    participationId,
    finalScore,
  );
}

async setTeamComment(
  userId: string,
  hackathonId: string,
  participationId: string,
  dto: AddCommentDto,
): Promise<void> {
  const jury = await this.juryRepository.getJuryByUserAndHackathon(
    userId,
    hackathonId,
  );

```

```

);
if (!jury) throw new NotFoundException('Jury member not found');

await this.evaluationRepository.upsertComment(
  jury.id,
  participationId,
  dto,
);
}

async findTeamReviews(
  hackathonId: string,
  userId: string,
): Promise<ReviewWithJuryData[]> {
  return this.participationRepository.findReviewsByMember(
    hackathonId,
    userId,
  );
}

async getInsights(id: string): Promise<HackathonInsightsResponse> {
  const hackathon = await this.hackathonRepository.getInsightsData(id);

  const participations = hackathon.participations;

  const totalTeams = participations.length;
  const totalParticipants = participations.reduce(
    (acc, p) => acc + p.team.members.length,
    0,
  );
  const submissions = participations.filter((p) => p.githubRepoUrl);
  const totalSubmissions = submissions.length;
  const avgScore =
    submissions.reduce((acc, p) => acc + p.finalScore, 0) /
    (totalSubmissions || 1);

  // role distribution
  const roleCount: Record<string, number> = {};
  participations.forEach((p) => {
    p.team.members.forEach((m) => {
      roleCount[m.roleName] = (roleCount[m.roleName] || 0) + 1;
    });
  });
  const roleDistribution = Object.entries(roleCount).map(
    ([label, value]) => ({ label, value }),
  );

  // submission timeline
  const subTimelineCount: Record<string, number> = {};
  submissions.forEach((p) => {
    const iso = p.updatedAt.toISOString();
    const hourKey = iso.substring(0, 13).replace('T', '') + ':00';

```

```

    subTimelineCount[hourKey] = (subTimelineCount[hourKey] || 0) + 1;
  });
const submissionTimeline = Object.entries(subTimelineCount)
  .map(([label, value]) => ({ label, value }))
  .sort((a, b) => a.label.localeCompare(b.label));

// jury activity
const juryTimelineCount: Record<string, number> = {};
participations.forEach((p) => {
  p.scores.forEach((s) => {
    const iso = s.createdAt.toISOString();
    const hourKey = iso.substring(0, 13).replace('T', '') + ':00';

    juryTimelineCount[hourKey] = (juryTimelineCount[hourKey] || 0) + 1;
  });
});
const juryActivityTimeline = Object.entries(juryTimelineCount)
  .map(([label, value]) => ({ label, value }))
  .sort((a, b) => a.label.localeCompare(b.label));

// score distribution
const scoreRanges = { '0-2': 0, '2-4': 0, '4-6': 0, '6-8': 0, '8-10': 0 };
submissions.forEach((p) => {
  if (p.finalScore <= 2) scoreRanges['0-2']++;
  else if (p.finalScore <= 4) scoreRanges['2-4']++;
  else if (p.finalScore <= 6) scoreRanges['4-6']++;
  else if (p.finalScore <= 8) scoreRanges['6-8']++;
  else scoreRanges['8-10']++;
});
const scoreDistribution = Object.entries(scoreRanges).map(
  ([label, value]) => ({ label, value }),
);

return {
  stats: {
    totalTeams,
    totalParticipants,
    totalSubmissions,
    averageScore: Number(avgScore.toFixed(1)),
  },
  charts: {
    roleDistribution,
    submissionTimeline,
    juryActivityTimeline,
    scoreDistribution,
  },
};
}
}

```

Реалізація перевірки ролі користувача відносно хакатону.

```
import { Injectable, CanActivate, ExecutionContext } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { ROLES_KEY, UserRole } from '../decorators/roles';
import { HackathonRepository } from '../../app/database/repositories/hackathon.repository';
import { JuryRepository } from '../../app/database/repositories/jury.repository';
```

```
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(
    private reflector: Reflector,
    private hackathonRepository: HackathonRepository,
    private juryRepository: JuryRepository,
  ) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    const requiredRoles = this.reflector.getAllAndOverride<UserRole[]>(
      ROLES_KEY,
      [context.getHandler(), context.getClass()],
    );

    if (!requiredRoles || requiredRoles.length === 0) {
      return true;
    }

    const request = context.switchToHttp().getRequest();
    const user = request.user;
    const hackathonId = request.params.id;

    if (!user) return false;

    const roleChecks: Record<string, () => Promise<boolean>> = {
      ADMIN: () => this.hackathonRepository.isCreator(hackathonId, user.id),
      JURY: () => this.juryRepository.isUserInJury(hackathonId, user.id),
    };

    const requiredCheckPromises = requiredRoles
      .filter((role) => roleChecks[role])
      .map((role) => roleChecks[role]());

    const results = await Promise.all(requiredCheckPromises);
    return results.some((res) => res);
  }
}
```

Файл jury-evaluation.component.ts

Реалізація функціональної задачі інтерфейсу оцінювання для членів журі.

```
import {
  Component,
  inject,
  OnInit,
  OnDestroy,
  signal,
  computed,
```

```

    effect,
  } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ActivatedRoute, RouterLink, Router } from '@angular/router';
import {
  FormBuilder,
  FormGroup,
  ReactiveFormsModule,
  Validators,
} from '@angular/forms';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { MatSliderModule } from '@angular/material/slider';
import { MatInputModule } from '@angular/material/input';
import { MatTooltipModule } from '@angular/material/tooltip';
import { MatProgressBarModule } from '@angular/material/progress-bar';
import { MatProgressSpinner } from '@angular/material/progress-spinner';
import {
  CriterionResponse,
  FullHackathonResponse,
  JurySubmissionItem,
} from '@h.linker/libs';
import { HackathonService } from '../services/hackathon.service';
import { AuthService } from '../services/auth.service';
import { Subscription, debounceTime, forkJoin, of } from 'rxjs';
import { NotificationService } from '../utils/notification.service';
import { toSignal } from '@angular/core/rxjs-interop';
import { BreadcrumbComponent } from '../breadcrumb/breadcrumb.component';

export interface EvaluationFormValues {
  summary: string;
  strengths: string;
  weaknesses: string;
  [criterionId: string]: string | number | null | undefined;
}

@Component({
  selector: 'app-jury-evaluation',
  standalone: true,
  imports: [
    CommonModule,
    ReactiveFormsModule,
    RouterLink,
    MatButtonModule,
    MatIconModule,
    MatSliderModule,
    MatInputModule,
    MatTooltipModule,
    MatProgressBarModule,
    MatProgressSpinner,
    BreadcrumbComponent,
  ],
  templateUrl: './jury-evaluation.component.html',

```

```

styleUrls: ['./jury-evaluation.component.scss'],
})
export class JuryEvaluationComponent implements OnInit, OnDestroy {
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private hackathonService = inject(HackathonService);
  private authService = inject(AuthService);
  private notificationService = inject(NotificationService);
  private fb = inject(FormBuilder);

  currentUser = toSignal(this.authService.user$);
  hackathon = signal<FullHackathonResponse | null>(null);
  submissions = signal<JurySubmissionItem[]>([]);
  currentIndex = signal<number>(0);
  isLoading = signal(true);

  showCompletionScreen = signal(false);

  evalForm!: FormGroup;
  calculatedScore = signal<number>(0);
  draftStatus = signal<string>('No changes detected');

  private formListenerSub?: Subscription;

  currentSub = computed(() => {
    const subs = this.submissions();
    return subs.length > 0 ? subs[this.currentIndex()] : null;
  });

  scoredCount = computed(() => {
    const userId = this.currentUser()?.id;
    if (!userId) return 0;
    return this.submissions().filter(s =>
      s.otherScores.some(os => os.userId === userId),
    ).length;
  });

  otherJuryScores = computed(() => {
    const h = this.hackathon();
    const sub = this.currentSub();
    const myId = this.currentUser()?.id;
    if (!h || !sub) return [];

    return (h.jury || []).
      .filter(j => j.userId !== myId)
      .map(j => {
        const scoreDoc = sub.otherScores.find(os => os.userId === j.userId);
        return {
          name: j.username || 'Juror',
          avatarUrl: j.avatarUrl || undefined,
          score: scoreDoc ? scoreDoc.score : null,
        };
      });
  });

```



```

});

constructor() {
  effect(() => {
    const h = this.hackathon();
    if (h?.criteria && this.evalForm) {
      this.evalForm.valueChanges.subscribe(
        (values: Partial<EvaluationFormValues>) => {
          let total = 0;
          h.criteria.forEach((c) => {
            const val = Number(values[c.id] ?? 0);
            total += val * (c.weight / 100);
          });
          this.calculatedScore.set(total);
        },
      );
    }
  });
}

ngOnInit() {
  const slug = this.route.snapshot.paramMap.get('slug');
  if (slug) {
    this.loadData(slug);
  }
}

ngOnDestroy() {
  this.formListenerSub?.unsubscribe();
}

private loadData(slug: string) {
  this.hackathonService.getBySlug(slug).subscribe({
    next: (h) => {
      this.hackathon.set(h);
      this.buildForm(h.criteria);

      this.hackathonService.getJurySubmissions(h.id).subscribe({
        next: (res) => {
          this.submissions.set(res.submissions);
          this.isLoading.set(false);

          if (res.submissions.length > 0) {
            if (this.scoredCount() === res.submissions.length) {
              this.showCompletionScreen.set(true);
              this.selectTeam(0);
            } else {
              const firstUnscored = res.submissions.findIndex(
                (s) => !this.isTeamEvaluatedByMe(s),
              );
              this.selectTeam(firstUnscored !== -1 ? firstUnscored : 0);
            }
          }
        }
      });
    }
  });
}

```

```

    },
    error: (err) => {
      if (err.status === 403) {
        this.notificationService.error(
          'Access denied. You are not a jury member for this event.',
        );
      } else {
        this.notificationService.error(
          'Failed to load participant submissions.',
        );
      }
      this.isLoading.set(false);
      this.router.navigate(['/events', slug]);
    },
  });
},
error: () => {
  this.notificationService.error('Failed to fetch hackathon details');
  this.isLoading.set(false);
  this.router.navigate(['/events']);
},
});
}

private buildForm(criteria: CriterionResponse[]) {
  const group: Record<string, unknown> = {
    summary: ['', Validators.maxLength(800)],
    strengths: ['', Validators.maxLength(500)],
    weaknesses: ['', Validators.maxLength(500)],
  };
  criteria.forEach((c) => {
    group[c.id] = [0, [Validators.min(1), Validators.max(c.maxValue)]];
  });
  this.evalForm = this.fb.group(group);
}

isTeamEvaluatedByMe(sub: JurySubmissionItem): boolean {
  const myId = this.currentUser()?.id;
  if (!myId) return false;
  return sub.otherScores.some((os) => os.userId === myId);
}

selectTeam(index: number) {
  this.formListenerSub?.unsubscribe();
  this.currentIndex.set(index);

  const sub = this.currentSub();
  const h = this.hackathon();
  if (!sub || !h) return;

  this.evalForm.reset(
    { summary: '', strengths: '', weaknesses: '' },
    { emitEvent: false },
  );
}

```

```

);

const storageKey = `h_linker_draft:${h.id}:${sub.participationId}`;
const savedDraft = localStorage.getItem(storageKey);

if (savedDraft) {
  try {
    const parsedData = JSON.parse(savedDraft) as EvaluationFormValues;
    this.evalForm.patchValue(parsedData, { emitEvent: false });
    this.draftStatus.set('Unsaved draft loaded from local storage');
  } catch (e) {
    console.error('Failed to parse draft data', e);
  }
} else if (
  sub.submittedScores &&
  Object.keys(sub.submittedScores).length > 0
) {
  const dbValues: Record<string, unknown> = {
    summary: sub.submittedComment || '',
    strengths: sub.submittedStrengths || '',
    weaknesses: sub.submittedWeaknesses || '',
    ...sub.submittedScores,
  };
  this.evalForm.patchValue(dbValues, { emitEvent: false });
  this.draftStatus.set('Saved evaluation loaded from database');
} else {
  this.draftStatus.set('All changes saved to database');
}

this.updateCalculatedScore(this.evalForm.value as EvaluationFormValues);

this.formListenerSub = this.evalForm.valueChanges.subscribe(
  (values: Partial<EvaluationFormValues>) => {
    this.updateCalculatedScore(values);
    this.draftStatus.set('Typing...');
  },
);

this.formListenerSub.add(
  this.evalForm.valueChanges
    .pipe(debounceTime(800))
    .subscribe((values: Partial<EvaluationFormValues>) => {
      localStorage.setItem(storageKey, JSON.stringify(values));
      const timeString = new Date().toLocaleTimeString([], {
        hour: '2-digit',
        minute: '2-digit',
      });
      this.draftStatus.set(`Draft saved automatically at ${timeString}`);
    }),
);
}

private updateCalculatedScore(values: Partial<EvaluationFormValues>) {

```

```

const h = this.hackathon();
if (!h?.criteria) return;

let total = 0;
h.criteria.forEach((c) => {
  const val = Number(values[c.id] ?? 0);
  total += val * (c.weight / 100);
});
this.calculatedScore.set(total);
}

onSaveAndNext() {
  const h = this.hackathon();
  const sub = this.currentSub();
  if (!h || !sub || this.evalForm.invalid) return;

  const formValues = this.evalForm.value as EvaluationFormValues;
  const { summary, strengths, weaknesses, ...criteriaScores } = formValues;

  const scores: Record<string, number> = {};
  Object.entries(criteriaScores).forEach(([key, val]) => {
    scores[key] = Number(val ?? 0);
  });

  this.draftStatus.set('Saving evaluation...');

  const hasComment =
    (summary && summary.trim().length > 0) ||
    (strengths && strengths.trim().length > 0) ||
    (weaknesses && weaknesses.trim().length > 0);

  const commentPayload = { summary, strengths, weaknesses };

  forkJoin({
    score: this.hackathonService.submitScores(
      h.id,
      sub.participationId,
      scores,
    ),
    comment: hasComment
      ? this.hackathonService.submitComment(
          h.id,
          sub.participationId,
          commentPayload,
        )
      : of(null),
  }).subscribe({
    next: () => {
      const storageKey = `h_linker_draft:${h.id}:${sub.participationId}`;
      localStorage.removeItem(storageKey);

      this.notificationService.success(
        `Evaluation for team "${sub.teamName}" successfully saved`,
      );
    }
  });
}

```

```

);

const myId = this.currentUser()?.id;
const myName = this.currentUser()?.username;
if (myId && myName) {
  this.submissions.update((subs) =>
    subs.map((s, idx) =>
      idx === this.currentIndex()
        ? {
            ...s,
            submittedScores: scores,
            submittedComment: hasComment ? summary : "",
            otherScores: [
              ...s.otherScores.filter((os) => os.userId !== myId),
              {
                userId: myId,
                username: myName,
                avatarUrl: this.currentUser()?.avatarUrl ?? undefined,
                score: Number(this.calculatedScore().toFixed(1)),
              },
            ],
          }
        : s,
    ),
  );
}

if (this.scoredCount() === this.submissions().length) {
  this.showCompletionScreen.set(true);
} else {
  if (this.currentIndex() < this.submissions().length - 1) {
    this.nextTeam();
  } else {
    const firstUnscored = this.submissions().findIndex(
      (s) => !this.isTeamEvaluatedByMe(s),
    );
    if (firstUnscored !== -1) {
      this.selectTeam(firstUnscored);
      this.notificationService.info('Redirected to skipped project');
    }
  }
},
error: (err) => {
  this.notificationService.error(err.message);
  this.draftStatus.set('Error saving changes');
},
});
}

nextTeam() {
  if (this.currentIndex() < this.submissions().length - 1) {
    this.selectTeam(this.currentIndex() + 1);
  }
}

```

```

    }
  }

  prevTeam() {
    if (this.currentIndex() > 0) {
      this.selectTeam(this.currentIndex() - 1);
    }
  }

  getMathFormula(): string {
    const h = this.hackathon();
    if (!h) return '';
    return h.criteria
      .map((c) => `${c.weight}% x ${this.evalForm.get(c.id)?.value || 0}`)
      .join(' + ');
  }
}

```

Файл `hackathon-constructor.component.ts`

Реалізація функціональної задачі створення та конфігурування хакатонів.

```

import {
  Component,
  inject,
  signal,
  computed,
  ViewChild,
  OnInit,
  ElementRef,
  DestroyRef,
} from '@angular/core';
import { CommonModule } from '@angular/common';
import {
  FormBuilder,
  Validators,
  ReactiveFormsModule,
  FormArray,
  FormGroup,
  FormControl,
} from '@angular/forms';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { MatStepper, MatStepperModule } from '@angular/material/stepper';
import { MatInputModule } from '@angular/material/input';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { MatSelectModule } from '@angular/material/select';
import { COMMA, ENTER } from '@angular/cdk/keycodes';

import { takeUntilDestroyed, toSignal } from '@angular/core/rxjs-interop';
import {
  debounceTime,
  distinctUntilChanged,

```

```

filter,
forkJoin,
map,
of,
switchMap,
} from 'rxjs';

import { HackathonService } from '../services/hackathon.service';
import { UserService } from '../services/user.service';
import { SettingsSectionComponent } from '../settings/settings-section.component';
import {
  HackathonStatus,
  FullHackathonResponse,
  CreateHackathonDTO,
  UpdateHackathonDTO,
  CriterionDTO,
  UserResponse,
  JuryDisplay,
} from '@h.linker/libs';
import { NotificationService } from '../utils/notification.service';
import { HackathonTimelineComponent } from '../hackathon-timeline/hackathon-timeline.component';
import { ImageUploadService } from '../services/image-upload.service';
import { MatTooltip } from '@angular/material/tooltip';
import { HackathonWeightPreviewComponent } from '../hackathon-weight-preview.component';
import { CategoryService } from '../services/category.service';
import { MatChipInputEvent, MatChipsModule } from '@angular/material/chips';
import {
  MatAutocompleteModule,
  MatAutocompleteSelectedEvent,
} from '@angular/material/autocomplete';
import { StepperSelectionEvent } from '@angular/cdk/stepper';
import { AuthService } from '../services/auth.service';
import { BreadcrumbComponent } from '../breadcrumb/breadcrumb.component';
import { generateSlug } from '../utils/slug';

@Component({
  selector: 'app-hackathon-constructor',
  standalone: true,
  imports: [
    CommonModule,
    ReactiveFormsModule,
    MatStepperModule,
    MatInputModule,
    MatButtonModule,
    MatIconModule,
    MatSelectModule,
    SettingsSectionComponent,
    HackathonTimelineComponent,
    MatTooltip,
    MatChipsModule,
    MatAutocompleteModule,
    HackathonWeightPreviewComponent,
    RouterLink,
  ],
})

```

```

    BreadcrumbComponent,
  ],
  templateUrl: './hackathon-creator.component.html',
  styleUrls: ['./hackathon-creator.component.scss'],
})
export class HackathonCreatorComponent implements OnInit {
  currentStep = signal(0);
  @ViewChild('stepper') stepper!: MatStepper;
  @ViewChild('categoryInput') categoryInput!: ElementRef<HTMLInputElement>;

  private destroyRef = inject(DestroyRef);
  private fb = inject(FormBuilder);
  private hackathonService = inject(HackathonService);
  private categoryService = inject(CategoryService);
  private userService = inject(UserService);
  private notificationService = inject(NotificationService);
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private imageUploadService = inject(ImageUploadService);
  private authService = inject(AuthService);

  readonly separatorKeysCodes = [ENTER, COMMA] as const;

  categoryCtrl = new FormControl("");
  filteredCategories = toSignal(
    this.categoryCtrl.valueChanges.pipe(
      debounceTime(300),
      distinctUntilChanged(),
      filter((value) => !!value && value.length >= 2),
      switchMap((query) => this.categoryService.searchCategories(query || "")),
      map((res) => res.categories),
    ),
    { initialValue: [] as string[] },
  );

  jurySearchCtrl = new FormControl("");
  selectedJury = signal<JuryDisplay[]>([]);
  filteredUsers = toSignal(
    this.jurySearchCtrl.valueChanges.pipe(
      debounceTime(300),
      distinctUntilChanged(),
      switchMap((val) =>
        val && val.length >= 2
          ? this.userService
              .getAll({ search: val })
              .pipe(map((res) => res.data))
          : of([]),
      ),
    ),
    { initialValue: [] as UserResponse[] },
  );

  hackathonId = signal<string | null>(null);

```



```

isLinearStepper = computed(() => !this.hackathonId());
isSaving = signal(false);
currentUser = toSignal(this.authService.user$);

infoForm = this.fb.group({
  title: [
    "",
    [Validators.required, Validators.minLength(5), Validators.maxLength(100)],
  ],
  slug: ["", [Validators.required, Validators.maxLength(120)]],
  prize: ["", [Validators.maxLength(200)]],
  description: ["", [Validators.maxLength(2000)]],
  imageUrl: [""],
  registrationStartDate: [null as Date | null, Validators.required],
  startDate: [null as Date | null, Validators.required],
  submissionDeadline: [null as Date | null, Validators.required],
  endDate: [null as Date | null, Validators.required],
});

detailsForm = this.fb.group({
  categories: [[] as string[]],
  criteria: this.fb.array<FormGroup>([]),
});

statusForm = this.fb.group({
  status: [HackathonStatus.DRAFT, Validators.required],
});

readonly statusOptions = [
  {
    value: HackathonStatus.DRAFT,
    label: 'Draft',
    icon: 'edit_note',
    desc: 'Visible only to organizers.',
  },
  {
    value: HackathonStatus.REGISTRATION,
    label: 'Registration',
    icon: 'how_to_reg',
    desc: 'Participants can join teams.',
  },
  {
    value: HackathonStatus.ACTIVE,
    label: 'Active',
    icon: 'rocket_launch',
    desc: 'Hackathon is ongoing. Submissions open.',
  },
  {
    value: HackathonStatus.FINISHED,
    label: 'Finished',
    icon: 'emoji_events',
    desc: 'Event ended. Judging phase.',
  },
]

```

```

];

addCategory(event: MatChipInputEvent): void {
  const value = (event.value || "").trim();
  const current = this.detailsForm.controls.categories.value || [];
  if (value && !current.includes(value)) {
    this.detailsForm.controls.categories.setValue([...current, value]);
    this.detailsForm.markAsDirty();
  }
  event.chipInput?.clear();
  this.categoryCtrl.setValue(null);
}

removeCategory(category: string): void {
  const current = this.detailsForm.controls.categories.value || [];
  this.detailsForm.controls.categories.setValue(
    current.filter((c) => c !== category),
  );
  this.detailsForm.markAsDirty();
}

selectedCategory(event: MatAutocompleteSelectedEvent): void {
  const value = event.option.viewValue;
  const current = this.detailsForm.controls.categories.value || [];
  if (!current.includes(value)) {
    this.detailsForm.controls.categories.setValue([...current, value]);
    this.detailsForm.markAsDirty();
  }
  this.categoryInput.nativeElement.value = "";
  this.categoryCtrl.setValue(null);
}

addJury(event: MatAutocompleteSelectedEvent): void {
  const user = event.option.value as UserResponse;
  const id = this.hackathonId();

  if (!id) {
    this.notificationService.error("Save identity first!");
    return;
  }

  if (this.selectedJury().some((u) => u.id === user.id)) return;

  this.isSaving.set(true);
  this.hackathonService.addJury(id, { userId: user.id }).subscribe({
    next: () => {
      this.selectedJury.update((prev) => [
        ...prev,
        {
          id: user.id,
          username: user.username,
          avatarUrl: user.avatarUrl ?? undefined,
          firstName: user.firstName,

```

```

        lastName: user.lastName ?? undefined,
      },
    ]);
    this.isSaving.set(false);
    this.notificationService.success(`${user.username} added to jury`);
  },
  error: (err) => {
    this.isSaving.set(false);
    this.notificationService.error(err.message);
  },
});

this.jurySearchCtrl.setValue("");
}

removeJury(userId: string): void {
  const id = this.hackathonId();
  if (!id) return;

  this.isSaving.set(true);
  this.hackathonService.removeJury(id, userId).subscribe({
    next: () => {
      this.selectedJury.update((prev) => prev.filter((u) => u.id !== userId));
      this.isSaving.set(false);
      this.notificationService.success('Jury member removed');
    },
    error: (err) => {
      this.isSaving.set(false);
      this.notificationService.error(err.message);
    },
  });
}

private detailsValue = toSignal(this.detailsForm.valueChanges, {
  initialValue: this.detailsForm.getRawValue(),
});

get criteriaArray(): FormArray<FormGroup> {
  return this.detailsForm.get('criteria') as FormArray<FormGroup>;
}

totalWeight = computed(() => {
  const criteria = this.detailsValue().criteria || [];
  return criteria.reduce(
    (acc: number, curr: Partial<{ weight: number | null }>) =>
      acc + (Number(curr?.weight) || 0),
    0,
  );
});

ngOnInit(): void {
  const params = this.route.snapshot.queryParamMap;
  const id = params.get('id');

```

```

const step = params.get('step');

if (id) {
  this.loadHackathonData(id);
}

if (step) {
  this.currentStep.set(Number(step));
}

this.infoForm
  .get('title')
  ?.valueChanges.pipe(takeUntilDestroyed(this.destroyRef))
  .subscribe((title) => {
    const slugControl = this.infoForm.get('slug');

    if (!this.hackathonId() && slugControl && !slugControl.dirty) {
      slugControl.setValue(generateSlug(title || ''));
    }
  });
}

onStepChange(event: StepperSelectionEvent): void {
  this.currentStep.set(event.selectedIndex);

  this.router.navigate([], {
    relativeTo: this.route,
    queryParams: {
      step: event.selectedIndex,
      id: this.hackathonId(),
    },
    queryParamsHandling: 'merge',
  });
}

triggerFileInput(fileInput: HTMLInputElement): void {
  fileInput.click();
}

onFileSelected(event: Event): void {
  const input = event.target as HTMLInputElement;
  const file = input.files?.[0];
  if (!file) return;

  this.isSaving.set(true);
  this.imageUploadService.upload(file).subscribe({
    next: (imageUrl: string) => {
      this.infoForm.patchValue({ imageUrl });
      this.infoForm.get('imageUrl')?.markAsDirty();
      this.notificationService.success('Banner uploaded!');
      this.isSaving.set(false);
    },
    error: () => {

```

```

    this.notificationService.error('Upload failed');
    this.isSaving.set(false);
  },
});
}

getStatusDetail(value: HackathonStatus | null | undefined) {
  return this.statusOptions.find((opt) => opt.value === value);
}

private loadHackathonData(id: string): void {
  this.hackathonService.getById(id).subscribe({
    next: (h: FullHackathonResponse) => {
      const user = this.currentUser();

      if (!user || h.creator.id !== user.id) {
        this.notificationService.error(
          'Access denied. You are not the creator of this event.',
        );
        this.router.navigate(['/events']);
        return;
      }

      this.hackathonId.set(h.id);
      this.infoForm.patchValue({
        title: h.title,
        slug: h.slug,
        description: h.description,
        prize: h.prize || '',
        imageUrl: h.imageUrl,
        registrationStartDate: new Date(h.registrationStartDate),
        startDate: new Date(h.startDate),
        submissionDeadline: new Date(h.submissionDeadline),
        endDate: new Date(h.endDate),
      });

      this.statusForm.patchValue({ status: h.status });
      this.detailsForm.patchValue({ categories: h.categories });

      const juryData: JuryDisplay[] = (h.jury || []).map((j) => ({
        id: j.userId,
        username: j.username,
        avatarUrl: j.avatarUrl ?? undefined,
      }));
      this.selectedJury.set(juryData);

      this.criteriaArray.clear();
      h.criteria?.forEach((c) => {
        this.criteriaArray.push(
          this.fb.group({
            name: [c.name, Validators.required],
            weight: [
              c.weight,

```

```

        [Validators.required, Validators.min(1), Validators.max(100)],
    ],
    maxValue: [
        c.maxValue || 10,
        [Validators.required, Validators.min(1)],
    ],
    }),
    );
    });
    },
    error: () => this.notificationService.error('Failed to load data'),
    });
    }

```

```

addCriterion(): void {
    this.criteriaArray.push(
        this.fb.group({
            name: ['', Validators.required],
            weight: [
                10,
                [Validators.required, Validators.min(1), Validators.max(100)],
            ],
            maxValue: [10, [Validators.required, Validators.min(1)]],
        }),
    );
}

```

```

removeCriterion(index: number): void {
    this.criteriaArray.removeAt(index);
    this.detailsForm.markAsDirty();
}

```

```

saveIdentity stepper: MatStepper): void {
    if (this.infoForm.invalid) return;
    const raw = this.infoForm.getRawValue();
    const {
        title,
        slug,
        registrationStartDate,
        startDate,
        submissionDeadline,
        endDate,
    } = raw;

```

```

    if (
        !title ||
        !slug ||
        !registrationStartDate ||
        !startDate ||
        !submissionDeadline ||
        !endDate
    )
        return;

```

```

this.isSaving.set(true);
const payload: CreateHackathonDTO = {
  title,
  slug,
  description: raw.description ?? undefined,
  prize: raw.prize || undefined,
  imageUrl: raw.imageUrl ?? undefined,
  registrationStartDate: registrationStartDate.toISOString(),
  startDate: startDate.toISOString(),
  submissionDeadline: submissionDeadline.toISOString(),
  endDate: endDate.toISOString(),
};

const id = this.hackathonId();
const request = id
  ? this.hackathonService.update(id, payload as UpdateHackathonDTO)
  : this.hackathonService.create(payload);

request.subscribe({
  next: (res: FullHackathonResponse) => {
    this.hackathonId.set(res.id);

    this.isSaving.set(false);
    this.infoForm.markAsPristine();

    this.notificationService.success('Identity saved');
    stepper.next();
  },
  error: (err) => {
    this.isSaving.set(false);
    this.notificationService.error(err.message);
  },
});
}

displayFn(): string {
  return '';
}

saveDetails(stepper: MatStepper): void {
  const id = this.hackathonId();
  if (!id || this.totalWeight() !== 100) return;

  this.isSaving.set(true);
  const raw = this.detailsForm.getRawValue();

  forkJoin({
    categories: this.hackathonService.setCategories(id, {
      categories: raw.categories || [],
    }),
    criteria: this.hackathonService.setCriteria(id, {
      criteria: raw.criteria as CriterionDTO[],
    })
  })

```

```

    }),
  ).subscribe({
    next: () => {
      this.isSaving.set(false);
      this.detailsForm.markAsPristine();
      this.notificationService.success('Details saved successfully');
      stepper.next();
    },
    error: (err) => {
      this.isSaving.set(false);
      this.notificationService.error(err.message);
    },
  });
}

```

```

updateStatus(): void {
  const id = this.hackathonId();
  const newStatus = this.statusForm.value.status;
  if (!id || !newStatus) return;

```

```

  this.isSaving.set(true);
  this.hackathonService.updateStatus(id, newStatus).subscribe({
    next: () => {
      this.isSaving.set(false);
      this.statusForm.markAsPristine();

      const option = this.statusOptions.find(
        (opt) => opt.value === newStatus,
      );
      const label = option ? option.label : newStatus;
      this.notificationService.success(
        `Hackathon status updated to: ${label}`,
      );
    },
    error: () => {
      this.isSaving.set(false);
      this.notificationService.error('Update failed');
    },
  });
}

```

```

goToView(): void {
  const slug = this.infoForm.get('slug')?.value;
  if (slug) {
    this.router.navigate(['/events', slug]);
  } else {
    this.notificationService.error(
      'Slug is missing, cannot navigate to view',
    );
  }
}
}

```


Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

«ЗАТВЕРДЖЕНО»

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ВЕБЗАСТОСУНОК ДЛЯ ОРГАНІЗАЦІЇ ХАКАТОНІВ З
ДИНАМІЧНИМ МОДУЛЕМ ОЦІНЮВАННЯ**

Програма та методика тестування

КПІ.ПІ-2419.045440.04.51

«ПОГОДЖЕНО»

Керівник проєкту:

_____ Олена МАРЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

Виконавець:

_____ Олександр МАЛИЙ

Київ – 2026

ЗМІСТ

1	ОБ'ЄКТ ВИПРОБУВАНЬ.....	3
2	МЕТА ТЕСТУВАННЯ	4
3	МЕТОДИ ТЕСТУВАННЯ.....	5
4	ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ.....	7

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є вебзастосунок для організації хакатонів з динамічним модулем оцінювання «h.linker». Програмне забезпечення функціонує в архітектурі клієнт-сервер та складається з двох основних підсистем: клієнтської частини (frontend), реалізованої на базі фреймворку Angular, та серверної частини (backend), побудованої за допомогою фреймворку NestJS з використанням середовища виконання Node.js.

Розроблене програмне забезпечення є кросплатформним вебзастосунком. Доступ до системи здійснюється через мережу Інтернет за допомогою стандартних веббраузерів. Цільовими платформами для коректної роботи клієнтської частини є сучасні браузери, зокрема:

- Google Chrome;
- Mozilla Firefox;
- Microsoft Edge;
- Safari.

Застосунок розрахований на роботу під управлінням десктопних операційних систем (Windows, macOS, Linux). Серверна частина розгортається у середовищі, що підтримує Node.js та має доступ до системи управління базами даних PostgreSQL.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка коректності обробки та надійності збереження даних;
- перевірка кросбраузерної сумісності вебзастосунку з останніми версіями сучасних браузерів;
- перевірка сумісності клієнтської частини застосунку з різними десктопними операційними системами (Windows, macOS, Linux);
- знаходження проблем, логічних помилок з метою їх усунення;
- перевірка інтуїтивності графічного інтерфейсу користувача;
- перевірка безпеки системи, включаючи механізми авторизації та коректність розмежування прав доступу для різних ролей;
- перевірка коректності роботи специфічної бізнес-логіки: динамічного модуля оцінювання, підрахунку турнірної таблиці (лідерборду) та алгоритму системи персональних рекомендацій на основі навичок;
- перевірка надійності механізмів валідації вхідних даних;
- перевірка підтримуваності, структурованості та чистоти коду;
- перевірка продуктивності серверної частини, пропускну здатності API та стабільності циклу подій (Event Loop) під час пікових навантажень;
- перевірка якості оптимізації клієнтського вебзастосунку (Angular SPA), швидкості завантаження, доступності (Accessibility) та дотримання стандартів SEO;
- підтвердження стабільності роботи бекенд-сервісів та клієнтських компонентів шляхом виконання автоматизованих модульних тестів (юніт-тестів).

3 МЕТОДИ ТЕСТУВАННЯ

Відповідно до піраміди тестування та архітектури платформи (монорепозиторій Nx, клієнтська частина на Angular, серверна на NestJS), було обрано комплексний підхід, що поєднує автоматизоване модульне (юніт) тестування нижнього рівня та мануальне тестування верхнього рівня.

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – перевіряється програма без виконання її коду. Здійснюється аналіз на предмет дотримання стандартів програмування за допомогою інструменту ESLint (перевірка типізації TypeScript, заборона невизначених змінних, контроль цикломатичної складності функцій);
- динамічне тестування – застосовується в процесі виконання програми. Коректність програмного засобу перевіряється під час запуску автоматизованих скриптів фреймворку Jest та під час мануального прогону користувацьких сценаріїв, збираючи дані про відхилення чи помилки;
- функціональне тестування – полягає у перевірці відповідності реальної поведінки програмного забезпечення очікуваній. Зокрема, перевіряється правильність роботи бізнес-логіки: процес авторизації, життєвий цикл хакатону, алгоритми системи персональних рекомендацій, робота динамічного модуля оцінювання та формування лідерборду;
- мануальне тестування – тестування без використання автоматизації. Здійснюється ручним виконанням попередньо підготовлених тест-кейсів, спрямованих на перевірку роботи інтерфейсу, безпеки (негативні тести доступу) та валідації даних;
- тестування «чорної скриньки» – об'єктом тестування тут є функції присутні у програмі. Перевіряється коректність вихідних даних при заданих вхідних без заглиблення у програмний код;

- тестування «білої скриньки» – об'єктом тестування тут є внутрішня поведінка програми. Метод реалізований через написання 293 юніт-тестів за допомогою інструментарію Jest для перевірки коректності побудови бекенд-сервісів, ключових ендпоінтів API, сервісів роботи з базою даних Prisma та ізольованих компонентів клієнтської частини;

- навантажувальне тестування (стрес-тестування) – перевірка поведінки системи під час симуляції масового паралельного доступу користувачів до платформи;

- профілювання продуктивності – аналіз використання пам'яті (Garbage Collector) та швидкодії асинхронних операцій вводу/виводу середовища Node.js.

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Під час проведення тестування будуть використовуватись наступні допоміжні засоби:

- фреймворк тестування Jest (для автоматизованої перевірки бізнес-логіки бекенду NestJS, а також для генерації HTML-звітів покриття коду);
- інструмент статичного аналізу коду ESLint;
- утиліта для генерації навантаження та стрес-тестування Autocannon;
- діагностичний інструмент профілювання мікросервісів Clinic.js;
- інструмент автоматизованого аудиту вебзастосунків Google Lighthouse;
- інструментарій управління монорепозіторієм Nx;
- сучасні веббраузери, такі як Google Chrome, Firefox, Safari тощо;
- сторонні сервіси та API, зокрема GitHub;
- система управління базами даних PostgreSQL та ORM Prisma.

Порядок проведення тестування буде наступним:

- підготовка тестового середовища, розгортання проєкту та підключення до бази даних;
- проведення статичного аналізу кодової бази;
- проведення навантажувального тестування бекенду з одночасним профілюванням циклу подій та використання оперативної пам'яті;
- виконання автоматизованого аудиту продуктивності, доступності та SEO ключових сторінок клієнтського застосунку;

- виконання автоматизованого модульного тестування за допомогою фреймворку Jest (перевірка ключових ендпоінтів API та сервісів роботи з базою даних);
- генерація та аналіз детального звіту покриття коду тестами у форматі HTML;
- динамічне мануальне тестування на відповідність функціональним вимогам згідно з розробленими сценаріями: процеси реєстрації хакатону, створення команди, подачі роботи, оцінювання та роботи лідерборду;
- тестування алгоритмів: перевірка системи персональних рекомендацій на основі навичок;
- тестування механізмів валідації та безпеки, включаючи негативні сценарії: перевірка блокування збереження невалідних критеріїв оцінювання та перевірка розмежування прав доступу до панелі журі після завершення змагання.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

«ЗАТВЕРДЖЕНО»

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ВЕБЗАСТОСУНОК ДЛЯ ОРГАНІЗАЦІЇ ХАКАТОНІВ З
ДИНАМІЧНИМ МОДУЛЕМ ОЦІНЮВАННЯ**

Керівництво користувача

КПІ.ПІ-2419.045440.05.34

«ПОГОДЖЕНО»

Керівник проєкту:

_____ Олена МАРЧЕНКО

Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

Виконавець:

_____ Олександр МАЛИЙ

Київ – 2026

ЗМІСТ

1	ПРИЗНАЧЕННЯ ПРОГРАМИ	3
2	ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ....	4
2.1	Системні вимоги для коректної роботи.....	4
2.2	Завантаження застосунку	5
2.3	Перевірка коректної роботи.....	7
3	ВИКОНАННЯ ПРОГРАМИ	9

1 ПРИЗНАЧЕННЯ ПРОГРАМИ

Вебзастосунок «h.linker» – це спеціалізована платформа, призначена для комплексної організації та проведення хакатонів, IT-змагань та різноманітних освітніх конкурсів. Його головне призначення полягає у наданні організаторам, розробникам та журі єдиного інтерактивного середовища для реєстрації, автоматизованого формування команд на основі навичок, зручної подачі проєктів та їх об'єктивного оцінювання за допомогою динамічно налаштовуваного математичного рушія. Очікувані сфери застосування охоплюють: університетські олімпіади з програмування, корпоративні воркшопи, глобальні IT-хакатони та будь-які інші проєктні заходи.

Кінцева збірка програмного забезпечення постачається у вигляді клієнту, серверу та БД, повністю підготовлених для розгортання на хмарних серверах за допомогою технології Docker. Вміст репозиторію проєкту організовано за принципом монорепозиторію з використанням інструментарію Nx. Він містить вихідний код клієнтської частини (Angular), серверної частини (NestJS), конфігураційні файли бази даних (схеми та міграції Prisma ORM), а також набір автоматизованих скриптів CI/CD. Інсталяційна версія застосунку не вимагає завантаження та встановлення виконуваних файлів на персональні комп'ютери користувачів. Програмне забезпечення функціонує як хмарний вебсервіс, доступ до якого здійснюється через браузер за відповідним доменним іменем.

2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ

2.1 Системні вимоги для коректної роботи

Для успішної та безперебійної роботи з вебзастосунком «h.linker» необхідне виконання ряду вимог щодо апаратного та програмного забезпечення. Оскільки система побудована за клієнт-серверною архітектурою та функціонує у хмарному середовищі, системні вимоги чітко розділяються на дві категорії: для кінцевого користувача та для середовища розгортання.

Для безпосереднього користування платформою зі сторони клієнта не потрібно потужного обладнання, оскільки всі обчислення виконуються на стороні сервера. Для повноцінної роботи користувачу необхідно лише:

- наявність будь-якого пристрою із встановленим сучасним браузером з повною підтримкою стандартів HTML5 і JavaScript;
- стабільне підключення до мережі Інтернет;

Натомість для розгортання, хостингу серверної частини застосунку та стабільної роботи комплексу Docker-контейнерів (модулів NestJS, бази даних PostgreSQL та вебсервера Nginx) потрібні виділені апаратні ресурси. Програмне забезпечення сервера повинно функціонувати на IBM-сумісних комп'ютерах або хмарних віртуальних машинах.

Мінімальна конфігурація технічних засобів для хостингу:

- операційна система: Windows 10 (дистрибутиви Linux/macOS);
- об'єм ОЗП: 4 Гб;
- браузер: наявність встановленого сучасного веббраузера;
- підключення до мережі Інтернет зі швидкістю від 20 Мбіт/с.

Рекомендована конфігурація технічних засобів для хостингу для забезпечення максимальної швидкодії системи:

- операційна система: Windows 10 (або новіша);

- тип процесору: Ryzen 7 5700X або його аналоги
- об'єм ОЗП: 32 Гб;
- браузер: найновіші версії браузерів (Google Chrome, Microsoft Edge, Opera) з повною підтримкою стандартів HTML5 і JavaScript;
- підключення до мережі Інтернет зі швидкістю від 100 Мбіт/с.

2.2 Завантаження застосунку

Оскільки вебзастосунок «h.linker» побудовано за хмарною клієнт-серверною архітектурою, процес отримання доступу до програмного забезпечення суттєво відрізняється залежно від технічної ролі особи (кінцевий користувач або розробник).

Для кінцевих користувачів (учасників хакатонів, членів журі та організаторів) відсутня необхідність завантажувати будь-які інсталяційні або виконувані файли на свій персональний пристрій. Робота з платформою здійснюється виключно через інтерфейс веббраузера. Для початку роботи достатньо мати підключення до мережі Інтернет та перейти за офіційною безпечною адресою розгорнутого застосунку: <https://hlinker.polandcentral.cloudapp.azure.com>.

Для системних адміністраторів, які планують розгорнути власну копію платформи на сервері, та для програмістів, які бажають дослідити чи модифікувати вихідний код, усі програмні файли системи доступні у системі контролю версій. Повна збірка програмного забезпечення, що включає вихідний код клієнтської та серверної частин, а також конфігураційні файли Docker, розміщена в офіційному репозиторії на платформі GitHub.

Щоб завантажити вихідний код на локальну машину або робочий сервер, необхідно мати встановлену утиліту Git. Отримання повної структури проєкту здійснюється шляхом виконання наступної команди у терміналі:

```
git clone https://github.com/JessFreak/h.linker hlinker
```

Після успішного виконання цієї команди у цільовій директорії буде створено папку `hlinker`, яка міститиме всі необхідні файли для подальшого налаштування та розгортання платформи за допомогою `Docker Compose`.

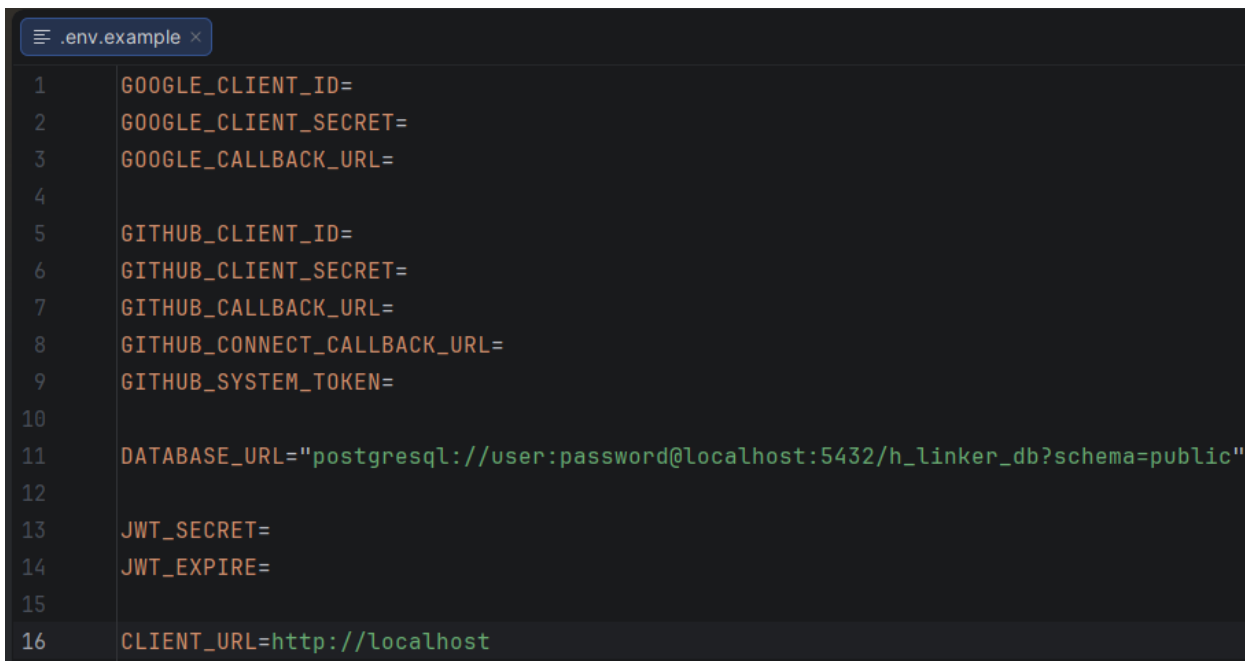
Перед безпосереднім запуском системи необхідно виконати конфігурацію файлів середовища. Насамперед налаштовується клієнтська частина платформи. Для цього у файлі за шляхом `apps/frontend/src/environments/environment.ts` необхідно вказати базові параметри підключення до API та інтеграції зі стороннім сервісом завантаження зображень `ImgBB` (рис. 2.1).



```
1 export const environment = {
2   production: false,
3   apiUrl: '/api',
4   imgbbApiKey: '',
5   imgbbApiUrl: 'https://api.imgbb.com/1/upload',
6 };
7
```

Рисунок 2.1 – Файл `environment.ts`

Далі необхідно налаштувати серверні секрети та ключі безпеки. У кореневій директорії проєкту слід створити файл конфігурації з назвою `.env`, який за замовчуванням ігнорується системою контролю версій для збереження конфіденційності даних. У цьому файлі заповнюються дані для підключення до бази даних `PostgreSQL`, криптографічні ключі для генерації `JWT`-токенів, а також секретні ключі (`Client ID` та `Client Secret`), отримані після реєстрації `OAuth`-застосунків у сервісах `Google` та `GitHub` (рис. 2.2).



```
.env.example x
1  GOOGLE_CLIENT_ID=
2  GOOGLE_CLIENT_SECRET=
3  GOOGLE_CALLBACK_URL=
4
5  GITHUB_CLIENT_ID=
6  GITHUB_CLIENT_SECRET=
7  GITHUB_CALLBACK_URL=
8  GITHUB_CONNECT_CALLBACK_URL=
9  GITHUB_SYSTEM_TOKEN=
10
11  DATABASE_URL="postgresql://user:password@localhost:5432/h_linker_db?schema=public"
12
13  JWT_SECRET=
14  JWT_EXPIRE=
15
16  CLIENT_URL=http://localhost
```

Рисунок 2.2 – Файл .env.example

Після успішного заповнення всіх конфігураційних параметрів виконується фінальний етап розгортання. Знаходячись у кореневій директорії проєкту, системний адміністратор повинен виконати в терміналі команду `docker compose up -d`. Ця команда автоматично завантажить необхідні образи, ізолює середовище, застосує міграції бази даних та запустить усі мікросервіси платформи у фоновому режимі, після чого застосунок стане повністю готовим до використання.

2.3 Перевірка коректної роботи

Оскільки вебзастосунок функціонує у хмарному середовищі, критерії перевірки його коректної роботи відрізняються для кінцевих користувачів та системних адміністраторів.

Для кінцевого користувача перевірка працездатності полягає у переході за офіційною вебадресою платформи через будь-який браузер. Якщо у вікні успішно завантажується головна сторінка застосунку з відображенням загальної інформації, кнопок авторизації та активних івентів, це свідчить про готовність системи до роботи та наявність стабільного з'єднання. У разі довгого завантаження або відображення системної помилки браузера, користувачу слід перевірити власне підключення до мережі Інтернет.

Для адміністратора, який виконував розгортання платформи на сервері або локальній машині, перевірка здійснюється у два етапи. Спочатку в терміналі необхідно виконати команду `docker ps`. У виведеному списку активних процесів мають відображатися запущеними три ключові контейнери (база даних, бекенд-сервер та фронтенд-сервер) зі статусом «Up». Якщо якийсь із контейнерів постійно перезавантажується, це свідчить про помилку конфігурації у файлі `.env`. Після успішної перевірки контейнерів адміністратору необхідно відкрити веббраузер та перейти за адресою локального хоста або налаштованим доменним іменем. Успішне завантаження стартової сторінки платформи без відображення помилок сервера остаточно підтверджує, що процес розгортання відбувся успішно.

Окрім цього, адміністратору настійно рекомендується додатково перевірити коректність роботи модулів авторизації через сторонні сервіси GitHub та Google. Оскільки ці функції взаємодіють із зовнішніми API, їхня працездатність є відносно незалежною від базового стану серверів платформи. У разі виникнення помилок під час спроби входу (наприклад, помилки перенаправлення або повідомлення про недійсного клієнта), це однозначно свідчить про некоректно задані параметри ключів безпеки у файлі `.env` або про помилку в налаштуваннях дозволених маршрутів переспрямування (Callback URL) безпосередньо на самих платформах Google Cloud Console чи GitHub Developer Settings.

3 ВИКОНАННЯ ПРОГРАМИ

Після переходу за адресою вебзастосунку користувач потрапляє на головну сторінку платформи «h.linker» (рис. 3.1). Вона містить базову інформацію про можливості системи, версію продукту та статистичні показники кількості студентів, команд, хакатонів і проєктів.

Для взаємодії із системою передбачено дві основні кнопки: «Get Started» (для переходу до реєстрації) та «Sign In» (для авторизації зареєстрованих користувачів). Нижня частина сторінки містить три інформаційні картки («Team Management», «GitHub Sync», «Hackathons»), які коротко описують ключові модулі платформи.

У самому низу посадкової сторінки (рис. 3.1) реалізовано панель статистичних даних, яка відображає кількісні показники залученості користувачів.

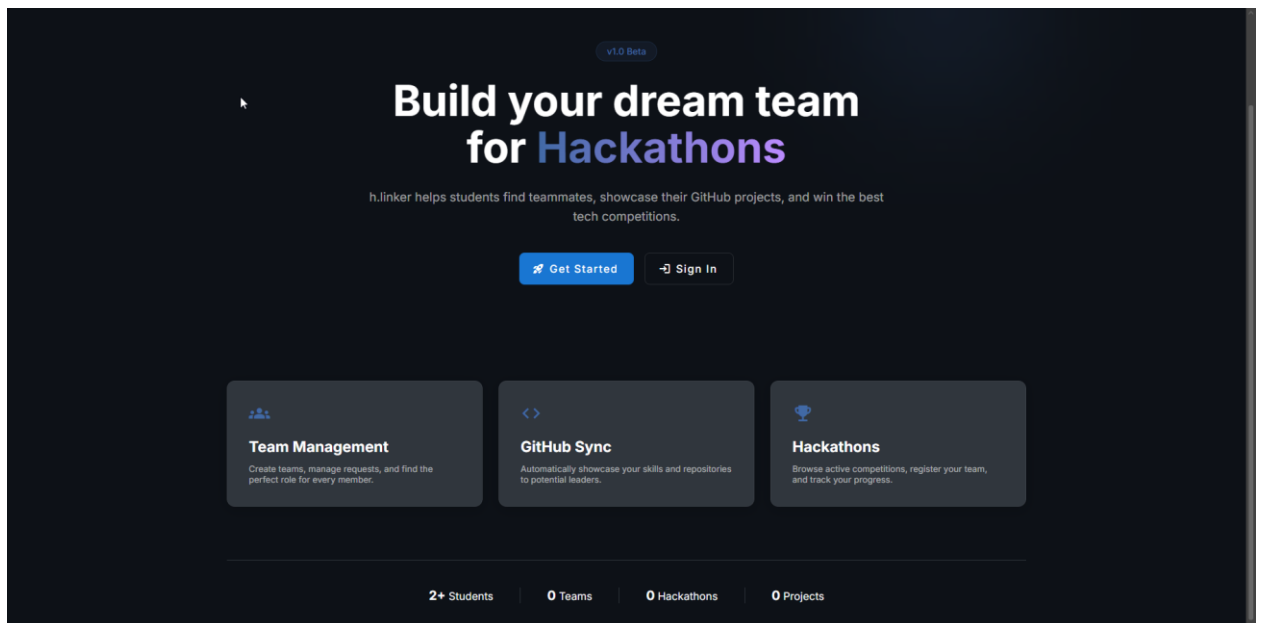


Рисунок 3.1 – Головна сторінка

На сторінці реєстрації (рис. 3.2) користувач має можливість створити новий обліковий запис. У верхній частині форми розташовані кнопки швидкої авторизації через сторонні сервіси: «Continue with GitHub» та «Continue with Google».

Для стандартної реєстрації передбачено текстові поля для введення імені, прізвища, унікального імені користувача, електронної пошти та пароля. Поля, що є обов'язковими для заповнення, позначені символом зірочки (*). Введення пароля підтримує режим приховування або відображення символів. Після заповнення всіх необхідних даних створення акаунту завершується натисканням кнопки «Create Account». Якщо користувач уже має обліковий запис, він може перейти до форми входу за допомогою посилання «Login instead» у самому низу картки.

The image shows a web browser displaying the h.linker registration page. The header includes the h.linker logo, navigation links for Events, Teams, and Showcase, a search bar, and links for Sign In and Sign Up. The main content area features a central registration card with the h.linker logo and tagline 'Hackathon Management System'. The card has 'Sign In' and 'Sign Up' buttons, followed by social login options for GitHub and Google. Below these are input fields for First Name, Last Name, Username, Email, and Password, each marked with an asterisk to indicate they are required. A prominent orange 'Create Account' button is at the bottom of the form, with a link 'Login instead' below it.

Рисунок 3.2 – Форма реєстрації

Форма входу в акаунт (рис. 3.3) призначена для автентифікації вже зареєстрованих користувачів. Для швидкого доступу у верхній частині картки розміщено кнопки авторизації через сторонні сервіси GitHub та Google.

Альтернативно користувач може заповнити стандартні текстові поля для введення логіна та пароля. Після заповнення даних автентифікація завершується натисканням кнопки «Sign In». Для переходу до створення нового облікового запису призначено посилання «Register instead» у нижній частині інтерфейсу.

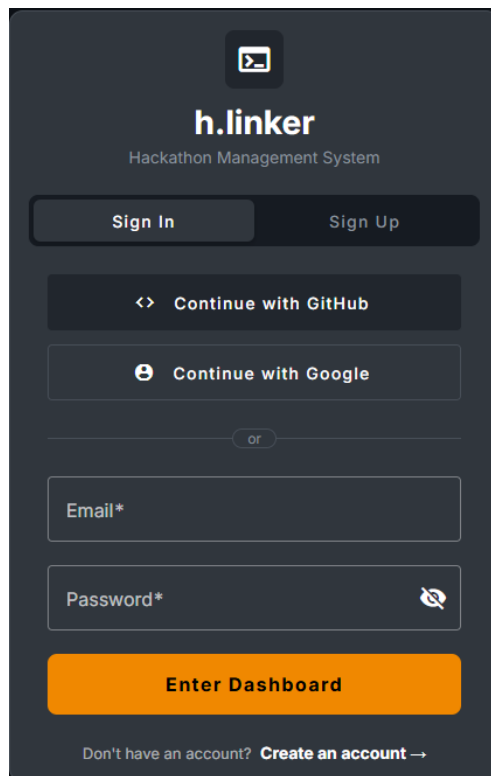


Рисунок 3.3 – Форма авторизації

Після заповнення форми автентифікації та натискання кнопки «Enter Dashboard» система виконує перевірку внесених даних. У разі успішної перевірки у правому нижньому кутку екрана з'являється спливаюче сповіщення «Logged in successfully» (рис. 3.4). Також хедер сайту оновлюється з даними користувача.

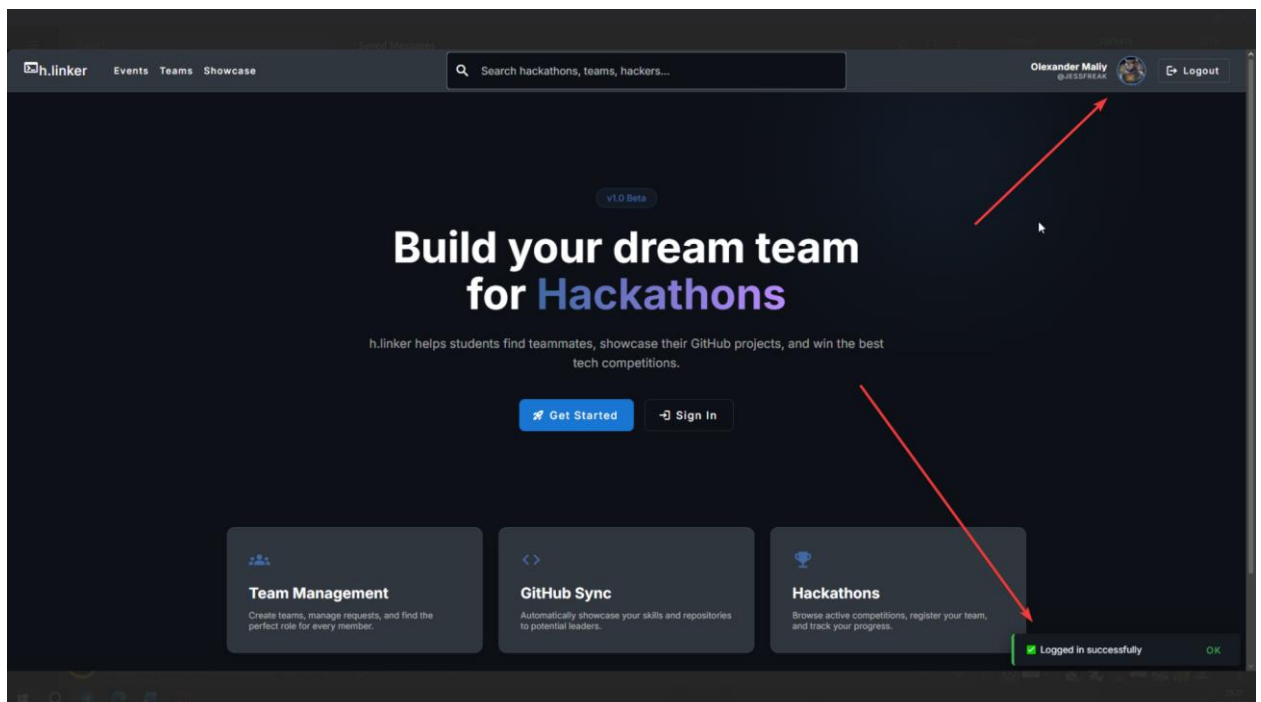


Рисунок 3.5 – Сповіщення про успішну авторизацію

Аналогічним чином у системі реалізовано зворотний зв'язок для всіх інших операцій. Будь-які успішні дії або помилки валідації супроводжуються подібними динамічними тоаст-повідомленнями у правому нижньому кутку екрана, що дозволяє користувачу миттєво фіксувати результат виконання поточного процесу.

Особистий профіль користувача (рис. 3.5) відображає зведену інформацію про розробника. У лівій панелі містяться аватар, ім'я, університет, перелік навичок, а також кнопка «Manage Profile» для редагування даних.

Основну частину сторінки займає аналітичний блок «GitHub Insights», який автоматично підтягує реальну статистику з акаунту GitHub: загальну кількість контрибуцій, отримані зірки, публічні репозиторії та діаграму основних мов програмування. Нижня частина сторінки містить вкладки для перегляду створених користувачем хакатонів, команд та поданих проєктів.

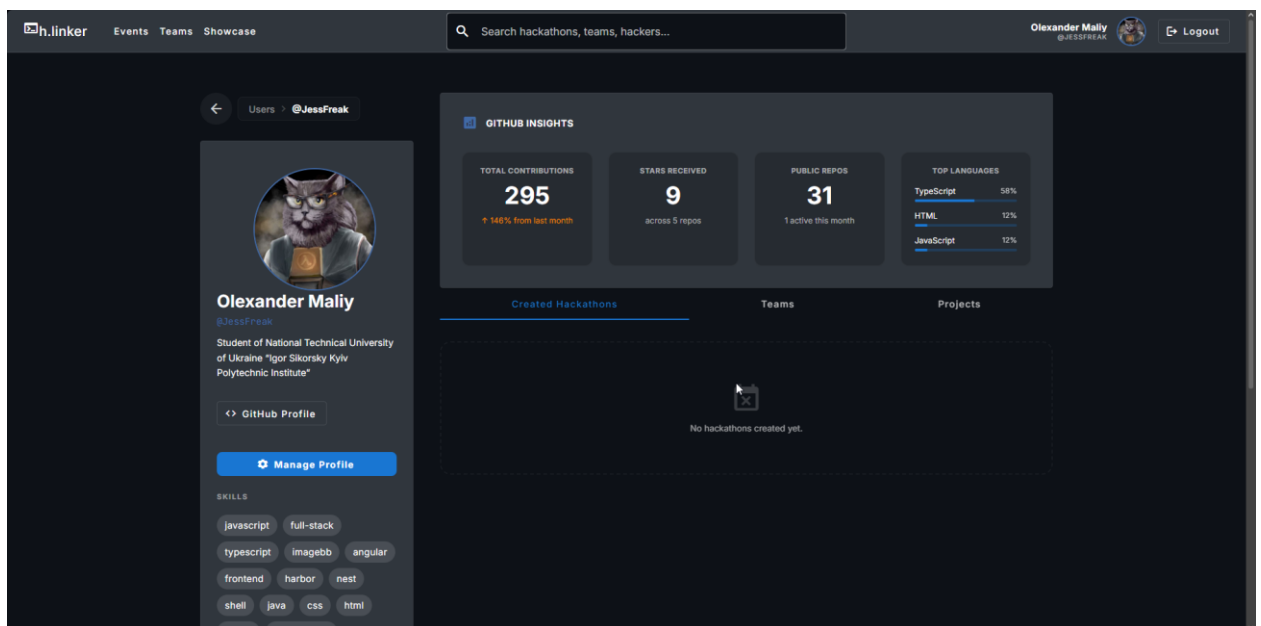


Рисунок 3.5 – Публічний профіль користувача

На сторінці налаштувань «Profile Settings» (рис. 3.6) користувач може редагувати персональні дані. У блоці «Public Profile» доступна зміна імені, прізвища, імені користувача та текстового опису про себе. Кнопка «Upload photo» дозволяє завантажити нове зображення профілю. Ліве меню забезпечує навігацію між категоріями налаштувань, а блок «Integrations» відображає

статус підключених сторонніх сервісів. Для збереження змін призначена кнопка «Save Changes».

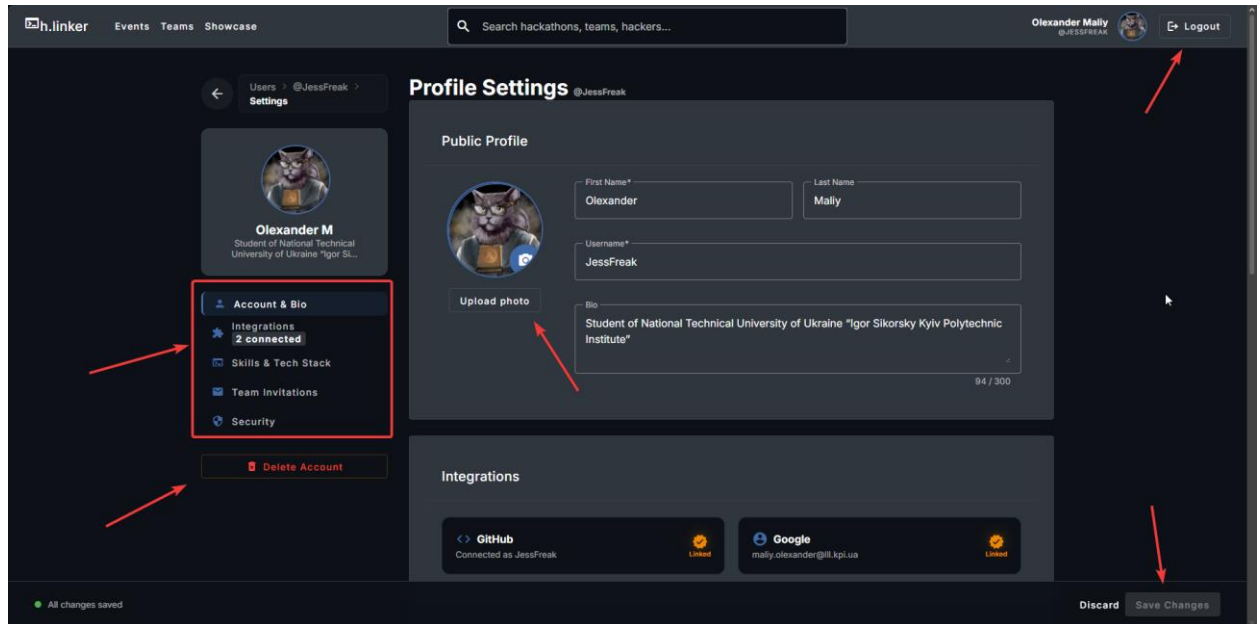


Рисунок 3.6 – Налаштування профілю користувача

Нижче на цій же сторінці або через відповідні вкладки меню (рис. 3.7) реалізовано керування технічними навичками та безпекою. У блоці «Your Skills & Tech Stack» користувач може додавати нові теги технологій через інтерактивний рядок вводу або видаляти наявні. Розділ «Team Invitations» відображає поточний статус запрошень до команд. Блок «Security & Password» призначений для зміни пароля, де після заповнення поточного та нового паролів необхідно натиснути кнопку «Update Password».

Для завершення поточної сесії користувач може скористатися кнопкою «Logout» у правому верхньому кутку панелі навігації (рис. 3.6). Окрім цього, на сторінці налаштувань профілю доступна функція повного видалення облікового запису за допомогою кнопки «Delete Account».

При натисканні на неї система відкриває модальне вікно підтвердження (рис. 3.8), яке попереджає про незворотність даної дії та втрату всіх пов'язаних даних розробника. Користувач може скасувати операцію кнопкою «Cancel» або безповоротно видалити профіль, натиснувши «Yes, Delete».

Your Skills & Tech Stack

Tech Stack

javascript full-stack typescript imagebb angular frontend harbor

nest shell java css html nestjs google-auth dockerfile

react admin-panel scss plpgsql handlebars system-modeling

hotel-management-system express jwt mongodb backend mongoose

microservices svelte python php nodejs rabbitmq postgresql

nextjs

Add skill (e.g. Angular, Python, AWS)...

Search for technologies or type your own and press Enter

Team Invitations

You have no pending invitations at the moment.

Security & Password

Current Password*

New Password* Confirm Password*

Update Password

Рисунок 3.7 – Нижня частина налаштувань профілю користувача

Delete Account

Are you sure you want to delete your account?
This action is permanent and all your data (skills, integrations, bio) will be lost forever.

Cancel Yes, Delete

Рисунок 3.8 – Сповіднення перед видаленням акаунту

Головна навігаційна панель та система навігаційних ланцюжків (рис. 3.9) забезпечують переміщення між основними розділами платформи двома незалежними способами.

Перший спосіб передбачає використання верхнього навігаційного меню, де за допомогою прямих кліків користувач може миттєво перемикатися між ключовими сторінками системи: переглядом хакатонів («Events»), списком сформованих команд («Teams») та каталогом успішних проєктів («Showcase»).

Другий спосіб базується на системі «хлібних крихт», яка інтегрована у верхній лівій частині робочої області на кожній сторінці платформи. Вони динамічно відображають поточний шлях користувача в ієрархії інтерфейсу, дозволяючи швидко повернутися на один або кілька логічних кроків назад, а розташована поруч кнопка зі стрілкою виконує стандартну функцію повернення до попереднього екрана.

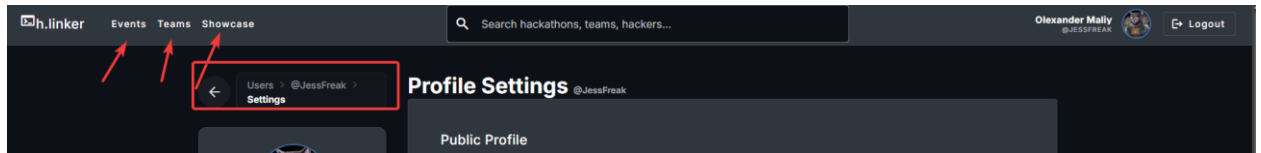


Рисунок 3.9 – Доступні варіанти навігації

Сторінка перегляду хакатонів «Events» (рис. 3.10) містить каталог усіх створених у системі змагань. У верхній частині відображається головний банер актуальної події з кнопкою «View Details» для перегляду її деталей.

Основну частину екрана займає сітка карток доступних заходів («Available Events»), де кожна картка містить логотип, назву, дату проведення, призовий фонд, теги технологій та поточний статус івенту. Для пошуку змагань у лівій панелі реалізовано блок фільтрації за назвою, стадією життєвого циклу, датами проведення та категоріями технологій, яка застосовується кнопкою «Apply Filters». Користувачі з правами організатора також мають доступ до верхньої кнопки «+ Create Hackathon» для створення нової події.

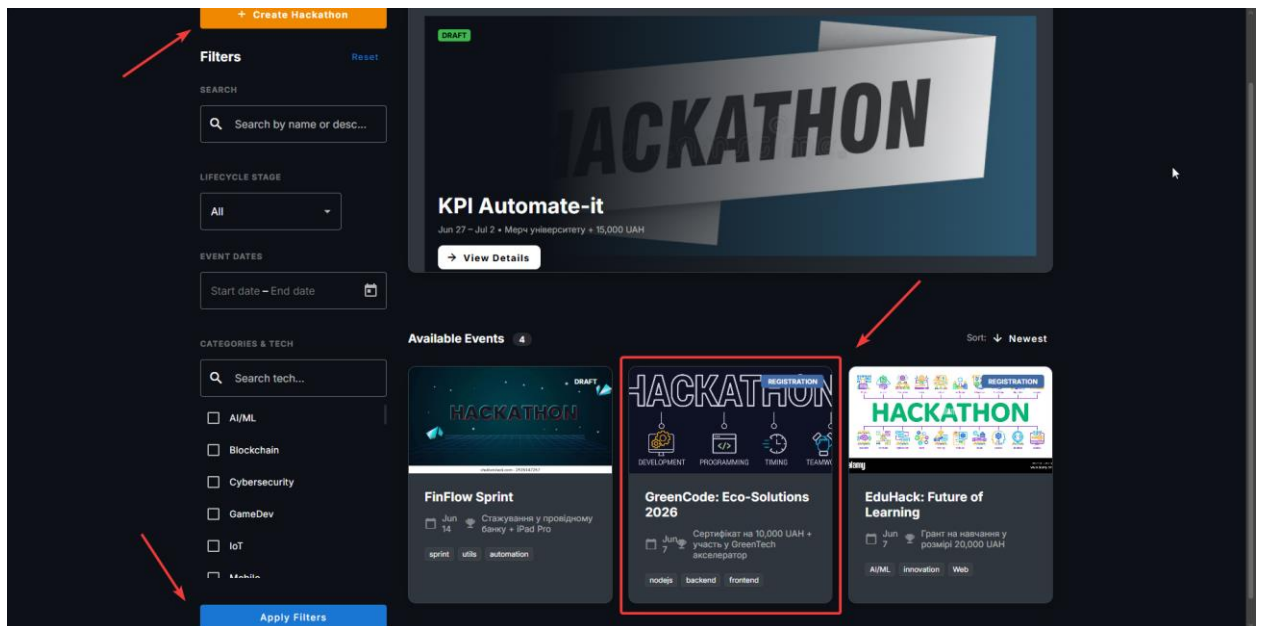


Рисунок 3.10 – Сторінка івентів

Сторінка детальної інформації про хакатон (рис. 3.11) відображає всі ключові дані обраного змагання. У верхній частині розміщено головний банер, назву заходу, дати проведення та призовий фонд. Для зручної навігації деталями події передбачено три інформаційні вкладки: «About» (загальний опис, нагороди та критерії оцінювання), «Timeline» (графік проведення) та «Jury» (склад журі).

У правій бічній панелі розташовано таймер зворотного відліку та блок взаємодії для учасників. Він містить кнопки «Register for Event» для подачі заявки на участь, «Find a Team» для пошуку команди та посилання «Live Standings» для перегляду поточної турнірної таблиці.

Крім того, якщо сторінку переглядає користувач із правами організатора, у верхній частині інтерфейсу відображаються додаткові кнопки керування: «Jury Workspace» для переходу в систему оцінювання робіт, «Event Insights» для перегляду аналітики змагання та «Edit Event» для редагування конфігурації хакатону. Перехід до редагування хакатону переносить користувача до заповненої сторінки створення хакатону (рис. 3.12-3.16).

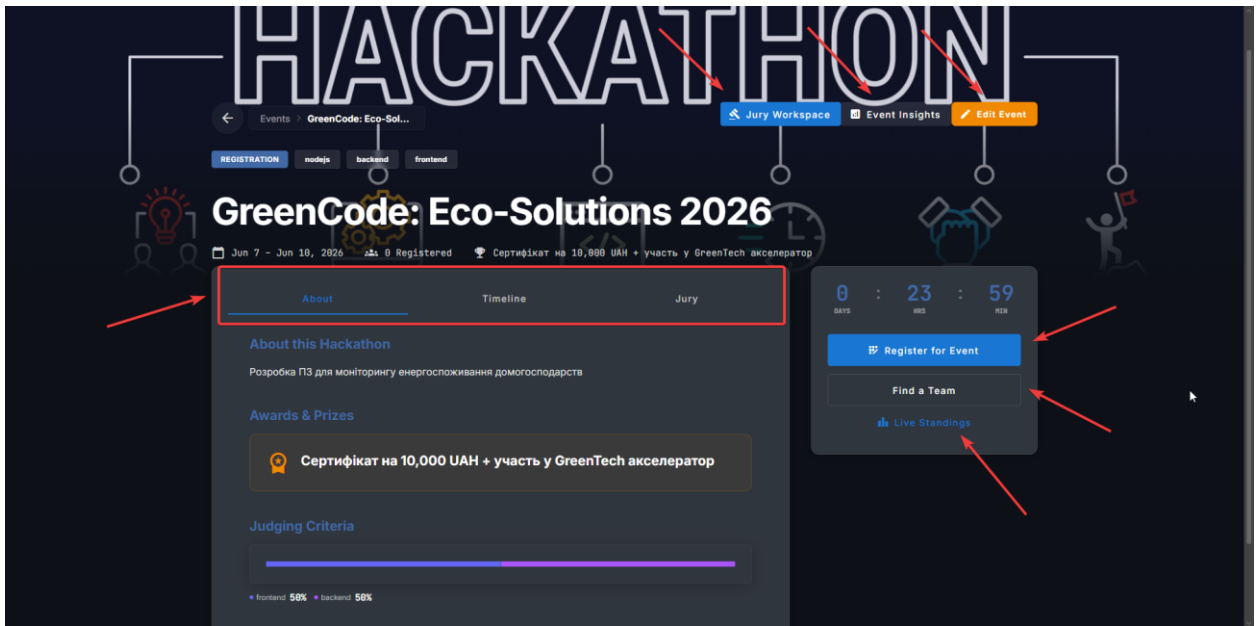


Рисунок 3.11 – Сторінка детальної інформації про хакатон

Процес створення нового хакатону починається з першого кроку під назвою «Identity & Timeline». У верхній частині форми (рис. 3.12) організатор заповнює ідентифікаційні дані змагання у блоці «Identity». Тут необхідно завантажити головний банер, вказати назву («Hackathon Title»), унікальний ідентифікатор для формування посилання («URL Slug»), описати призовий фонд («Prize Pool / Awards») та додати короткий опис заходу.

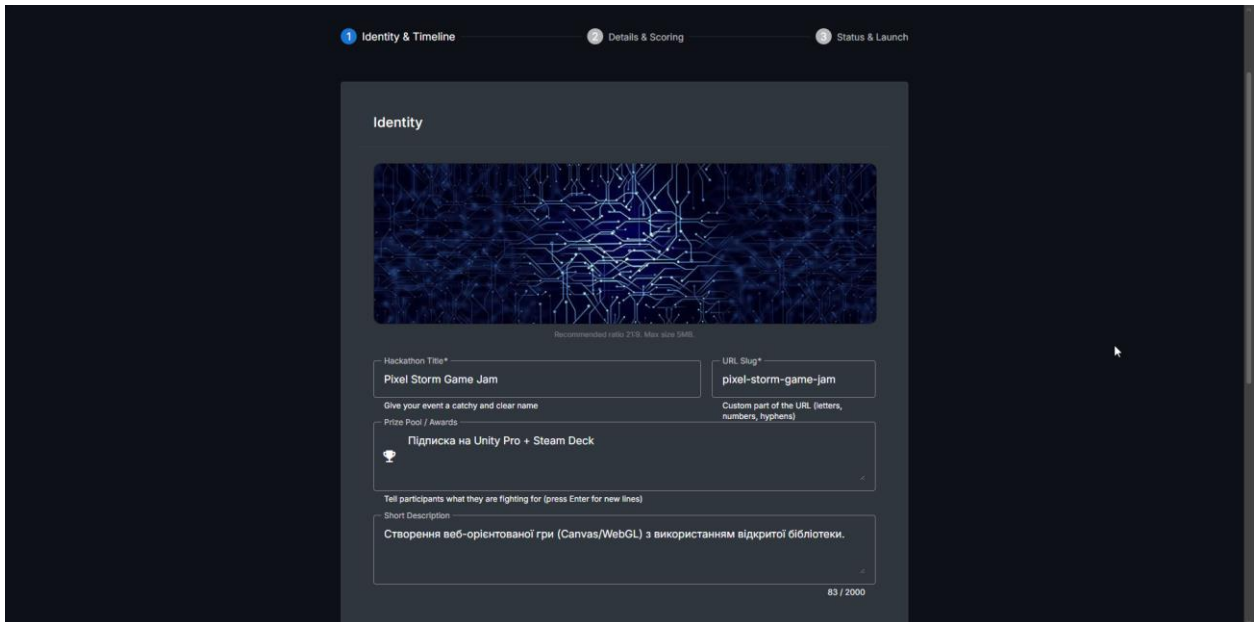


Рисунок 3.12 – Перший блок створення хакатону

Нижня частина сторінки (рис. 3.13) містить блок «Timeline & Deadlines» для налаштування графіку проведення події. За допомогою вбудованих

календарів задаються чотири обов'язкові дати: відкриття реєстрації, початок хакатону, дедлайн подачі проєктів та дата завершення заходу. Для зручності всі обрані дати автоматично вишиковуються на інтерактивній візуальній шкалі, що дозволяє легко перевірити тривалість кожного етапу. Після заповнення всіх полів першої секції необхідно натиснути кнопку «Initialize Hackathon» для збереження базових налаштувань та переходу до наступних кроків конфігурації.

Рисунок 3.13 – Налаштування дат хакатону

На другому етапі створення хакатону під назвою «Details & Scoring» (рис. 3.14) організатор деталізує тематику та формує суддівську колегію. У блоці «Focus Areas» за допомогою інтерактивного рядка вводу задаються категорії та ключові технології змагання, які відображаються у вигляді тегів із можливістю їх видалення. Розділ «Jury Members» дозволяє знаходити зареєстрованих користувачів платформи за ім'ям або юзернеймом (@username) та призначати їх суддями. Сформований список членів журі виводиться нижче, де навпроти кожного запису є кнопка для скасування призначення.

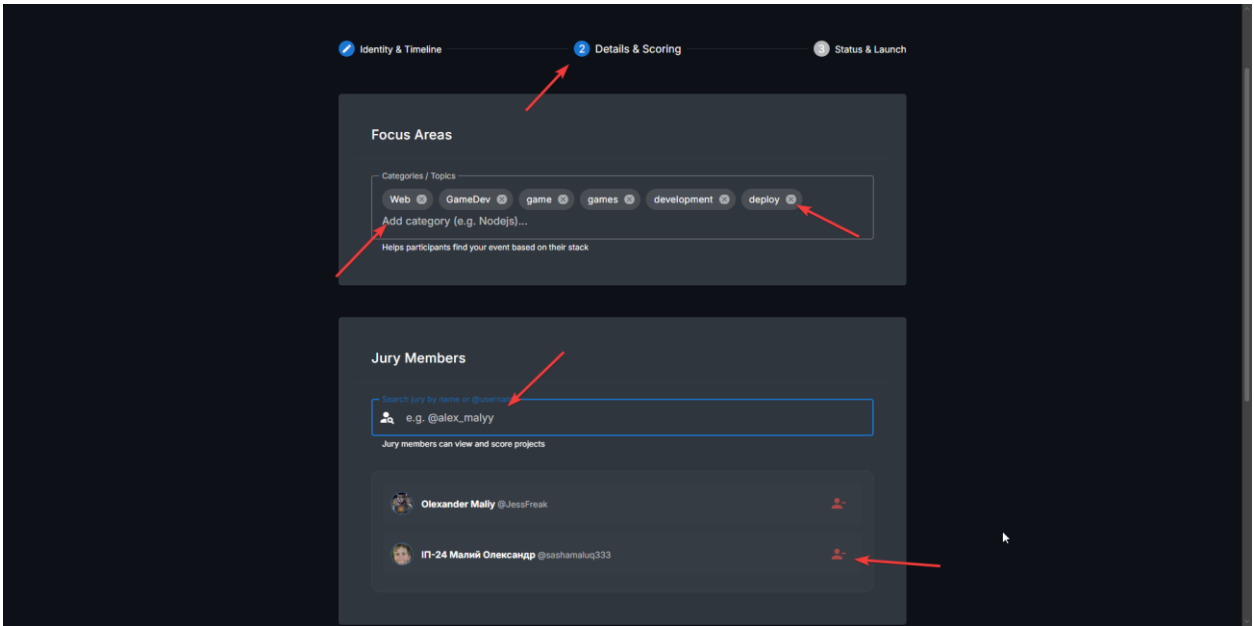


Рисунок 3.14 – Налаштування журі та категорій

Нижня частина цього етапу містить блок «Scoring Criteria» (рис. 3.15) для гнучкого налаштування системи оцінювання. Організатор може додавати власні критерії, задаючи для кожного назву, максимальний бал («Max Pts») та ваговий коефіцієнт у відсотках («Weight»). Над списком критеріїв генерується динамічна кольорова шкала, яка візуалізує поточний розподіл часток для зручного балансування оцінок. Після завершення налаштувань перехід до фінального кроку здійснюється кнопкою «Save and Continue», а для повернення назад призначена кнопка «Back».

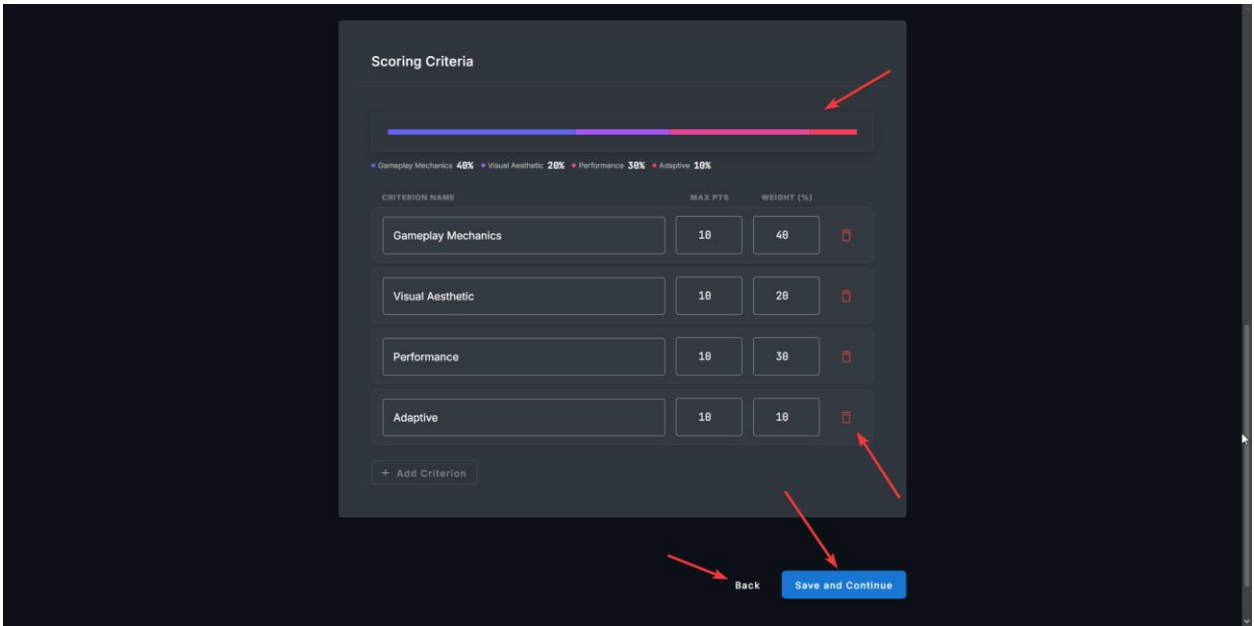


Рисунок 3.15 – Налаштування критеріїв оцінювання

Третій, фінальний етап створення хакатону має назву «Status & Launch» (рис. 3.16). На цій сторінці організатор безпосередньо керує життєвим циклом створеного заходу.

У блоці «Hackathon Lifecycle» розташовано випадючий список, за допомогою якого встановлюється поточна стадія змагання (наприклад, переведення з режиму чернетки у статус «Registration»). Цей параметр є ключовим, оскільки він визначає рівень видимості події на платформі та перелік доступних дій для учасників. Будь-яка зміна стадії фіксується натисканням кнопки «Apply Status Change». Після успішної ініціалізації всіх налаштувань організатор може миттєво перейти до перегляду готової сторінки змагання, натиснувши кнопку «View Public Page», або за необхідності повернутися до попереднього кроку кнопкою «Back».

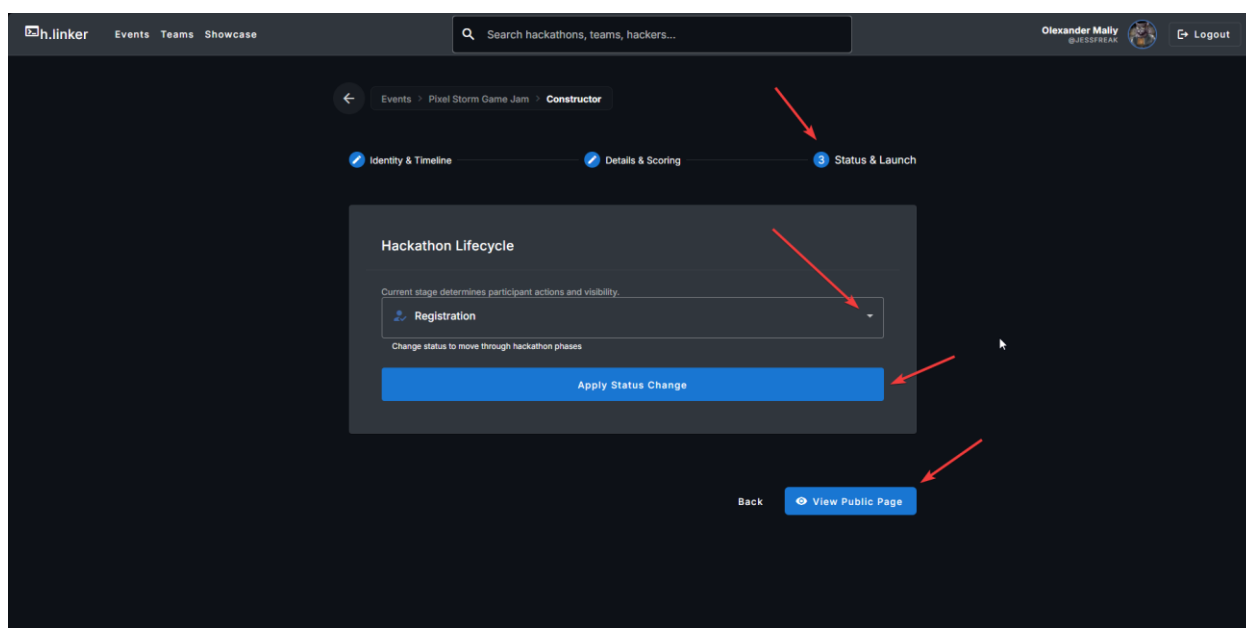


Рисунок 3.16 – Фінальна секція створення хакатону

Для участі у хакатоні користувач має подати заявку від імені своєї команди. При натисканні на кнопку «Register for Event» система автоматично перевіряє, чи є розробник капітаном (лідером) хоча б одного колективу. Якщо користувач ще не створив власну команду, на екрані з'являється відповідне модальне вікно (рис. 3.17). Воно інформує про неможливість продовження реєстрації та пропонує перейти до розділу формування команд за допомогою кнопки-посилання «Go to Teams». Скасувати дію та просто закрити вікно

можна кнопкою «Cancel», при цьому основна кнопка підтвердження реєстрації («Register Team») залишається системно заблокованою до виконання необхідних умов.

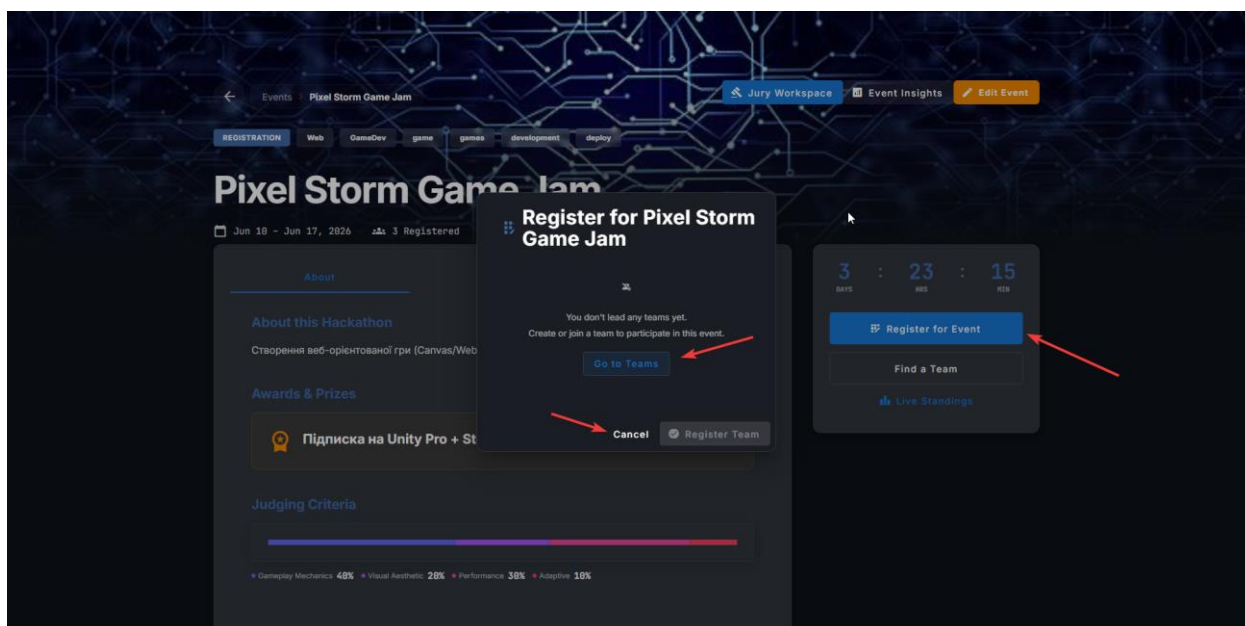


Рисунок 3.17 – Відсутність команд в учасника

Сторінка перегляду команд (рис. 3.18) призначена для пошуку та взаємодії з існуючими колективами. Основну область екрана займає сітка карток команд, де відображається назва, опис і кількість учасників. На кожній картці передбачено посилання «Details» для перегляду детальної інформації та кнопка «Join Team» для подачі заявки на вступ до колективу. У правому верхньому кутку списку реалізовано меню для сортування виведених результатів.

У лівій бічній панелі розміщено блок інструментів пошуку та фільтрації («Discovery Filters»). Він містить текстове поле для пошуку за назвою, фільтри за типом участі, кнопку їх застосування («Apply Filters») та посилання для скидання параметрів («Reset»). Під блоком фільтрів знаходиться яскрава кнопка «+ Create Team», яка ініціює процес створення нової команди.

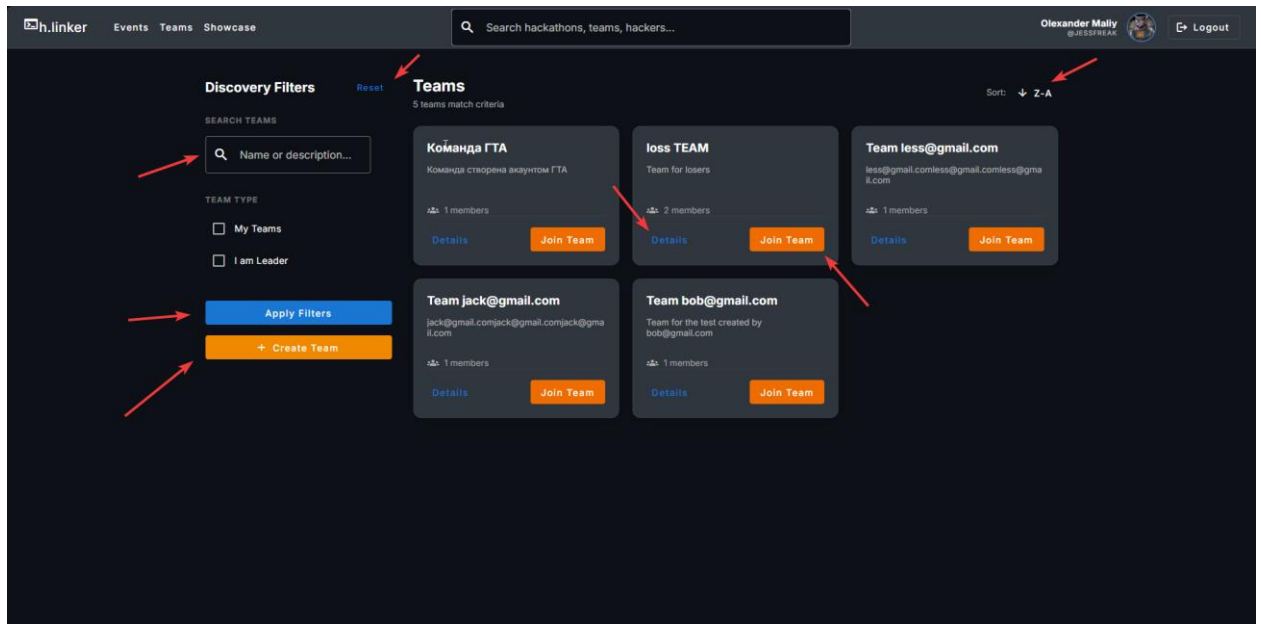


Рисунок 3.18 – Сторінка перегляду команд

Після натискання кнопки «Join Team» відкривається модальне вікно формування запиту на вступ до обраного колективу (рис. 3.19). Користувачу необхідно обрати бажану роль із випадального списку та додати коротке супровідне повідомлення для лідера команди у відповідному текстовому полі. Надсилання заявки виконується кнопкою «Send Request», а скасування операції – кнопкою «Cancel».

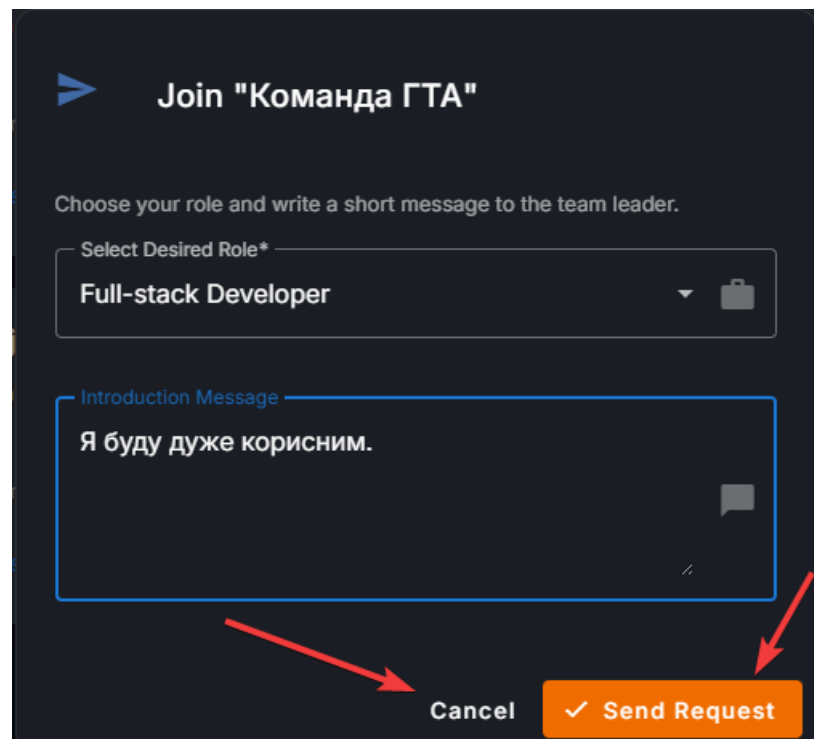


Рисунок 3.19 – Модальне вікно запиту

Сторінка детальної інформації про команду (рис. 3.20) відображає публічні дані колективу для зовнішніх користувачів. У верхній частині інтерфейсу розташована кнопка «Join Team» для швидкої подачі заявки на вступ.

Основна робоча область розділена на логічні блоки. Зліва подано загальний опис, список прив'язаних проєктів із посиланнями «View on GitHub» та історію участі колективу в змаганнях, де фіксуються статуси подій та отримані бали. Права панель містить інструмент комунікації «Open Team Chat» (який автоматично деактивується, якщо лідер команди не додав посилання на зовнішній месенджер або користувач не є учасником команди) та повний перелік поточних учасників із зазначенням їхніх внутрішніх ролей.

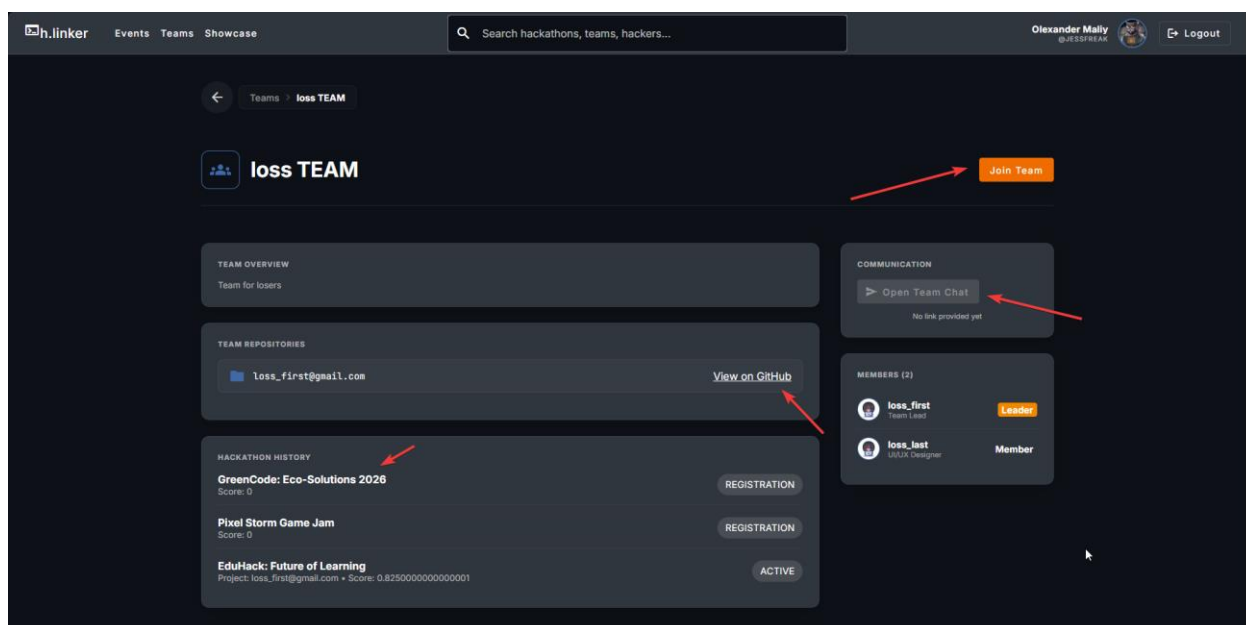


Рисунок 3.20 – Сторінка повної інформації про команду

Натискання кнопки «+ Create Team» у лівій бічній панелі викликає модальне вікно створення нового колективу (рис. 3.21).

У цій формі користувачу необхідно заповнити обов'язкове текстове поле з назвою команди. Додатково передбачено поля для введення загального опису та посилання на зовнішній ресурс для комунікації учасників, наприклад, сервер у Discord чи Telegram. Після введення всіх даних процес створення підтверджується натисканням кнопки «+ Create Team» у нижній частині вікна.

Для переривання операції та закриття форми використовується кнопка «Cancel».

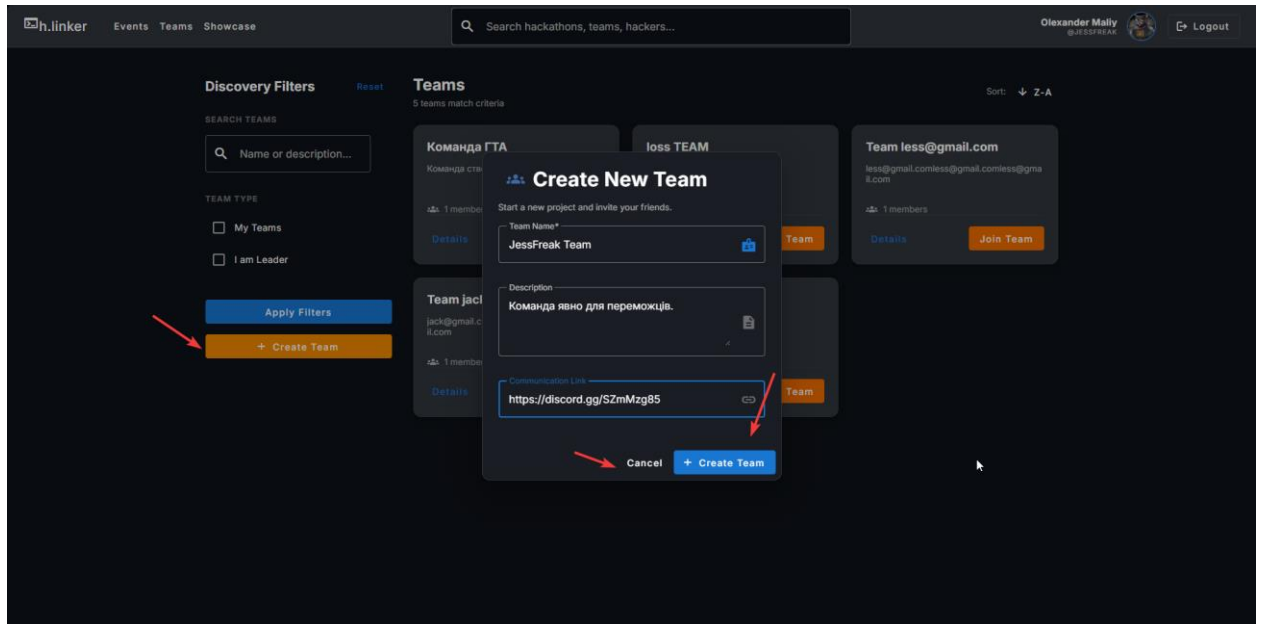


Рисунок 3.21 – Меню створення команди

Коли користувач переглядає сторінку власної команди, де він є капітаном (лідером), інтерфейс правої панелі змінюється (рис. 3.22).

У верхній частині панелі з'являється додаткова функціональна кнопка «Manage Team», яка забезпечує перехід до розділу конфігурації команди. Поруч відображається тег поточного статусу користувача. У блоці «Communication» кнопка «Open Team Chat» стає активною, оскільки лідер надав відповідне посилання. Нижче, у списку учасників («Members»), капітану стає доступна «+ Invite Members» для запрошення нових розробників.

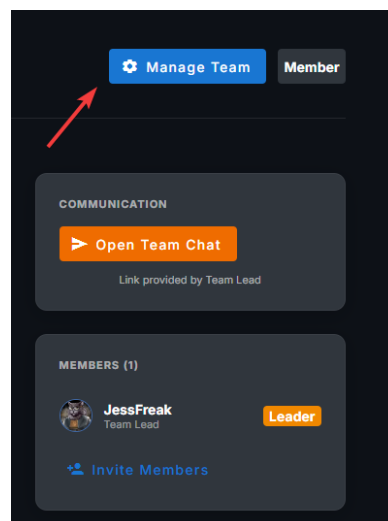


Рисунок 3.22 – Перехід до налаштувань команди

Сторінка управління командою (рис. 3.23) розділена на кілька функціональних блоків, навігація між якими здійснюється через ліве бічне меню: «General Info», «Members & Requests» та «Danger Zone». У розділі «General Information» лідер команди може редагувати базові дані колективу: змінювати назву («Team Name»), оновлювати загальний опис («Description») та актуалізувати посилання на платформу для комунікації («Communication Link»).

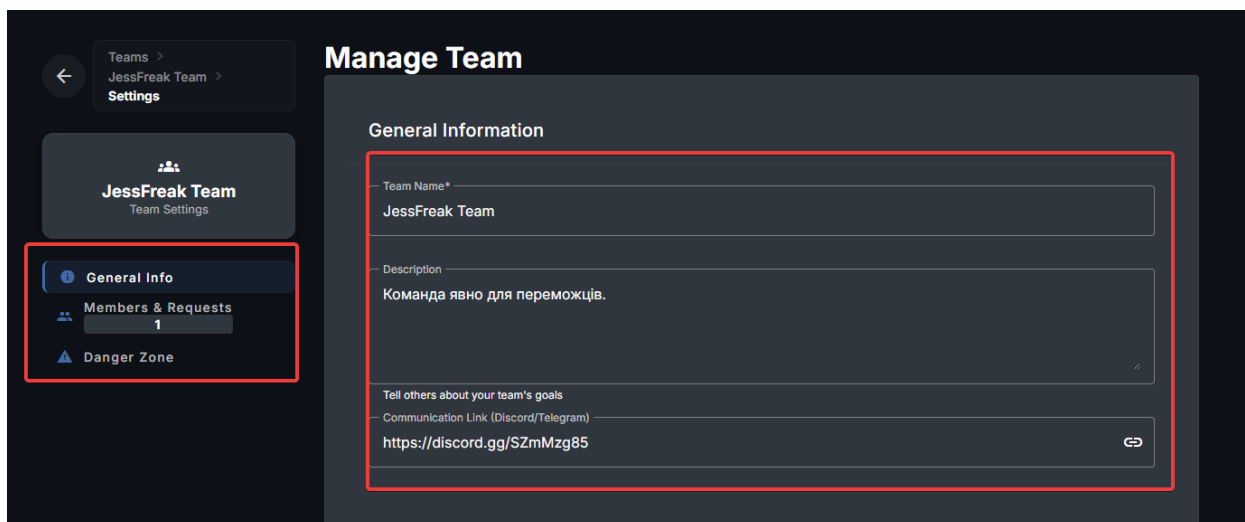


Рисунок 3.23 – Налаштування команди

Нижче на сторінці (рис. 3.24) реалізовано управління складом та критичними налаштуваннями. Блок «Team Members» містить перелік поточних учасників та секцію вхідних заявок на вступ. Для кожної заявки відображається ім'я кандидата, бажана роль, супровідне повідомлення та кнопки для її миттєвого схвалення (синя галочка) або відхилення (червоний хрестик).

У розділі «Danger Zone» доступні адміністративні операції: передача прав капітана іншому учаснику за допомогою кнопки «Transfer» та безповоротний розпуск колективу кнопкою «Disband Team». Будь-які внесені на сторінці зміни фіксуються натисканням кнопки «Save Changes» на нижній закріпленій панелі.

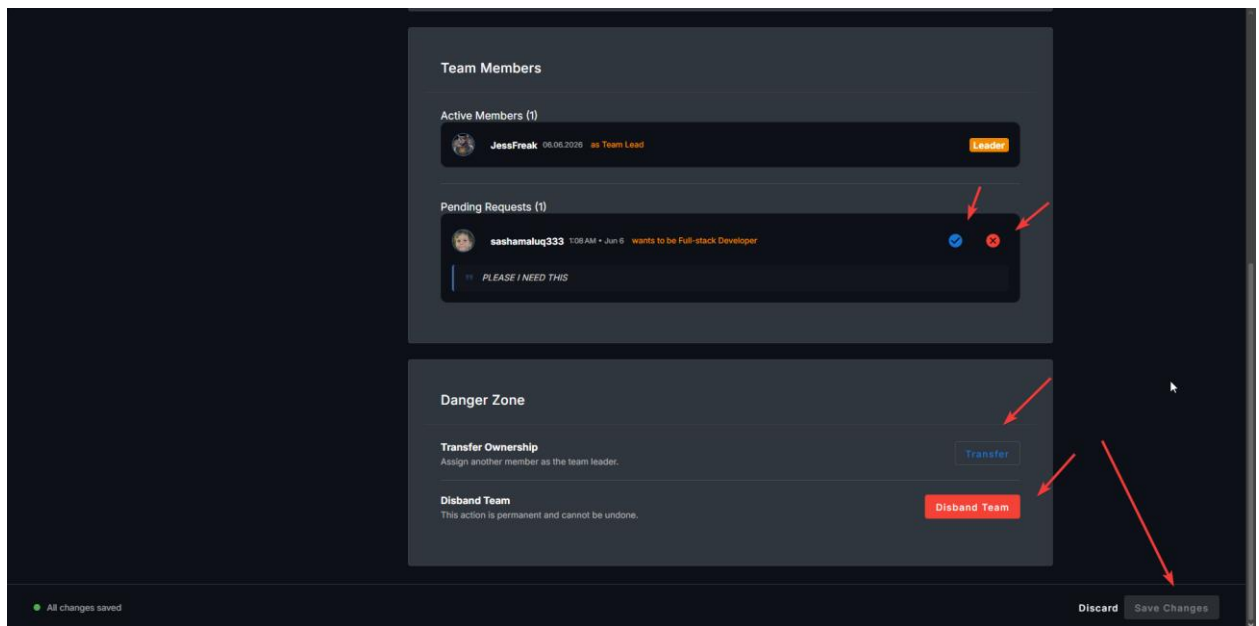


Рисунок 3.24 – Секції учасників та небезпечна зона налаштувань команди

Виконання критичних операцій в управлінні командою супроводжується обов'язковими модальними вікнами підтвердження для запобігання випадковим змінам.

При ініціації передачі прав лідера з'являється вікно «Transfer Ownership» (рис. 3.25), де поточний капітан повинен обрати свого наступника з випадуючого списку наявних учасників. Система додатково попереджає про повну втрату адміністративних прав після підтвердження цієї операції. Схожий механізм захисту реалізовано і для функції розпуску колективу: вікно «Disband Team» (рис. 3.26) чітко інформує про незворотність цієї дії та автоматичне скасування членства для всіх учасників.

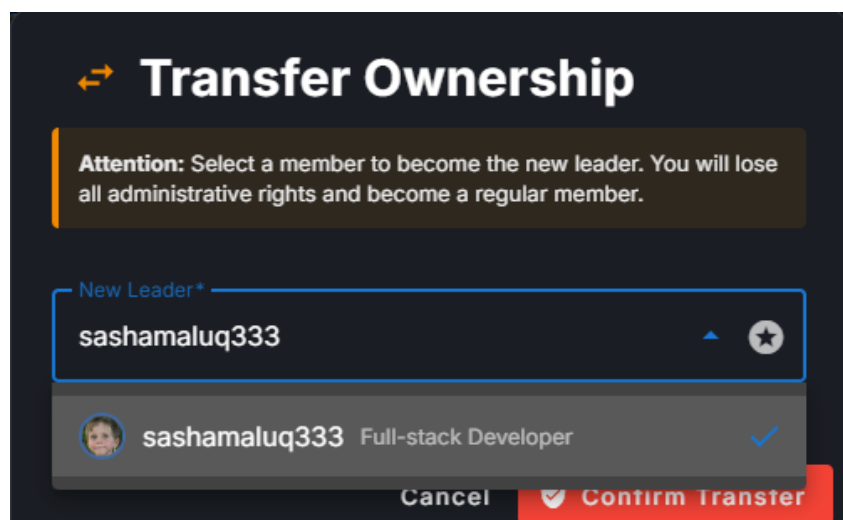


Рисунок 3.25 – Меню трансферу ролі лідера

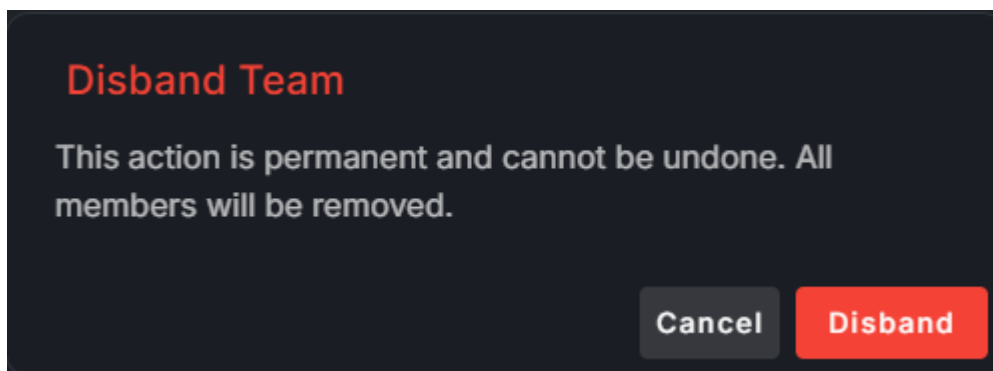


Рисунок 3.26 – Модальне вікно видалення команди

Окремий сценарій передбачено для звичайних учасників колективу (які не мають статусу лідера). На головній сторінці команди для них відображається кнопка «Leave Team» (рис. 3.27). При її натисканні викликається відповідне вікно підтвердження виходу, яке попереджає користувача, що дія є остаточною, і для можливого повернення до цієї команди знадобиться нове запрошення або повторне схвалення заявки. У кожному з цих вікон передбачено кнопку «Cancel» для безпечного скасування розпочатої дії.

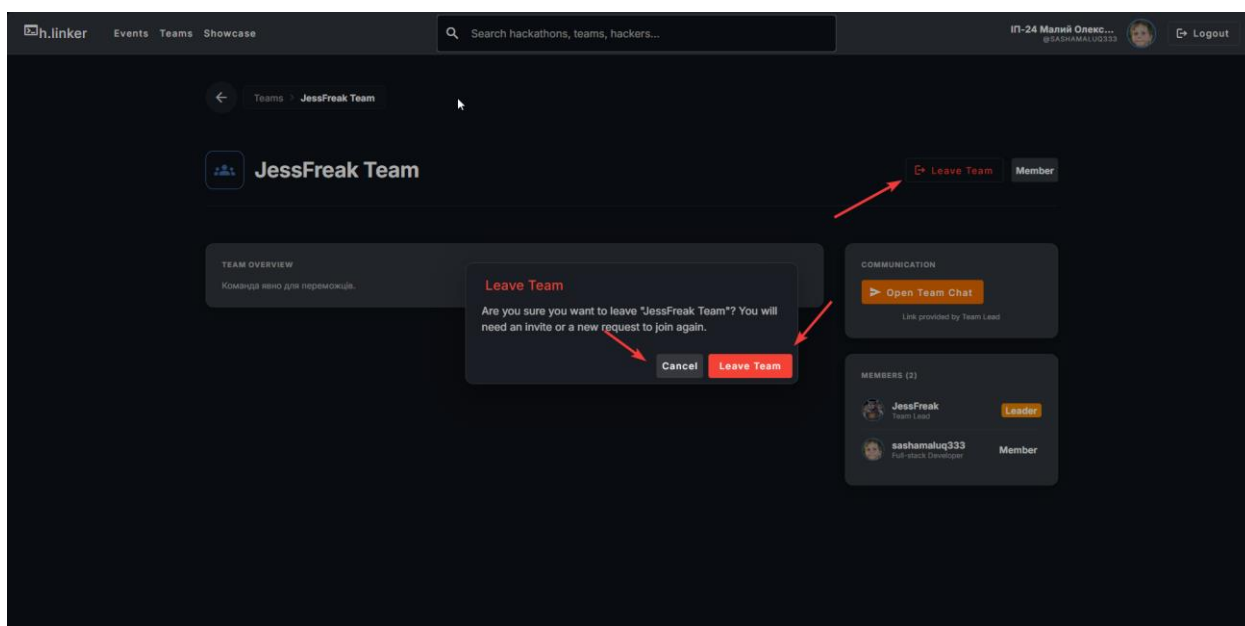


Рисунок 3.27 – Попередження перед виходом з команди

Сторінка пошуку спеціалістів «Find Talent» (рис. 3.28) виконує функцію загального каталогу розробників платформи. Основну частину інтерфейсу займає сітка карток користувачів. Кожна картка містить аватар, ім'я, короткий опис профілю та перелік підтверджених технічних навичок у вигляді

візуальних тегів. Для безпосередньої взаємодії з кандидатом передбачено посилання «View Profile» та кнопку «+ Invite». У лівій бічній панелі розташовано блок фільтрації з можливістю пошуку за текстовим запитом, вибором стеку технологій та додатковими параметрами, а у верхній правій частині – інструмент стандартного сортування списку.

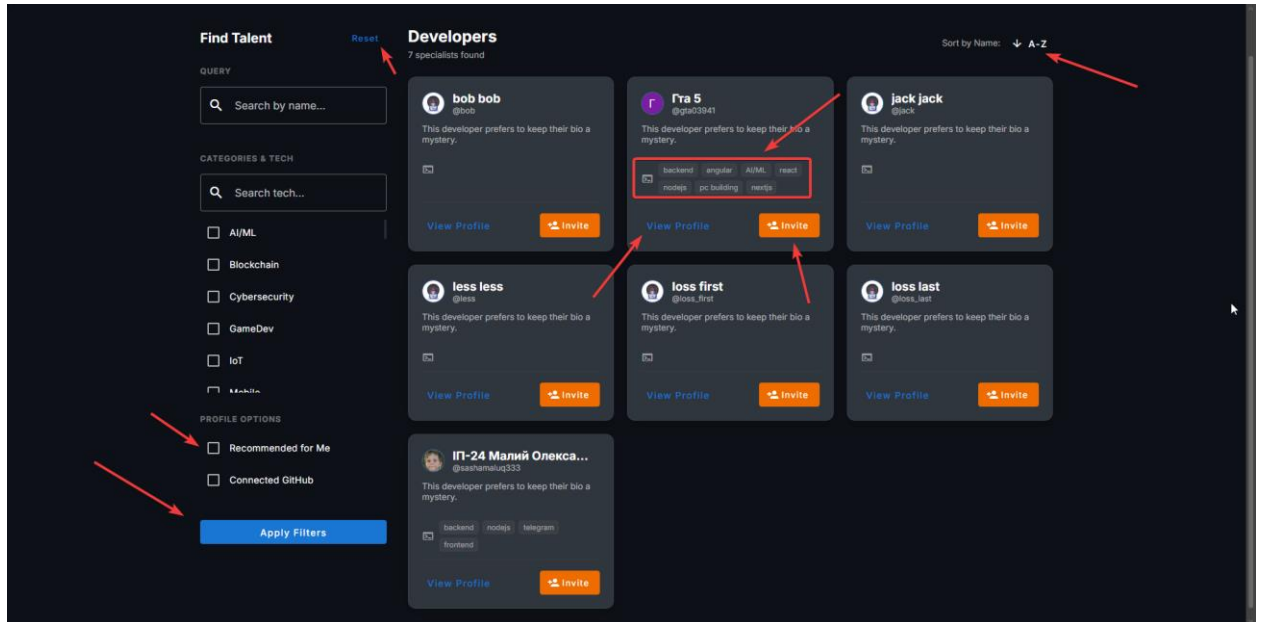


Рисунок 3.28 – Сторінка пошуку користувачів

Друга варіація цієї ж сторінки (рис. 3.29) демонструє активацію розумної системи рекомендацій на основі навичок. При виборі опції «Recommended for Me» у блоці «Profile Options» та застосуванні фільтрів, система кардинально змінює логіку видачі результатів. Вбудований алгоритм аналізує власний стек технологій користувача (або потреби його команди) і порівнює його з профілями інших розробників. У результаті список пересортовується за рівнем релевантності, а на картках найбільш підходящих кандидатів з'являється спеціальний фіолетовий індикатор із відсотком технічного збігу (наприклад, 14% або 9%). Цей інструмент суттєво оптимізує процес рекрутингу на платформі, дозволяючи лідерам швидко знаходити фахівців, які найкраще доповнять існуючу експертизу команди.

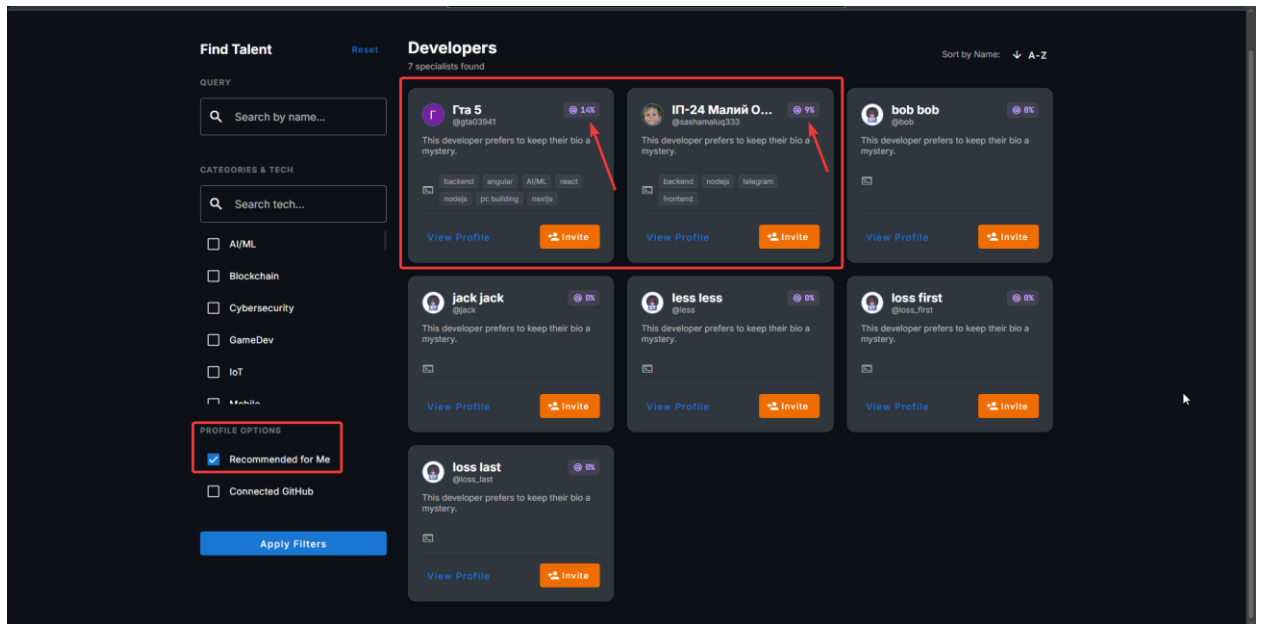


Рисунок 3.29 – Сторінка пошуку користувачів з використанням рекомендацій

Для залучення нового учасника до роботи над проєктами капітан команди може скористатися інструментом прямого запрошення. При натисканні на кнопку «Invite» на профілі користувача відкривається відповідне модальне вікно (рис. 3.30), яке дозволяє керувати процесом рекрутингу.

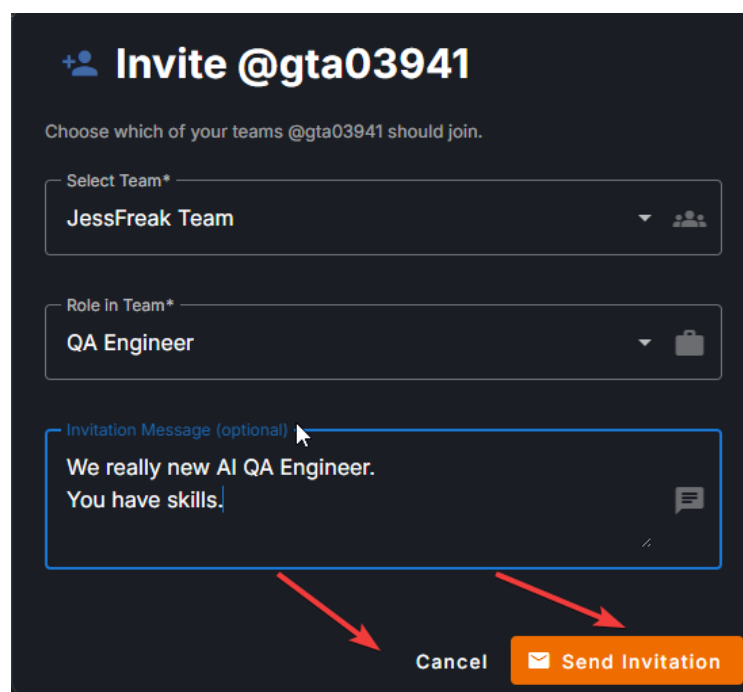


Рисунок 3.30 – Меню запрошення користувача до команди

Блок «Team Invitations» (рис. 3.31) у налаштуваннях профілю відображає всі вхідні запрошення до команд. Кожна картка містить назву команди,

запропоновану роль та супровідне повідомлення від лідера. Користувач може прийняти запрошення, натиснувши на синю галочку, або відхилити його за допомогою червоного хрестика.

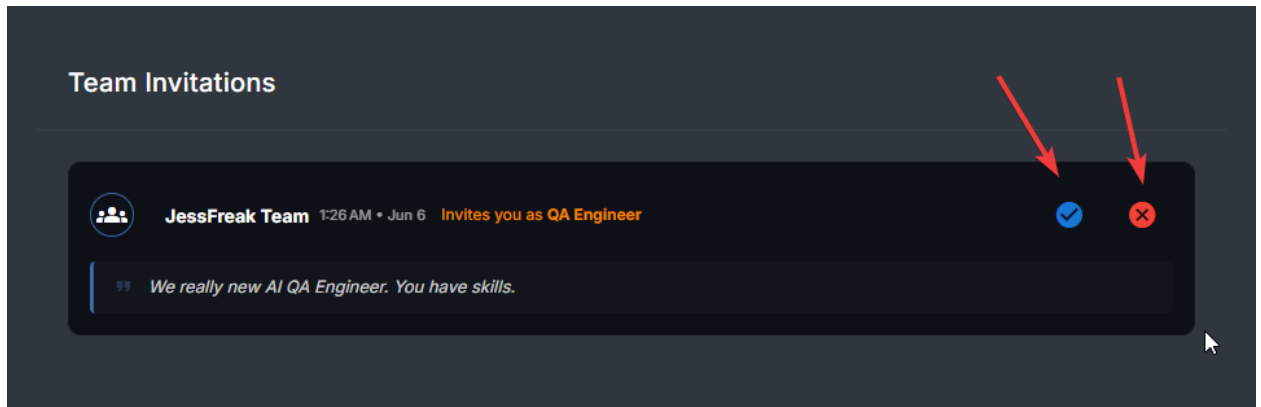


Рисунок 3.31 – Нове запрошення до команди

Після того, як користувач має сформовану команду, процес реєстрації на хакатон стає доступним через модальне вікно «Register for [Event Name]» (рис. 3.32). У цьому інтерфейсі лідеру необхідно обрати команду, яка представлятиме його в змаганні, із випадаючого списку «Select Team».

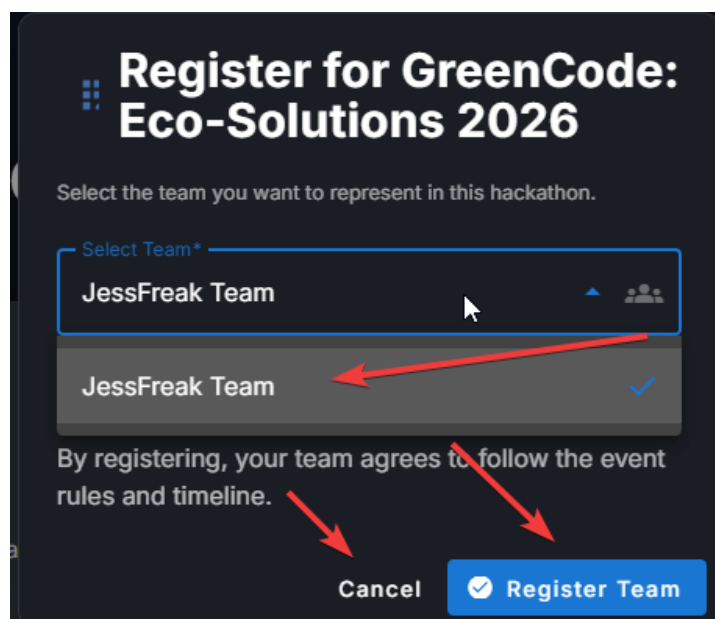


Рисунок 3.32 – Реєстрація на хакатон з командою

Дешборд учасника хакатону (рис. 3.33) є центральним хабом для моніторингу прогресу та керування процесом подачі конкурсного проєкту.

У верхній частині сторінки розміщено динамічний таймер зворотного відліку до «Project Submission Deadline» та ключову кнопку «SUBMIT PROJECT», яка активує форму для здачі роботи.

Основний простір дашборду структуровано за допомогою чотирьох інформаційних блоків:

- Your Team: містить назву команди, список учасників з їхніми ролями та кнопку швидкого переходу до налаштувань колективу («Team Settings»);
- Submission Status: відображає поточний стан подачі проєкту;
- Leaderboard: демонструє поточну турнірну з відображенням позиції (рангу) команди та посиланням «View Full Standings»;
- Your Score: показує отримані бали та поточне місце команди в загальному рейтингу.

Додатково на сторінці відображаються встановлені організаторами «Judging Criteria», де графічно представлено розподіл вагових коефіцієнтів між окремими аспектами оцінювання.

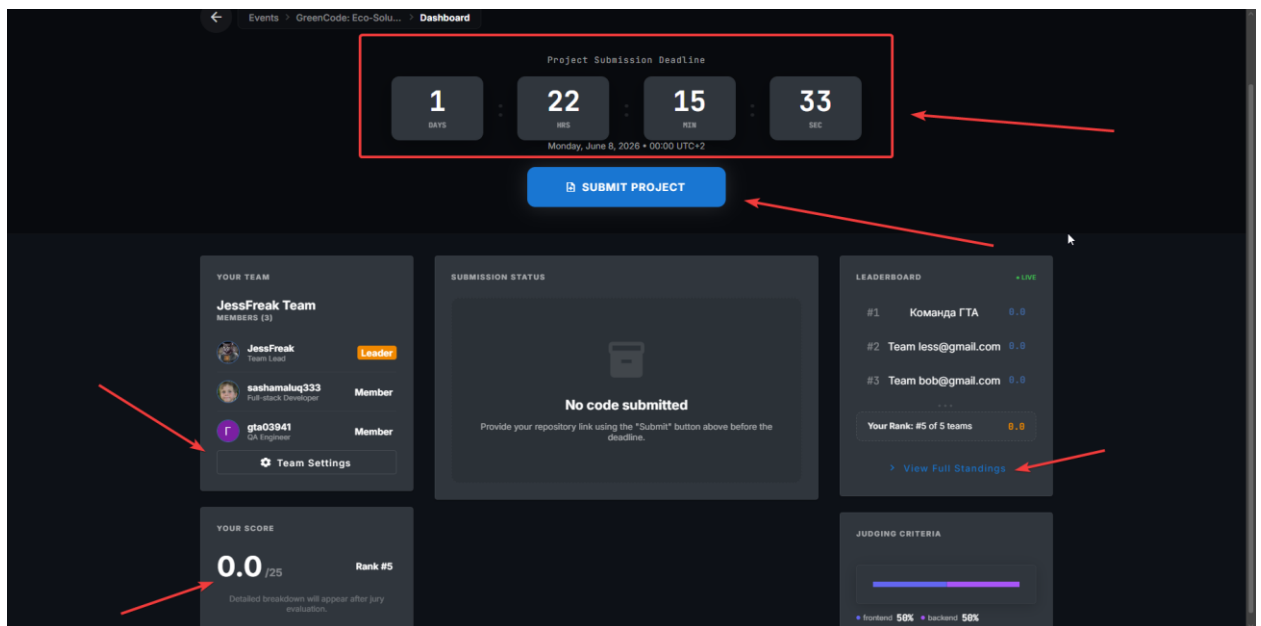


Рисунок 3.33 – Дешборд учасника хакатону

Процес подачі конкурсного проєкту організовано як послідовний трикроковий майстер:

- крок 1: Project Info (рис. 3.34) – користувач вказує базову інформацію: офіційну назву проєкту та його короткий опис. Далі треба натиснути кнопку «Continue to Repository»;

- крок 2: Repository (рис. 3.35) – на цьому етапі передбачено два варіанти вибору репозиторію: інтеграція через GitHub, де користувач може переглянути та вибрати власні публічні репозиторії учасників команди, або введення посилання на репозиторій вручну. Перехід до останнього кроку виконується кнопкою «Review & Finalize»;
- крок 3: Finalize (рис. 3.36) – фінальний етап, на якому організатор подачі перевіряє узагальнені дані: назву проєкту, посилання на репозиторій та актуальний склад команди. Для завершення процесу користувач має поставити відмітку про підтвердження готовності проєкту до оцінювання та його відповідність правилам хакатону, після чого натиснути кнопку «SUBMIT PROJECT».

The screenshot shows a web interface for 'Project Submission'. At the top, a breadcrumb trail reads 'Events > GreenCode: Eco-Solu... > Dashboard > Submission'. Below this is the title 'Project Submission' and the team name 'Team: JessFreak Team'. A progress bar at the top indicates three steps: '1 Project Info' (highlighted with a red box), '2 Repository', and '3 Finalize'. The main form area is titled 'GENERAL INFORMATION' and contains two text input fields. The first field, 'Project Title*', contains the text 'h.linker'. The second field, 'Short Description*', contains the text 'Build your dream team for Hackathons h.linker helps students find teammates, showcase their GitHub projects, and win the best tech competitions.' A character count '144 / 2000' is visible at the bottom right of the description field. A red arrow points from the bottom right of the description field to a blue button labeled '→ Continue to Repository'.

Рисунок 3.34 – Перший крок здачі проєкту

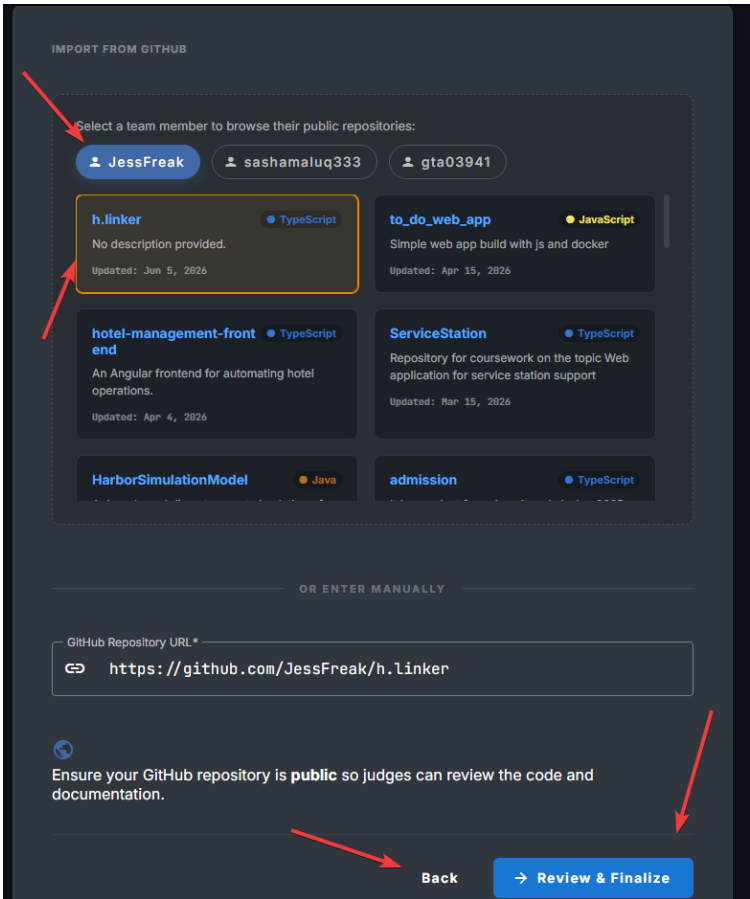


Рисунок 3.35 – Другий крок задачі проєкту

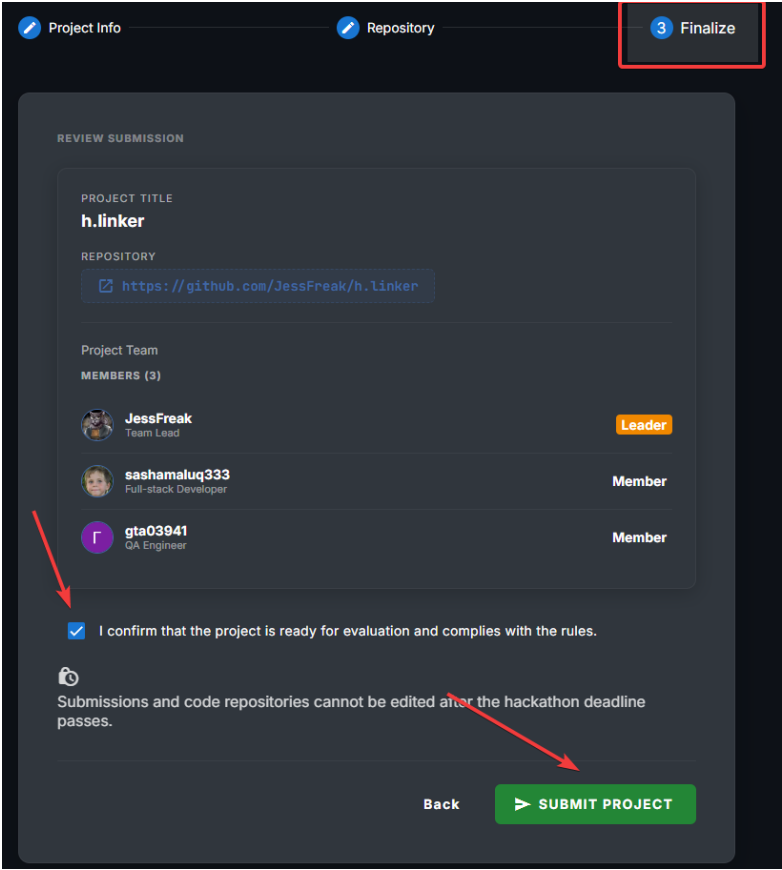


Рисунок 3.36 – Фінальний крок задачі проєкту

Сторінка оцінювання журі (рис. 3.37) є інструментом для виставлення балів та надання зворотного зв'язку учасникам хакатону. У верхній частині інтерфейсу розташована навігаційна панель, що відображає прогрес оцінювання всіх команд, виділяючи ті, що вже отримали оцінки, та виокремлюючи проєкт, який перебуває на стадії розгляду в даний момент. Ліворуч знаходяться дані про проєкт, включаючи назву, можливість переходу до репозиторію на GitHub, опис та розділ з результатами інших суддів, що доступні лише для читання.

Права частина сторінки містить форму оцінювання, де суддя може виставляти бали за кожним із визначених критеріїв за допомогою інтерактивних повзунків, що автоматично враховують налаштовану вагу категорій. Нижче передбачено текстове поле для розгорнутого відгуку, а також місця для фіксації сильних сторін проєкту та областей для вдосконалення. У нижній частині форми динамічно розраховується фінальна оцінка на основі зважених балів, що дозволяє уникнути помилок у підрахунках. Завершується робота з командою кнопкою збереження результатів та переходу до наступного проєкту, при цьому система також підтримує можливість збереження проміжного варіанту як чернетки.

The screenshot shows the h.linker evaluation interface. The top navigation bar includes 'h.linker', 'Events', 'Teams', and 'Showcase'. A search bar is present. The team list at the top shows 'Команда ГТА', 'Team less@gmail.com', 'Team bob@gmail.com', and 'JessFreak Team' (selected). The 'Submitted' timestamp is 'Jun 6, 02:09'. The left sidebar contains the project name 'JessFreak Team • Hackathon Project', a link to 'Open GitHub Repository', a project description, and a section for 'OTHER JURY SCORES' with a score of 24.0 from @sashamak333. The main area is the 'Evaluation Form' for 'JessFreak Team'. It has sliders for 'frontend' and 'backend' scores, each with a weight of 50%. The 'frontend' score is 25, and the 'backend' score is 25. Below the sliders is a text box for 'EXPERT REVIEW / COMMENTS'. At the bottom, a 'CALCULATED SCORE' of 25.0/10 is displayed, along with 'Key Strengths' and 'Areas for Improvement' sections. Navigation buttons like 'Prev Team', 'Next Team', 'Save as Draft', and 'Save & Next Team' are at the bottom.

Рисунок 3.37 – Сторінка оцінювання робіт

Після того, як суддя виставляє оцінки для останнього проєкту, на екрані з'являється вікно сповіщення «Evaluation Complete!» (рис. 3.38). Воно містить підтвердження успішного завершення процесу оцінювання всіх команд, візуалізоване зеленою іконкою галочки, а також кнопку «Review Scores», що надає судді можливість перейти до перегляду та фінальної перевірки виставлених балів.

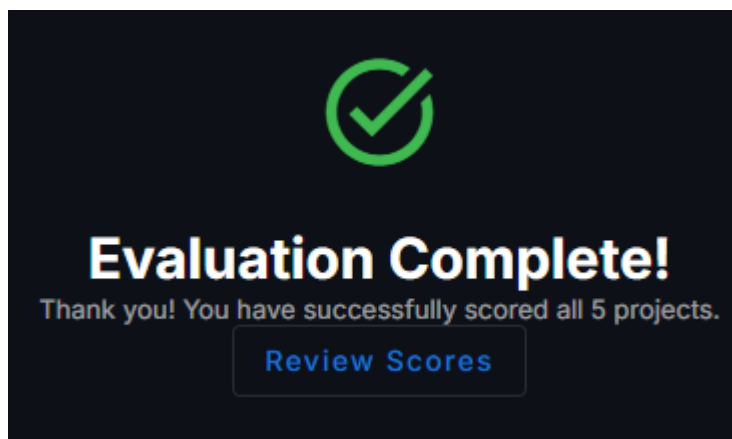


Рисунок 3.38 – Сповіщення про завершення оцінювання

Оновлений дашборд учасника (рис. 3.39) відображає стан проєкту після завершення етапу оцінювання журі. Ключові зміни торкнулися блоку статусу подачі: тепер там зафіксовано назву проєкту з посиланням на репозиторій та кнопку для швидкого переходу до нього, а також можливість коригування даних через кнопку «EDIT SUBMISSION».

Інформативність блоку «Your Score» суттєво зросла: він демонструє загальний бал та рейтинг за окремими категоріями, що дозволяє команді проаналізувати свої успіхи. Найбільш змістовним став новий розділ «Jury Reviews & Feedback», де наведено загальний відгук судді, а також структуровано вказано сильні сторони («Strengths») та зони для покращення («Areas for Improvement»). Якщо оцінювання проводили кілька суддів, передбачено навігацію між їхніми відгуками за допомогою кнопок перемикавання. Також оновилося турнірна таблиця «Leaderboard», яка в реальному часі демонструє поточне лідерство команди з відповідними балами.

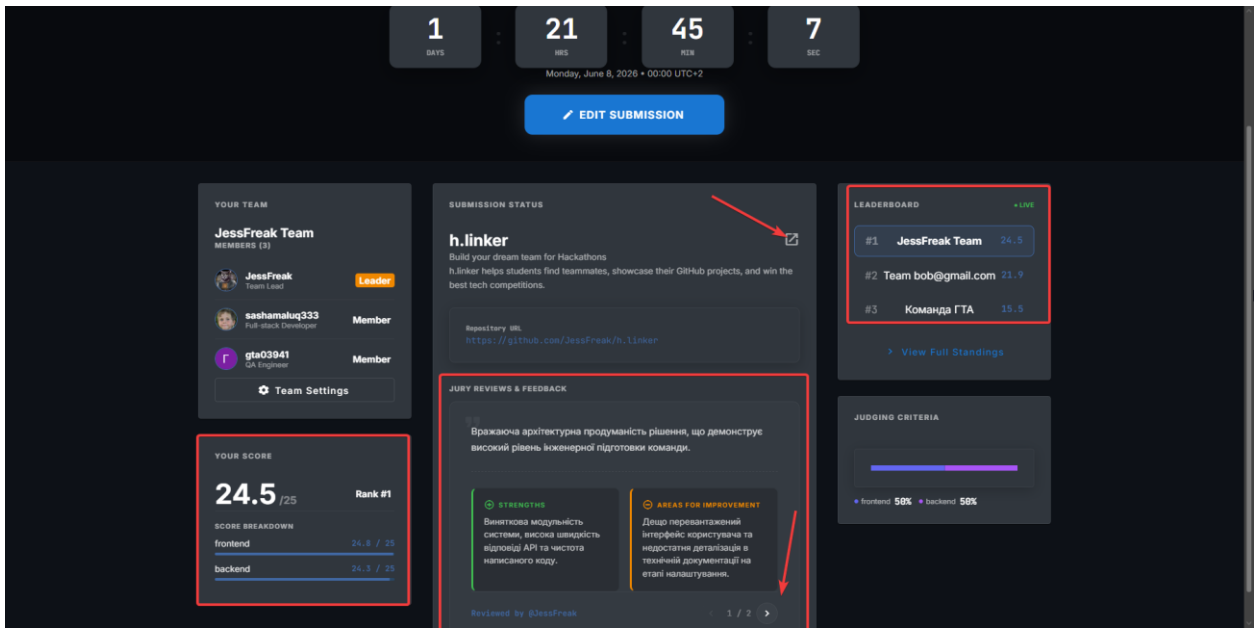


Рисунок 3.39 – Оновлений дашборд учасника

Сторінка «Leaderboard» (рис. 3.40) забезпечує наочну візуалізацію поточних результатів хакатону. Верхня частина екрана присвячена трьом лідерам змагання: команди, що посіли перші три місця, виділені картками різного розміру з акцентом на переможця (Rank #1). Кожна картка містить назву команди, кількість учасників та набрану кількість балів.

Нижче розташовано блок «All Standings», де у вигляді компактного списку відображаються решта учасників змагання, їхні порядкові номери (ранги), назви команд, кількість людей у складі та фінальні оцінки.

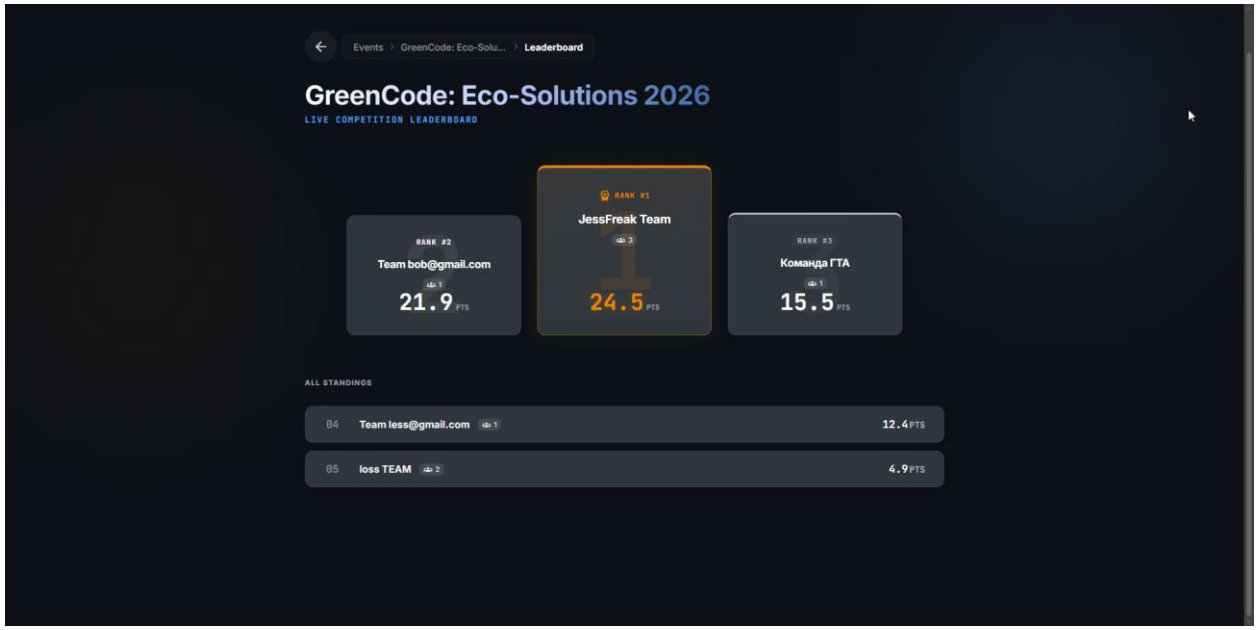


Рисунок 3.40 – Сторінка лідерборду

Сторінка «Event Analytics» (рис. 3.41) є спеціалізованим дашбордом для організаторів хакатону, який забезпечує комплексний моніторинг та аналіз статистичних показників проведеного заходу. У верхній частині інтерфейсу розташовано панель із чотирма ключовими метриками, що дають миттєве розуміння масштабів події: загальна кількість залучених учасників, кількість сформованих колективів, кількість успішно поданих рішень та середній бал за підсумками суддівського оцінювання.

Основний простір сторінки займають чотири аналітичні віджети з візуалізацією даних:

- Submission Timeline: лінійний графік, що відображає динаміку завантаження проєктів учасниками з прив'язкою до часу;
- Participant Roles: кільцева діаграма, яка наочно демонструє розподіл залучених фахівців за їхніми ролями в командах;
- Score Distribution: стовпчаста гістограма, що показує частоту виставлення тих чи інших балів (згрупованих за діапазонами), дозволяючи організаторам оцінити загальний рівень якості та успішності поданих робіт.
- Jury Activity: графік, який відстежує інтенсивність роботи суддівської колегії в процесі оцінювання проєктів.

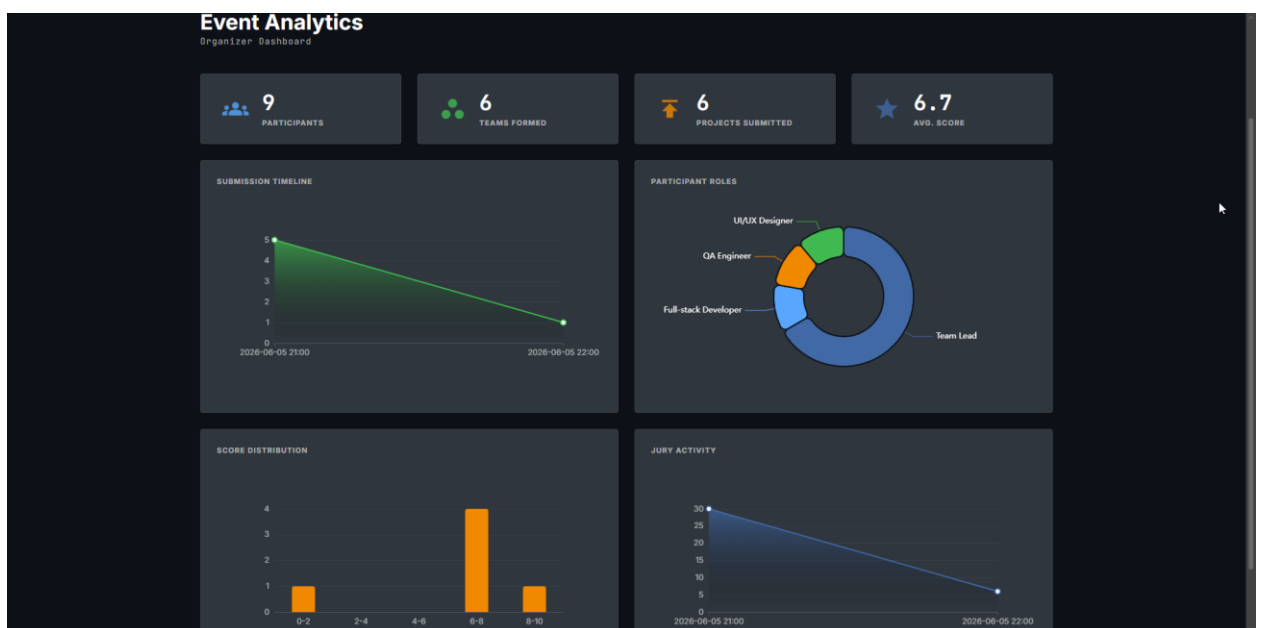


Рисунок 3.41 – Сторінка статистики хакатону

Розділ «Project Showcase» (рис. 3.42) слугує публічною вітриною найкращих програмних рішень, створених командами під час минулих змагань, та одночасно виконує роль відкритого каталогу портфоліо учасників платформи.

У лівій бічній панелі під назвою «Project Filters» зосереджено інструменти гнучкого пошуку. Користувач може знаходити потрібні роботи через текстове поле за назвою чи ключовими словами, шукати за конкретним стеком технологій, а також використовувати набір чекбоксів для фільтрації за доменними напрямками. Для застосування обраних параметрів використовується кнопка «Apply Filters», а для миттєвого очищення форми – посилання «Reset».

У верхній частині основної робочої області розташовано лічильник загальної кількості знайдених робіт та меню сортування, яке дозволяє впорядкувати видачу, наприклад, за найвищим рейтингом суддів. Основний простір сторінки заповнено сіткою деталізованих карток проєктів. Кожна така картка містить тематичний банер, тег із назвою відповідного хакатону, бейдж із зайнятим призовим місцем та відсоток успішності проєкту. Крім того, на картці відображаються назва команди-автора із зазначенням кількості розробників, короткий опис реалізованого рішення та візуальні теги задіяних технологій. У самому низу картки зафіксовано абсолютну оцінку від журі та розміщено інтерактивну кнопку «Repository» для прямого переходу до відкритого вихідного коду. Натискання на саму ж карточку переносить до сторінки івенту, де була здана ця робота.

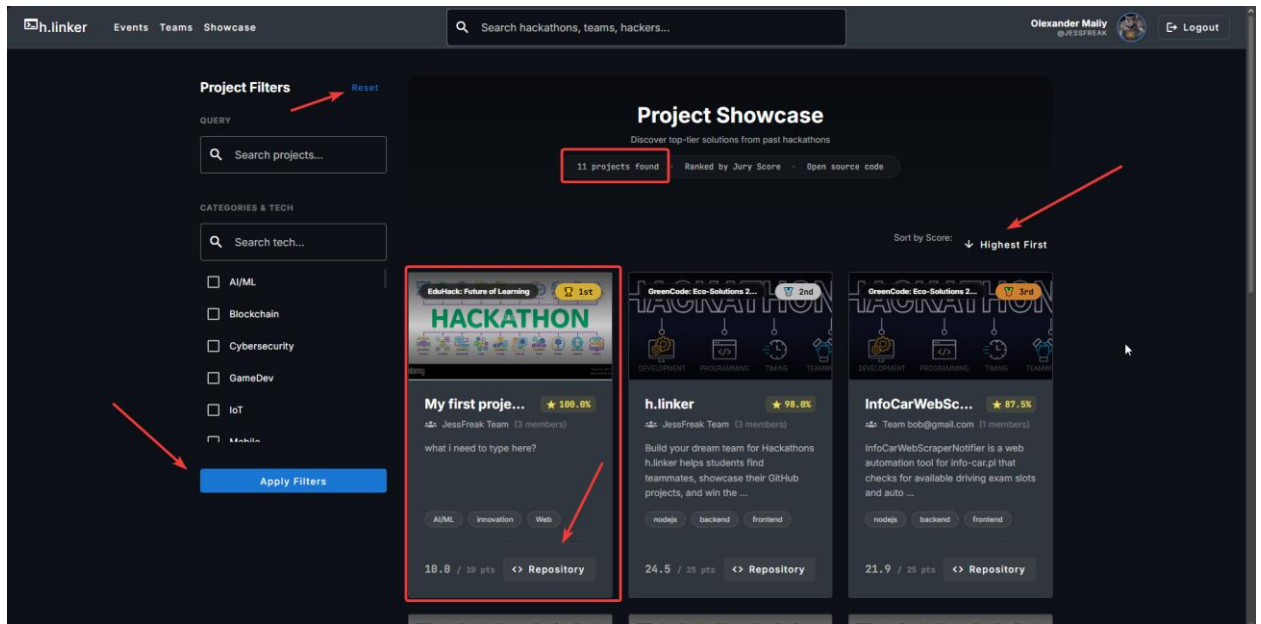


Рисунок 3.42 – Сторінка «Project Showcase»

Також є глобальний рядок пошуку, розміщений у верхній навігаційній панелі (рис. 3.43). Цей інструмент забезпечує наскрізний пошук по всій системі. Під час введення текстового запиту система аналізує дані та формує зручний випадаючий список результатів, який автоматично структурується за логічними категоріями: хакатони (події), команди та профілі розробників. Натискання на будь-який із знайдених елементів у цьому списку миттєво перенаправляє користувача на відповідну сторінку з детальною інформацією про обраний об'єкт, що значно спрощує та прискорює загальну навігацію платформою.

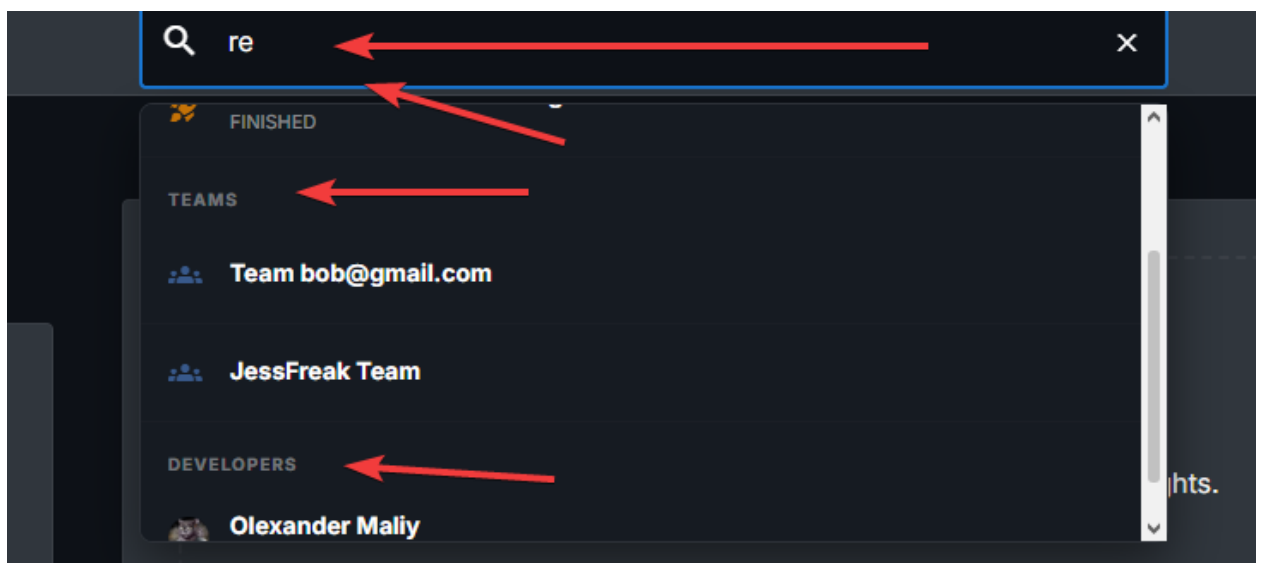


Рисунок 3.43 – Робота глобального пошуку

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

«ЗАТВЕРДЖЕНО»

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ВЕБЗАСТОСУНОК ДЛЯ ОРГАНІЗАЦІЇ ХАКАТОНІВ З
ДИНАМІЧНИМ МОДУЛЕМ ОЦІНЮВАННЯ**

Графічний матеріал

КПІ.ПІ-2419.045440.06.99

«ПОГОДЖЕНО»

Керівник проєкту:

_____ Олена МАРЧЕНКО

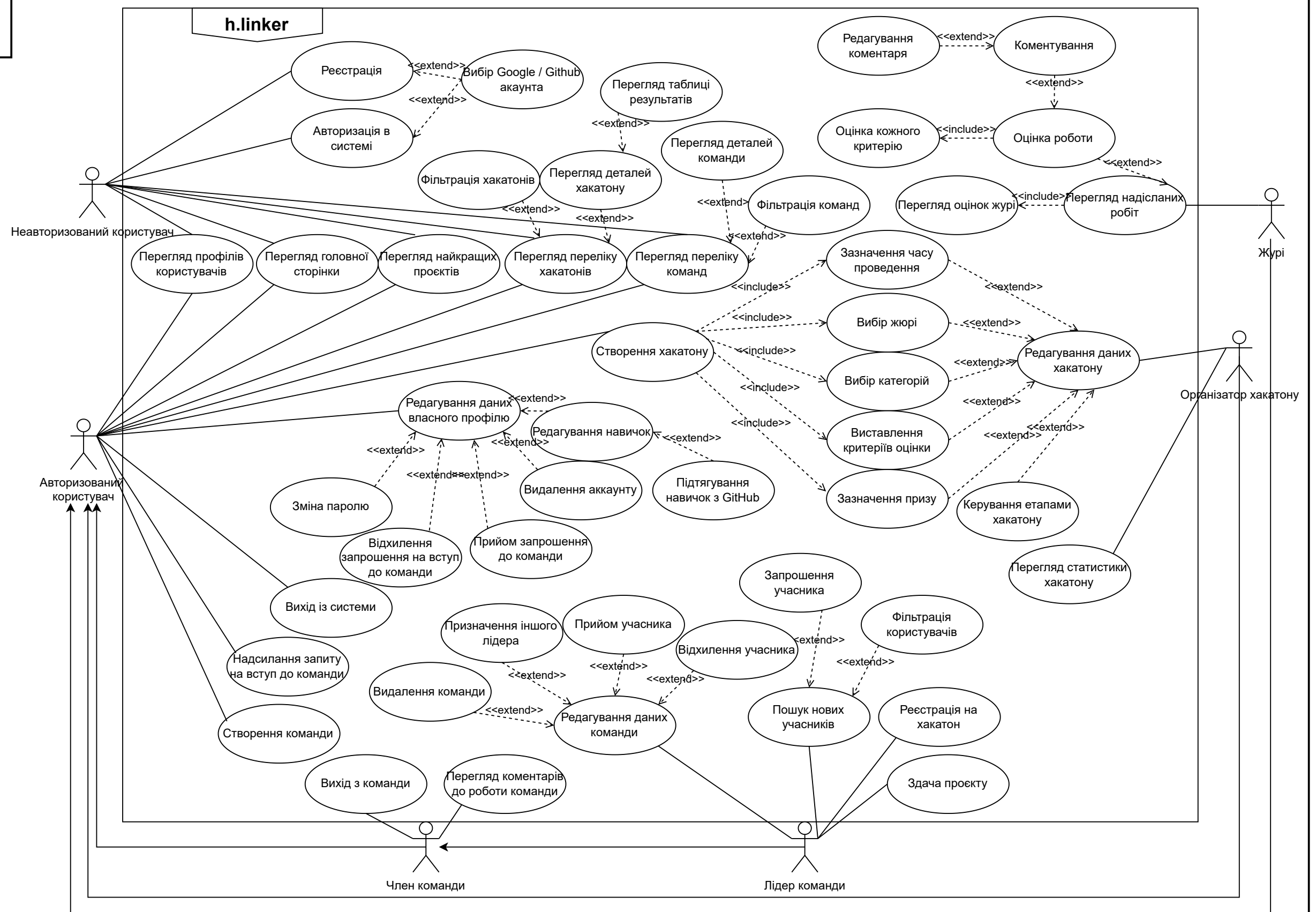
Нормоконтроль:

_____ Тетяна ШУЛЬКЕВИЧ

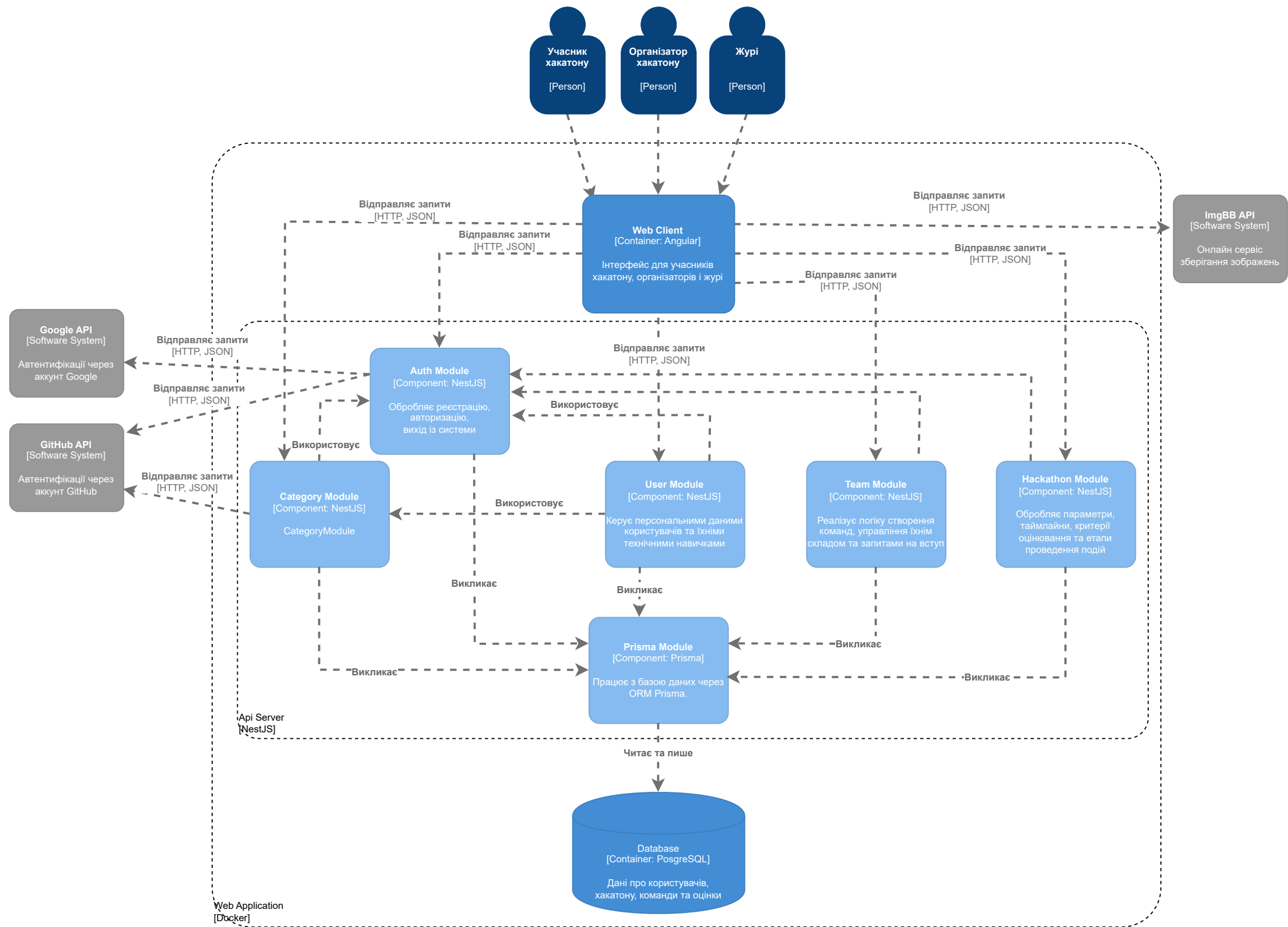
Виконавець:

_____ Олександр МАЛИЙ

Київ – 2026

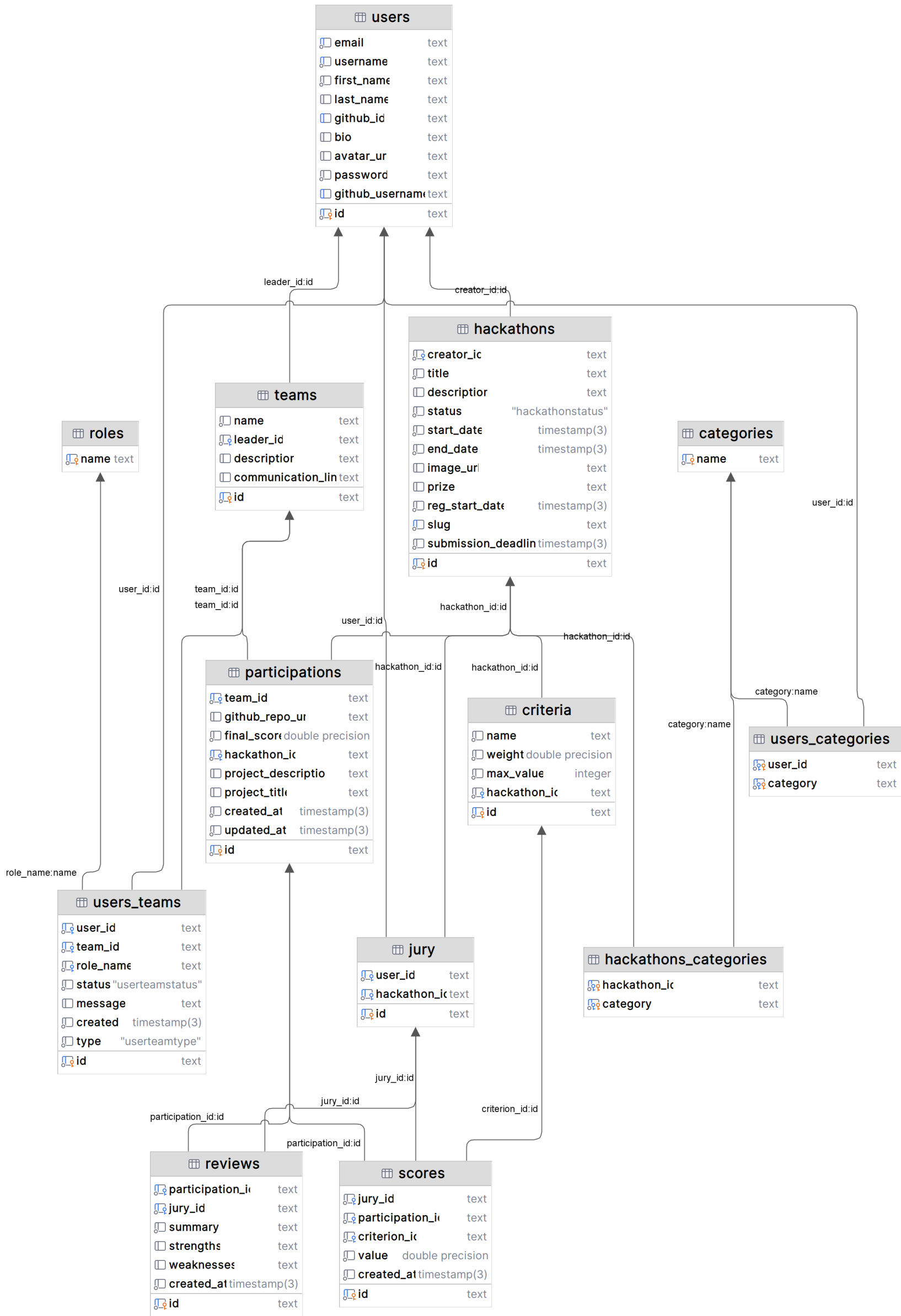


					КПІ.ІП-2419.045440.06.99.CCB						
					Схема структурна варіантів використань	Літера			Маса	Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата							
Розробив	Малий О.С.										
Перевірів	Марченко О. І.										
Т. контр.						Аркуш 1			Аркушів 1		
					Вебзастосунок для організації хакатонів із динамічним модулем оцінювання	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-24					
Н. контр.	Шулькевич Т.В.										
Затвердив	Жаріков Е.В.										

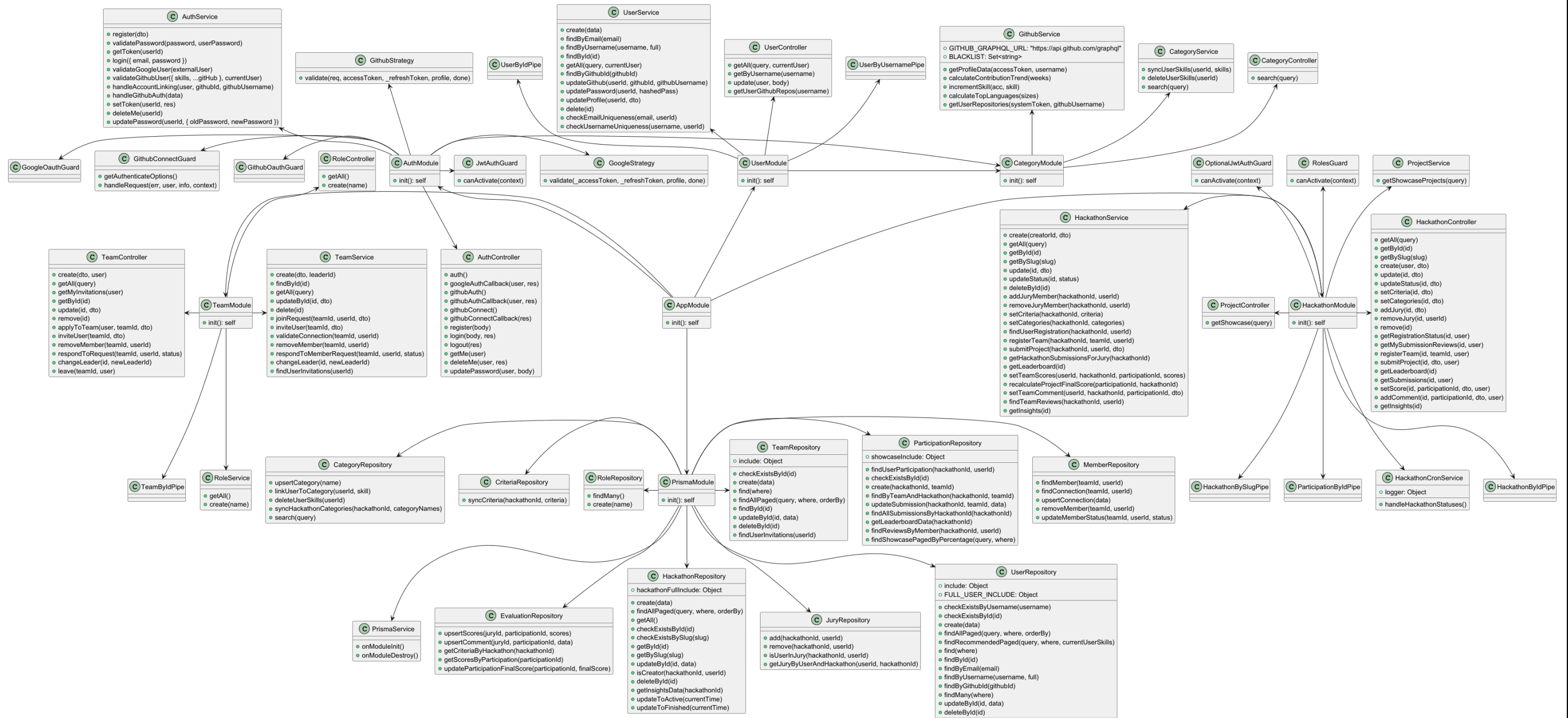


					КПІ.ІП-2419.045440.06.99.CCM			
					Схема структурна компонентів програмного забезпечення			
Зм.	Арк.	№ документа	Підпис	Дата				
Розробив		Малий О.С.			Вебзастосунок для організації хакатонів із динамічним модулем оцінювання			
Перевірів		Марченко О. І.						
Т. контр.								
Н. контр.		Шулькевич Т.В.			КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-24			
Затвердив		Жаріков Е.В.						

Літера			Маса	Масштаб
Аркуш 1		Аркушів 1		



					КПІ.ІП-2419.045440.06.99.СБД								
					Схема бази даних				Лит.		Маса	Масштаб	
Зм.	Арк.	№ докум.	Підп.	Дата									
Розробив		Малий О.С.											
Перевірів		Марченко О. І.											
Т. контр.									Аркуш 1		Аркушів 1		
					Вебзастосунок для організації хакатонів із динамічним модулем оцінювання				КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-24				
Н. контр.		Шулькевич Т.В.											
Затвердив		Жаріков Е.В.											



					КПІ.ІП-2419.045440.06.99.ССК				
					Схема структурна класів програмного забезпечення	Літера		Маса	Масштаб
Зм.	Арк.	№ документа	Підпис	Дата					
Розробив	Малий О.С.								
Перевірів	Марченко О. І.								
Т. контр.						Аркуш 1		Аркушів 1	
					КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-24				
Н. контр.	Шулькевич Т.В.				Вебзастосунок для організації хакатонів із динамічним модулем оцінювання				
Затвердив	Жаріков Е.В.								